UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Ingeniería de Requisitos

Sílabo y material de estudio

Código de la asignatura: **ISR-401**

Cuarto nivel — 3 créditos — 96 horas presenciales

*Material de estudio elaborado conforme al estándar internacional **IEEE/ISO/IEC 29148:2018**, la guía **SWEBOK v4.0 (2024)**, los modelos de calidad **ISO/IEC 25010:2023** y la segunda edición del manual de Pohl (Springer, 2024).*

Período académico: 2025 – 2026

Quevedo – Los Ríos – Ecuador

«La especificación de requisitos es el momento de la ingeniería del software en el que se decide, en última instancia, si el sistema va a resolver el problema correcto. Todo lo que viene después es construcción; lo que viene antes es ingeniería de requisitos. »

— adaptado de Klaus Pohl [1]

Aviso: Este material acompaña al *Programa Analítico* oficial de la asignatura aprobado por la Facultad de Ciencias de la Computación. Los contenidos están enmarcados en los estándares y guías internacionales que se citan a lo largo del texto y se sustentan en bibliografía verificable, según las normas IEEE.

Índice general

Lista de acrónimos **XVI**

1. Encuadre académico y fundamentos de requisitos de software **1**

1.1. Tema 1. Encuadre académico de la asignatura 1

1.1.1. Datos generales de la asignatura 2

1.1.2. Mapa global del semestre 2

1.1.3. Metodología y estrategias didácticas 2

1.1.4. Sistema de evaluación 3

1.1.5. Introducción a la Ingeniería de Requisitos 3

1.2. Tema 2. Fundamentos de requisitos de software 5

1.2.1. Subtema 2.1. Definición de requisito de software. Requisitos de productos vs. requisitos de procesos 5

1.2.2. Subtema 2.2. Requisitos funcionales y no funcionales. Propiedades emergentes. Requisitos cuantificables 8

1.2.3. Subtema 2.3. Requisitos del sistema vs. requisitos del software. Restricciones y dependencias 11

1.2.4. Subtema 2.4. Taxonomía de requisitos de calidad (ISO/IEC 25010:2023). Los llamados NFR 15

1.3. Mapa síntesis de la Semana 1 20

1.4. Práctica experimental y trabajo autónomo 20

1.5. Resumen ejecutivo de la semana 21

2. Proceso de ingeniería de requisitos y contexto del sistema **23**

2.1. Tema 3. Proceso de requisitos 23

2.1.1. Subtema 3.1. Modelos de proceso de requisitos. Actores 23

2.1.2. Subtema 3.2. Administración y soporte de procesos. Interrelación con el ciclo de vida del software 26

2.1.3. Subtema 3.3. Perfeccionamiento y calidad del proceso de IR 27

2.1.4. Subtema 3.4. El framework de IR: actividades core, actividades transversales y artefactos 29

2.2. Tema 4. Contexto, tipos de sistemas y visión 30

2.2.1. Subtema 4.1. Tipos de sistemas y consecuencias para IR 30

2.2.2. Subtema 4.2. Límite del sistema, descomposición y visión 32

2.2.3. Subtema 4.3. Las ocho perspectivas de contexto de Pohl 34

2.3. Mapa síntesis de la Semana 2 37

2.4. Práctica experimental y trabajo autónomo 38

2.5. Resumen ejecutivo de la semana 39

3. Elicitación de requisitos: técnicas clásicas **41**

3.1. Tema 5. Elicitación de requisitos . . . 41
3.1.1. Subtema 5.1. Fundamentos y fuentes de la elicitación . . . 41
3.1.2. Subtema 5.2. Entrevistas y cuestionarios . . . 43
3.1.3. Subtema 5.3. Talleres de requisitos y grupos focales . . . 45
3.1.4. Subtema 5.4. Observación y etnografía . . . 46
3.1.5. Subtema 5.5. Análisis de documentos y sistemas existentes . . . 48
3.2. Mapa síntesis de la Semana 3 . . . 50
3.3. Práctica experimental y trabajo autónomo . . . 50
3.4. Resumen ejecutivo de la semana . . . 51

4. Técnicas avanzadas de elicitación y modelado de metas 52
4.1. Tema 6. Técnicas avanzadas de elicitación . . . 52
4.1.1. Subtema 6.1. Prototipado como técnica de elicitación . . . 52
4.1.2. Subtema 6.2. Modelado orientado a metas: i* y KAOS . . . 54
4.1.3. Subtema 6.3. Modelado de procesos de negocio con BPMN . . . 56
4.1.4. Subtema 6.4. Historias de usuario y técnicas ágiles de elicitación . . . 57
4.2. Mapa síntesis de la Semana 4 . . . 60
4.3. Práctica experimental y trabajo autónomo . . . 60
4.4. Resumen ejecutivo de la semana . . . 61

5. Análisis de requisitos: conflictos, priorización y calidad 62
5.1. Tema 7. Análisis de requisitos . . . 62
5.1.1. Subtema 7.1. Análisis y clasificación de requisitos . . . 62
5.1.2. Subtema 7.2. Detección y resolución de conflictos entre requisitos . . . 64
5.1.3. Subtema 7.3. Técnicas de priorización de requisitos . . . 65
5.1.4. Subtema 7.4. Calidad de los requisitos y trazabilidad . . . 67
5.2. Mapa síntesis de la Semana 5 . . . 70
5.3. Práctica experimental y trabajo autónomo . . . 70
5.4. Resumen ejecutivo de la semana . . . 71

6. Metas, escenarios y requisitos orientados a solución 72
6.1. Tema 7. Metas, escenarios y requisitos orientados a solución . . . 72
6.1.1. Subtema 7.1. Metas: concepto, descomposición y documentación . . . 72
6.1.2. Subtema 7.2. Escenarios: tipos, documentación y once reglas . . . 75
6.1.3. Subtema 7.3. Diagramas de CU UML y Message Sequence Charts . . . 79
6.1.4. Subtema 7.4. Requisitos orientados a solución: los tres modelos . . . 81
6.2. Mapa síntesis de la Semana 6 . . . 83
6.3. Práctica experimental y trabajo autónomo . . . 83
6.4. Resumen ejecutivo de la semana . . . 84

7. Documentación de requisitos 85
7.1. Tema 8. Documentación de requisitos . . . 85
7.1.1. Subtema 8.1. Propósito de la documentación y tipos de artefactos . . . 85

7.1.2. Subtema 8.2. Formatos de representación y técnicas de precisión lingüística 86
7.1.3. Subtema 8.3. Atributos de requisito: la ficha de especificación individual 89
7.1.4. Subtema 8.4. Criterios de calidad y criterios de aceptación 91
7.2. Mapa síntesis de la Semana 7 93
7.3. Práctica experimental y trabajo autónomo 93
7.4. Resumen ejecutivo de la semana 94

8. El documento ERS/SRS 95
8.1. Tema 9. El documento ERS/SRS 95
8.1.1. Subtema 9.1. Tipos de especificaciones en la familia IEEE 29148:2018 . 95
8.1.2. Subtema 9.2. Estructura de referencia del ERS según IEEE 29148:2018 97
8.1.3. Subtema 9.3. Lenguaje natural: verbos modales y técnicas de precisión . 100
8.1.4. Subtema 9.4. Combinación de modelos UML y texto en el ERS 101
8.2. Mapa síntesis de la Semana 8 104
8.3. Práctica experimental y trabajo autónomo 105
8.4. Resumen ejecutivo de la semana 105

9. Modelos de especificación UML: datos y función 107
9.1. Subtema 10.1. Modelos de datos: ER y clases UML 107
9.1.1. El modelo Entidad-Relación como puerta de entrada a los datos 107
9.2. Subtema 10.2. Modelos funcionales: DFD y actividades UML 110
9.2.1. El diagrama de flujo de datos en IR 111
9.2.2. Diagramas de actividades UML 113
9.3. Mapa síntesis de la Semana 9 115
9.4. Práctica experimental y trabajo autónomo 115
9.5. Resumen ejecutivo de la semana 116

10. Modelos de comportamiento e interrelaciones 117
10.1. Subtema 10.3. Autómatas finitos, statecharts y máquinas de estado UML . . . 117
10.1.1. De los autómatas finitos a los statecharts 117
10.2. Subtema 10.4. Interrelaciones entre las tres perspectivas 121
10.2.1. El principio de coherencia de Pohl 121
10.2.2. Checklist de coherencia del ERS (vista de modelos) 122
10.2.3. Integración de los cuatro modelos en el ERS del PFC 123
10.3. Mapa síntesis de la Semana 10 123
10.4. Práctica experimental y trabajo autónomo 124
10.5. Resumen ejecutivo de la semana 124

11. Validación de requisitos 126
11.1. Subtema 11.1. Motivación y principios de validación 126
11.1.1. ¿Por qué validar los requisitos? 126
11.1.2. Los seis principios de validación de Pohl 127
11.2. Subtema 11.2. Walkthroughs, revisiones estructuradas e inspecciones 127

11.2.1. Las tres modalidades de revisión de requisitos 128
11.3. Subtema 11.3. Prototipado como técnica de validación 130
11.3.1. El prototipo de validación vs. el prototipo de elicitación 130
11.3.2. Validación de modelos UML 131
11.4. Subtema 11.4. Pruebas de aceptación y relación con requisitos 131
11.4.1. Tipos de pruebas de aceptación 132
11.4.2. Vínculo entre GWT, RTM y plan de pruebas 133
11.5. Mapa síntesis de la Semana 11 134
11.6. Práctica experimental y trabajo autónomo 134
11.7. Resumen ejecutivo de la semana 135

12. Gestión de requisitos **136**
12.1. Subtema 12.1. Naturaleza iterativa de la IR y gestión del contexto 136
12.1.1. La IR no termina cuando empieza el desarrollo 136
12.1.2. Ciclo de vida iterativo de la IR 137
12.1.3. Gestión del contexto: fuentes de cambio en PFC agropecuarios 137
12.2. Subtema 12.2. Gestión del cambio: CCB, proceso y documentación 138
12.2.1. El proceso de gestión del cambio 138
12.3. Subtema 12.3. Seguimiento y trazabilidad de requisitos 139
12.3.1. Los tres tipos de trazabilidad 139
12.3.2. Trazabilidad dirigida por uso 140
12.4. Subtema 12.4. Medición, priorización, configuración y baselines 140
12.4.1. Métricas de gestión de requisitos 141
12.4.2. Gestión de la configuración del ERS 141
12.4.3. Baselines del ERS en el PFC 143
12.5. Mapa síntesis de la Semana 12 143
12.6. Práctica experimental y trabajo autónomo 144
12.7. Resumen ejecutivo de la semana 144

13. Herramientas CASE y estándares para IR **146**
13.1. Subtema 13.1. Herramientas CASE para IR 146
13.1.1. Panorama de herramientas 147
13.2. Subtema 13.2. IEEE 29148:2018 e ISO/IEC 25010:2023 en proyectos reales 148
13.2.1. IEEE 29148:2018: más allá del ERS 148
13.2.2. ISO/IEC 25010:2023 como criterio de RNF 149
13.3. Subtema 13.3. Atributos y trazabilidad en herramientas CASE 150
13.3.1. Esquema de atributos configurado en Jira para el PFC 150
13.4. Subtema 13.4. Gestión de la configuración del ERS en Git 151
13.4.1. Flujo de trabajo Git para el ERS del PFC 151
13.5. Mapa síntesis de la Semana 13 152
13.6. Práctica experimental y trabajo autónomo 153
13.7. Resumen ejecutivo de la semana 153

14. Ingeniería de Requisitos en procesos ágiles **154**
14.1. Subtema 14.1. IR en procesos ágiles: backlog, historias y criterios 154
14.1.1. El product backlog como ERS ágil 154
14.2. Subtema 14.2. BDD: especificación ejecutable de requisitos 156
14.2.1. La anatomía completa de un escenario Gherkin 156
14.2.2. De Gherkin a prueba ejecutable con Pytest-BDD 158
14.3. Subtema 14.3. Integración de IR con Scrum 158
14.3.1. El rol del Product Owner en la IR ágil 158
14.3.2. IR en cada evento Scrum 159
14.4. Subtema 14.4. XP y comparación IR tradicional vs. IR ágil 159
14.4.1. IR en Extreme Programming (XP) 159
14.4.2. Comparación IR tradicional vs. IR ágil 160
14.5. Mapa síntesis de la Semana 14 161
14.6. Práctica experimental y trabajo autónomo 161
14.7. Resumen ejecutivo de la semana 162

15. Proyecto integrador: planificación, levantamiento y análisis **163**
15.1. Subtema 15.1. Selección del sistema, stakeholders y plan de IR 163
15.1.1. Criterios para seleccionar el sistema objetivo del PFC 163
15.1.2. Mapa de stakeholders del PFC 164
15.1.3. Plan de IR del PFC 165
15.2. Subtema 15.2. Elicitación con usuarios reales 166
15.2.1. Guía de entrevista real para el PFC agropecuario 166
15.2.2. Taller de requisitos real: estructura para el PFC 166
15.3. Subtema 15.3. Clasificación, priorización y negociación 167
15.3.1. Clasificación de los candidatos a requisitos del PFC 167
15.3.2. Negociación de conflictos reales del PFC 168
15.4. Mapa síntesis de la Semana 15 168
15.5. Actividades del proyecto integrador 169
15.6. Resumen ejecutivo de la semana 169

16. Especificación, modelado, validación y defensa del PFC **170**
16.1. Subtema 16.1. ERS/SRS completo según IEEE 29148 170
16.1.1. Lista de verificación del ERS del PFC 170
16.2. Subtema 16.2. Modelos UML: revisión final y coherencia 171
16.2.1. Revisión de coherencia de los modelos del PFC 171
16.3. Subtema 16.3. Revisión cruzada entre equipos 172
16.3.1. Protocolo de peer review entre PFC 172
16.4. Subtema 16.4. Métricas, trazabilidad en CASE y rubrica de evaluación 173
16.4.1. Métricas de calidad del ERS del PFC con ISO 25010 173
16.5. Subtema 17.1–17.3. Defensa final del PFC 174
16.5.1. Estructura de la presentación oral del ERS 174

16.5.2. Simulacro de solicitudes de cambio en la defensa 175
16.6. Mapa síntesis de la Semana 16 y cierre del curso 175
16.7. Actividades finales del proyecto integrador 176
16.8. Resumen ejecutivo de la semana y del curso 176

17. Métricas, trazabilidad y preparación de la defensa 177
17.1. Subtema 17.1. Trazabilidad en CASE y configuración final del ERS 177
17.1.1. Verificación final de la trazabilidad en la herramienta CASE 177
17.1.2. Estado final del repositorio Git del PFC 179
17.2. Subtema 17.2. Evaluación del ERS con rúbrica: autoevaluación final 179
17.2.1. Autoevaluación del ERS con la rúbrica institucional 179
17.2.2. Errores más frecuentes detectados en defensas anteriores 180
17.2.3. Completitud, consistencia y verificabilidad: checklist final 181
17.3. Subtema 17.3. Preparación y estructura de la defensa final 182
17.3.1. Estructura de la presentación: los 20 minutos más importantes 183
17.3.2. Simulacro de respuesta a solicitudes de cambio 184
17.4. Mapa síntesis de la Semana 17 185
17.5. Actividades de la semana 17 185
17.6. Resumen ejecutivo de la semana 186

18. Defensa formal, evaluación y cierre del semestre 187
18.1. Protocolo de la defensa formal del PFC 187
18.1.1. Estructura de la sesión de defensa 187
18.1.2. Rúbrica del tribunal: cómo se aplica la calificación 188
18.1.3. Preguntas más frecuentes del tribunal 189
18.2. Evaluación final y coevaluación entre equipos 191
18.2.1. Coevaluación: los estudiantes como tribunal 191
18.3. Reflexión final: el proceso de IR visto desde el final del semestre 191
18.3.1. ¿Qué hemos aprendido sobre IR en 18 semanas? 191
18.3.2. Las diez lecciones del semestre 192
18.3.3. El camino hacia la certificación IREB 193
18.4. Cierre del semestre: evaluación del proceso de aprendizaje 194
18.4.1. Autoevaluación y coevaluación del proceso 194
18.4.2. Mensaje de cierre 194
18.5. Mapa de cierre del semestre completo 195

Glosario de términos 196

Referencias bibliográficas 197

Índice de figuras

1.1. Secuencia de las cinco unidades de la asignatura y su distribución aproximada en el calendario semestral. . . . 2
1.2. Framework de Ingeniería de Requisitos según Pohl: tres clases de artefactos, tres actividades centrales y dos actividades transversales, todo enmarcado por el contexto del sistema. Adaptado de [1]. . . . 4
1.3. Cadena de transformación de la necesidad al requisito de software. Adaptado de [1-3]. . . . 5
1.4. Pirámide invertida de niveles de requisitos. Cada nivel se deriva del anterior mediante un proceso de *refinamiento y asignación* (allocation) [2]. . . . 12
1.5. Las nueve características de calidad de ISO/IEC 25010:2023 [4]. La novena característica, **Safety**, constituye la novedad principal de la edición 2023. . . . 16

2.1. Tres modelos de proceso de Ingeniería de Requisitos (IR). (a) En cascada: actividades ejecutadas una vez en secuencia. (b) Iterativo: ciclos sucesivos refinan el conjunto de requisitos. (c) Ágil: requisitos descubiertos y desarrollados sprint a sprint. Adaptado de [11, 32-34]. . . . 24
2.2. La IR en el ciclo de vida del software. Adaptado de [1, 3, 23]. . . . 27
2.3. Framework completo de IR de Pohl 2024: contexto, tres artefactos, tres actividades core y dos actividades transversales [1]. . . . 29
2.4. Sistema, contexto y entorno: los tres niveles que la IR debe distinguir. Adaptado de [1]. . . . 33
2.5. Las ocho perspectivas de contexto de Pohl 2024 [1]. . . . 35

3.1. Ciclo de cuatro pasos de la elicitación de requisitos. La flecha roja indica que la validación suele generar nuevas preguntas y una nueva iteración. Adaptado de [1, 52]. . . . 43

4.1. Modelo SD simplificado en i* para el sistema de turnos del Ministerio de Salud Pública del Ecuador (MSP). Las dependencias muestran lo que cada actor necesita de los demás. Adaptado de [1, 58]. . . . 55

5.1. Curvas del Modelo Kano. Los requisitos básicos no elevan la satisfacción aunque se implementen bien; los atractivos generan un salto de satisfacción no anticipado. Adaptado de [10, 64]. . . . 67

6.1. Grafo KAOS del sistema de turnos MSP. OR-refinement en nivel 1 (alternativas de diseño); AND-refinement en nivel 2 (todas las sub-metas son necesarias). Las meta hojas verdes operacionalizan en RF; el óvalo dorado es una suposición sobre el entorno. Adaptado de [1, 42]. . . . 74
6.2. MSC del CU-01: Reservar turno médico (flujo básico completo). Flechas sólidas = mensajes entrantes; flechas discontinuas = respuestas. El rectángulo azul en la lifeline del sistema indica el período de procesamiento. Adaptado de [1, 70]. . 80

6.3. Diagrama de clases UML del dominio del sistema de turnos MSP. Se modela el *problema*, no el diseño. Las multiplicidades expresan reglas de negocio verificables. Adaptado de [1, 71]. 81
6.4. Statechart del ciclo de vida de la entidad `Turno`. Cada transición muestra el evento disparador y la acción ejecutada por el sistema (separados por "/"). Adaptado de [1, 72]. 82

7.1. Continuum de formalidad de los formatos de documentación de requisitos. Las tres flechas muestran los tres atributos que varían a lo largo del eje. Adaptado de [1, 2]. 86

8.1. Jerarquía de documentos de requisitos de la familia IEEE 29148:2018. Las flechas indican el flujo de derivación. En proyectos pequeños a medianos, el Especificación de Requisitos de Software (ERS)/SRS (verde) puede ser el único documento formal. Adaptado de [2, 66]. 96
8.2. Ciclo de vida del ERS: desde el primer borrador hasta el baseline y la gestión de cambios posterior. El Change Control Board (Comité de Control de Cambios) (CCB) es el árbitro de todos los cambios al baseline. Adaptado de [1, 2, 10]. . . 103

9.1. Diagrama ER del sistema de gestión ganadera (PFC — Los Ríos). Notación simplificada Crow's Foot. El arete es la clave primaria del bovino, conforme al sistema de identificación AGROCALIDAD. Adaptado de [1, 76]. 109
9.2. Diagrama de clases UML (dominio) del sistema de clínica veterinaria (PFC — Quevedo). La composición entre Consulta y Receta indica que la receta no existe sin una consulta. Adaptado de [70, 71]. 110
9.3. DFD Nivel 0 (diagrama de contexto) del sistema de gestión de camaroneras. Las entidades externas identifican las interfaces del sistema y generan directamente los RF de integración. Adaptado de [78, 79]. 112
9.4. DFD Nivel 1 del sistema de camaroneras. Cada burbuja de proceso genera al menos un RF; cada depósito de datos genera entidades en el ER. Adaptado de [1, 78, 79]. 112
9.5. Diagrama de actividades UML del proceso de cosecha de palma africana. El fork indica ejecución paralela (el cortador descarga mientras el pesador registra). La decisión modela la regla de negocio del peso mínimo de cierre de jornada. Cada acción genera RF; el fork genera un RNF de concurrencia. Adaptado de [71, 77]. 114

10.1. Autómata finito determinista del nodo sensor IoT de la piscina camaronera. Cada transición genera un RF de comportamiento: el evento es el disparador; la acción tras `/` es lo que el sistema ejecuta. Adaptado de [1, 2]. 118
10.2. Statechart (Harel) del ciclo de vida del bovino en el PFC de ganadería. El estado compuesto *Etapa Reproductiva* agrupa los sub-estados reproductivos, evitando repetir transiciones. Las guardas (entre corchetes) generan reglas de negocio; las acciones (tras /) generan RF. Adaptado de [1, 72]. 120

10.3. Diagrama de máquina de estados UML para la entidad `Cita` en el sistema de clínica veterinaria. Los estados ATENDIDA, NO ASISTIDA y CANCELADA son estados finales distintos, necesarios para las estadísticas de gestión de la clínica. Adaptado de [72, 77]. 121

11.1. Cadena de trazabilidad desde el requisito hasta el resultado de la prueba de aceptación, pasando por la RTM. Adaptado de [2, 60]. 133

12.1. Ciclo de vida iterativo de la IR. La gestión del cambio (centro) actúa como regulador de todas las iteraciones posteriores al primer baseline. Adaptado de [1, 2]. 137

13.1. Flujo de trabajo Git recomendado para el ERS del PFC. La rama `main` contiene solo versiones aprobadas por el CCB; los cambios entran via Pull Request revisado. Los tags marcan los baselines. Adaptado de [1, 2]. 152

15.1. Mapa de stakeholders del PFC de gestión ganadera. Los stakeholders en el cuadrante superior derecho (involucrar activamente) deben participar en todas las sesiones de elicitación y revisión del ERS. Adaptado de [1, 52]. 165

Índice de tablas

1.1. Datos generales de la asignatura Ingeniería de Requisitos. . . . 2
1.2. Componentes del sistema de evaluación de la asignatura. . . . 3
1.3. Las nueve características de un requisito bien formado, con ejemplos contextualizados. Adaptado de [1, 2]. . . . 6
1.4. Requisitos de producto vs. requisitos de proceso. . . . 8
1.5. Las tres familias de requisitos no funcionales según [11]. . . . 9
1.6. Conversión de enunciados vagos en requisitos cuantificables. . . . 11
1.7. Niveles canónicos de requisitos según IEEE 29148:2018 [2]. . . . 12
1.8. Asignación (allocation) de requisitos del sistema al software, con ejemplos ecuatorianos. . . . 13
1.9. Tipos comunes de restricciones, con ejemplos en Ecuador. . . . 14
1.10. Tipos de dependencias entre requisitos. . . . 15
1.11. Cambios principales entre ISO/IEC 25010:2011 e ISO/IEC 25010:2023 [4, 20]. . 16
1.12. Las nueve características de calidad ISO/IEC 25010:2023 con sus subcaracterísticas [4]. . . . 17
1.13. Mapa de cierre de la Semana 1. . . . 20

2.1. Comparación de los tres modelos de proceso de IR según varios autores [1, 11, 21, 22, 31]. . . . 25
2.2. Roles típicos en el proceso de IR. . . . 26
2.3. Modelo Requirements Engineering Process Maturity Model (REPM) de Sawyer, Sommerville y Viller [24]. . . . 28
2.4. Tipos de sistemas y sus consecuencias para la IR. . . . 31
2.5. Plantilla de documento de visión adaptada de [10]. . . . 34
2.6. Las ocho perspectivas de contexto, sus preguntas guía y ejemplos. . . . 35
2.7. Plantilla de análisis de contexto por perspectivas. . . . 37
2.8. Mapa de cierre de la Semana 2. . . . 38

3.1. Fuentes de requisitos y técnicas de elicitación asociadas. Adaptado de [1, 2, 10]. 42
3.2. Tipos de entrevista y sus características [1, 10, 52]. . . . 43
3.3. Técnicas de formulación de preguntas para entrevistas de elicitación [1, 10, 52]. 44
3.4. Estructura de un taller de requisitos de un día. Adaptado de [10, 52]. . . . 45
3.5. Modalidades de observación en IR [1, 54, 55]. . . . 47
3.6. Tipos de documentos útiles para la elicitación en proyectos ecuatorianos. . . . 48
3.7. Comparación de las técnicas clásicas de elicitación por criterios relevantes [1, 10, 50, 52]. . . . 49
3.8. Mapa de cierre de la Semana 3. . . . 50

4.1. Taxonomía de prototipos en IR por fidelidad y técnica. Adaptado de [1, 10]. . . 53

4.2. Operadores de descomposición de metas en KAOS [42]. 56
4.3. Elementos BPMN 2.0 más relevantes para la elicitación de requisitos [59]. . . . 57
4.4. Criterios INVEST para evaluar la calidad de una historia de usuario [37]. . . . 58
4.5. Jerarquía de artefactos de requisitos en metodologías ágiles. Adaptado de [37, 38]. 59
4.6. Mapa de cierre de la Semana 4. 60

5.1. Ejemplo de clasificación multidimensional de requisitos del sistema de turnos MSP [1, 2]. 63
5.2. Las cinco categorías de defectos más frecuentes en los requisitos, con ejemplos y correcciones [10, 62, 63]. 64
5.3. Categorías Must-have, Should-have, Could-have, Won't-have (técnica de priorización) (MoSCoW) y su interpretación [10]. 66
5.4. Categorías del Modelo Kano y su impacto en la satisfacción del cliente. Adaptado de [10, 64]. 66
5.5. Ejemplo de priorización por valor/costo/riesgo para el sistema de matrícula Universidad Técnica Estatal de Quevedo (UTEQ). Ponderaciones: $w_V = 1, w_P = 1, w_C = 1, w_R = 0{,}5$. 67
5.6. Características de calidad del conjunto de requisitos según [2]. 68
5.7. Ejemplo de Requirements Traceability Matrix (Matriz de Trazabilidad de Requisitos) (RTM) para el sistema de matrícula UTEQ. 69
5.8. Mapa de cierre de la Semana 5. 70

6.1. Taxonomía de metas según [1, 2, 42]. 73
6.2. Las siete reglas de Pohl para la documentación textual de metas [1]. Se incluye contraejemplo y corrección para cada regla. 74
6.3. Elementos del modelo SR en iStar 2.0 [58, 68]. 75
6.4. Tipos de escenario y sus características [1, 37, 52, 69, 73]. 76
6.5. CU-01: Reservar turno médico — plantilla Cockburn *fully dressed* [1, 69]. . . . 77
6.6. MC-01: Suplantación de identidad para reservar turno masivamente. Adaptado de [1, 14, 73]. 79
6.7. Relaciones en los diagramas de CU UML [1, 69, 70]. 80
6.8. Las tres perspectivas de los modelos orientados a solución [1, 71, 72]. 81
6.9. Tabla de coherencia entre los tres modelos ROS del sistema de turnos MSP. Cada fila verifica que una entidad o evento aparezca consistentemente en los tres modelos [1, 2]. 82
6.10. Mapa de cierre de la Semana 6. 83

7.1. Tipos de artefactos de documentación de requisitos [1, 2]. 86
7.2. Los ocho *requirement smells* más frecuentes según Femmer et al. [63] y Tiwari et al. [75]. 88
7.3. Los cinco patrones de la plantilla EARS con ejemplos ecuatorianos [1, 15]. . . . 89
7.4. Los diez atributos de un requisito individual según [2, 10, 52]. 90
7.5. Ficha Volere para RF-01: Reservar turno médico. Adaptado de [52]. 91

7.6. Criterios de aceptación en formato GWT para RF-01 y RNF-03. Adaptado de [2, 60]. 92
7.7. Métricas de calidad del proceso de IR más usadas en proyectos medianos [1, 10]. 92
7.8. Mapa de cierre de la Semana 7. 93

8.1. Familia de documentos IEEE/ISO/IEC 29148:2018 y su posición en la jerarquía de requisitos [2, 66]. 96
8.2. Criterios de selección del tipo de especificación según el contexto del proyecto [1, 2]. 97
8.3. Estructura de referencia canónica del ERS/SRS según [2, 10, 52]. 97
8.4. Capítulo 4 del ERS del sistema de matrículas UTEQ. Ejemplo de organización de RF por subsistema funcional. 99
8.5. Jerarquía de verbos modales en la redacción de requisitos según [1, 2, 74]. . . . 100
8.6. Estrategias de integración de modelos UML y texto en el ERS [1, 70, 71]. . . . 102
8.7. RTM extendida para los RF más críticos del sistema de matrícula UTEQ. La trazabilidad es bidireccional en todas las columnas [2, 66]. 103
8.8. Proceso mínimo del CCB para proyectos universitarios y medianos [2, 10]. . . . 104
8.9. Mapa de cierre de la Semana 8. 104

9.1. Comparación de las dos notaciones ER más usadas en IR [71, 76]. 108
9.2. Reglas de mapeo de ER a diagrama de clases UML [70, 71, 77]. 109
9.3. Niveles del DFD y su uso en la especificación de requisitos [3, 78, 79]. 111
9.4. Mapa de cierre de la Semana 9. 115

10.1. Tabla de transiciones del AFD del sensor IoT y los RF derivados. 119
10.2. Tabla de interrelaciones entre los tres modelos del PFC de ganadería. Cada celda vacía indica una posible omisión que el analista debe justificar o corregir [1, 2, 66]. 122
10.3. Checklist de coherencia de los tres modelos del ERS [1, 2, 10]. 122
10.4. Mapa de cierre de la Semana 10. 123

11.1. Costo relativo de corrección de defectos de requisitos según la fase de detección [7, 9, 35]. 127
11.2. Comparación de las tres modalidades de revisión de requisitos [1, 10, 52, 65]. . 128
11.3. Las cuatro fases de la inspección Fagan aplicada al ERS del PFC de camaroneras [10, 65]. 129
11.4. Comparación entre prototipo de elicitación y de validación [1, 10, 57]. 130
11.5. Tipos de pruebas de aceptación y su relación con los artefactos del ERS [2, 10, 60]. 132
11.6. Fragmento del plan de pruebas de aceptación del PFC de aulas IoT. Cada caso de prueba está vinculado a un requisito del ERS con trazabilidad directa [2, 60]. 133
11.7. Mapa de cierre de la Semana 11. 134

12.1. Formulario de solicitud de cambio (CR) del PFC de clínica veterinaria. Este es el formato mínimo recomendado [2, 10]. 138
12.2. Proceso del CCB adaptado para proyectos de fin de carrera (PFC) de la UTEQ [1, 10]. 139

12.3. Los tres tipos de trazabilidad de requisitos según [1, 2, 66]. 139
12.4. RTM completa con los tres tipos de trazabilidad para el PFC de mascotas a domicilio. Cada fila es un RF trazado en las seis dimensiones [1, 2, 66]. 140
12.5. Métricas clave de gestión de requisitos y sus valores de referencia [1, 10, 80]. . . 141
12.6. Política de versionado semántico para el ERS del PFC [1, 2, 10]. 142
12.7. Los tres baselines del ERS recomendados para un PFC de la UTEQ [1, 2, 10]. . 143
12.8. Mapa de cierre de la Semana 12 y de la Unidad 4. 143

13.1. Comparación de herramientas CASE para IR relevantes para PFC de la UTEQ [1, 3, 10]. 147
13.2. Recomendación de herramienta CASE según el tipo de PFC de la UTEQ. . . . 148
13.3. Cláusulas de IEEE 29148:2018 y su implementación práctica en el PFC de aulas IoT [1, 2]. 149
13.4. Aplicación de las características de ISO 25010:2023 al PFC de gestión de ganado bovino [1, 4, 17]. 150
13.5. Mapeo de los 10 atributos IEEE 29148:2018 a campos de Jira. Este esquema es el mínimo exigible en el PFC de la UTEQ. 151
13.6. Mapa de cierre de la Semana 13. 152

14.1. Jerarquía de artefactos del backlog ágil y su equivalente en IR tradicional [1, 37, 38]. 155
14.2. Palabras clave de Gherkin y su semántica [60, 61]. 156
14.3. Actividades de IR en cada evento de Scrum [1, 38, 56]. 159
14.4. Comparación sistemática entre IR tradicional e IR ágil [1, 37, 38, 56]. 160
14.5. Mapa de cierre de la Semana 14. 161

15.1. Criterios de selección del sistema objetivo para el PFC de la UTEQ con ejemplos de los seis dominios propuestos. 164
15.2. Plan de IR mínimo para el PFC de la UTEQ (adaptable a los 6 dominios propuestos) [1, 2, 52]. 165
15.3. Agenda del taller de requisitos del PFC (adaptación de la estructura de Robertson & Robertson [52]). 167
15.4. Mapa de cierre de la Semana 15 — inicio del Proyecto Integrador. 168

16.1. Lista de verificación del ERS del PFC — criterios de completitud. Todos los ítems deben estar marcados como ✓antes de la entrega. 171
16.2. Elementos UML críticos a revisar por dominio de PFC. 172
16.3. Rúbrica de evaluación del ERS del PFC — Ingeniería de Requisitos UTEQ. Puntaje total: 100 puntos. 174
16.4. Estructura recomendada de la presentación oral del ERS del PFC. 174
16.5. Mapa de cierre del curso ISR-401 Ingeniería de Requisitos. 175

17.1. Verificación de la cobertura de trazabilidad por tipo — PFC semana 17. Toda celda en rojo indica una brecha que debe cerrarse antes de la defensa [1, 2, 66]. 178

17.2. Rúbrica de autoevaluación del ERS del PFC — semana 17. El equipo completa las columnas de puntaje propio y evidencia antes de la defensa. 180
17.3. Los diez errores más frecuentes detectados por tribunales en defensas de ERS de PFC, organizados por frecuencia. 181
17.4. Verificación de completitud, consistencia y verificabilidad del ERS del PFC — checklist final de la semana 17 [1, 2, 10]. 182
17.5. Guión detallado de la presentación oral del ERS del PFC por minuto. La columna "Evidencia" indica el artefacto que debe estar visible en la pantalla durante ese segmento. 183
17.6. Mapa de cierre de la Semana 17 — semana de verificación y ensayo. 185

18.1. Protocolo completo de la sesión de defensa del PFC de IR — Semana 18. . . . 188
18.2. Niveles de logro de la rúbrica del tribunal para la defensa del ERS del PFC. . . 189
18.3. Las doce preguntas más frecuentes del tribunal en defensas de ERS de PFC, organizadas por tema. 190
18.4. Competencias de IR adquiridas en el semestre y su valoración en el mercado laboral del sector tecnológico ecuatoriano. 192
18.5. Correspondencia entre los contenidos del semestre ISR-401 y los bloques de conocimiento del IREB CPRE v3.2.0 [1, 56]. 193
18.6. Mapa completo del semestre ISR-401 — de la Semana 1 a la Semana 18. 195

Lista de acrónimos

ABP Aprendizaje Basado en Problemas

ABProy Aprendizaje Basado en Proyectos

ATDD Acceptance Test-Driven Development

BPMN Business Process Model and Notation

CASE Computer-Aided Software Engineering

CCB Change Control Board (Comité de Control de Cambios)

CMMI Capability Maturity Model Integration

ConOps Concept of Operations (Concepto de Operaciones)

CRM Customer Relationship Management

DFD2 Data Flow Diagram (Diagrama de Flujo de Datos)

EARS Easy Approach to Requirements Syntax

ECU Electronic Control Unit (Unidad de Control Electrónico)

ER Entidad-Relación (modelo de datos)

ERP Enterprise Resource Planning

ERS Especificación de Requisitos de Software

FG Focus Group (Grupo Focal)

GAD Gobierno Autónomo Descentralizado

IESS Instituto Ecuatoriano de Seguridad Social

IR Ingeniería de Requisitos

IS Ingeniería de Software

KAOS Knowledge Acquisition in autOmated Specification

LLM Large Language Model (Modelo Grande de Lenguaje)

LOPDP Ley Orgánica de Protección de Datos Personales (Ecuador, 2021)

MAPE-K Monitor-Analyze-Plan-Execute over a shared Knowledge

MoSCoW Must-have, Should-have, Could-have, Won't-have (técnica de priorización)

MSC Message Sequence Chart (Diagrama de Secuencia de Mensajes)

MSP Ministerio de Salud Pública del Ecuador

MVP Minimum Viable Product (Producto Mínimo Viable)

NaPiRE Naming the Pain in Requirements Engineering

NFR Non-Functional Requirement (sinónimo en inglés de RNF)

OMG Object Management Group

REPM Requirements Engineering Process Maturity Model

RNF Requisito No Funcional

RTM Requirements Traceability Matrix (Matriz de Trazabilidad de Requisitos)

SENESCYT Secretaría Nacional de Educación Superior, Ciencia, Tecnología e Innovación

SERCOP Servicio Nacional de Contratación Pública

SRI Servicio de Rentas Internas (Ecuador)

SRS Software Requirements Specification (Especificación de Requisitos de Software)

StRS Stakeholder Requirements Specification (Especificación de Requisitos de Stakeholder)

SyRS System Requirements Specification (Especificación de Requisitos del Sistema)

UTEQ Universidad Técnica Estatal de Quevedo

Semana 1

Encuadre académico y fundamentos de requisitos de software

Los requisitos no son lo que el cliente dice que quiere; son lo que el sistema debe hacer para resolver, de forma verificable y acordada, un problema real bien delimitado.
adaptado de Pohl [1]

Objetivos de aprendizaje de la semana

Al finalizar esta primera semana, el estudiante estará en capacidad de:

1. Comprender el alcance, la metodología y el sistema de evaluación de la asignatura **Ingeniería de Requisitos (ISR-401)** y su articulación con el proyecto integrador del semestre.
2. Definir con precisión qué es un **requisito de software** a la luz de la norma **IEEE/ISO/IEC 29148:2018** y de la guía **SWEBOK v4.0** [2, 3].
3. Distinguir entre **requisitos de producto** y **requisitos de proceso**, y aplicar la diferencia a casos reales del entorno productivo ecuatoriano.
4. Diferenciar **requisitos funcionales** de **no funcionales**, identificar **propiedades emergentes** y redactar requisitos **cuantificables** y verificables.
5. Distinguir entre **requisitos del sistema** y **requisitos del software**, y modelar las **restricciones** y **dependencias** que los condicionan.
6. Aplicar el modelo de calidad **ISO/IEC 25010:2023** para clasificar y especificar requisitos no funcionales con base en sus nueve características de calidad [4].

1.1 Tema 1. Encuadre académico de la asignatura

La asignatura **Ingeniería de Requisitos** ocupa un lugar singular en el plan de estudios de la Carrera de Ingeniería de Software de la UTEQ: es la primera materia profesionalizante en la que el estudiante no construye software sino que decide qué software se va a construir. Esta diferencia, aparentemente sutil, tiene consecuencias profundas en la formación: exige desarrollar competencias comunicativas, analíticas y éticas que no se ejercitan con la misma intensidad en las asignaturas de programación [5, 6].

Diversos estudios han mostrado que entre el 40 % y el 60 % de los defectos encontrados en sistemas en producción tienen su origen en una deficiente IR; corregir un defecto en producción cuesta hasta cien veces más que detectarlo en la fase de requisitos [7-9]. La consecuencia es directa: un proyecto de software que descuide los requisitos tiende a fracasar, mientras que un equipo que domine la IR incrementa de manera notable la probabilidad de éxito y reduce el costo total del ciclo de vida [1, 10].

1.1.1 Datos generales de la asignatura

La tabla 1.1 sintetiza la información administrativa oficial de la asignatura conforme al programa analítico de la UTEQ [5].

Tabla 1.1. *Datos generales de la asignatura Ingeniería de Requisitos.*

Atributo	Detalle
Carrera	Ingeniería de Software (rediseño)
Nivel	4to nivel
Modalidad	Presencial
Código	ISR-401
Créditos	3.0
Horas presenciales	96.0 horas teórico-prácticas
Aprendizaje autónomo	48.0 horas
Prerrequisito	Fundamentos de Ingeniería de Software (3er nivel)
Eje de formación	Formación profesional
Cátedra integradora	Sí; integra Calidad, Seguridad y Deontología

1.1.2 Mapa global del semestre

El semestre se organiza en **cinco unidades** que evolucionan desde la teoría hacia la práctica integradora. La figura 1.1 representa esta secuencia.

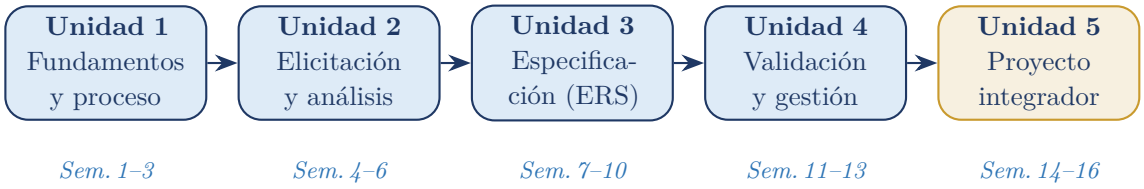


Figura 1.1. *Secuencia de las cinco unidades de la asignatura y su distribución aproximada en el calendario semestral.*

La quinta unidad, **Proyecto integrador**, no introduce nueva teoría: recoge y aplica todo lo aprendido en las cuatro primeras unidades a un caso real de la región (un Gobierno Autónomo Descentralizado (GAD) cantonal, una cooperativa de ahorro, una clínica privada, un sistema agroindustrial bananero, etc.) y culmina con la defensa pública del ERS ante un tribunal docente [5].

1.1.3 Metodología y estrategias didácticas

La metodología combina exposición teórica activa con un fuerte componente práctico en equipo. Se articulan ocho estrategias coordinadas [5, 6]:

- **Clase magistral activa**: exposición de fundamentos teóricos (IEEE 29148, SWEBOK v4, ISO/IEC 25010) con preguntas y análisis de casos en tiempo real.

- **Aprendizaje Basado en Problemas (ABP)**: análisis de proyectos reales fallidos por mala gestión de requisitos –por ejemplo, sistemas gubernamentales y de salud públicos del Ecuador y de la región– en los que el estudiante diagnostica la causa raíz desde el ángulo de la IR.
- **Aprendizaje Basado en Proyectos (ABProy)**: desarrollo durante el semestre de un ERS completo de un sistema real, en equipos de 3 a 5 estudiantes, con roles definidos.
- **Aprendizaje colaborativo**: revisión cruzada (peer review) de documentos ERS entre equipos.
- **Estudios de casos**: análisis de pliegos del Servicio Nacional de Contratación Pública (SERCOP), proyectos de banca y de salud pública con requisitos mal especificados.
- **Design Thinking**: aplicado en la elicitación para identificar necesidades latentes mediante empatización y prototipado rápido.
- **Aula invertida**: lectura previa de fragmentos de IEEE 29148 y SWEBOK v4 para que el aula se dedique a discusión y modelado.
- **Gamificación**: dinámicas de puntos en priorización (MoSCoW, Modelo Kano) y detección de defectos en ERS.

1.1.4 Sistema de evaluación

La evaluación es continua, formativa y sumativa. La distribución típica del puntaje –ajustable por el docente al inicio de semestre– se muestra en la tabla 1.2.

Tabla 1.2. *Componentes del sistema de evaluación de la asignatura.*

Componente	Tipo	Peso	Cuándo
Actividades en clase y autónomas	Formativa	20 %	Todo el semestre
1ª evaluación parcial	Sumativa	20 %	Cierre Unidad 2
2ª evaluación parcial	Sumativa	20 %	Cierre Unidad 4
Proyecto integrador (artefactos)	Sumativa	25 %	Entregas iterativas
Defensa final ante tribunal	Sumativa	15 %	Última semana

1.1.5 Introducción a la Ingeniería de Requisitos

¿Qué es la Ingeniería de Requisitos?

La IR es la disciplina que se ocupa de **descubrir, documentar, acordar y gestionar** lo que un sistema de software debe hacer y bajo qué condiciones debe hacerlo [1, 3, 11]. La guía SWEBOK v4 lo expresa con claridad: la IR es la rama de la Ingeniería de Software (IS) dedicada al desarrollo, especificación y gestión de los requisitos a lo largo de todo el ciclo de vida del producto [3]. La norma [2] la entiende como un conjunto de procesos del ciclo de vida de los sistemas y del software dedicados a transformar las necesidades de los stakeholders en requisitos de stakeholder, requisitos del sistema y requisitos del software.

Definición

Ingeniería de Requisitos. Conjunto sistemático y disciplinado de actividades dedicadas a *elicitar, documentar, validar, acordar y gestionar* los requisitos de un sistema (frecuentemente intensivo en software), de modo que sirvan como base contractual y técnica para todas las actividades posteriores del ciclo de vida [1, 2].

Pohl, en su segunda edición de 2024, propone tres **metas centrales** de la IR [1]:

1. **Conocer los requisitos relevantes y alcanzar consenso entre los stakeholders**, expresar los requisitos sin ambigüedad y documentarlos según los formatos pactados.
2. **Asegurar la calidad del proceso** para que los requisitos documentados cumplan los criterios establecidos.
3. **Gestionar el contexto** en el que la IR tiene lugar y soportar la evolución de los requisitos a lo largo del ciclo de vida.

La figura 1.2 resume el **framework de IR** que Pohl utiliza como hilo conductor del libro y que esta asignatura adoptará como esquema de referencia. Este marco se compone de tres tipos de *actividades core*, dos *actividades transversales* y tres tipos de *artefactos de requisitos*.

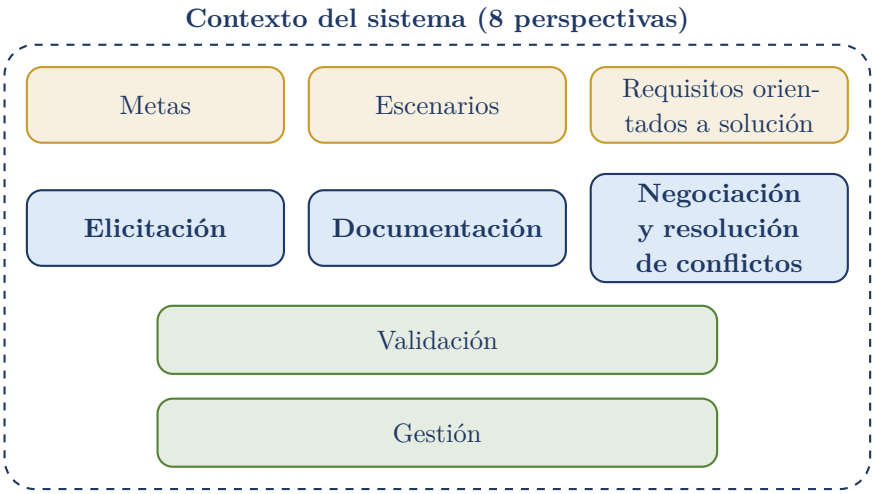


Figura 1.2. *Framework de Ingeniería de Requisitos según Pohl: tres clases de artefactos, tres actividades centrales y dos actividades transversales, todo enmarcado por el contexto del sistema. Adaptado de [1].*

¿Por qué la IR importa especialmente en Ecuador?

Ecuador atraviesa un proceso sostenido de transformación digital en sectores estratégicos: gobierno electrónico, banca, salud pública, educación superior, agroindustria y comercio. Cada uno de esos sectores ha experimentado proyectos retrasados o cancelados por requisitos mal entendidos. El *Plan Nacional de Gobierno Electrónico* y los lineamientos de soberanía tecnológica exigen procesos formales de IR [12, 13], y la *Ley Orgánica de Protección de Datos Personales* de 2021 incorpora obligaciones explícitas que se traducen, ineludiblemente, en requisitos no funcionales de seguridad y privacidad para todos los sistemas que traten datos personales [14].

Ejemplo en Ecuador

Caso: matrícula universitaria en la UTEQ. Cuando un estudiante se matricula en línea, intervienen requisitos como: «el sistema deberá permitir matricularse en máximo cinco asignaturas por nivel», «deberá validar que el estudiante haya aprobado los prerrequisitos», «deberá responder en menos de 3 segundos en hora pico», «deberá mantener registros por al menos 10 años por exigencia de la Secretaría Nacional de Educación Superior, Ciencia, Tecnología e Innovación (SENESCYT)». Si cualquiera de estos requisitos no se levanta correctamente, el sistema falla en producción durante el período de matrículas, justo cuando más se necesita.

1.2 Tema 2. Fundamentos de requisitos de software

1.2.1 Subtema 2.1. Definición de requisito de software. Requisitos de productos vs. requisitos de procesos

Definición formal según IEEE/ISO/IEC 29148:2018

La cláusula 3.1.19 de la norma [2] ofrece una definición **tripartita** que conviene memorizar y comprender en cada una de sus acepciones complementarias. Un requisito es:

1. Un enunciado que traduce o expresa una necesidad y sus **restricciones y condiciones** asociadas.
2. Una **condición o capacidad** que debe poseer un sistema, producto, servicio, resultado o componente para satisfacer un acuerdo, estándar, especificación u otros documentos formalmente impuestos.
3. Una **representación documentada** de una condición o capacidad como en (1) o (2).

Tres ideas atraviesan esta definición y conviene fijarlas:

- Todo requisito **nace de una necesidad**: alguien quiere o requiere algo.
- Esa necesidad debe expresarse como **condición o capacidad observable y verificable**, no como un deseo difuso.
- Debe quedar **documentada**, porque solo lo escrito puede ser revisado, acordado y trazado.

Del problema al requisito: la cadena de evolución

SWEBOK v4 [3] y Pohl [1] coinciden en que un requisito atraviesa varias etapas antes de estabilizarse: empieza como una necesidad latente del stakeholder, se convierte en una preocupación expresada, luego en una meta, después en un requisito de stakeholder, y finalmente en un requisito del sistema o del software. La figura 1.3 muestra esta cadena.



Figura 1.3. *Cadena de transformación de la necesidad al requisito de software. Adaptado de [1-3].*

Anatomía de un buen requisito

La cláusula 5.2.4 de IEEE 29148 establece nueve **características** que todo requisito bien formado debe cumplir individualmente [2]. La tabla 1.3 las sintetiza con ejemplos en el contexto ecuatoriano.

Tabla 1.3. *Las nueve características de un requisito bien formado, con ejemplos contextualizados. Adaptado de [1, 2].*

Característica	Significado	Ejemplo (matrícula)
Necesario	Sin él, los stakeholders no aceptan el sistema.	El sistema deberá registrar la matrícula del estudiante.
Apropiado al nivel	Adecuado al nivel del documento (negocio/sistema/software).	Si es de software: «el sistema deberá validar la cédula del estudiante».
No ambiguo	Una sola interpretación posible.	«en menos de 3 segundos», en lugar de «rápidamente».
Completo	Sin información implícita o supuesta.	Especifica entradas, comportamiento y salidas.
Singular	Un único requisito por enunciado.	Separar «registrar»de «notificar por correo».
Factible	Realizable técnica, legal y económicamente.	No exigir respuesta en 1 ms si la red no lo permite.
Verificable	Comprobable por inspección, análisis, demostración o prueba.	Existe una prueba que demuestre el tiempo de respuesta.
Correcto	Refleja con precisión la necesidad real.	Validado con la responsable de Secretaría Académica.
Conforme	Sigue plantilla, estilo y formato del ERS.	Numeración **RF-NN**, redacción en voz activa.

Una plantilla básica para principiantes, recomendada por [1, 15], es:

Plantilla básica:

[Sujeto] [Verbo modal: deberá / no deberá] [Acción que el sistema realiza] [Objeto / Condición / Restricción medible].

Cuando los requisitos contienen lógica condicional o eventos, conviene adoptar plantillas más estructuradas, como Easy Approach to Requirements Syntax (EARS) [15], que se estudiarán a fondo en la Unidad 3.

Advertencia / Error frecuente

Error frecuente del estudiante. Confundir un requisito con una *decisión de diseño* o con una *solución técnica*. Si en lugar de «el sistema deberá autenticar al usuario»se escribe «el sistema deberá usar JWT con expiración de 30 minutos», ya no se está especificando un requisito sino una decisión de implementación. La regla de oro es: *el requisito describe* ***qué****; el diseño describe* ***cómo*** [10, 11].

Requisitos de producto vs. requisitos de proceso

Una distinción fundamental introducida por SWEBOK v4 [3] y reforzada por Sommerville [11] es la que separa los **requisitos de producto** de los **requisitos de proceso**. Confundirlos es causa frecuente de discusiones estériles entre cliente y equipo técnico.

Requisitos de producto. Describen el *sistema de software que se va a construir*: lo que hace, cómo se comporta, las cualidades que debe exhibir y las restricciones que debe respetar. Son los protagonistas del ERS y los que generalmente preocupan más al usuario final.

Ejemplos en el contexto ecuatoriano:

- «El sistema deberá calcular el aporte mensual al Instituto Ecuatoriano de Seguridad Social (IESS) aplicando el porcentaje vigente sobre el sueldo bruto del trabajador.»
- «El sistema deberá emitir comprobantes electrónicos en el formato XML exigido por el Servicio de Rentas Internas (Ecuador) (SRI)»[16].
- «El sistema deberá soportar al menos 1 500 transacciones concurrentes durante la temporada alta de matrículas.»

Requisitos de proceso. Regulan *cómo se desarrollará el software*, no qué hace el software. Imponen condiciones al equipo de desarrollo, no al producto. Provienen casi siempre del cliente, de la organización contratante o de regulaciones [3].

Ejemplos típicos en proyectos ecuatorianos:

- «El proveedor deberá usar metodología Scrum con sprints de dos semanas.»
- «El proveedor deberá entregar el código fuente en un repositorio Git institucional bajo licencia GPL.»
- «La documentación deberá entregarse en español, conforme a la plantilla IEEE 29148:2018»[2].

Cuadro comparativo. La tabla 1.4 resume las diferencias.

Tabla 1.4. *Requisitos de producto vs. requisitos de proceso.*

Dimensión	Producto	Proceso
¿Qué describe?	El sistema construido (qué hace, cómo se comporta).	El proceso de construcción (cómo se desarrolla).
¿A quién obliga?	Al sistema en operación.	Al equipo de desarrollo o al proveedor.
¿Cuándo se verifica?	En pruebas de aceptación del software.	En auditorías de proceso e hitos contractuales.
Origen típico	Usuarios, clientes, regulaciones del dominio.	Contrato, estándares corporativos, política institucional.
Ejemplo Ecuador	Generación del Anexo Transaccional del SRI.	Uso de GitLab institucional para el versionado.

Ejemplo en Ecuador

Ejemplo integrador: licitación pública del SERCOP. Cuando una entidad pública del Ecuador convoca, vía SERCOP, a una licitación para desarrollar un sistema de gestión documental, los pliegos suelen contener mezcladas estas dos clases de requisitos. Por ejemplo: **(a)** «el sistema deberá firmar electrónicamente los oficios con certificado digital»(producto); **(b)** «el oferente deberá entregar manuales en español, en versión digital y física, y con licencia GPL»(proceso). El analista debe *separarlos explícitamente*, pues se gestionan, verifican y aceptan de manera diferente: el primero con pruebas funcionales, el segundo con auditoría de entregables.

1.2.2 Subtema 2.2. Requisitos funcionales y no funcionales. Propiedades emergentes. Requisitos cuantificables

Requisitos funcionales (RF)

Definición

Requisito funcional (Requisito funcional). Enunciado que describe una *función, servicio o tarea* que el sistema debe ejecutar. Responde a la pregunta: *¿qué debe hacer el sistema?* [1, 11].

Sommerville [11] caracteriza los RF como «las declaraciones de los servicios que el sistema debe proveer, cómo debe reaccionar ante entradas particulares y cómo debe comportarse en situaciones específicas». Para mantener uniformidad en el ERS, conviene adoptar una plantilla y un identificador único; una plantilla muy usada en proyectos ecuatorianos es:

RF-NN: *El sistema deberá [acción] [objeto] [cuando condición] [con el fin de propósito].*

Ejemplo en Ecuador

Ejemplo: sistema de turnos en un consultorio médico de Cuenca.

- **RF-01**: El sistema deberá permitir al paciente reservar turno en línea seleccionando especialidad, médico, fecha y hora disponibles.
- **RF-02**: El sistema deberá enviar confirmación por correo electrónico y mensaje SMS al paciente al confirmar la reserva.
- **RF-03**: El sistema deberá impedir reservar más de un turno activo simultáneo por paciente para la misma especialidad.
- **RF-04**: El sistema deberá generar un reporte mensual de citas atendidas, agrupado por médico y por especialidad, exportable a PDF y XLSX.

Requisitos no funcionales (Requisito no funcional / Non-Functional Requirement (sinónimo en inglés de RNF) (NFR))

Los **requisitos no funcionales** (RNF) describen *cualidades, restricciones y atributos* del sistema. No describen una función específica sino propiedades transversales: cuán rápido, cuán seguro, cuán mantenible, cuán portable. Pese a su nombre, su impacto suele ser *tan determinante para el éxito* como el de los requisitos funcionales [17-19].

Definición

Requisito no funcional. Atributo o restricción de un sistema que no describe una función concreta sino una *propiedad transversal* del producto, del proceso de desarrollo o del entorno. Suele expresarse mediante *métricas y umbrales* que permiten su verificación [1, 4, 17].

La taxonomía clásica los agrupa en tres familias [11]:

Tabla 1.5. *Las tres familias de requisitos no funcionales según [11].*

Familia	¿Qué cubre?	Ejemplos típicos
Requisitos de producto	Atributos que el sistema mismo debe exhibir.	Rendimiento, fiabilidad, usabilidad, seguridad.
Requisitos organizacionales	Políticas y procedimientos del cliente o desarrollador.	Estándar de codificación, plataforma, idioma de la UI.
Requisitos externos	Restricciones legales, éticas, regulatorias.	Cumplir con LOPDP, formatos del SRI, IESS.

Ejemplos de RNF en el contexto ecuatoriano.

- **RNF-01** (Rendimiento): «El sistema deberá responder a la consulta de matrícula en menos de 3 s para el percentil 95 de las solicitudes durante la semana pico de inscripciones.»
- **RNF-02** (Seguridad): «El sistema deberá cifrar las contraseñas con bcrypt, factor de costo ≥ 12, y nunca almacenarlas en texto plano.»
- **RNF-03** (Disponibilidad): «El sistema de banca móvil deberá presentar una disponibilidad mensual no menor al 99,8 %, equivalente a un máximo de 87,7 minutos de indisponibilidad por mes.»

- **RNF-04** (Usabilidad): «Un usuario sin entrenamiento previo deberá completar el registro de un comprobante electrónico en menos de 5 minutos en su primer intento.»
- **RNF-05** (Cumplimiento legal): «El sistema deberá cumplir con los lineamientos de la LOPDP del Ecuador (Registro Oficial 459 del 26 de mayo de 2021)»[14].

Propiedades emergentes

Definición

Propiedad emergente. Cualidad del sistema considerado como un *todo*, que solo aparece cuando todos los componentes interactúan en su entorno operativo. *No puede atribuirse a ningún componente aislado* [11].

Ignorar las propiedades emergentes es uno de los errores más serios y costosos de la IR, porque son justamente las que determinan si el sistema funciona en la realidad. Son ejemplos clásicos:

- **Fiabilidad global**: aunque cada componente sea fiable al 99 %, la fiabilidad del sistema total no es 99 %, sino el resultado de cómo se acoplan todos los componentes y sus modos de fallo.
- **Tiempo de respuesta extremo a extremo**: depende de la red, los servidores, las bases de datos y los servicios de terceros (SRI, IESS, Banco Central). Optimizar solo el frontend no garantiza el resultado.
- **Seguridad**: ningún componente «es seguro»por sí solo. La seguridad emerge de la combinación de autenticación, autorización, cifrado, auditoría, hardening y políticas.
- **Usabilidad**: emerge de la interacción entre la interfaz, la terminología, los tiempos de respuesta y la consistencia entre módulos.

Ejemplo en Ecuador

Caso: banca móvil ecuatoriana. Una aplicación de banca móvil de un banco ecuatoriano puede tener componentes individualmente óptimos: una app rápida, un backend potente, una integración eficiente con el switch transaccional. Sin embargo, su disponibilidad emergente puede caer al 95 % si los procesos de respaldo, los enlaces con el Banco Central y la operación humana de atención no se diseñan en conjunto. La propiedad emergente «disponibilidad real para el cliente»no se logra sumando atributos aislados, se diseña pensando el sistema como un todo [8].

Requisitos cuantificables

Un requisito es **cuantificable** cuando contiene una métrica objetiva que permite verificarlo sin ambigüedad. Tanto Pohl como SWEBOK v4 insisten en que todo RNF debe ser, en la medida de lo posible, cuantificable; de lo contrario es solo un deseo [1, 3]. La transformación de un enunciado vago en uno cuantificable suele requerir tres elementos:

1. Una **métrica** concreta (segundos, porcentaje, número de fallos por mes, transacciones por segundo, etc.).
2. Una **unidad clara y un punto de referencia** o instrumento de medición.
3. Una **condición de operación** (carga, contexto, percentil).

Tabla 1.6. *Conversión de enunciados vagos en requisitos cuantificables.*

Enunciado vago (incorrecto)	Enunciado cuantificable (correcto)
El sistema debe ser rápido.	El sistema deberá responder a una consulta de saldo en menos de 1,5 s para el 95 % de las solicitudes con 500 usuarios concurrentes.
La interfaz debe ser amigable.	Un usuario sin experiencia previa deberá poder completar la operación de transferencia en menos de 90 segundos en su primer intento.
Debe ser seguro.	El sistema deberá bloquear la cuenta tras 5 intentos fallidos de autenticación en una ventana de 10 minutos.
Debe estar siempre disponible.	El sistema deberá tener una disponibilidad mensual $\geq$ 99,8 %, medida sobre la ventana de servicio 24×7.
Debe ser fácil de mantener.	El equipo deberá poder corregir un defecto crítico (severidad 1) y desplegarlo en menos de 4 horas hábiles.

Advertencia / Error frecuente

Regla de oro. Si un requisito *no se puede medir*, no se puede verificar. Y si no se puede verificar, técnicamente no es un requisito: es un *deseo*. Antes de aprobar el ERS, el analista debe preguntarse por cada RNF: ¿qué prueba lo demostraría?, ¿con qué número se compararía el resultado? [10, 15].

1.2.3 Subtema 2.3. Requisitos del sistema vs. requisitos del software. Restricciones y dependencias

Niveles de requisitos: una jerarquía indispensable

La norma [2] distingue, dentro de un mismo proyecto, varios niveles de requisitos que se construyen unos a partir de otros y se trazan entre sí. No reconocer estos niveles es causa habitual de confusión entre clientes, analistas y desarrolladores. La tabla 1.7 resume los cuatro niveles canónicos.

Tabla 1.7. *Niveles canónicos de requisitos según IEEE 29148:2018 [2].*

Nivel	Documento	Pregunta	Audiencia
Negocio / Misión	Business Reqs. – ConOps	¿Por qué se construye?	Alta dirección.
Stakeholder	StRS	¿Qué necesitan los stakeholders?	Usuarios, gerentes.
Sistema	SyRS	¿Qué hace el sistema completo?	Arquitectos.
Software	SRS / ERS	¿Qué hace el software?	Desarrolladores.

La figura 1.4 representa esta jerarquía como una pirámide invertida: en la cumbre se encuentran las metas de negocio (las más abstractas) y, descendiendo, los requisitos cada vez más detallados y asignables a componentes concretos.

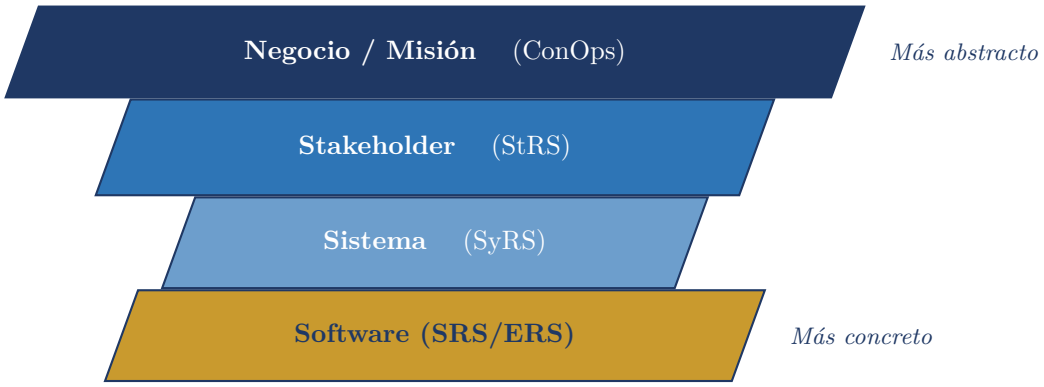


Figura 1.4. *Pirámide invertida de niveles de requisitos. Cada nivel se deriva del anterior mediante un proceso de refinamiento y asignación (allocation) [2].*

Requisitos del sistema

Los requisitos del sistema describen el comportamiento del sistema completo, entendido como la combinación de *hardware, software, personas, datos, instalaciones, materiales y procedimientos* [1, 2]. Un sistema, en este sentido amplio, no es solo un programa: es una solución completa que incluye al usuario, al ambiente y a los procesos organizacionales.

Ejemplo en Ecuador

Caso: pagos del IESS para empleadores. Como sistema, no es solo un portal web. Incluye: el portal del afiliado (software), los servidores en el datacenter (hardware), los integradores con bancos privados y el Banco del Pacífico (subsistemas externos), los lineamientos legales del Reglamento del IESS (procedimientos) y los empleadores que deben cargar planillas (personas). Si solo se especifica el software, se ignoran restricciones críticas como horarios de *cutoff* bancario, requerimientos de respaldo del datacenter o capacitación al empleador.

Requisitos del software

Los requisitos del software, documentados en el SRS o ERS en español, son una refinación de los requisitos del sistema *asignados al componente de software*. Describen únicamente lo que el software hace, ignorando explícitamente al hardware, al usuario y al entorno externo [2, 11].

Cómo se derivan unos de otros. La derivación de requisitos del sistema a requisitos del software se realiza mediante un proceso de *asignación* o *allocation*: se decide qué porción de cada requisito del sistema será cubierta por software y qué porción por hardware, personas o procesos [2]. Cada decisión de asignación debe quedar trazada.

Tabla 1.8. *Asignación (allocation) de requisitos del sistema al software, con ejemplos ecuatorianos.*

Requisito del sistema	Asignación a software	Asignación a HW / personas
Pesar carga de banano para exportación con precisión $\pm 50\,\text{g}$.	El módulo deberá leer y registrar el peso recibido del puerto serial.	Balanza certificada por la Agencia de Regulación; operario calibrado.
Facturar electrónicamente al SRI.	El software deberá generar el XML, firmarlo y transmitirlo al SRI [16].	Token criptográfico o HSM con certificado digital; contador con clave.
Permitir cobros sin conexión en zonas rurales.	El software deberá almacenar transacciones en cola local y sincronizar al volver la conexión.	Dispositivo POS con batería $\geq 8\,\text{h}$; liquidación diaria.

Restricciones

Definición

Restricción. Condición externa al equipo de desarrollo que *limita las opciones de diseño y de implementación* del sistema. A diferencia de un requisito –que el equipo intenta cumplir mediante una solución–, una restricción *no se negocia*: viene impuesta por el contexto [1, 2].

Reconocer las restricciones a tiempo es vital, porque cambian el espacio de soluciones posibles. La tabla 1.9 clasifica los tipos de restricción más comunes con ejemplos de Ecuador.

Tabla 1.9. *Tipos comunes de restricciones, con ejemplos en Ecuador.*

Tipo	Descripción	Ejemplo en Ecuador
Tecnológicas	Tecnologías obligatorias o prohibidas.	El portal del Ministerio debe correr sobre PostgreSQL y Linux por política de soberanía tecnológica.
Legales / Normativas	Leyes, reglamentos o normas obligatorias.	Cumplir con la LOPDP [14] y los esquemas del SRI [16].
Económicas	Limitaciones de presupuesto o costos.	El proyecto cuenta con USD 45 000 para todo el ciclo de vida del primer año.
Temporales	Plazos de entrega impuestos.	Debe estar operativo antes del 1 de enero por entrada en vigor de una resolución.
De interfaz	Sistemas con los que debe convivir.	Debe integrarse con el ESIGEF (Min. Finanzas) y con el portal del SERCOP.
Organizacionales	Políticas internas, estructura o cultura.	Debe usar el directorio activo institucional para la autenticación.
Físicas / Ambientales	Condiciones del entorno operativo.	El POS debe operar a 35 °C y 90 % de humedad en zonas costeras.

Dependencias entre requisitos

Definición

Dependencia. Relación entre dos requisitos en la que el cumplimiento, la prioridad o el comportamiento de uno depende del otro. Detectar dependencias evita inconsistencias y guía la planificación de *releases* [1, 3].

SWEBOK v4 las clasifica en cinco categorías [3], que se sintetizan en la tabla 1.10.

Tabla 1.10. *Tipos de dependencias entre requisitos.*

Tipo	Significado	Ejemplo (matrícula)
Requiere (Requires)	RF-A no tiene sentido si no existe RF-B.	**RF-12** «pagar matrícula»requiere **RF-08** «generar orden de pago».
Conflicto (Conflicts)	Dos requisitos no pueden cumplirse simultáneamente.	Maximizar disponibilidad 24×7 vs. ventanas de mantenimiento programado de 4 h.
Refina (Refines)	Un requisito detalla a otro de mayor nivel.	**RF-08.1** «calcular descuento de beca»refina a **RF-08** «generar orden de pago».
Implica (Implies)	Cumplir RF-A obliga a cumplir RF-B aunque no se mencione.	Aceptar pagos con tarjeta implica cumplir PCI-DSS.
Excluye (Excludes)	RF-A solo se cumple si RF-B no se selecciona.	Pago en efectivo en taquilla excluye recargo por uso de pasarela.

Ejemplo en Ecuador

Caso: dependencia mal gestionada en un proyecto público. En un sistema regional de salud, el equipo aprobó **RF-23** «el sistema deberá enviar receta electrónica al farmacéutico»sin notar que dependía de **RF-09** «el sistema deberá identificar inequívocamente al farmacéutico colegiado». **RF-09** fue desestimado por «menos prioritario». El resultado: en producción, las recetas llegaban a destinatarios genéricos, sin trazabilidad legal, exponiendo a la institución a responsabilidades sanitarias. Una matriz de dependencias, elaborada en la fase de IR, habría evitado el problema.

1.2.4 Subtema 2.4. Taxonomía de requisitos de calidad (ISO/IEC 25010:2023). Los llamados NFR

La importancia de un modelo de calidad estandarizado

ISO/IEC 25010 define un *modelo de calidad* como un marco que descompone la calidad del producto software en **características y subcaracterísticas medibles** [4]. Esta norma reemplazó al ISO/IEC 9126 y es hoy la referencia internacional más usada para estructurar los requisitos no funcionales. Como recuerda SWEBOK v4 [3], los NFR no se inventan al azar: se derivan de un modelo de calidad.

Novedad: ISO/IEC 25010:2023 (segunda edición)

En noviembre de 2023, ISO publicó la **segunda edición** del estándar, con cambios sustantivos respecto de la versión 2011 que conviene conocer [4]:

- Se incorpora **Safety** (seguridad funcional o operacional) como novena característica de calidad. La versión 2011 solo tenía ocho. Esta es la novedad más importante.
- *Usability* fue **renombrada como Interaction capability**. El cambio refleja que el modelo

se aplica ahora a sistemas con múltiples modos de interacción (voz, gestos, IoT) y no solo a interfaces gráficas.

- *Portability* fue **renombrada como Flexibility** y se le añadió la subcaracterística *scalability*.
- Se añadieron las subcaracterísticas *inclusivity* (en Interaction capability), *resistance* (en Security) y *user engagement* y *faultlessness* (que reemplazan a *aesthetics* y *maturity*).

La tabla 1.11 resume las principales diferencias entre ambas ediciones.

Tabla 1.11. *Cambios principales entre ISO/IEC 25010:2011 e ISO/IEC 25010:2023 [4, 20].*

Aspecto	Edición 2011	Edición 2023
N.º de características	8	9 (se añade Safety)
Usability	Sí (presente)	Renombrada como *Interaction capability*
Portability	Sí (presente)	Renombrada como *Flexibility*
Subcaracterísticas nuevas	—	inclusivity, resistance, scalability, faultlessness
Foco	Productos de software	Productos de software *y otros productos TIC*

Las nueve características de calidad de ISO/IEC 25010:2023

La figura 1.5 resume gráficamente las nueve características y la tabla 1.12 las describe con sus subcaracterísticas.

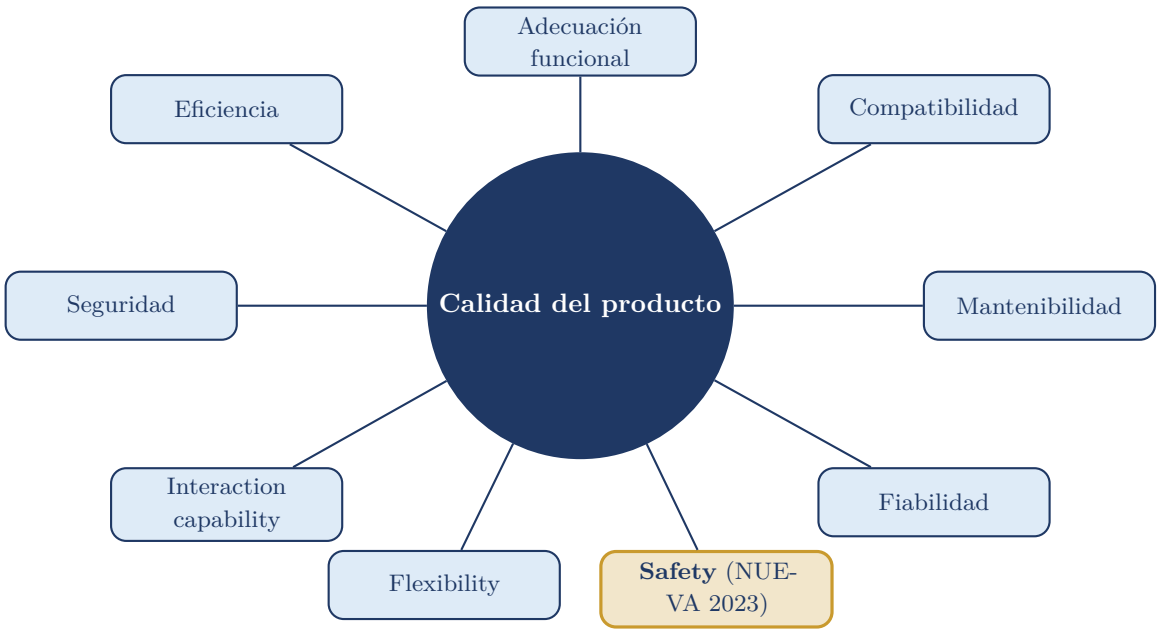


Figura 1.5. *Las nueve características de calidad de ISO/IEC 25010:2023 [4]. La novena característica,* ***Safety****, constituye la novedad principal de la edición 2023.*

Tabla 1.12. *Las nueve características de calidad ISO/IEC 25010:2023 con sus subcaracterísticas [4].*

#	Característica	Idea central	Subcaracterísticas
1	Adecuación funcional	Hace lo que debe hacer.	Completitud, corrección, pertinencia funcional.
2	Eficiencia de desempeño	Lo hace usando recursos razonables.	Comportamiento temporal, utilización de recursos, capacidad.
3	Compatibilidad	Convive con otros sistemas.	Coexistencia, interoperabilidad.
4	Interaction capability	Es fácil y satisfactorio de usar (antes Usability).	Reconocibilidad, aprendizaje, operabilidad, protección frente a errores, *user engagement*, accesibilidad, *inclusivity*, autodescriptividad.
5	Fiabilidad	Funciona sin fallar bajo condiciones acordadas.	*Faultlessness*, disponibilidad, tolerancia a fallos, recuperabilidad.
6	Seguridad	Protege información y servicios.	Confidencialidad, integridad, no repudio, responsabilidad, autenticidad, *resistance*.
7	Mantenibilidad	Se modifica con bajo costo.	Modularidad, reusabilidad, analizabilidad, modificabilidad, capacidad de prueba.
8	Flexibility	Se adapta a otros entornos (antes Portabilidad).	Adaptabilidad, instalabilidad, reemplazabilidad, *scalability*.
9	**Safety** (nueva)	No causa daño a personas, propiedad o entorno.	Restricción operacional, identificación de riesgos, *fail safe*, alerta de peligro, integración segura.

Detalle de cada característica con ejemplos ecuatorianos

1. Adecuación funcional. Mide si el software hace lo que debe hacer y lo hace correctamente. Sus subcaracterísticas son completitud, corrección y pertinencia funcional [4].

Ejemplo en Ecuador

Facturación electrónica. Un sistema contable ecuatoriano debe generar comprobantes con todos los campos exigidos por el SRI (completitud), debe calcular correctamente el IVA al porcentaje vigente y aplicar bien las retenciones (corrección), y debe permitir al contador realizar las tareas usuales de cierre mensual sin pasos innecesarios (pertinencia) [16].

2. Eficiencia de desempeño. Cantidad de recursos consumidos y tiempos de respuesta entregados, bajo condiciones específicas. Subcaracterísticas: comportamiento temporal, utilización

de recursos y capacidad.

Ejemplo en Ecuador

Plataforma de matrícula UTEQ. «El sistema deberá soportar 1 500 inscripciones concurrentes con un tiempo de respuesta promedio menor a 3 s y consumo de CPU por instancia inferior al 75 % durante la semana pico». Aquí se cuantifican las tres subcaracterísticas: tiempo, recursos y capacidad.

3. Compatibilidad. Capacidad para coexistir con otros productos y para interoperar con ellos. Subcaracterísticas: coexistencia (compartir recursos sin afectar a otros) e interoperabilidad (intercambiar y usar información con otros sistemas).

Ejemplo en Ecuador

Banca móvil. Una solución de banca móvil ecuatoriana debe interoperar con el Banco Central (transferencias SPI), con BANRED para tarjetas, con el SRI para retenciones y con cooperativas ecuatorianas, sin afectar el rendimiento de los servicios de los demás.

4. Interaction capability (antes Usability). Grado en que el software puede ser usado por usuarios específicos para alcanzar metas con efectividad, eficiencia y satisfacción en un contexto de uso. La edición 2023 añadió *inclusivity* y *user engagement* como subcaracterísticas [4].

Ejemplo en Ecuador

App municipal en zonas rurales. Una app del GAD Municipal para reportar baches o incidentes urbanos en zonas rurales de Los Ríos debe ser comprensible para un usuario con baja exposición tecnológica (reconocibilidad), enseñar su uso en pocos minutos (aprendizaje), resistir errores como subir fotos demasiado grandes (protección frente a errores), cumplir con pautas WCAG 2.1 AA (accesibilidad e inclusivity) y contener guías contextuales (autodescriptividad).

5. Fiabilidad. Capacidad para mantener un nivel especificado de desempeño bajo condiciones acordadas. Subcaracterísticas: *faultlessness*, disponibilidad, tolerancia a fallos y recuperabilidad.

Ejemplo en Ecuador

Sistema de turnos del MSP. «El sistema deberá tener una disponibilidad mensual $\geq 99{,}9\,\%$ (disponibilidad), tolerar la caída de un nodo sin interrumpir el servicio (tolerancia a fallos) y recuperarse en menos de 15 minutos tras una falla mayor (recuperabilidad)».

6. Seguridad. Capacidad de proteger información y datos. Subcaracterísticas: confidencialidad, integridad, no repudio, responsabilidad, autenticidad y *resistance* (nueva en 2023).

Ejemplo en Ecuador

Historia clínica electrónica. Un sistema de historia clínica de un hospital del IESS debe asegurar que solo el médico tratante acceda al diagnóstico (confidencialidad), evitar adulteraciones (integridad), guardar evidencia firmada de la prescripción (no repudio), registrar cada acceso (responsabilidad), autenticar con doble factor (autenticidad) y resistir intentos de inyección SQL (*resistance*) [14].

7. Mantenibilidad. Eficiencia con que se puede modificar el software. Subcaracterísticas: modularidad, reusabilidad, analizabilidad, modificabilidad y capacidad de prueba.

Ejemplo en Ecuador

Sistema académico universitario. Cuando la SENESCYT cambia las reglas de homologación de asignaturas, el sistema académico de una universidad ecuatoriana debe poder modificar el módulo afectado sin tocar los demás (modularidad y modificabilidad), permitir analizar el impacto del cambio (analizabilidad) y verificarlo con pruebas automatizadas (capacidad de prueba).

8. Flexibility (antes Portabilidad). Efectividad con que el software puede transferirse de un entorno a otro o escalar en el actual. Subcaracterísticas: adaptabilidad, instalabilidad, reemplazabilidad y *scalability* (nueva en 2023).

Ejemplo en Ecuador

Migración a la nube gubernamental. Un sistema desarrollado para servidores on-premise del Ministerio puede migrarse a la nube gubernamental del Ecuador. Si fue diseñado pensando en flexibilidad, puede ejecutarse sobre Linux y Docker (adaptabilidad), instalarse con un script reproducible (instalabilidad), reemplazar al sistema legado sin pérdida de datos (reemplazabilidad) y escalar horizontalmente al crecer la carga (*scalability*).

9. Safety (NUEVA en 2023). Capacidad del software de no provocar daño a las personas, a la propiedad o al entorno. Subcaracterísticas: restricción operacional, identificación de riesgos, *fail safe*, alerta de peligro e integración segura [4].

Ejemplo en Ecuador

Sistema agroindustrial automatizado. En una empacadora bananera de Quevedo, el software que controla las cintas y elevadores debe detenerse automáticamente si detecta una condición anormal (*fail safe*), advertir al operario sobre obstrucciones (alerta de peligro), respetar los límites de velocidad de cintas (restricción operacional) e integrarse de forma segura con sensores y PLC industriales (integración segura).

Cómo escribir un NFR usando ISO/IEC 25010:2023

La taxonomía no es un fin en sí misma: es una guía de redacción. Una buena receta práctica para el estudiante, alineada con [1] y [10], es la siguiente:

1. **Identificar** el atributo de calidad (por ejemplo, Eficiencia → Comportamiento temporal).
2. **Definir** un escenario operativo (carga, contexto, percentil).

3. **Indicar** la métrica y el valor objetivo.
4. **Indicar** el método de verificación.

RNF-XX *[Eficiencia – Comportamiento temporal]*: «Bajo una carga concurrente de 800 usuarios, el sistema deberá responder a la consulta de saldo en menos de 1,5 s para el percentil 95 de las solicitudes. Verificación: prueba de carga con JMeter durante 30 minutos.»

1.3 Mapa síntesis de la Semana 1

Cerramos la semana con un mapa que conecta los cuatro grandes pilares vistos. La idea es que el estudiante visualice cómo encajan entre sí cuando aborde su primer ERS.

Tabla 1.13. *Mapa de cierre de la Semana 1.*

Pilar	Aporte conceptual	Documento donde aparece
Definición y tipos (2.1)	Vocabulario base; producto vs. proceso.	Glosario y sección de tipos del ERS.
Funcionales y NF (2.2)	Diferenciar el qué del cómo de bien; cuantificar.	Secciones 3 y 4 del ERS (IEEE 29148).
Sistema vs. software (2.3)	Niveles, restricciones, dependencias.	Diagrama de contexto y matriz de trazabilidad.
ISO/IEC 25010:2023 (2.4)	Taxonomía estructurada para los NFR.	Sección de requisitos de calidad del ERS.

1.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 1 (en clase, 2 horas).

Objetivo: aplicar la plantilla de redacción y las nueve características de IEEE 29148 a un caso real ecuatoriano.

Caso: la UTEQ desea construir un sistema en línea de préstamo de libros para sus 17 carreras.

Tareas en equipos de 3–4 estudiantes:

1. Identificar al menos cinco stakeholders distintos.
2. Redactar diez requisitos funcionales con la plantilla **RF-NN** y verificar cada uno contra las nueve características de IEEE 29148 [2].
3. Identificar tres requisitos vagos suministrados por el docente y reescribirlos como requisitos cuantificables.
4. Discutir en plenaria los hallazgos.

Práctica / Actividad

Tarea autónoma n.º 1 (4 horas, individual).

Caso: un GAD cantonal del Ecuador desea contratar el desarrollo de un sistema de cobro de impuestos prediales en línea, integrado con el portal del SRI y con un proveedor de pasarela de pagos local.

Tareas:

1. Identificar al menos tres stakeholders y sus necesidades preliminares.
2. Redactar cinco RF y cinco RNF, siguiendo la plantilla y las características de [2].
3. Clasificar cada RNF según las **nueve** características de ISO/IEC 25010:2023 [4].
4. Identificar tres restricciones y dos dependencias entre requisitos, justificando cada una.
5. Para cada RNF, indicar la métrica y el método de verificación.

Entregable: documento PDF de máximo 6 páginas. La rúbrica ponderará especialmente la verificabilidad y la cuantificación de los RNF.

Práctica / Actividad

Lectura previa para la Semana 2 (aula invertida).

- Pohl, K. *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.ª ed., capítulos 1–3 [1].
- IEEE/ISO/IEC 29148:2018, cláusulas 5 y 6 [2].
- Daun et al. (2023), *A systematic literature review of requirements engineering education*, Requirements Engineering Journal [6].

1.5 Resumen ejecutivo de la semana

- La IR es la actividad sobre la cual se sostiene toda la construcción posterior del software. Errores aquí cuestan hasta cien veces más en producción [7].
- Un **requisito** es, según IEEE 29148:2018, un enunciado de una necesidad y sus restricciones; una capacidad observable que debe poseer el sistema; o su representación documentada [2].
- Los **requisitos de producto** describen el sistema; los **de proceso** describen cómo se construye [3].
- Los **requisitos funcionales** dicen qué hace el sistema; los **no funcionales** describen sus cualidades transversales [11, 17].
- Las **propiedades emergentes** solo aparecen cuando todos los componentes interactúan; no se pueden atribuir a un componente aislado [11].
- Todo RNF debe ser **cuantificable**: si no se puede medir, no se puede verificar [1].
- Existen cuatro **niveles** de requisitos (negocio, stakeholder, sistema, software) que se trazan entre sí [2].
- Las **restricciones** no se negocian; las **dependencias** deben mapearse explícitamente para evitar inconsistencias [1, 3].
- ISO/IEC 25010:2023 introduce **nueve características de calidad** (frente a las ocho de la

versión 2011), añadiendo **Safety** y renombrando Usabilidad y Portabilidad como *Interaction capability* y *Flexibility* respectivamente [4].

Semana 2

Proceso de ingeniería de requisitos y contexto del sistema

Un proceso de Ingeniería de Requisitos sin un marco que lo organice se reduce a entrevistas dispersas y documentos inconexos; un marco sin proceso es solo teoría. La Ingeniería de Requisitos madura cuando ambos confluyen en un ciclo repetible, medible y mejorable.

adaptado de Pohl [1]

Objetivos de aprendizaje de la semana

Al finalizar esta segunda semana, el estudiante estará en capacidad de:

1. Comprender la naturaleza *procesual* de la IR, identificando sus actividades, artefactos y actores [1, 3].
2. Comparar los **tres grandes modelos de proceso** de IR (cascada, iterativo y ágil), reconociendo sus ventajas, desventajas y contextos de aplicación [11, 21, 22].
3. Articular el proceso de IR con el **ciclo de vida del software** normalizado por *ISO/IEC/IEEE 12207:2017* [23].
4. Aplicar criterios de **calidad y madurez** al proceso de IR, conforme al modelo REPM y a los principios de *ISO/IEC 33001:2015* [24-26].
5. Operar el **framework de IR de Pohl**: tres actividades core, dos transversales y tres tipos de artefactos [1].
6. Distinguir los principales **tipos de sistemas de software** –información, embebidos, software-intensivos y autoadaptativos– y sus consecuencias para la IR [1, 27, 28].
7. Establecer correctamente el **límite del sistema** y formular una **visión** de proyecto trazable y comunicable [1, 10].
8. Analizar el contexto de un sistema mediante las **ocho perspectivas** de Pohl: uso, sujeto, TI, desarrollo, negocio, innovación, sociedad y técnica [1].

2.1 Tema 3. Proceso de requisitos

2.1.1 Subtema 3.1. Modelos de proceso de requisitos. Actores

¿Por qué hablamos de «proceso»de IR?

La palabra **proceso** no es decorativa: cuando hablamos de «proceso de IR»estamos afirmando que las actividades de elicitar, documentar, negociar, validar y gestionar requisitos *no se realizan por casualidad ni en cualquier orden*, sino que constituyen un **flujo ordenado, repetible y mejorable** cuya ejecución puede medirse y comparase entre proyectos [1, 3, 11].

Definición

Proceso de Ingeniería de Requisitos. Conjunto estructurado de actividades –y de las prácticas y artefactos asociados– que transforman las necesidades de los Stakeholders en requisitos especificados, acordados, validados y mantenidos a lo largo del ciclo de vida del sistema [1, 3, 29].

Pensar en proceso aporta tres ventajas concretas: (i) permite **repetir** buenos resultados de un proyecto a otro, (ii) habilita la **medición y mejora** del trabajo y (iii) hace posible **comparar y certificar** a los equipos contra estándares como Capability Maturity Model Integration (CMMI) o *ISO/IEC 33001:2015* [25, 26, 30].

Los tres grandes modelos de proceso de IR

La literatura distingue, de forma estable y consensuada, tres grandes familias de modelos de proceso de IR: el modelo **en cascada (lineal)**, el modelo **iterativo** y el modelo **ágil** [1, 11, 22, 31]. La figura 2.1 sintetiza visualmente sus diferencias.

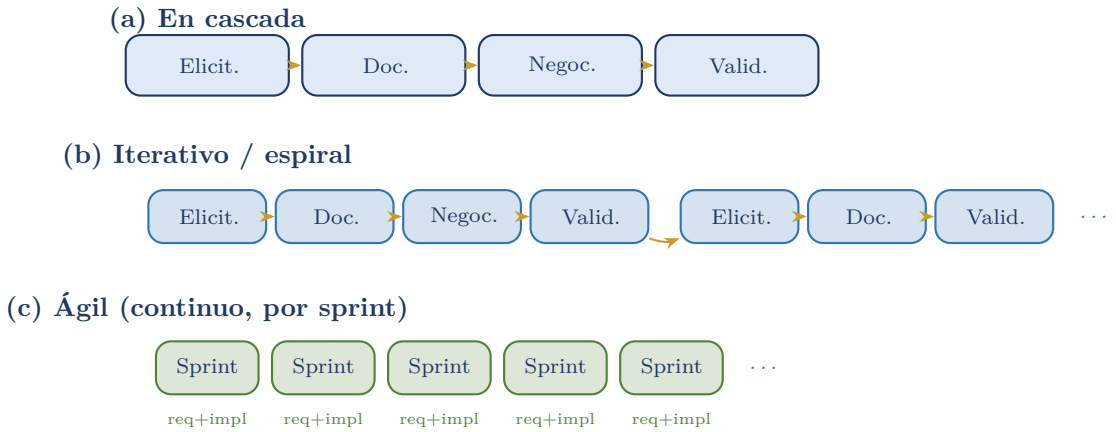


Figura 2.1. *Tres modelos de proceso de IR. (a) En cascada: actividades ejecutadas una vez en secuencia. (b) Iterativo: ciclos sucesivos refinan el conjunto de requisitos. (c) Ágil: requisitos descubiertos y desarrollados sprint a sprint. Adaptado de [11, 32-34].*

Modelo en cascada (lineal). Inspirado en el ciclo de vida clásico propuesto por Royce [32], organiza la IR como una secuencia de fases que se ejecutan *una sola vez* y producen un ERS casi definitivo antes de pasar al diseño. Es el modelo más antiguo y aún sigue vigente en proyectos altamente regulados (defensa, banca central, sistemas médicos críticos), donde un cambio tardío resulta inaceptable [11, 31, 35].

Modelo iterativo. Reconoce que la IR no se cierra de un solo intento. La idea, que Boehm formalizó con su modelo en espiral [33], es producir una versión preliminar del ERS, validarla con Stakeholders, detectar omisiones y ambigüedades, y **volver al ciclo** para refinarla. Cada iteración disminuye la incertidumbre. Suele combinarse con prototipos.

Modelo ágil. Inspirado en el *Manifesto for Agile Software Development* [36], propone descubrir y elaborar los requisitos de forma *continua*, dentro de cada *sprint*, junto con el código

y las pruebas. Las historias de usuario reemplazan a los párrafos formales, y el Product backlog sustituye al ERS monolítico [21, 22, 34, 37, 38].

Cuadro comparativo de los tres modelos

Tabla 2.1. *Comparación de los tres modelos de proceso de IR según varios autores [1, 11, 21, 22, 31].*

Dimensión	Cascada	Iterativo	Ágil
Documento principal	ERS extenso, único.	ERS evolutivo en versiones.	Backlog vivo de historias.
Comunicación stakeholder	Inicio y aprobación final.	Al cierre de cada iteración.	Continua, diaria, en sprint.
Cambios	Costosos, vía CCB.	Asumibles entre iteraciones.	Esperados, parte del proceso.
Riesgo de descubrimiento tardío	Alto.	Medio.	Bajo.
Trazabilidad	Estricta y formal.	Formal por iteración.	Más débil; depende de herramienta.
Ideal para	Sistemas críticos, regulados.	Proyectos medianos, sistemas con incertidumbre acotada.	Web/móvil, productos de mercado.
Ventajas	Predictible; auditable; contractual.	Equilibrio entre rigor y flexibilidad.	Rápido, adaptable, centrado en valor.
Desventajas	Rígido; costoso de cambiar; supone conocimiento total.	Más complejo de planificar.	Riesgo de *deuda de requisitos*; trazabilidad débil.

Idea clave. Los tres modelos no son mutuamente excluyentes: *coexisten* en proyectos reales. La banca ecuatoriana, por ejemplo, suele aplicar cascada para módulos centrales (core bancario, contabilidad) y ágil para los canales digitales y aplicaciones móviles [22, 39].

Actores del proceso de IR

Las normas [2] y [23] definen un conjunto mínimo de roles que participan en el proceso. Pohl los presenta de manera operativa [1]: el **ingeniero de requisitos** (analista) es el rol central, pero opera en una red de actores con responsabilidades complementarias.

Tabla 2.2. *Roles típicos en el proceso de IR.*

Rol	Responsabilidad principal
Patrocinador (sponsor)	Provee recursos y respaldo institucional. Aprueba alcance y baselines.
Stakeholder primario	Beneficiario directo del sistema; aporta requisitos y los valida.
Stakeholder secundario	Afectado indirecto (auditores, reguladores). Aporta restricciones.
Ingeniero de requisitos / analista	Conduce el proceso; elicita, documenta, negocia y mantiene el ERS.
Arquitecto / diseñador	Recibe el ERS; aporta retroalimentación de factibilidad técnica.
Tester	Deriva pruebas a partir de los requisitos; participa en su verificación.
Product Owner (ágil)	Custodio del backlog; prioriza historias y representa al cliente.
Comité de Control de Cambios (CCB)	Aprueba o rechaza cambios al ERS o al backlog congelado.

Ejemplo en Ecuador

Caso: equipo del proyecto integrador del semestre. En el proyecto integrador de la asignatura, el equipo de 3–5 estudiantes asume los roles centrales: un **analista líder** (responsable del ERS), un **modelador** (diagramas UML), un **documentador** (control de versiones del ERS) y un **enlace con el cliente real** (un GAD local, una microempresa de Quevedo, una clínica del cantón). El profesor actúa como **patrocinador y CCB**; el cliente real cumple el rol de **stakeholder primario**. Esta distribución refleja, a escala, la estructura típica de un proyecto profesional.

2.1.2 Subtema 3.2. Administración y soporte de procesos. Interrelación con el ciclo de vida del software

Administración del proceso de IR

Un proceso de IR no se ejecuta solo: requiere ser *planificado, asignado, monitoreado y ajustado*. Pohl distingue tres dimensiones de administración [1]:

1. **Administración del contexto**: reconocer y documentar el contexto del sistema (las ocho perspectivas que veremos en el Tema 4) y mantenerlo actualizado.
2. **Administración de las actividades core**: planificar la elicitación, la documentación y la negociación, asignar responsables y establecer hitos.
3. **Administración de los artefactos**: controlar versiones, configuraciones y baselines del ERS y de sus modelos asociados.

Soporte: técnicas, herramientas y plantillas

El soporte al proceso se concreta en **tres recursos** principales que el equipo debe organizar antes de empezar [1, 10]:

- **Técnicas estandarizadas**: entrevistas, talleres, observación, prototipos, modelado UML, EARS, MoSCoW, Kano, Fagan, walkthroughs.
- **Herramientas Computer-Aided Software Engineering (CASE)**: Jira, Trello, Enterprise Architect, Sparx EA, Figma, draw.io, Confluence, GitLab Issues, ReqIF para intercambio.
- **Plantillas y guías**: plantilla IEEE 29148:2018 para el ERS, plantillas EARS para requisitos, plantilla del documento de visión, plantillas de matrices de trazabilidad.

Interrelación con el ciclo de vida del software

La IR no es una etapa aislada del ciclo de vida del software: *atraviesa todo el ciclo*, desde la concepción hasta el retiro. La norma *ISO/IEC/IEEE 12207:2017* define los procesos del ciclo de vida del software y ubica a la IR explícitamente como un proceso técnico clave [23]. La figura 2.2 muestra cómo la IR se relaciona con las demás actividades.

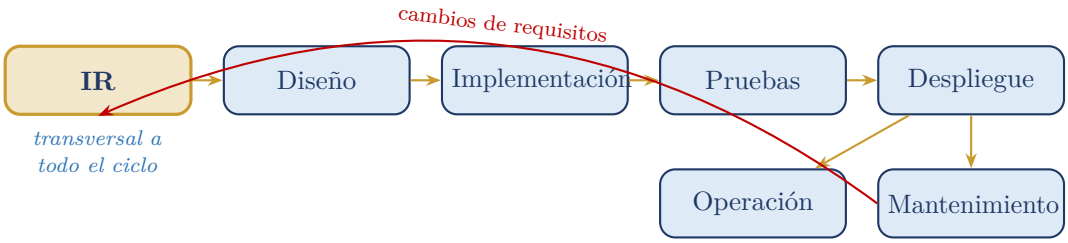


Figura 2.2. *La IR en el ciclo de vida del software. Adaptado de [1, 3, 23].*

Tres ideas clave se desprenden [11, 23]:

- La IR **precede y condiciona** todo lo que viene después: si un requisito está mal, el diseño, el código y la prueba heredan el error.
- La IR **recibe retroalimentación** continua: durante el diseño se descubren ambigüedades; durante las pruebas, omisiones; durante la operación, requisitos emergentes.
- Por eso la IR se considera **transversal y dinámica**, no una fase cerrada.

Ejemplo en Ecuador

Caso: ciclo de vida del sistema de matrícula UTEQ. Cuando la SENESCYT emite una nueva resolución de homologación de asignaturas, el sistema universitario debe ajustarse. Esto inicia un *ciclo de retroalimentación* hacia la IR: se identifican nuevos requisitos, se actualiza el ERS, se redice riseñan los módulos y se prueban. Sin un proceso de IR establecido, este cambio se «parcha»en código y termina degradando la mantenibilidad del sistema [39, 40].

2.1.3 Subtema 3.3. Perfeccionamiento y calidad del proceso de IR

¿Por qué medir y mejorar el proceso?

El estudio *Naming the Pain in Requirements Engineering* (Naming the Pain in Requirements Engineering (NaPiRE)) [39], basado en encuestas a más de 200 empresas en 10 países, identificó que las cuatro causas más frecuentes de problemas en proyectos de software son *deficiencias*

del proceso de IR: comunicación inadecuada con stakeholders, requisitos incompletos, requisitos cambiantes y especificaciones ambiguas. Esto motivó toda una corriente de modelos de **perfeccionamiento del proceso** [24, 26, 35].

Modelos de madurez aplicados a IR

Existen al menos tres marcos de referencia complementarios:

1. CMMI (Capability Maturity Model Integration). Modelo general de madurez de procesos software cuyo nivel de capacidad se evalúa de 1 (inicial) a 5 (en optimización) [30]. La IR aparece principalmente en el área de proceso «Requirements Development»y «Requirements Management».

2. *ISO/IEC 33001:2015*. Define el marco de evaluación de procesos software como sucesor de ISO/IEC 15504 (SPICE) [25]. Establece atributos de proceso (process attributes) y niveles de capacidad de proceso de 0 a 5.

3. REPM (Requirements Engineering Process Maturity). Propuesto por Sawyer, Sommerville y Viller [24], es un modelo *específico para IR* que organiza las prácticas en tres niveles según se introduzcan en la organización (tabla 2.3).

Tabla 2.3. *Modelo REPM de Sawyer, Sommerville y Viller [24].*

Nivel	Carácter	Prácticas típicas
1. Inicial	Caótico, dependiente del individuo.	No hay procedimientos definidos; los analistas hacen «lo que pueden».
2. Repetible	Prácticas básicas adoptadas.	Estructura del ERS, identificación de stakeholders, control de versiones, definición de plantillas.
3. Definido	Prácticas avanzadas e integradas.	Trazabilidad de extremo a extremo, análisis formal, métricas, mejora continua del proceso.

Métricas para evaluar la calidad del proceso de IR

Para «perfeccionar»el proceso es indispensable medirlo. Las métricas más usadas en literatura y práctica son [26, 35, 39, 40]:

- **Volatilidad de requisitos**: cantidad de cambios al ERS después de su línea base, normalizado por mes o por sprint.
- **Cobertura de trazabilidad**: porcentaje de requisitos con trazabilidad bidireccional documentada.
- **Densidad de defectos detectados en revisión de ERS**: número de defectos por página de ERS detectados en inspección.
- **Tiempo medio de resolución de un cambio de requisito**: desde su solicitud hasta su aprobación o rechazo formal.
- **Satisfacción de los stakeholders**: encuesta tras la validación o tras una iteración.

Advertencia / Error frecuente

Cuidado con medir lo fácil y olvidar lo importante. Una organización puede tener números brillantes en «cantidad de requisitos documentados»y, simultáneamente, fracasar en producción por requisitos no relevantes. Las métricas del proceso de IR deben mirar *calidad*, no solo *cantidad*: requisitos completos, verificables y trazables [39].

2.1.4 Subtema 3.4. El framework de IR: actividades core, actividades transversales y artefactos

Estructura del framework de Pohl 2024

La segunda edición del libro de Pohl (2024) presenta un framework de IR compuesto por cuatro componentes que se relacionan entre sí [1]:

1. **El contexto del sistema**, descrito mediante *ocho perspectivas* (uso, sujeto, TI, desarrollo, negocio, innovación, sociedad y técnica). Es el «marco»dentro del cual ocurre todo.
2. **Tres actividades core**: elicitación, documentación, y negociación / resolución de conflictos.
3. **Dos actividades transversales (cross-sectional)**: validación y gestión.
4. **Tres tipos de artefactos de requisitos**: metas, escenarios y requisitos orientados a solución.

La figura 2.3 muestra el framework con sus relaciones. (Una versión simplificada se introdujo en la Semana 1; aquí se presenta con todos los elementos.)

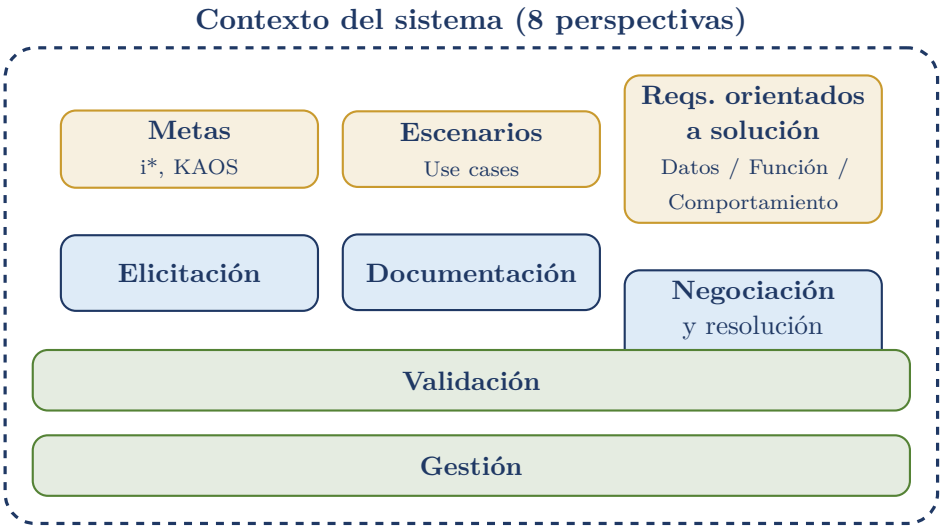


Figura 2.3. *Framework completo de IR de Pohl 2024: contexto, tres artefactos, tres actividades core y dos actividades transversales [1].*

Las tres actividades core

1. Elicitación de requisitos. Actividad de descubrir, identificar y obtener requisitos a partir de las fuentes (stakeholders, documentos, sistemas existentes, regulaciones, contexto). Se profundizará en la Unidad 2, Semanas 4–6.

2. Documentación de requisitos. Actividad de registrar los requisitos elicitados de forma adecuada (lenguaje natural, modelos, plantillas estructuradas). Se profundizará en la Unidad 3, Semanas 7–10.

3. Negociación y resolución de conflictos. Actividad de *acordar* entre los stakeholders qué requisitos son los correctos, dadas las restricciones del proyecto. Pohl da en su edición 2024 mucho más peso a esta actividad que en la primera, porque es donde se generan los conflictos clave [1].

Las dos actividades transversales

1. Validación de requisitos. Confirma que los requisitos representan correctamente las necesidades de los stakeholders y son adecuados para servir de base al desarrollo. Se profundizará en la Unidad 4, Semanas 11–13.

2. Gestión de requisitos. Mantiene el conjunto de requisitos consistente y actualizado a lo largo del ciclo de vida: trazabilidad, control de cambios, baselines, priorización dinámica.

Lo importante es que ambas son *transversales*: se ejecutan **en paralelo** a las tres actividades core, no después de ellas [1]. Cada vez que se elicita o se documenta un requisito, inmediatamente se valida y se gestiona.

Los tres tipos de artefactos

1. Metas (goals). Objetivos prescriptivos de alto nivel que el sistema o la organización deben alcanzar. Se descomponen jerárquicamente con operadores AND/OR. Se modelan en lenguajes como i* 2.0 y Knowledge Acquisition in autOmated Specification (KAOS) [41, 42].

2. Escenarios (scenarios). Descripciones concretas de secuencias de interacción entre actores y el sistema. Tipos: caso de uso, historia de usuario, escenario narrativo, *misuse case*. Se profundizan en la Unidad 2, Semana 6.

3. Requisitos orientados a solución. Documentan *qué* debe hacer el sistema desde tres perspectivas: **datos** (qué información maneja), **función** (qué procesa) y **comportamiento** (cómo reacciona en el tiempo). Se modelan con diagramas Entidad-Relación (modelo de datos) (ER), clases UML, Data Flow Diagram (Diagrama de Flujo de Datos) (DFD2), statecharts y máquinas de estado.

Ventaja del framework de Pohl. Sirve como una *lista de verificación arquitectónica del proyecto de IR*: al iniciar un proyecto, el equipo se pregunta para cada elemento (contexto, artefactos, actividades) ¿lo hemos cubierto? Esta sola pregunta evita la mayor parte de los olvidos típicos en IR [1].

2.2 Tema 4. Contexto, tipos de sistemas y visión

2.2.1 Subtema 4.1. Tipos de sistemas y consecuencias para IR

Una clasificación práctica

No todos los sistemas de software son iguales: ignorar las diferencias es una causa frecuente de fracasos. Pohl 2024 [1] y SWEBOK v4 [3] distinguen al menos cuatro grandes tipos, que imponen retos distintos a la IR.

Tabla 2.4. *Tipos de sistemas y sus consecuencias para la IR.*

Tipo	Características	Reto principal para la IR
Sistemas de información (SI)	Soportan procesos administrativos; UI rica; bases de datos.	Comprender procesos de negocio; gestión del cambio organizacional.
Sistemas embebidos	Software dentro de un dispositivo; restricciones de tiempo, energía y memoria.	Requisitos no funcionales estrictos; integración HW/SW; certificaciones.
Sistemas software-intensivos	Software interactúa intensivamente con otros sistemas y dispositivos.	Volumen y complejidad; interfaces múltiples; cadena de proveedores.
Sistemas auto-adaptativos	Reconfiguran su comportamiento en runtime ante cambios del entorno.	Requisitos contextuales y adaptativos; incertidumbre; loop MAPE-K.

Sistemas de información

Son los sistemas más comunes en empresas, gobierno y academia [11]. Capturan, almacenan, procesan y entregan información a usuarios humanos. Ejemplos: Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), sistemas académicos, facturación, contabilidad.

Consecuencias para la IR: el reto está en **entender el negocio**. Si el analista no comprende cómo trabaja Tesorería del SRI, los requisitos serán superficiales. La elicitación demanda mucha entrevista, observación y análisis documental [1, 10].

Ejemplo en Ecuador

Sistema de tesorería de un GAD cantonal. Un sistema que gestione cobros del GAD de Quevedo (impuestos prediales, patentes, permisos, tasas) es un sistema de información típico. La IR debe capturar reglas de cálculo de tasas, exenciones legales locales, formatos de comprobantes válidos para el SRI y procesos de conciliación bancaria. Si se omite alguno, el sistema falla en producción durante el cobro masivo.

Sistemas embebidos

Son sistemas software-hardware integrados en un dispositivo físico que controlan o supervisan procesos del mundo real con restricciones de tiempo, energía y memoria [1, 3]. Ejemplos: Electronic Control Unit (Unidad de Control Electrónico) (ECU) automotriz, marcapasos, contador de luz inteligente, controlador industrial.

Consecuencias para la IR: dominio de los NFR; las certificaciones (*Safety* de ISO 26262 en

automoción, IEC 62304 en dispositivos médicos) generan requisitos adicionales muy específicos. El ERS suele incluir requisitos de tiempo de respuesta en milisegundos y de energía en miliamperios.

Ejemplo en Ecuador

Empacadora bananera automatizada. En las empacadoras de banano de Los Ríos hay sistemas embebidos que controlan cintas, brazos y balanzas con tolerancias estrictas (peso ±50 g, sincronización de cintas en ±10 ms). La IR debe especificar esos umbrales con precisión y referenciar los reglamentos de exportación, porque un error aritmético se traduce en pérdida económica directa.

Sistemas software-intensivos

Son sistemas en los que el software interactúa intensivamente con otros sistemas, dispositivos y personas [27]. Una banca móvil moderna, un sistema hospitalario integrado o un «smart city»son ejemplos. La complejidad surge no del software individual, sino de las *interacciones* entre numerosos componentes.

Consecuencias para la IR: aumento del volumen de interfaces; cadena extensa de proveedores; alta incertidumbre. Se requieren *requisitos de integración*, *requisitos de operación distribuida* y mucha trazabilidad.

Sistemas autoadaptativos

Son sistemas capaces de **monitorear su entorno y modificar su propio comportamiento** en runtime para mantener sus metas frente a cambios o incertidumbre [27, 28]. La arquitectura canónica es **MAPE-K**: Monitor, Analyze, Plan, Execute, sobre una base de Knowledge compartida. Ejemplos: vehículos autónomos, aplicaciones de «smart vehicles», balanceadores en la nube, asistentes basados en Large Language Model (Modelo Grande de Lenguaje)s (LLMs).

Consecuencias para la IR: aparece una nueva categoría de requisitos –los **requisitos contextuales y adaptativos**– que describen condiciones del entorno y reglas de adaptación [27, 28]. Tradicionalmente, los requisitos se asumían *conocidos al inicio*; en sistemas adaptativos hay que asumir que *surgirán durante la operación*. Esto exige nuevos enfoques como los modelos KAOS extendidos o las *contextual goal models*.

Tendencia actual (2023–2025). La IR para sistemas adaptativos –especialmente para sistemas con componentes de LLM y de aprendizaje automático– es una de las áreas de investigación más activas del momento, con avances importantes documentados en [43-47].

2.2.2 Subtema 4.2. Límite del sistema, descomposición y visión

El límite del sistema

Definición

Límite del sistema (system boundary). Frontera conceptual que separa lo que pertenece al sistema a construir (*el sistema*) de lo que pertenece a su entorno (el contexto). Definirlo correctamente es una de las primeras y más críticas decisiones de la IR [1, 2, 48].

Un límite mal definido produce dos errores opuestos pero igualmente costosos:

- **Sub-especificación**: dejar fuera del sistema componentes que en realidad le competen (por ejemplo, omitir el módulo de firma electrónica en un sistema de comprobantes electrónicos del SRI).
- **Sobre-especificación**: incluir como propios componentes que pertenecen al entorno (por ejemplo, especificar el motor de bases de datos de Oracle dentro del sistema, en lugar de considerarlo una restricción de plataforma).

La figura 2.4 muestra los tres elementos que la IR debe distinguir desde el inicio: el sistema, el contexto relevante y el entorno externo [1].

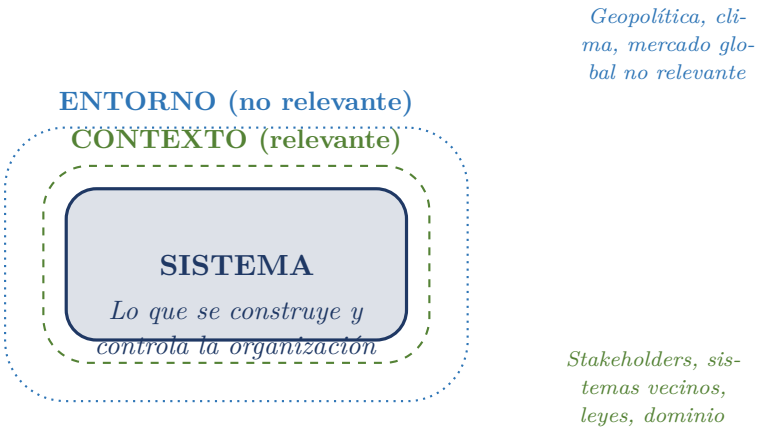


Figura 2.4. *Sistema, contexto y entorno: los tres niveles que la IR debe distinguir. Adaptado de [1].*

Descomposición del sistema

Una vez establecido el límite, la IR suele descomponer el sistema en *subsistemas* y *componentes lógicos*, cada uno con sus requisitos asignados [1, 2, 48]. Esta descomposición se documenta en el ERS o, mejor, en el documento de arquitectura preliminar.

Ejemplo en Ecuador

Sistema de salud pública del MSP. Un sistema integrado del MSP para gestión de turnos, recetas e historia clínica suele descomponerse en subsistemas: gestión de turnos, gestión de pacientes, historia clínica electrónica, recetas, integración con farmacias y analítica. La IR debe asignar a cada subsistema sus requisitos y gestionar las interfaces entre ellos.

La visión del sistema

Definición

Visión del sistema. Descripción concisa, motivadora y compartida de *lo que el sistema debe llegar a ser y del valor que aportará.* Suele documentarse en una o dos páginas y guía las decisiones a lo largo del proyecto [1, 10].

Wiegers & Beatty proponen una plantilla muy difundida en la industria para el documento de visión [10], sintetizada en la tabla 2.5.

Tabla 2.5. *Plantilla de documento de visión adaptada de [10].*

Sección	Contenido orientativo
1. Oportunidad de negocio	¿Qué problema o necesidad existe?
2. Declaración del problema	¿A quién afecta y cuál es su impacto?
3. Declaración de la posición	Frase de una línea: «Para X, que requiere Y, el sistema Z hace W».
4. Stakeholders y usuarios	Quiénes son y qué les interesa.
5. Beneficios principales	Tres a cinco beneficios cuantificables.
6. Funcionalidades de alto nivel	Lista corta de capacidades clave.
7. Restricciones y supuestos	Tecnológicas, legales, de tiempo, económicas.
8. Criterios de éxito	Cómo se sabrá que el proyecto fue exitoso.

Ejemplo en Ecuador

Plantilla de visión: sistema de matrícula UTEQ.

Para los estudiantes y la Secretaría Académica de la UTEQ, que requieren un proceso de matrícula rápido, transparente y trazable durante los períodos críticos del calendario académico, el Sistema Integrado de Matrícula «MatrículaUTEQ»permitirá inscribirse, consultar prerrequisitos, generar órdenes de pago y obtener el comprobante oficial en menos de cinco minutos por estudiante, soportando 1 500 inscripciones concurrentes en hora pico.

Esta única frase orienta todo el proyecto: público, dolor a resolver, solución y atributos clave.

2.2.3 Subtema 4.3. Las ocho perspectivas de contexto de Pohl

¿Por qué ocho perspectivas?

En su primera edición, Pohl introdujo *cuatro* facetas de contexto. La segunda edición de 2024 [1] las expandió a **ocho perspectivas**, reflejando que el contexto de los sistemas modernos

–autoadaptativos, distribuidos, alimentados por datos– es mucho más rico que hace una década. Cada perspectiva aporta información necesaria para construir requisitos completos.

La figura 2.5 representa las ocho perspectivas en torno al sistema.

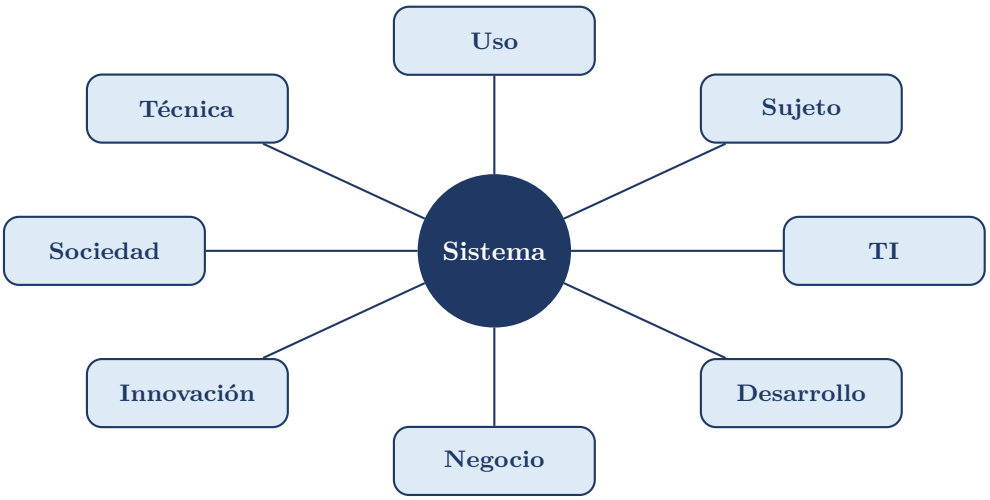


Figura 2.5. *Las ocho perspectivas de contexto de Pohl 2024 [1].*

Detalle de cada perspectiva con ejemplos ecuatorianos

Tabla 2.6. *Las ocho perspectivas de contexto, sus preguntas guía y ejemplos.*

Perspectiva	Pregunta guía	Ejemplo (sistema de turnos del)
1. Uso	¿Quiénes lo usan, en qué tareas, en qué condiciones?	Pacientes con diferente nivel de alfabetización digital, médicos en consulta, personal administrativo.
2. Sujeto	¿Sobre qué entidades del mundo opera?	Pacientes, citas, tratamientos, recetas, médicos.
3. TI	¿Con qué sistemas de TI debe interactuar?	Historia clínica, sistema de farmacia, identificación digital.
4. Desarrollo	¿En qué proceso, con qué metodología, con qué equipo se construye?	Equipo distribuido, metodología ágil, despliegue continuo.
5. Negocio	¿Qué procesos de negocio soporta? ¿Qué políticas?	Política de turnos del MSP, atención emergente vs. programada, copagos.
6. Innovación	¿Qué oportunidades de innovación hay?	Predicción de no-asistencia con IA; turnos por canal de mensajería.
7. Sociedad	¿Qué impacto social tiene? ¿Qué normas?	Igualdad de acceso en zonas rurales; protección de datos según LOPDP.
8. Técnica	¿Qué restricciones técnicas hay? (HW, red, etc.)	Conectividad limitada en hospitales rurales; equipos heterogéneos.

Perspectiva de uso. Caracteriza a los usuarios y sus tareas. Recoge perfiles, nivel de alfabetización, frecuencia de uso, contextos de uso (oficina, calle, operación quirúrgica). Es la fuente directa de requisitos de *Interaction capability* (ISO 25010:2023) [4].

Perspectiva del sujeto. Caracteriza el dominio del problema: las entidades del mundo real sobre las que opera el sistema. En un sistema bancario son cuentas, transacciones, clientes, divisas. Esta perspectiva alimenta los *modelos de datos* y diagramas ER.

Perspectiva TI. Caracteriza los sistemas existentes con los que debe interoperar. En un proyecto público ecuatoriano, esto incluye típicamente el ESIGEF (Ministerio de Finanzas), el portal del SERCOP, el sistema del SRI, el del IESS, y la Plataforma Gubernamental.

Perspectiva de desarrollo. Caracteriza el proceso, la metodología y el equipo: si será cascada o ágil, si el equipo es distribuido o local, si hay subcontratación, qué herramientas se usarán. Influye en los requisitos de proceso (Semana 1, Subtema 2.1).

Perspectiva de negocio. Caracteriza los procesos de negocio que el sistema debe soportar y las políticas que debe respetar. En la UTEQ, por ejemplo: política de matrículas, política de exámenes recuperatorios, reglamento de homologación.

Perspectiva de innovación. «¿Qué podríamos hacer diferente o mejor con este sistema?». Es la perspectiva más prospectiva: identifica oportunidades de aplicar nuevas tecnologías (IA, IoT, blockchain) o nuevos modelos operativos. En la edición 2024 de Pohl tiene mayor protagonismo [1].

Perspectiva de sociedad. Caracteriza el impacto social, ético y legal del sistema. Cubre accesibilidad, equidad, sostenibilidad, protección de datos (LOPDP, GAD), inclusión cultural y lingüística [14, 49]. Es donde aparecen los *stakeholders indirectos*.

Perspectiva técnica. Caracteriza las restricciones tecnológicas: red, hardware, sistemas operativos, plataformas en la nube, dispositivos. En zonas rurales del Ecuador, esta perspectiva impone con frecuencia restricciones de conectividad y de potencia que no aparecen en proyectos urbanos.

Cómo aplicar las ocho perspectivas en la práctica

Pohl recomienda usar las perspectivas como un **checklist estructurado** al inicio del proyecto [1]: para cada una se agendan al menos una entrevista o un taller con stakeholders relevantes y se documentan los hallazgos. La tabla 2.7 muestra una plantilla práctica.

Tabla 2.7. *Plantilla de análisis de contexto por perspectivas.*

Perspectiva	Stakeholder a entrevistar	Hallazgos clave	Requisitos derivados (preliminares)
Uso	Usuarios finales	...	RF/RNF de Interaction capability
Sujeto	Experto del dominio	...	Modelo conceptual de datos
TI	Arquitecto / DBA	...	RF de integración / restricciones
Desarrollo	Project Manager / scrum master	...	Requisitos de proceso
Negocio	Product Owner / dueño del proceso	...	Reglas de negocio, RF
Innovación	Líderes de área	...	Oportunidades de mejora
Sociedad	Compliance, legal, RR.HH.	...	Restricciones legales, accesibilidad
Técnica	Operaciones / DevOps	...	Restricciones técnicas, RNF de plataforma

Ejemplo en Ecuador

Aplicación práctica al proyecto integrador del semestre. Cada equipo del curso tomará su sistema objetivo (un GAD cantonal, una microempresa de Quevedo, una clínica privada, un sistema agroindustrial) y completará la plantilla anterior con al menos un hallazgo concreto y un requisito preliminar por perspectiva. El resultado se presentará en el primer hito del proyecto integrador.

2.3 Mapa síntesis de la Semana 2

La tabla 2.8 conecta los conceptos vistos durante la semana y muestra dónde aparecerán de nuevo a lo largo del semestre.

Tabla 2.8. *Mapa de cierre de la Semana 2.*

Pilar	Aporte conceptual	Dónde se profundiza
Modelos de proceso (3.1)	Cascada, iterativo, ágil; actores.	Unidad 5 (proyecto integrador).
Administración / ciclo (3.2)	La IR como proceso transversal del ciclo de vida.	Unidad 4 (gestión).
Calidad y madurez (3.3)	CMMI, ISO 33001, REPM, métricas de proceso.	Unidad 4 (gestión, métricas).
Framework de Pohl (3.4)	Tres core, dos transversales, tres artefactos.	Unidades 2, 3 y 4 (todo el resto del curso).
Tipos de sistemas (4.1)	SI, embebidos, software-intensivos, autoadaptativos.	Unidad 2 (elicitación contextual).
Límite y visión (4.2)	Sistema vs. contexto vs. entorno; doc. de visión.	Unidad 5 (kickoff del proyecto).
Ocho perspectivas (4.3)	Checklist sistemático para análisis de contexto.	Unidad 2 (elicitación) y todo el ERS.

2.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 2 (en clase, 2 horas).

Objetivo: aplicar el framework de Pohl 2024 y las ocho perspectivas de contexto a un caso real ecuatoriano.

Caso: la UTEQ desea modernizar el sistema de matrícula en línea para soportar matrículas concurrentes durante la semana pico.

Tareas en equipos de 3–4 estudiantes:

1. Decidir y argumentar qué *modelo de proceso* es más apropiado (cascada, iterativo, ágil) y por qué.
2. Identificar los actores del proceso (al menos 6) con su rol principal.
3. Para cada una de las ocho perspectivas de contexto, identificar un stakeholder a entrevistar y al menos un requisito preliminar que se derivaría.
4. Redactar el documento de visión usando la plantilla de Wiegers [10], no más de una página.
5. Discutir en plenaria los hallazgos.

Práctica / Actividad

Tarea autónoma n.º 2 (4 horas, individual).

Caso: usted ha sido contratado como analista para un proyecto de un GAD parroquial rural del Ecuador que desea construir una *aplicación móvil* para reportar incidencias urbanas (baches, luminarias, basura).

Tareas:

1. Clasifique el sistema según los tipos de la sección 4.1 (información, embebido, software-intensivo, autoadaptativo) y argumente la clasificación.
2. Defina explícitamente el límite del sistema y diga qué queda dentro y qué queda fuera.
3. Llene la plantilla de las ocho perspectivas de contexto con al menos un hallazgo y un requisito preliminar por perspectiva.
4. Redacte el documento de visión (una página).
5. Identifique al menos dos riesgos del proceso de IR para este caso y proponga cómo gestionarlos.

Entregable: documento PDF de máximo 8 páginas. La rúbrica ponderará especialmente la calidad del análisis de contexto.

Práctica / Actividad

Lectura previa para la Semana 3 (aula invertida).

- Pohl, K. *Requirements Engineering*, 2.ª ed., Parte II completa (capítulos sobre contexto y perspectivas) [1].
- Méndez Fernández et al. (2017), *Naming the pain in requirements engineering* [39].
- Inayat et al. (2015), *Systematic literature review on agile requirements engineering practices and challenges* [21].

2.5 Resumen ejecutivo de la semana

- La IR es un **proceso**: una secuencia ordenada, repetible y mejorable de actividades, no un conjunto de tareas improvisadas [1, 3].
- Existen **tres modelos** de proceso de IR: en cascada, iterativo y ágil. No son excluyentes; coexisten en proyectos reales según el contexto [11, 22, 34].
- El proceso de IR se administra en tres dimensiones: *contexto*, *actividades core* y *artefactos* [1].
- La IR es **transversal** al ciclo de vida del software normalizado por *ISO/IEC/IEEE 12207:2017*: precede al diseño, recibe retroalimentación de las pruebas y absorbe los cambios del mantenimiento [23].
- Modelos de madurez como CMMI, *ISO/IEC 33001* y REPM permiten medir y mejorar el proceso de IR [24-26, 30].
- El **framework de Pohl 2024** estructura la IR en cuatro componentes: contexto, tres actividades core (elicitación, documentación, negociación), dos transversales (validación,

gestión) y tres artefactos (metas, escenarios, requisitos orientados a solución) [1].

- Existen al menos cuatro **tipos de sistemas** con consecuencias distintas para la IR: información, embebidos, software-intensivos y autoadaptativos [1, 27, 28].
- Establecer el **límite del sistema** (sistema vs. contexto vs. entorno) es una de las primeras decisiones críticas de la IR [1, 48].
- El **documento de visión** (una a dos páginas) condensa el propósito del sistema en términos de problema, solución y beneficios [10].
- Las **ocho perspectivas de contexto** de Pohl 2024 (uso, sujeto, TI, desarrollo, negocio, innovación, sociedad, técnica) son un checklist sistemático para construir requisitos completos [1].

Semana 3

Elicitación de requisitos: técnicas clásicas

Los requisitos no se recolectan como si fueran manzanas caídas de un árbol; se descubren, se negocian y se construyen conjuntamente entre el analista y los stakeholders. La elicitación es, ante todo, un acto de comunicación.
adaptado de Pohl [1]

Objetivos de aprendizaje de la semana

Al finalizar esta tercera semana, el estudiante estará en capacidad de:

1. Explicar la naturaleza comunicativa de la **elicitación de requisitos** y sus fuentes principales [1-3].
2. Distinguir las principales **barreras** a la elicitación (de comunicación, de dominio, organizacionales y cognitivas) y aplicar estrategias para superarlas [50, 51].
3. Diseñar y conducir **entrevistas de elicitación** estructuradas, semiestructuradas y abiertas [1, 10, 52].
4. Diseñar y facilitar **talleres de requisitos** aplicando técnicas de dinámicas grupales [2, 52].
5. Aplicar **grupos focales** para validar preferencias y prioridades en proyectos del contexto ecuatoriano [1, 53].
6. Conducir observación directa y **etnografía** para descubrir requisitos tácitos [54, 55].
7. Extraer requisitos a partir del **análisis de documentos** y de sistemas existentes (ingeniería inversa de requisitos) [10, 52].

3.1 Tema 5. Elicitación de requisitos

3.1.1 Subtema 5.1. Fundamentos y fuentes de la elicitación

¿Qué es la elicitación y por qué es difícil?

Definición

Elicitación de requisitos. Actividad central de la IR dedicada a *descubrir, identificar y obtener* los requisitos a partir de las distintas fuentes disponibles: stakeholders, documentos, sistemas existentes, normativas y el contexto organizacional. La raíz etimológica del término (*elicere* en latín: «extraer, hacer salir») refleja su naturaleza: los requisitos no se toman, se *sacan* [1-3].

Nuseibeh y Easterbrook [51] sintetizaron los hallazgos de dos décadas de investigación en

una afirmación lapidaria: los mayores problemas en el desarrollo de software no son técnicos sino comunicativos. La elicitación es el punto donde la comunicación falla con más frecuencia y con consecuencias más costosas. Los estudios del *Standish Group* estiman que entre el 37 % y el 56 % de los proyectos fracasados reportan requisitos inadecuados como causa primaria [8, 9].

El reto no es que los stakeholders no quieran cooperar: es que **el conocimiento que necesita el analista existe en formas que no son directamente accesibles**. Zowghi y Coulin [50] identifican cuatro dimensiones de este problema:

1. **Conocimiento tácito**: los usuarios saben cómo hacen su trabajo pero no siempre saben articularlo en palabras. La etnografía y la observación revelan lo que la entrevista no puede.
2. **Vocabulario diferente**: el dominio del negocio tiene su propio léxico; el dominio del software tiene el suyo. Sin un glosario compartido, los requisitos se malinterpretan.
3. **Stakeholders múltiples con intereses contradictorios**: el jefe de contabilidad y el cajero quieren cosas distintas del mismo sistema; el analista debe descubrirlas a ambas.
4. **Requisitos latentes**: necesidades que el stakeholder no identifica como tales hasta que ve un prototipo o experimenta un problema en producción.

Fuentes de requisitos

La norma *IEEE/ISO/IEC 29148:2018* [2] y Pohl [1] distinguen las siguientes fuentes primarias de requisitos:

Tabla 3.1. *Fuentes de requisitos y técnicas de elicitación asociadas. Adaptado de [1, 2, 10].*

Fuente	Descripción	Técnicas de elicitación
Stakeholders	Personas con interés en el sistema. Fuente primaria.	Entrevistas, talleres, FG, cuestionarios.
Documentos existentes	Normativas, contratos, manuales, pliegos del SERCOP.	Análisis documental, revisión de normativa.
Sistemas existentes	El sistema «AS-IS»que el nuevo sistema reemplazará.	Ingeniería inversa de requisitos, observación.
Contexto organizacional	Procesos de negocio, cultura, estructura.	Etnografía, análisis de procesos, BPMN.
Estándares y regulaciones	Leyes, normas ISO, resoluciones del SRI, LOPDP.	Análisis de normativa, consulta legal.
Sistemas de la competencia	Productos similares en el mercado.	Análisis comparativo (benchmarking).

Proceso de elicitación: un ciclo de cuatro pasos

Aunque las técnicas de elicitación son diversas, el proceso sigue siempre un ciclo reconocible de cuatro pasos [1, 52]:

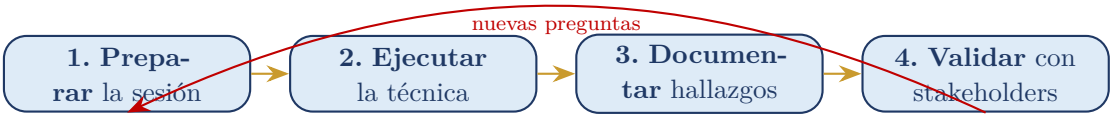


Figura 3.1. *Ciclo de cuatro pasos de la elicitación de requisitos. La flecha roja indica que la validación suele generar nuevas preguntas y una nueva iteración. Adaptado de [1, 52].*

3.1.2 Subtema 5.2. Entrevistas y cuestionarios

La entrevista de elicitación

Definición

Entrevista de elicitación. Conversación planificada y estructurada entre el analista y uno o más stakeholders con el propósito de obtener información sobre necesidades, objetivos, procesos y restricciones del sistema [1, 10, 56].

La entrevista es la técnica más antigua, más versátil y más estudiada de la elicitación. Su efectividad depende casi enteramente de la preparación y la habilidad comunicativa del analista. SWEBOK v4 [3] la sitúa como primera opción cuando se necesita información rica, contextual y matizada.

Tipos de entrevista

Tabla 3.2. *Tipos de entrevista y sus características [1, 10, 52].*

Tipo	Características	Ventajas	Cuándo usar
Estructurada	Preguntas fijas, secuencia predefinida.	Reproducible; fácil de analizar.	Cuando se necesita comparar entre stakeholders.
Semiestructurada	Guión base + espacio para derivar en temas emergentes.	Equilibra profundidad y consistencia.	Caso general; recomendado como punto de partida.
Abierta (no estructurada)	Preguntas amplias; el entrevistado dirige.	Descubre temas insospechados.	Dominio poco conocido; exploración inicial.

Cómo preparar una entrevista eficaz

Robertson & Robertson [52] y Wiegers & Beatty [10] coinciden en que la preparación es el factor que más distingue a una entrevista exitosa de una improductiva. La siguiente lista de verificación sintetiza las mejores prácticas:

- **Identificar al entrevistado correcto**: el stakeholder debe tener autoridad y conocimiento sobre el tema concreto de la sesión. No entrevistar a intermediarios cuando se puede acceder a la fuente.

- **Establecer el objetivo de la sesión**: una sola entrevista no puede cubrir todo. Definir un foco claro (por ejemplo, «flujo de emisión de facturas electrónicas») en lugar de «el sistema en general».
- **Preparar la guía de preguntas**: mínimo 10 preguntas ordenadas de general a específico. Incluir al menos tres preguntas de tipo *walk-me-through*: «¿Puede describirme paso a paso cómo hace X hoy?»
- **Investigar el dominio previamente**: consultar manuales, documentos previos, normativa vigente. Llegar con el vocabulario básico del dominio.
- **Acordar logística**: lugar, duración (60–90 min. ideal), grabación (con consentimiento), nota de seguimiento.

Técnicas de pregunta para entrevistas

Tabla 3.3. *Técnicas de formulación de preguntas para entrevistas de elicitación [1, 10, 52].*

Técnica	Ejemplo	Propósito
Walk-me-through	«¿Puede describir cómo registra hoy una receta electrónica?»	Revelar el flujo real, no el oficial.
¿Qué pasaría si…?	«¿Qué ocurre si el SRI cae a medianoche?»	Descubrir casos excepcionales.
5 porqués (5 Whys)	«¿Por qué el proceso tarda tres días?»(y continuar preguntando por qué)	Llegar a la causa raíz de un problema.
Escala de satisfacción	«Del 1 al 10, ¿qué tan satisfecho está con el sistema actual?»	Cuantificar el dolor o la necesidad.
Contraste AS-IS / TO-BE	«¿Cómo le gustaría que funcionara en comparación con hoy?»	Identificar metas y brechas.

Cuestionarios

Cuando el número de stakeholders es grande (por ejemplo, 200 empleados de un GAD provincial) o están distribuidos geográficamente, la entrevista individual no escala. El **cuestionario** permite recoger información de muchos stakeholders en paralelo y con costo reducido [3, 10, 56].

Sus limitaciones son simétricas a sus ventajas: no permite profundizar en respuestas interesantes, la tasa de respuesta suele ser baja, y los errores de comprensión no pueden corregirse en el momento. Por ello se recomienda usarlo como *complemento* de las entrevistas, no como sustituto [52].

Advertencia / Error frecuente

Error frecuente. Elaborar cuestionarios muy largos (más de 20 preguntas) o con preguntas ambiguas. La regla de oro: cada pregunta debe ser comprendida unívocamente por el 95 % de los destinatarios. Probarlo con al menos tres personas del dominio antes de distribuirlo [10].

Ejemplo en Ecuador

Caso: sistema de atención al contribuyente del SRI. El SRI tiene más de 18 000 contribuyentes registrados en la provincia de Los Ríos. Para elicitar requisitos del portal de consulta tributaria en línea, resulta inviable entrevistar a todos. La estrategia combinada fue: (i) entrevistas en profundidad con 12 contadores y 5 auditores tributarios para entender flujos complejos; (ii) cuestionario digital (Google Forms) con 15 preguntas enviado a 400 contribuyentes; (iii) análisis de los 200 tickets de soporte más frecuentes. El cuestionario reveló que el 67 % de los encuestados esperaba poder descargar el Anexo Transaccional en formato CSV, requisito que no había aparecido en las entrevistas.

3.1.3 Subtema 5.3. Talleres de requisitos y grupos focales

Talleres de requisitos

Definición

Taller de requisitos. Sesión facilitada, de duración definida (habitualmente medio día a dos días), en la que representantes de varios grupos de stakeholders trabajan juntos para elicitar, discutir, priorizar y acordar requisitos de forma colaborativa [2, 10, 52].

El taller tiene una ventaja que ninguna entrevista individual puede ofrecer: permite que los *conflictos entre stakeholders afloren y se resuelvan en tiempo real*, en lugar de descubrirse tardíamente durante la revisión del ERS [1]. Robertson & Robertson [52] lo denominan «el evento de requisitos»(*requirements event*) y lo consideran la técnica de mayor rendimiento por hora invertida.

Estructura de un taller efectivo

Tabla 3.4. *Estructura de un taller de requisitos de un día. Adaptado de [10, 52].*

Bloque	Duración	Actividades
Apertura	30 min	Presentación del objetivo, normas de trabajo, roles (facilitador, scribe, timekeeper).
Visión compartida	45 min	Lectura y validación del documento de visión; ajuste de alcance.
Elicitación abierta	90 min	Brainstorming de requisitos funcionales por área; técnica de post-its o *event storming*.
Almuerzo / descanso	60 min	—
Análisis y conflictos	90 min	Agrupación, deduplicación, detección de conflictos. Primer nivel de priorización (MoSCoW).
NFR y restricciones	45 min	Identificación de atributos de calidad con criterios cuantificables (ISO 25010).
Cierre y acuerdos	30 min	Lectura de acuerdos, pendientes, responsables y fecha de la próxima sesión.

Rol del facilitador. El facilitador es *neutral en cuanto al contenido* pero *activo en cuanto al proceso*: su función no es aportar requisitos sino asegurar que todos los participantes puedan expresarse, que se respete el tiempo, que los conflictos se traten de forma constructiva y que los acuerdos queden documentados [10, 52].

Grupos focales

Definición

Grupo focal (FG). Entrevista grupal moderada, de 6 a 12 participantes, en la que se exploran actitudes, percepciones y preferencias ante una propuesta de sistema, una maqueta o un prototipo [1, 3, 53].

La diferencia clave con el taller es el enfoque: el taller elicita requisitos mediante trabajo colaborativo; el grupo focal *valida* concepciones y detecta preferencias. Es especialmente valioso para requisitos de *Interaction capability* (ex-Usability) y de *Safety* en el modelo ISO 25010:2023 [4].

Kontio et al. [53] demostraron que los grupos focales, cuando están bien moderados, aportan más información sobre NFR de usabilidad que diez entrevistas individuales, por el efecto de *groupthink reverso*: un participante articula una experiencia que resuena en otros que no la habrían mencionado solos.

Ejemplo en Ecuador

Caso: app móvil del GAD de Quevedo para reportar incidencias. Antes de especificar los requisitos de interfaz, el equipo organizó dos grupos focales: uno con ciudadanos de entre 18 y 35 años (nativos digitales) y otro con adultos mayores de 55 años con acceso limitado a smartphones. El primer grupo priorizó la geolocalización automática y la carga de fotos; el segundo demandó instrucciones de voz y botones grandes. Sin ambos grupos focales, el ERS habría asumido implícitamente solo el perfil del primer grupo, excluyendo al 38 % de la población objetivo.

3.1.4 Subtema 5.4. Observación y etnografía

La observación como técnica de elicitación

Definición

Observación de campo. Técnica en la que el analista estudia el trabajo de los usuarios en su entorno real, sin interrumpirles, con el objetivo de descubrir requisitos tácitos: lo que los usuarios *hacen* en lugar de lo que *dicen que hacen* [1, 54, 55].

La brecha entre el trabajo declarado y el trabajo real es uno de los hallazgos más robustos de la investigación en IR [51, 54, 55]. Cuando un contador describe su proceso de conciliación bancaria, tiende a describir el proceso oficial, no las hojas de cálculo auxiliares que usa en realidad. Solo la observación puede capturar esas prácticas informales que se convierten en requisitos ocultos del nuevo sistema.

Modalidades de observación

Tabla 3.5. *Modalidades de observación en IR [1, 54, 55].*

Modalidad	Descripción	Cuándo usar
Pasiva (no participante)	El analista observa sin interactuar.	Cuando la interacción distorsionaría el comportamiento normal.
Activa (participante)	El analista realiza las tareas junto al usuario.	Para entender procesos complejos que requieren hacerlos.
Etnografía	Observación prolongada (días/-semanas) con notas de campo, entrevistas informales y análisis cultural.	Sistemas con alto componente organizacional o cultural.
Observación remota	Grabación de sesiones de usuario con herramientas de *screen recording* o *eye-tracking*.	Validación de prototipos; UX.

Etnografía aplicada a la IR

Sharp et al. [55] introdujeron la etnografía como técnica formal de elicitación en IR, adaptando metodologías de las ciencias sociales al contexto de la ingeniería de software. Su estudio en un centro de control de tráfico aéreo demostró que la observación etnográfica reveló más del 60 % de los requisitos de coordinación que ninguna entrevista individual había capturado, porque esos requisitos emergían de interacciones espontáneas entre controladores.

Millen [54] propuso la **etnografía rápida** como adaptación para proyectos con plazos cortos: observación focalizada de pocas horas en los momentos más críticos del flujo de trabajo, combinada con entrevistas de seguimiento inmediato.

Ejemplo en Ecuador

Caso: sistema de despacho en una cooperativa de transporte interprovincial. Antes de especificar el sistema de boletería en línea, el equipo pasó un día observando el proceso en la terminal terrestre de Quevedo. Descubrieron que los despachadores usaban un cuaderno físico para anotar pasajeros sin asiento asignado («en lista») —un proceso que no aparecía en ningún manual— y que era fundamental para el despacho en temporada alta. El nuevo sistema debía incluir ese concepto como requisito funcional, aunque ningún stakeholder lo había mencionado en las entrevistas previas.

Guía para planificar una sesión de observación

- **Seleccionar momentos y lugares clave**: el flujo de trabajo tiene picos (cierre mensual, temporada alta, hora pico) que concentran los casos más reveladores.
- **Solicitar permiso explícito**: tanto de la gerencia como del usuario observado. La observación sin consentimiento es éticamente inaceptable y legalmente cuestionable en el marco de la LOPDP [14].
- **Llevar plantilla de notas de campo**: quién, qué, cuándo, artefactos usados (papel, Excel

auxiliar, pantallas), excepciones observadas, preguntas para el seguimiento.

- **No interrumpir**; anotar y preguntar al finalizar el episodio de trabajo.
- **Triangular**: comparar lo observado con lo declarado en entrevistas previas para identificar brechas.

3.1.5 Subtema 5.5. Análisis de documentos y sistemas existentes

El análisis documental como fuente de requisitos

Wiegers & Beatty [10] recuerdan que toda organización tiene documentos que codifican —de forma explícita o implícita— los requisitos del sistema: manuales de procedimiento, formularios, contratos, pliegos de licitación, actas de reuniones previas, informes de problemas. Analizarlos tiene tres ventajas: (i) no depende de la disponibilidad de los stakeholders, (ii) capta el estado real del sistema (no la versión recordada) y (iii) identifica inconsistencias entre lo documentado y lo practicado.

Documentos clave en el contexto ecuatoriano

Tabla 3.6. *Tipos de documentos útiles para la elicitación en proyectos ecuatorianos.*

Tipo de documento	Requisitos que revela
Pliego de licitación del SERCOP	Requisitos funcionales, legales y de proceso del cliente.
Ficha técnica del SRI (facturación electrónica)	Formato XML, campos obligatorios, validaciones, tiempos de respuesta.
Manual de usuario del sistema «AS-IS»	Funcionalidades actuales, flujos existentes, excepciones.
Reglamento interno / normativa del cliente	Reglas de negocio, políticas de acceso, restricciones operacionales.
Informes de problemas / tickets de soporte	Defectos recurrentes = candidatos a requisitos de mejora.
Resoluciones del organismo rector (SENESCYT, IESS, SRI)	Restricciones legales que se convierten en RNF.
Actas de reuniones anteriores del cliente	Decisiones ya tomadas, restricciones acordadas, alcance previo.

Ingeniería inversa de requisitos desde sistemas existentes

Cuando el nuevo sistema debe reemplazar a uno existente, es posible aplicar **ingeniería inversa de requisitos**: a partir del comportamiento observable del sistema AS-IS, derivar los requisitos implícitos que cumple. Esta técnica es especialmente útil cuando la documentación original se ha perdido —situación frecuente en sistemas gubernamentales ecuatorianos con décadas de antigüedad [10, 52].

El proceso consta de cuatro pasos:

1. **Exploración del sistema existente**: mapear las funcionalidades, las entidades de datos y los flujos mediante navegación guiada y captura de pantallas.
2. **Extracción de reglas de negocio**: identificar validaciones, cálculos y flujos condicionales desde el código fuente (si está disponible) o desde el comportamiento observable.
3. **Documentación como requisitos**: expresar cada comportamiento identificado como un enunciado de requisito verificable.
4. **Validación con usuarios**: confirmar que los requisitos derivados son correctos y completos; identificar aquellos que el nuevo sistema debe *conservar* vs. los que debe *mejorar o eliminar*.

Ejemplo en Ecuador

Caso: modernización del sistema de inventarios de un hospital del IESS. El sistema existente (desarrollado en Visual Basic 6.0, año 2001) no tenía documentación funcional. Mediante ingeniería inversa: se navegó el sistema durante tres días, se capturaron 340 pantallas, se extrajeron 89 reglas de negocio del código VB6 y se documentaron como requisitos. La revisión con los farmacéuticos reveló que 23 de esas 89 reglas eran obsoletas (procedimientos que ya no se usaban) y que faltaban 12 funcionalidades que se gestionaban en hojas de Excel paralelas. El ERS resultante fue mucho más completo que si se hubiera empezado desde cero.

Comparación de las cinco técnicas de elicitación

Tabla 3.7. *Comparación de las técnicas clásicas de elicitación por criterios relevantes [1, 10, 50, 52].*

Criterio	Entrevista	Cuestiona	Taller	Observaci	Análisis doc.	
Profundidad	Alta	Baja	Media	Alta	Media	
Cobertura (N stake.)	Baja	Alta	Media	Baja	Alta	
Costo por dato	Alto	Bajo	Medio	Alto	Bajo	
Req. tácitos	Medio	Nulo	Bajo	Muy alto	Bajo	
Conflictos visibles	Bajo	Nulo	Alto	Bajo	Nulo	
Formación requerida	Media	Baja	Alta	Alta	Baja	

La tabla 3.7 confirma el consenso de la literatura [1, 10, 52]: **ninguna técnica es superior en todos los criterios**. La estrategia óptima combina al menos tres técnicas complementarias en todo proyecto.

3.2 Mapa síntesis de la Semana 3

Tabla 3.8. *Mapa de cierre de la Semana 3.*

Subtema	Concepto clave	Aplicación en el proyecto
5.1 Fundamentos	Conocimiento tácito; 4 dimensiones del reto.	Selección de técnicas para el kickoff.
5.2 Entrevistas	Walk-me-through; 5 Whys; guía de preguntas.	Primera ronda de entrevistas con el cliente.
5.3 Talleres	Facilitador neutral; estructura de un día.	Taller de consenso en Semana 5.
5.4 Observación	Brecha AS-IS/TO-BE; etnografía rápida.	Visita de campo al cliente.
5.5 Documentos	Ingeniería inversa; docs. SRI/-SERCOP/IESS.	Recopilación documental antes del taller.

3.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 3 (en clase, 2 horas).

Objetivo: practicar la entrevista semiestructurada en un escenario real.

Caso: un comerciante informal de Quevedo desea un sistema de inventario y ventas básico para su minimarket.

Tareas en parejas (analista / stakeholder):

1. El analista prepara una guía de 10 preguntas semiestructuradas; el stakeholder prepara 5 «sorpresas»(requisitos ocultos que solo revelará si le preguntan correctamente).
2. Conducir la entrevista durante 20 minutos; el resto del equipo observa y toma notas.
3. Documentar los requisitos elicitados en la plantilla IEEE 29148 [2]: mínimo 8 RF y 3 RNF.
4. Discusión plenaria: ¿cuántas «sorpresas»descubrió cada pareja?, ¿qué técnica de pregunta fue más efectiva?

Práctica / Actividad

Tarea autónoma n.º 3 (4 horas, en equipo del proyecto integrador).

Entregable para el hito 1 del proyecto integrador:

1. **Plan de elicitación**: fuentes identificadas, técnicas seleccionadas (justificadas), cronograma de sesiones, roles.
2. **Guía de entrevista**: mínimo 12 preguntas organizadas por tema para la primera sesión con el cliente real.
3. **Resultados de la primera sesión**: acta de la entrevista con el cliente real, incluyendo: stakeholders presentes, duración, principales hallazgos y lista preliminar de al menos 15 candidatos a requisitos.
4. **Análisis documental**: revisión de al menos 3 documentos del cliente (manual, reglamento, formularios), con la lista de requisitos que se derivan de cada uno.

Entregable: informe PDF de máximo 10 páginas, más los documentos de soporte (acta de entrevista, guía, análisis documental).

Práctica / Actividad

Lectura previa para la Semana 4 (aula invertida).

- Pohl, K. [1], capítulos sobre metas y modelado orientado a objetivos (Parte IV).
- Robertson & Robertson [52], cap. 10 (*Business Events*).
- SWEBOK v4 [3], área de conocimiento de IR, sección sobre modelado de requisitos.

3.4 Resumen ejecutivo de la semana

- La **elicitación** no es recolección pasiva: es descubrimiento activo de conocimiento tácito, declarado y latente [50, 51].
- Las cuatro barreras principales son el conocimiento tácito, el vocabulario diferente, los intereses contradictorios y los requisitos latentes [1, 50].
- El proceso de elicitación sigue un ciclo de cuatro pasos: preparar, ejecutar, documentar y validar [1, 52].
- La **entrevista semiestructurada** es la técnica de mayor rendimiento por hora; la técnica *walk-me-through* es la más efectiva para descubrir flujos reales [10].
- Los **talleres** son insustituibles para resolver conflictos entre stakeholders en tiempo real [52].
- La **observación etnográfica** revela requisitos tácitos que ninguna entrevista puede capturar [54, 55].
- El **análisis documental** y la ingeniería inversa de requisitos permiten recuperar el saber acumulado del sistema AS-IS [10, 52].
- **Ninguna técnica es suficiente por sí sola**: la estrategia óptima combina al menos tres técnicas complementarias en todo proyecto [1, 56].

Semana 4

Técnicas avanzadas de elicitación y modelado de metas

Un prototipo vale más que mil palabras de requisito: muestra lo que el analista ha entendido y le permite al stakeholder señalar exactamente lo que está mal. La representación visual convierte las suposiciones en afirmaciones verificables.
adaptado de Wiegers y Beatty [10]

Objetivos de aprendizaje de la semana

Al finalizar esta cuarta semana, el estudiante estará en capacidad de:

1. Aplicar el **prototipado** —de baja y alta fidelidad— como técnica activa de elicitación y validación temprana [1, 10, 57].
2. Construir **modelos i*** (i-estrella) de actores y dependencias para analizar el contexto organizacional y derivar metas de sistema [1, 56, 58].
3. Construir **grafos de metas KAOS** para descomponer objetivos de negocio en requisitos operacionalizables [1, 42].
4. Modelar procesos de negocio con **Business Process Model and Notation (BPMN) 2.0** e identificar los eventos de negocio que generan requisitos funcionales [52, 59].
5. Redactar **historias de usuario** completas con criterios de aceptación en formato Acceptance Test-Driven Development (ATDD)/Gherkin para proyectos ágiles [37, 60, 61].
6. Seleccionar y combinar técnicas avanzadas de elicitación según el tipo de sistema y contexto del proyecto [1, 3].

4.1 Tema 6. Técnicas avanzadas de elicitación

4.1.1 Subtema 6.1. Prototipado como técnica de elicitación

El prototipo como herramienta de comunicación

Definición

Prototipo (en IR). Representación parcial, simplificada y construida rápidamente del sistema o de alguno de sus aspectos, cuyo propósito no es ser el sistema final sino *hacer visibles las suposiciones del analista* y provocar retroalimentación temprana y concreta de los stakeholders [1, 10, 57].

Floyd [57] fue quien formalizó el concepto de prototipado como técnica de exploración de requisitos, diferenciándolo del prototipado como técnica de diseño. La distinción es crucial:

en IR, el prototipo se construye para *aprender sobre los requisitos*, no para convertirse en el sistema. Esta orientación tiene una consecuencia práctica directa: un prototipo de elicitación puede —y debe— ser descartado en cuanto cumple su función [1].

Taxonomía de prototipos

Tabla 4.1. *Taxonomía de prototipos en IR por fidelidad y técnica. Adaptado de [1, 10].*

Tipo	Descripción	Costo/tiempo	Cuándo usar
Paper mock-up (baja fidelidad)	Dibujos en papel de pantallas o flujos.	Horas; cero costo digital.	Exploración inicial; validar estructura de navegación.
Wireframe digital (baja-media)	Esquemas en Figma, Balsamiq, draw.io, sin colores ni estilos reales.	1–3 días.	Validar flujo y arquitectura de información.
Mock-up (alta fidelidad)	Diseño visual completo pero no funcional.	3–7 días.	Validar apariencia, usabilidad (pruebas A/B).
Prototipo funcional (evolutivo)	Código real, funcionalidad parcial; puede evolucionar.	Semanas.	Sistemas con alta incertidumbre de dominio.
Spike / POC	Experimento técnico muy limitado para verificar factibilidad.	1–3 días.	Validar que una integración o tecnología es posible.

Prototipado desechable vs. evolutivo

La literatura [1, 10, 57] distingue dos estrategias opuestas cuya confusión genera problemas frecuentes en proyectos:

Prototipado desechable (*throwaway*). El prototipo se construye con el único propósito de elicitar o validar requisitos. Una vez que los requisitos quedan claros, el prototipo se descarta. Ventaja: no hay presión de calidad de código. Riesgo: el cliente puede pedir «quedar con el prototipo», lo que produce un sistema frágil construido sobre una base exploratoria.

Prototipado evolutivo. El prototipo se convierte progresivamente en el sistema final. Ventaja: elimina el desperdicio de reescribir. Riesgo: acumula deuda técnica si no se refactoriza activamente. Requiere prácticas de ingeniería rigurosas desde el inicio.

Advertencia / Error frecuente

Error frecuente. El cliente ve un prototipo de alta fidelidad y asume que el sistema está «casi listo». Un wireframe interactivo de dos días de trabajo puede verse como una semana de desarrollo para alguien sin experiencia en software. Gestionar las expectativas es tan importante como construir el prototipo [10].

Ejemplo en Ecuador

Caso: sistema de gestión de becas SENESCYT. El equipo de análisis construyó en tres días un *paper mock-up* del flujo de postulación a becas. Al presentarlo a los directores de la unidad beneficiaria, descubrieron que el proceso real tenía una etapa de «preselección técnica»que ningún documento mencionaba y que era gestionada informalmente por los técnicos mediante correo electrónico. Esa etapa se convirtió en cinco nuevos requisitos funcionales del sistema.

4.1.2 Subtema 6.2. Modelado orientado a metas: i* y KAOS

¿Por qué modelar metas?

El modelado orientado a metas (*goal-oriented requirements engineering*) aborda una limitación fundamental de la IR clásica: los requisitos funcionales describen *qué hace el sistema*, pero no responden a *por qué ese sistema existe* ni *para quién*. Esta falta de conexión entre los requisitos y las metas organizacionales es una de las causas más frecuentes de sistemas técnicamente correctos pero estratégicamente inútiles [1, 42, 58].

Definición

Modelado orientado a metas. Paradigma de IR que centra el proceso en la identificación, representación y refinamiento de los *objetivos* (metas) de los stakeholders y de la organización, para derivar de ellos los requisitos del sistema de forma trazable [1, 42].

Existen dos frameworks consolidados con comunidades de práctica activas y soporte en herramientas: el **framework i*** (i-estrella) de Eric Yu [58] y el framework **KAOS** de van Lamsweerde [42]. Ambos se incluyen en el corpus del IREB CPRE v3.2.0 y en SWEBOK v4 [3].

El framework i* (i-estrella)

Definición

Framework i* (i-estrella). Marco de modelado orientado a actores sociales, propuesto por Eric Yu [58], que concibe el sistema como un conjunto de **actores** (agentes, roles, posiciones) que mantienen **dependencias** entre sí: de recursos, de tareas, de metas y de *softgoals* (metas blandas).

El framework i* se usa en la IR con dos modelos complementarios [1, 58]:

- **Modelo SD** (*Strategic Dependency*): muestra las dependencias entre actores externos al sistema, sin entrar en sus intenciones internas. Responde a la pregunta: *¿quién depende de quién, y en qué?*

- **Modelo SR** (*Strategic Rationale*): abre la «frontera»de cada actor y muestra sus metas, tareas, recursos y *softgoals* internos, y cómo las contribuciones de otros actores satisfacen (o inhiben) esas metas.

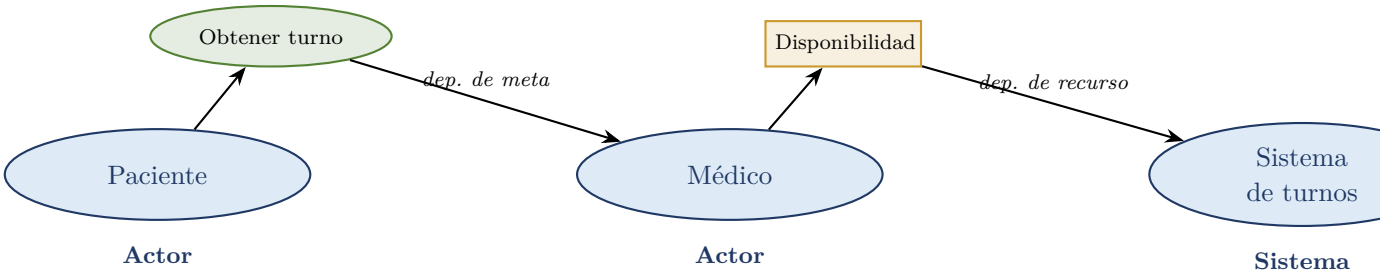


Figura 4.1. *Modelo SD simplificado en i* para el sistema de turnos del MSP. Las dependencias muestran lo que cada actor necesita de los demás. Adaptado de [1, 58].*

Notación básica de i*. Los cuatro tipos de dependencias del modelo SD son [58]:

- **Dependencia de meta** (círculo): el *depender* depende del *dependee* para que se alcance una meta.
- **Dependencia de tarea** (hexágono): el *depender* necesita que el *dependee* ejecute una tarea específica.
- **Dependencia de recurso** (rectángulo): se necesita una entidad física o de información.
- **Dependencia de softgoal** (nube): se necesita que se satisfaga un atributo de calidad (NFR).

El framework KAOS

Definición

KAOS (*Keep All Objectives Satisfied*). Framework de modelado orientado a metas propuesto por van Lamsweerde y la Université catholique de Louvain. Organiza los requisitos en una *jerarquía de metas* que se descompone hasta llegar a metas operacionalizables (requisitos concretos) o expectativas sobre el entorno (suposiciones) [1, 42].

La jerarquía KAOS utiliza cuatro operadores de descomposición [42]:

Tabla 4.2. *Operadores de descomposición de metas en KAOS [42].*

Operador	Significado	Ejemplo
AND-refinement	La meta padre se cumple solo si *todas* las sub-metas se cumplen.	«Gestionar turnos»= «Reservar turno»AND «Cancelar turno»AND «Reprogramar turno».
OR-refinement	La meta padre se cumple si *alguna* sub-meta se cumple (alternativas).	«Autenticar usuario»= «Contraseña»OR «Biometría»OR «Token OTP».
Operacionalización	La sub-meta ya es un requisito atómico asignable al sistema.	«El sistema deberá enviar confirmación por SMS al confirmar la reserva.»
Suposición (assumption)	Condición del entorno que el sistema asume verdadera.	«El paciente tiene número de celular activo en el registro.»

Ejemplo en Ecuador

Aplicación KAOS: sistema de turnos MSP de Quevedo. Meta de negocio de alto nivel: «Reducir la espera del paciente a menos de 30 minutos en el 90 % de los casos»(OR-refinement):

- Alternativa A: Gestión eficiente de turnos (AND): reserva en línea + recordatorio automático + gestión de no asistencia.
- Alternativa B: Redistribución de carga entre médicos (AND): monitoreo de tiempos + reasignación dinámica.

El equipo eligió la Alternativa A porque la Alternativa B requería cambios organizacionales fuera del alcance del proyecto de software. Esta decisión queda trazada en el grafo KAOS y justifica el alcance del ERS.

4.1.3 Subtema 6.3. Modelado de procesos de negocio con BPMN

BPMN como fuente de requisitos funcionales

Definición

BPMN 2.0. Notación estándar de la Object Management Group (OMG) para modelar procesos de negocio mediante diagramas de flujo enriquecidos con semántica de colaboración y coreografía. En IR, los diagramas BPMN sirven para capturar el proceso *AS-IS*, identificar los **eventos de negocio** que disparan acciones del sistema y derivar de ellos los requisitos funcionales correspondientes [1, 52, 59].

Robertson & Robertson [52] proponen el concepto de **evento de negocio** como punto de partida de la elicitación: cada cosa que ocurre en el mundo del cliente y que requiere una respuesta del sistema es un evento de negocio. Identificar estos eventos de forma exhaustiva garantiza que no se olvide ninguna funcionalidad principal.

Elementos BPMN esenciales para IR

Tabla 4.3. *Elementos BPMN 2.0 más relevantes para la elicitación de requisitos [59].*

Elemento	Descripción	Requisito derivado
Evento de inicio	Algo que dispara el proceso.	RF de inicio de flujo.
Tarea de usuario	Trabajo manual realizado por una persona.	RF de interfaz de usuario.
Tarea de servicio	Acción automática del sistema.	RF de procesamiento o integración.
Gateway exclusivo (XOR)	El flujo toma un camino u otro, no ambos.	Lógica condicional en RF; casos alternativos.
Gateway paralelo (AND)	Los flujos se ejecutan en paralelo.	RF de procesamiento concurrente.
Evento intermedio	Algo que ocurre en mitad del proceso.	RF de notificación, espera o excepción.
Lane (carril)	Responsabilidad de un actor o sistema.	Asignación de RF al actor correcto.

Técnica de derivación de requisitos desde BPMN. El procedimiento para convertir un diagrama BPMN en candidatos a requisitos funcionales es el siguiente [1, 52]:

1. Por cada **tarea de usuario**, se genera un RF de tipo «el sistema deberá permitir al actor X realizar Y».
2. Por cada **tarea de servicio**, se genera un RF de tipo «el sistema deberá ejecutar automáticamente Y».
3. Por cada **gateway XOR**, se generan RF para cada camino, más un RF de evaluación de la condición.
4. Por cada **evento intermedio de tipo mensaje o timer**, se genera un RF de notificación o temporización.
5. Por cada **lane**, se revisa si el actor tiene acceso adecuado: RF de seguridad / autorización.

Ejemplo en Ecuador

Proceso de facturación electrónica SRI. El proceso BPMN tiene: evento inicio (el contador emite una factura), tarea de usuario (ingresa datos del receptor), tarea de servicio (sistema genera XML firmado), tarea de servicio (sistema envía al SRI), gateway XOR (¿respuesta de autorización?), evento final (factura autorizada / factura rechazada). De este único proceso BPMN se derivan al menos 8 requisitos funcionales y 3 no funcionales (tiempo de respuesta del SRI, disponibilidad del certificado digital, reintentos ante fallo de red) [16].

4.1.4 Subtema 6.4. Historias de usuario y técnicas ágiles de elicitación

Historias de usuario

Definición

Historia de usuario. Formato breve de descripción de un requisito, propio del desarrollo ágil, con la estructura: *«Como [rol], quiero [acción/capacidad], para [beneficio/valor]»*. Complementada con **criterios de aceptación** que permiten saber cuándo la historia está *terminada* (done) [37, 38].

Cohn [37] introduce el acrónimo **INVEST** como criterio de calidad de una historia de usuario:

Tabla 4.4. *Criterios INVEST para evaluar la calidad de una historia de usuario [37].*

Criterio	Significado
Independent	La historia puede desarrollarse y entregarse de forma independiente.
Negotiable	Su contenido se negocia entre el equipo y el Product Owner hasta que se desarrolle.
Valuable	Aporta valor real al usuario o al negocio; si no aporta valor, no es una historia.
Estimable	El equipo puede estimar el esfuerzo necesario para implementarla.
Small	Cabe en un sprint (habitualmente menos de 8 puntos de historia).
Testable	Tiene criterios de aceptación que permiten verificar si está hecha correctamente.

Criterios de aceptación en formato Gherkin (ATDD)

El **ATDD** (Acceptance Test-Driven Development) propone escribir los criterios de aceptación de una historia antes de implementarla, en un lenguaje comprensible tanto para técnicos como para no técnicos. La notación **Gherkin** (usada por Cucumber y Behave) sigue la estructura **Given / When / Then** [60, 61]:

Estructura Gherkin (Given/When/Then):

Scenario: [nombre del escenario]
Given [contexto / precondición]
When [acción del usuario o del sistema]
Then [resultado esperado / poscondición]
And [condición adicional opcional]

Ejemplo en Ecuador

Historia de usuario y criterios de aceptación: matrícula UTEQ.
Historia: «Como estudiante de cuarto nivel de Ingeniería de Software, quiero matricularme en línea en mis asignaturas del semestre, para evitar desplazarme a las instalaciones de la universidad.»
Criterio de aceptación 1:
Given el estudiante ha iniciado sesión con su cédula y contraseña
When selecciona la asignatura «Ingeniería de Requisitos»
Then el sistema valida que cumple el prerrequisito (Fundamentos de IS)
And registra la matrícula y genera el comprobante en PDF.
Criterio de aceptación 2 (flujo alternativo):
Given el estudiante ha iniciado sesión
When intenta matricularse en «Diseño de Software»sin haber aprobado el prerrequisito
Then el sistema muestra el mensaje: «No cumple el prerrequisito: Fundamentos de IS (reprobada en período 2024-I)»
And no registra la matrícula.

Épicas, temas e historias: la jerarquía ágil

Tabla 4.5. *Jerarquía de artefactos de requisitos en metodologías ágiles. Adaptado de [37, 38].*

Nivel	Descripción	Ejemplo (sistema de matrículas)
Tema (Theme)	Área de capacidad de alto nivel.	Matrícula académica.
Épica (Epic)	Historia muy grande; debe desglosarse en sub-historias.	«Como estudiante, quiero gestionar mi matrícula completa.»
Historia de usuario	Unidad entregable en un sprint.	«Como estudiante, quiero matricularme en línea.»
Tarea (Task)	Unidad de trabajo técnico dentro de la historia.	Crear endpoint REST /api/matricula, diseñar UI del formulario.

Técnicas complementarias ágiles de elicitación

Más allá de las historias de usuario, el ecosistema ágil dispone de técnicas específicas de elicitación que complementan al Product Owner [37, 56, 60]:

- **Story mapping**: técnica visual de Jeff Patton que organiza las historias en un mapa bidimensional (flujo de usuario horizontal × nivel de detalle vertical), permitiendo identificar el Minimum Viable Product (Producto Mínimo Viable) (MVP) y planificar releases.
- **Event storming**: taller de exploración del dominio propuesto por Alberto Brandolini, basado en post-its de colores que representan eventos de dominio, comandos, agregados y políticas. Muy efectivo para descubrir el modelo de dominio y los eventos de negocio.
- **Impact mapping**: técnica de Gojko Adzic [60] que vincula las historias con los objetivos de negocio a través de actores e impactos, garantizando que solo se construye lo que tiene impacto medible.

- **Specification by Example (SbE)**: definir los criterios de aceptación como ejemplos concretos en lugar de reglas abstractas, usando tablas o escenarios Gherkin [60].

4.2 Mapa síntesis de la Semana 4

Tabla 4.6. *Mapa de cierre de la Semana 4.*

Subtema	Concepto clave	Aplicación en el proyecto
6.1 Prototipado	Throwaway vs. evolutivo; wireframe / mock-up.	Prototipo de baja fidelidad antes del taller.
6.2 i* y KAOS	Actores, dependencias, jerarquía de metas.	Modelo i* SD del contexto organizacional.
6.3 BPMN	Eventos de negocio → RF; lanes → autorización.	Diagrama AS-IS del proceso principal.
6.4 Historias	INVEST; Gherkin Given/When/Then; épicas.	Backlog inicial si el proyecto es ágil.

4.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 4 (en clase, 2 horas).

Objetivo: construir un modelo i* SD y un diagrama BPMN simple a partir de un caso dado.

Caso: un GAD municipal quiere automatizar el proceso de pago de impuesto predial, actualmente realizado en ventanilla con intervención manual de recaudadores, jefes de rentas y el sistema del Banco del Pacífico.

Tareas en equipos de 3–4 estudiantes:

1. Construir el modelo i* SD: identificar todos los actores (al menos 5), sus dependencias y sus tipos (meta, tarea, recurso, softgoal).
2. Construir el diagrama BPMN AS-IS del proceso actual en 3 lanes.
3. Derivar al menos 10 candidatos a RF y 3 a RNF a partir del diagrama BPMN.
4. Reescribir los 3 RF más importantes como historias de usuario con sus criterios de aceptación en formato Gherkin.

Práctica / Actividad

Tarea autónoma n.º 4 (4 horas, en equipo del proyecto integrador).

Entregable para el hito 1 (continuación):

1. **Modelo i* SD** del contexto organizacional del proyecto real.
2. **Diagrama BPMN AS-IS** del proceso principal que el sistema soportará (en la herramienta que el equipo elija: draw.io, Camunda, Lucidchart).
3. **Lista de eventos de negocio**: tabla con evento → RF derivado.
4. Si el proyecto es ágil: **story map** con épicas y las 5 primeras historias de usuario con criterios de aceptación Gherkin.

4.4 Resumen ejecutivo de la semana

- El **prototipado** convierte suposiciones implícitas en afirmaciones verificables; su tipo (desechable / evolutivo) debe decidirse antes de construirlo [10, 57].
- El framework **i*** modela las dependencias entre actores sociales y permite analizar el contexto organizacional antes de definir los requisitos del sistema [58].
- **KAOS** organiza los requisitos como una jerarquía de metas descompuestas con operadores AND/OR hasta llegar a requisitos operacionalizables [42].
- Los diagramas **BPMN 2.0** permiten derivar sistemáticamente RF a partir de eventos, tareas y gateways del proceso de negocio [52, 59].
- Las **historias de usuario** con criterios de aceptación en formato **Gherkin** son el puente entre la elicitación ágil y las pruebas de aceptación automatizadas [37, 60, 61].
- El criterio **INVEST** garantiza la calidad de cada historia de usuario antes de incorporarla al backlog [37].

Semana 5

Análisis de requisitos: conflictos, priorización y calidad

El análisis de requisitos es el momento en que el analista deja de tomar nota y empieza a pensar: ¿Qué tiene sentido? ¿Qué falta? ¿Qué se contradice? Sin este paso, el ERS es un inventario de deseos; con él, es la base contractual del sistema.
adaptado de Robertson y Robertson [52]

Objetivos de aprendizaje de la semana

Al finalizar esta quinta semana, el estudiante estará en capacidad de:

1. Analizar y clasificar un conjunto de requisitos elicitados aplicando dimensiones de **tipo, nivel y perspectiva** [1-3].
2. Detectar las cinco categorías de defectos de requisitos más frecuentes y proponer correcciones [10, 62, 63].
3. Identificar y resolver **conflictos entre requisitos** mediante las técnicas de compromiso, priorización y eliminación [1, 52].
4. Aplicar las técnicas de priorización **MoSCoW**, **modelo Kano** y **priorización por valor/costo/riesgo** [10, 37, 52].
5. Evaluar la calidad de un ERS mediante las características de calidad de **IEEE 29148:2018** aplicadas tanto a requisitos individuales como al conjunto [2].
6. Construir una **Matriz de Trazabilidad de Requisitos (RTM)** y mantener la trazabilidad bidireccional [1, 2, 10].

5.1 Tema 7. Análisis de requisitos

5.1.1 Subtema 7.1. Análisis y clasificación de requisitos

¿Qué es el análisis de requisitos?

Definición

Análisis de requisitos. Conjunto de actividades que, a partir de la lista bruta de candidatos a requisitos obtenida en la elicitación, producen un conjunto estructurado, consistente y clasificado de requisitos que pueden servir de base para la especificación. Incluye la detección de defectos, la resolución de conflictos, la clasificación, la priorización y la verificación de completitud [1-3].

La diferencia entre un *candidato a requisito* (lo que el stakeholder dijo) y un *requisito analizado* (lo que el analista valida, clasifica y articula correctamente) es el resultado del análisis.

Esta distinción es la que convierte la elicitación en ingeniería [1].

Dimensiones de clasificación de requisitos

Pohl [1] y SWEBOK v4 [3] proponen clasificar cada requisito a lo largo de **tres dimensiones ortogonales**:

1. **Por tipo**: funcional, no funcional / de calidad, restricción.
2. **Por nivel**: negocio, stakeholder, sistema, software (la pirámide de la Semana 1).
3. **Por perspectiva**: datos (qué entidades maneja), función (qué procesa), comportamiento (cómo reacciona en el tiempo).

La tabla 5.1 muestra cómo aplicar estas dimensiones a un conjunto de requisitos del sistema de turnos del MSP.

Tabla 5.1. *Ejemplo de clasificación multidimensional de requisitos del sistema de turnos MSP [1, 2].*

Enunciado	Tipo	Nivel	Perspectiv
El sistema deberá permitir al paciente reservar un turno.	Funcional	Software	Función
El sistema deberá responder en menos de 2 s al 95 %.	Calidad	Software	Función
El sistema deberá almacenar datos de paciente con nombre, cédula, fecha de nacimiento.	Funcional	Software	Datos
El sistema deberá bloquear la reserva tras 3 intentos fallidos.	Calidad	Software	Comportamiento
El sistema deberá ejecutarse en servidores del datacenter del MSP.	Restricción	Sistema	—

Defectos frecuentes de los requisitos

Wiegers & Beatty [10], Berry [62] y Femmer et al. [63] identifican categorías de defectos que el análisis debe detectar y corregir antes de redactar el ERS:

Tabla 5.2. *Las cinco categorías de defectos más frecuentes en los requisitos, con ejemplos y correcciones [10, 62, 63].*

Defecto	Ejemplo defectuoso	Corrección
Ambigüedad	«El sistema debe responder rápidamente.»	«El sistema deberá responder en menos de 2 s para el 95 % de las solicitudes con 500 usuarios concurrentes.»
Incompletitud	«El sistema deberá calcular el impuesto.»(¿qué impuesto? ¿qué tasa?)	Especificar el tipo de impuesto, la base de cálculo y la fuente normativa (SRI).
Inconsistencia	RF-12: «El sistema solo permitirá a administradores cancelar turnos»+ RF-34: «Cualquier usuario puede cancelar su turno.»	Resolver el conflicto: ¿quién puede cancelar qué turnos?
No verificable	«El sistema deberá ser fácil de usar.»	«Un usuario sin entrenamiento deberá completar la reserva en menos de 5 minutos en su primer intento.»
No factible	«El sistema deberá procesar 1 millón de transacciones por segundo.»	Validar con arquitectura la capacidad real; ajustar o eliminar.

5.1.2 Subtema 7.2. Detección y resolución de conflictos entre requisitos

Tipos de conflictos

Definición

Conflicto de requisitos. Situación en la que dos o más requisitos son inconsistentes entre sí: no pueden satisfacerse simultáneamente con el mismo diseño o dentro de las restricciones del proyecto [1, 51].

Pohl [1] distingue tres tipos de conflicto:

- **Conflicto de contenido**: un requisito afirma algo que otro niega. Ejemplo: un RF exige cifrar todas las comunicaciones con AES-256; una restricción de presupuesto prohíbe la infraestructura necesaria para esa velocidad de cifrado.
- **Conflicto de prioridad**: dos stakeholders declaran requisitos incompatibles en cuanto al orden de importancia. Ejemplo: el área de Tesorería quiere que el cierre mensual sea el módulo más robusto; el área de Atención al Cliente quiere que el portal web tenga prioridad.
- **Conflicto de proceso**: dos grupos proponen metodologías de desarrollo opuestas que afectan indirectamente a los requisitos.

Estrategias de resolución

La negociación de conflictos es la actividad *core* más exigente social y emocionalmente de la IR [1, 52]. Las estrategias disponibles, de menor a mayor intensidad de intervención, son:

1. **Aclaración**: muchos conflictos aparentes se deben a ambigüedad lingüística. Al precisar el enunciado, el conflicto desaparece.
2. **Priorización**: si dos requisitos conflictivos no pueden coexistir, se elige el de mayor valor para el negocio y se descarta o pospone el otro.
3. **Compromiso (trade-off)**: se busca un punto intermedio que satisfaga parcialmente a ambas partes. Ejemplo: en lugar de AES-256 (caro) o sin cifrado (inaceptable), se adopta AES-128 (suficiente para el nivel de riesgo del proyecto).
4. **Escalada**: cuando los stakeholders no pueden llegar a un acuerdo, el conflicto escala a la autoridad superior (patrocinador, director de proyecto, CCB). El analista documenta las posturas y presenta las implicaciones de cada opción.
5. **Win-win de Boehm**: propuesto por Barry Boehm, es una negociación principalista en la que se identifican los *intereses* reales de cada parte (no sus posiciones), se buscan opciones que satisfagan los intereses de todos, y se elige la que maximiza el valor total [1].

Ejemplo en Ecuador

Conflicto real en un proyecto de hospital privado en Guayaquil. El gerente médico requería que todas las historias clínicas fueran visibles para cualquier médico de guardia (disponibilidad máxima). El departamento jurídico requería que solo el médico tratante pudiera acceder al historial completo (privacidad máxima, LOPDP). El conflicto era de contenido, no de prioridad. La solución win-win: distinción de dos niveles de historia clínica — *resumen de urgencias* (accesible para médicos de guardia) e *historial completo* (solo para el médico tratante). Ambas partes aceptaron porque ninguna cedió en su interés esencial [14].

5.1.3 Subtema 7.3. Técnicas de priorización de requisitos

¿Por qué priorizar?

La priorización reconoce una verdad ineludible de todos los proyectos reales: **nunca hay tiempo ni presupuesto para implementar todos los requisitos en el primer release** [10, 37]. La priorización decide qué se hace primero. Una mala priorización lleva a entregar funcionalidades poco importantes antes que las esenciales, lo que desperdicia recursos y decepciona a los stakeholders.

Técnica MoSCoW

Propuesta por Dai Clegg para el método DSDM, la técnica MoSCoW clasifica cada requisito en cuatro categorías [10, 56]:

Tabla 5.3. *Categorías MoSCoW y su interpretación [10].*

Categoría	Interpretación	Límite recomendado
Must-have	Sin este requisito el sistema no puede entregarse. Crítico para el negocio o la ley.	Máximo 60 % del backlog.
Should-have	Importante, pero hay alternativas aceptables si no se incluye en esta versión.	20–30 %.
Could-have	Deseable; si hay capacidad restante se incluye; si no, no pasa nada.	10–20 %.
Won't-have	Explícitamente excluido de *esta versión* (no de forma permanente).	Documentar.

⚠ Advertencia / Error frecuente

Antipatrón frecuente. Cuando se le pide a un cliente que clasifique sus requisitos con MoSCoW, tiende a poner el 90 % en Must-have. La técnica funciona solo cuando el facilitador aplica restricciones de tiempo y capacidad: «Tienen un presupuesto equivalente a 6 meses de trabajo. Solo el 60 % del tiempo es para Must-haves. ¿Cuáles eligen?»[10, 52].

Modelo Kano de satisfacción del cliente

El modelo Kano, propuesto por Noriaki Kano en 1984 y ampliamente adoptado en IR [1, 10, 64], clasifica los requisitos según el *tipo de satisfacción* que producen en el cliente, no por su importancia declarada:

Tabla 5.4. *Categorías del Modelo Kano y su impacto en la satisfacción del cliente. Adaptado de [10, 64].*

Categoría Kano	Efecto en el cliente	Ejemplo
Básico (*must-be*)	Si está, no sorprende; si falta, causa rechazo inmediato.	El sistema no pierde datos del paciente.
Desempeño (*performance*)	Cuanto más, mayor satisfacción de forma lineal.	Tiempo de respuesta: cuanto más rápido, mejor.
Atractivo (*delighter*)	No se espera, pero deleita y diferencia.	El sistema notifica al paciente 30 min. antes del turno.
Indiferente	No afecta la satisfacción en ninguna dirección.	El color del botón de confirmación.
Contradictorio	Su presencia causa insatisfacción.	Publicar el número de cédula del paciente en la confirmación.

La figura 5.1 ilustra el modelo de curvas de Kano.

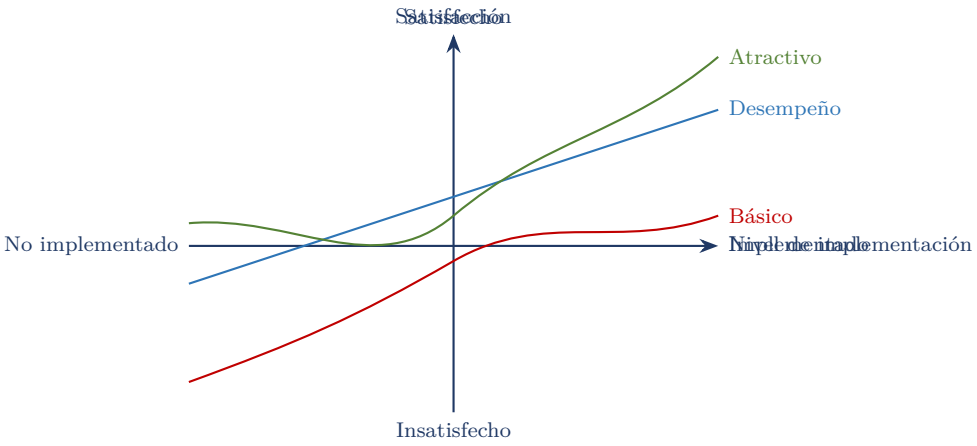


Figura 5.1. *Curvas del Modelo Kano. Los requisitos básicos no elevan la satisfacción aunque se implementen bien; los atractivos generan un salto de satisfacción no anticipado. Adaptado de [10, 64].*

Priorización por valor, costo y riesgo

Una técnica cuantitativa propuesta por Wiegers & Beatty [10] asigna a cada requisito tres puntuaciones en una escala de 1 a 9 y calcula una prioridad ponderada:

$$\text{Prioridad} = \frac{w_V \cdot V + w_P \cdot P}{w_C \cdot C + w_R \cdot R} \tag{5.1}$$

Donde:

- V = beneficio si el requisito se incluye (1–9).
- P = penalización si se excluye (1–9).
- C = costo relativo de implementar el requisito (1–9).
- R = riesgo técnico (1–9).
- w_V, w_P, w_C, w_R = ponderaciones definidas por el equipo.

La tabla 5.5 muestra un ejemplo aplicado.

Tabla 5.5. *Ejemplo de priorización por valor/costo/riesgo para el sistema de matrícula UTEQ. Ponderaciones: $w_V = 1, w_P = 1, w_C = 1, w_R = 0{,}5$.*

Requisito	V	P	C	R	Prioridad
RF-01 Matricular en línea	9	9	4	2	3,60
RF-02 Verificar prerrequisitos	9	8	3	1	4,43
RF-03 Generar comprobante PDF	7	6	2	1	4,60
RF-04 Portal de estadísticas admin.	4	2	5	4	0,86
RF-05 Integración con pasarela de pago	8	7	7	6	1,50

5.1.4 Subtema 7.4. Calidad de los requisitos y trazabilidad

Calidad individual vs. calidad del conjunto

La norma *IEEE/ISO/IEC 29148:2018* [2] distingue dos niveles de calidad:

- **Calidad individual**: las nueve características que debe cumplir cada requisito por separado (revisadas en la Semana 1: necesario, apropiado al nivel, no ambiguo, completo, singular, factible, verificable, correcto, conforme).
- **Calidad del conjunto**: seis características que el conjunto de requisitos debe cumplir como un todo: completo, consistente, asequible, delimitado, no redundante, comprensible.

Tabla 5.6. *Características de calidad del conjunto de requisitos según [2].*

Característica	Significado	Síntoma de incumplimiento
Completo	No falta ningún requisito necesario.	Los usuarios encuentran funcionalidades indispensables que no están en el ERS.
Consistente	No hay requisitos que se contradigan entre sí.	Los desarrolladores reciben instrucciones incompatibles al implementar.
Asequible	Todos los requisitos pueden implementarse dentro de las restricciones de costo, tiempo y tecnología.	El proyecto se desvía en cronograma o presupuesto.
Delimitado	El alcance está claramente definido; nada fuera de alcance se especifica.	Se implementan funciones no solicitadas (*gold-plating*).
No redundante	Ningún requisito repite a otro (aunque pueden estar relacionados).	Los cambios deben hacerse en múltiples lugares.
Comprensible	El ERS puede ser leído y entendido por todos sus destinatarios.	Los desarrolladores hacen suposiciones sin consultar al analista.

Técnicas de revisión de requisitos

Definición

Revisión de requisitos. Examen sistemático de los artefactos de requisitos por personas distintas al autor, con el objetivo de detectar defectos de forma temprana y económica [1, 2, 10].

Las tres modalidades principales de revisión, ordenadas de menor a mayor formalidad, son [1, 10]:

- **Walkthrough**: el autor guía a un grupo de revisores a través del documento. Informal, sin roles definidos, sin métricas. Efectivo para documentos cortos y equipos pequeños.
- **Revisión estructurada por pares**: cada revisor lee el documento de forma independiente con una lista de verificación, y el grupo discute los hallazgos sin que el autor defienda el documento.
- **Inspección Fagan**: proceso formal con seis roles (moderador, autor, lector, inspectores), cuatro fases (planning, overview, inspection meeting, rework + follow-up) y métricas de

densidad de defectos por página. Requiere capacitación previa del equipo pero es la técnica más efectiva por unidad de costo [65].

Trazabilidad de requisitos

Definición

Trazabilidad de requisitos. Capacidad de seguir la vida de un requisito en **ambas direcciones**: hacia arriba, vinculándolo con la necesidad del stakeholder o el objetivo de negocio que lo origina; hacia abajo, vinculándolo con los elementos de diseño, código, pruebas y manuales que lo satisfacen [1, 2, 66].

La norma *ISO/IEC/IEEE 15288:2023* eleva la trazabilidad a **proceso obligatorio** del ciclo de vida de sistemas [66]. Sin trazabilidad, es imposible:

- Evaluar el impacto de un cambio de requisito en el diseño y las pruebas existentes.
- Auditar si todos los requisitos tienen cobertura de prueba.
- Rastrear la causa de un defecto hasta el requisito que lo originó.
- Demostrar conformidad regulatoria (p.ej., dispositivos médicos o sistemas de banca).

La Matriz de Trazabilidad de Requisitos (RTM)

La herramienta práctica de trazabilidad más usada en proyectos medianos es la **Matriz de Trazabilidad de Requisitos** [2, 10]. La tabla 5.7 muestra su estructura básica aplicada al proyecto de matrícula UTEQ.

Tabla 5.7. *Ejemplo de RTM para el sistema de matrícula UTEQ.*

ID Req.	Enunciado resumido	Origen (StRS)	Diseño	Caso de prueba	Estado
RF-01	Matricular en línea.	StRS-12	UC-03	CP-01, CP-02	Verificado
RF-02	Verificar prerrequisitos.	StRS-14	UC-03.2	CP-03	En revisión
RF-03	Generar comprobante PDF.	StRS-15	UC-04	CP-04	Pendiente
RNF-01	Respuesta < 3 s al P95.	StRS-20	Arch-NFR-01	PR-01	Pendiente
RNF-02	Cifrado de contraseñas con bcrypt.	StRS-22	Arch-SEC-02	SP-01	Pendiente

Trazabilidad bidireccional. La RTM permite navegar en dos direcciones: dado un requisito, encontrar todos los artefactos que lo satisfacen (*forward tracing*); dado un artefacto de diseño o prueba, encontrar el requisito que lo originó (*backward tracing*). Las herramientas como Jira, ReqIF, DOORS o incluso hojas de cálculo bien mantenidas pueden implementar ambas direcciones [2, 10].

5.2 Mapa síntesis de la Semana 5

Tabla 5.8. *Mapa de cierre de la Semana 5.*

Subtema	Concepto clave	Aplicación en el proyecto
7.1 Clasificación	3 dimensiones; 5 defectos frecuentes.	Revisión de los candidatos a requisitos del hito 1.
7.2 Conflictos	Win-win; 4 estrategias; CCB.	Acta de resolución de conflictos del taller.
7.3 Priorización	MoSCoW; Kano; Valor/Costo/-Riesgo.	Backlog priorizado del proyecto integrador.
7.4 Calidad y RTM	9 caract. individuales + 6 del conjunto; RTM.	RTM inicial del ERS en construcción.

5.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 5 (en clase, 2 horas).

Objetivo: aplicar MoSCoW, detectar defectos y construir una RTM mínima.

Caso: el docente entrega un listado de 20 candidatos a requisitos *defectuosos* (con los cinco tipos de defectos mezclados) para un sistema de reservas de la biblioteca UTEQ.

Tareas en equipos de 3–4 estudiantes:

1. Clasificar cada candidato en las 3 dimensiones (tipo, nivel, perspectiva).
2. Identificar el defecto de cada candidato y redactar la versión corregida.
3. Aplicar MoSCoW a los 20 requisitos corregidos (respetar los límites del 60 % para Must-have).
4. Construir la RTM con los 5 requisitos más prioritarios, indicando al menos: ID, origen, artefacto de diseño propuesto, caso de prueba.
5. Presentación ante la clase: 5 minutos por equipo.

Práctica / Actividad

Tarea autónoma n.º 5 (5 horas, en equipo del proyecto integrador).

Hito 1 del proyecto integrador – entrega completa:

1. **Documento de visión** (plantilla Wiegers, 1–2 páginas).
2. **Actas de elicitación** (mínimo 2 sesiones documentadas: entrevista + observación o taller).
3. **Lista de candidatos a requisitos clasificada**: mínimo 30 candidatos, clasificados en las 3 dimensiones y con defectos corregidos.
4. **Resolución de conflictos**: al menos 2 conflictos identificados y resueltos, con su acta de decisión.
5. **Lista priorizada MoSCoW**: con justificación de las categorías asignadas.
6. **RTM inicial**: para los 15 requisitos más prioritarios.

Entregable: informe PDF de máximo 15 páginas + anexos. Se entrega en el repositorio del equipo y se defiende oralmente en clase la semana siguiente.

Práctica / Actividad

Lectura previa para la Semana 6 (aula invertida – inicio Unidad 3).

- IEEE/ISO/IEC 29148:2018 [2], cláusulas 7 a 9 (estructura del SRS / ERS).
- Wiegers & Beatty [10], capítulo 10 (*Documentation*).
- Pohl [1], Parte III (*Core Activities: Documentation*).
- IREB CPRE v3.2.0 [56], Unidad educativa 3 (*Documentation*).

5.4 Resumen ejecutivo de la semana

- El **análisis** transforma candidatos a requisitos en un conjunto estructurado, clasificado, consistente y verificable [1, 3].
- Las tres dimensiones de clasificación son **tipo** (funcional, calidad, restricción), **nivel** (negocio, stakeholder, sistema, software) y **perspectiva** (datos, función, comportamiento) [1, 2].
- Los cinco defectos más frecuentes son **ambigüedad, incompletitud, inconsistencia, no verificable y no factible** [10, 62, 63].
- Los conflictos se resuelven con **aclaración, priorización, compromiso, escalada o win-win**; el CCB es la autoridad de última instancia [1].
- Las técnicas de priorización más usadas son **MoSCoW**, **Modelo Kano** y **Valor/Costo/-Riesgo** [10, 37, 64].
- La calidad del ERS se evalúa a dos niveles: **9 características individuales y 6 del conjunto**, según IEEE 29148:2018 [2].
- La **RTM** implementa la trazabilidad bidireccional requerida por IEEE 29148:2018 e ISO 15288:2023 [2, 66].

Semana 6

Metas, escenarios y requisitos orientados a solución

Las metas articulan el por qué *del sistema; los escenarios articulan el* cómo *desde la perspectiva del usuario; los modelos UML articulan el* qué *desde la perspectiva del sistema. Un ERS de calidad necesita los tres niveles actuando en coherencia.*
adaptado de Lamsweerde [67]

Objetivos de aprendizaje de la semana

Al finalizar esta sexta semana, el estudiante estará en capacidad de:

1. Explicar el concepto de **meta de sistema** y aplicar las siete reglas de Pohl para su documentación textual [1-3].
2. Construir **grafos KAOS** con AND/OR-refinement y derivar requisitos operacionalizados [1, 42].
3. Construir modelos **iStar 2.0** (SD y SR) para analizar el contexto organizacional [1, 58, 68].
4. Redactar escenarios de los cuatro tipos con las **once reglas de calidad** [1, 52, 69].
5. Elaborar **diagramas de CU UML** y **MSC** vinculados a los escenarios [1, 69, 70].
6. Construir los tres **modelos orientados a solución**: clases, DFD/actividades y statecharts [1, 71, 72].
7. Verificar la coherencia entre metas, escenarios y modelos [1, 2, 66].

6.1 Tema 7. Metas, escenarios y requisitos orientados a solución

6.1.1 Subtema 7.1. Metas: concepto, descomposición y documentación

¿Qué es una meta en IR?

Definición

Meta de sistema. Objetivo que el sistema debe contribuir a alcanzar, expresado en términos del dominio del problema y sin presuponer una solución tecnológica concreta. Las metas se sitúan entre las necesidades de negocio (nivel estratégico) y los requisitos del sistema (nivel operacional), y son el mecanismo que hace explícita la *intención* detrás de cada requisito [1, 2, 42].

Van Lamsweerde [42] y Pohl [1] coinciden en que la mayor parte de los defectos en los requisitos funcionales se deben a que el analista no ha comprendido ni documentado las metas de los stakeholders. Sin metas explícitas, los requisitos son huérfanos: no se puede verificar si

son necesarios, si están completos o si entran en conflicto con otros.

Taxonomía de metas

Tabla 6.1. *Taxonomía de metas según [1, 2, 42].*

Categoría	Descripción	Ejemplo (sistema turnos MSP)
Meta de negocio	Objetivo estratégico de la organización; el sistema es un medio.	Reducir espera media del paciente a menos de 30 min.
Meta de sistema	Lo que el sistema debe lograr dentro del alcance acordado.	El sistema deberá asignar turnos en menos de 60 segundos.
Meta de calidad (*softgoal*)	NFR sin criterio preciso; se satisface en grado.	El sistema deberá ser fácil de usar para ciudadanos sin formación informática.
Suposición	Condición del entorno que el sistema asume verdadera pero no controla.	El paciente tiene número de celular activo y acceso a internet.

Las siete reglas de Pohl para documentar metas

Tabla 6.2. *Las siete reglas de Pohl para la documentación textual de metas [1]. Se incluye contraejemplo y corrección para cada regla.*

#	Regla	Violación	Corrección
1	Nombrar con sustantivo verbal.	«Turnos»	«Asignar turno al paciente»
2	Incluir el actor responsable.	«Asignar turno»	«El sistema asigna turno al paciente.»
3	Nivel de abstracción correcto.	«El sistema mostrará el botón Confirmar en pantalla 3.»	«El sistema confirmará la reserva.»
4	Verificable o cuantificable.	«Mejorar la UX.»	«Reducir pasos de reserva de 12 a menos de 4.»
5	Atómica: una sola intención.	«El sistema asignará y cancelará turnos.»	Dos metas separadas.
6	Consistente con las demás metas.	Meta A: solo médicos cancelan. Meta B: pacientes cancelan.	Resolver conflicto antes de documentar.
7	Toda meta hoja tiene requisito derivado.	Meta hoja sin RF.	Operacionalizar en al menos un RF o RNF.

Descomposición AND/OR: el grafo de metas KAOS

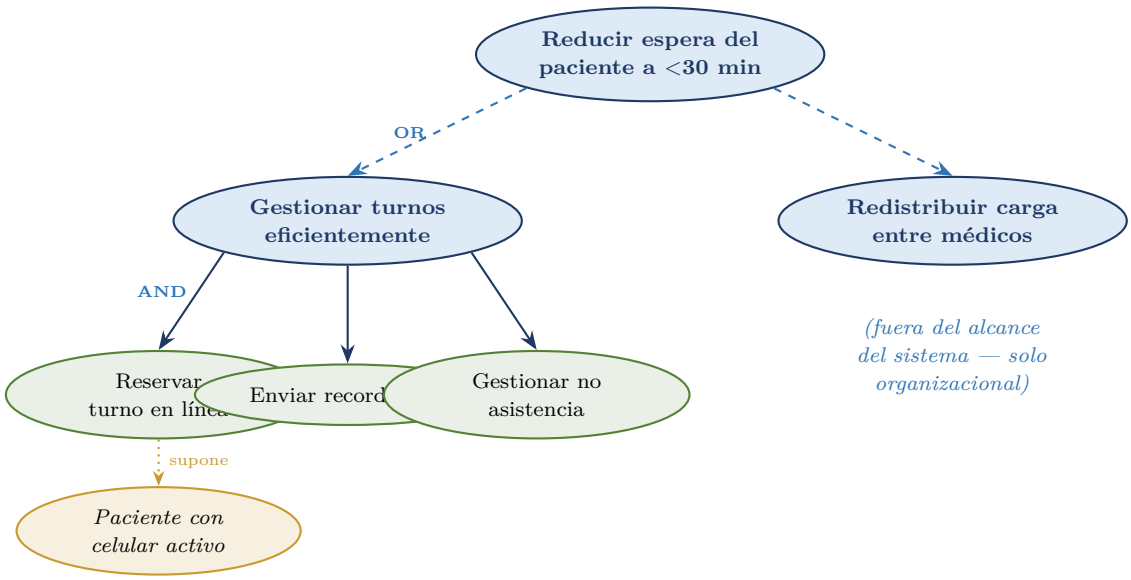


Figura 6.1. *Grafo KAOS del sistema de turnos MSP. OR-refinement en nivel 1 (alternativas de diseño); AND-refinement en nivel 2 (todas las sub-metas son necesarias). Las meta hojas verdes operacionalizan en RF; el óvalo dorado es una suposición sobre el entorno. Adaptado de [1, 42].*

iStar 2.0: el modelo SR de rationale estratégico

La especificación iStar 2.0 [68] actualiza el framework original de Yu [58] con precisiones sintácticas y semánticas. El **modelo SR** (*Strategic Rationale*) abre la frontera interna de cada actor y muestra sus metas, tareas, recursos y calidades internas, más las contribuciones entre ellos.

Tabla 6.3. *Elementos del modelo SR en iStar 2.0 [58, 68].*

Elemento	Figura	Significado	Ejemplo en MSP
Meta (*goal*)	Círculo	Estado que el actor quiere alcanzar.	Obtener atención oportuna.
Tarea (*task*)	Hexágono	Forma concreta de hacer algo.	Reservar turno en línea.
Recurso (*resource*)	Rectángulo	Entidad necesaria para una tarea.	Cédula de ciudadanía.
Calidad (*quality*)	Nube	NFR / meta blanda.	Con mínimo esfuerzo.
Contribución	Arco "+" o "-"	Grado de satisfacción de un elemento a otro.	La tarea contribuye + a la calidad.

Ejemplo en Ecuador

Modelo SR del Paciente — derivación de RNF.
El actor *Paciente* tiene en su modelo SR: meta «Obtener atención oportuna»→ descompuesta (medio-fin) en tarea «Reservar turno en línea». La meta blanda «Con mínimo esfuerzo»contribuye «+»a la tarea. De esa meta blanda se deriva directamente:

> **RNF-UX-01**: El proceso de reserva de turno no deberá requerir más de cuatro pasos a partir de la pantalla principal. El 90 % de los usuarios sin experiencia previa deberá completarlo en menos de 3 minutos en su primer intento.

Este RNF es verificable (4 pasos, 3 minutos, 90 %) y trazable directamente a la meta blanda del modelo SR [1, 68].

6.1.2 Subtema 7.2. Escenarios: tipos, documentación y once reglas

¿Por qué usar escenarios?

Definición

Escenario. Descripción narrativa o semiestructurada de una secuencia de interacciones entre actores y el sistema, situada en un contexto concreto, que ilustra la forma en que el sistema satisface (o falla en satisfacer) una meta [1, 52, 69].

Los escenarios son comprensibles por los stakeholders sin formación técnica y suficientemente precisos para los desarrolladores. Son, de hecho, la *única* clase de artefacto de requisitos que un cliente puede leer, validar y firmar sin necesitar instrucción previa en notaciones formales [1, 10].

Los cuatro tipos de escenario

Tabla 6.4. *Tipos de escenario y sus características [1, 37, 52, 69, 73].*

Tipo	Descripción	Propósito principal	Cuándo usar
Caso de uso *fully dressed* [69]	Plantilla estructurada con actor, precondiciones, flujos básico/alternativo/excepción, poscondición.	Capturar RF verificables.	Siempre que haya flujos significativos.
Historia de usuario [37]	Como X, quiero Y, para Z + Gherkin.	Comunicación ágil equipo-PO.	Proyectos Scrum/-Kanban.
Misuse case [73]	Flujo malicioso de un *misactor*.	Derivar RF de seguridad.	Todo sistema con datos sensibles.
Escenario narrativo	Texto libre de contexto y flujo.	Validar la visión.	Etapas tempranas, pre-CU.

Caso de uso CU-01: plantilla Cockburn *fully dressed*

Tabla 6.5. *CU-01: Reservar turno médico — plantilla Cockburn fully dressed [1, 69].*

CU-01: Reservar turno médico	
ID / Nombre	CU-01 / Reservar turno médico
Alcance	Sistema de Gestión de Turnos del MSP — Módulo Ciudadano
Nivel	Objetivo de usuario (*sea level*)
Actor primario	Paciente (ciudadano ecuatoriano con cédula vigente)
Actores secundarios	Servidor SMS; Registro Civil (validación cédula)
Precondiciones	(P1) El paciente dispone de número de cédula válido. (P2) Existe al menos un médico con disponibilidad en el período. (P3) El paciente tiene acceso a internet.
Garantía de éxito	El turno queda registrado y el paciente recibe confirmación por SMS y correo.
Flujo básico	1. El paciente accede al portal web e ingresa su número de cédula. 2. El sistema valida la cédula con el MSP/Registro Civil. 3. El sistema muestra especialidades disponibles con tiempo de espera estimado. 4. El paciente selecciona especialidad, fecha y hora preferida. 5. El sistema verifica disponibilidad del horario seleccionado. 6. El sistema registra el turno y genera el código único de confirmación. 7. El sistema envía confirmación por SMS al número registrado. 8. CU termina: turno confirmado, código entregado.
Flujo alt. A	**2a — Cédula inválida:** El sistema muestra mensaje de error. El paciente reintenta (máx. 3 intentos); al 4.º, se bloquea la sesión por 10 min.
Flujo alt. B	**5b — Horario ocupado:** El sistema ofrece los 3 horarios más cercanos disponibles. El paciente elige uno o cancela.
Flujo excepción C	**7c — Falla SMS:** El sistema reintenta a los 5 y 10 minutos. Si fallan los 3 intentos, notifica por correo y registra el incidente.
Requisitos derivados	RF-01 (reservar), RF-02 (validar cédula), RF-07 (enviar SMS), RNF-03 (respuesta < 2 s), RNF-05 (disponibilidad $\geq 99{,}5\ \%$).
Frecuencia	800–2 000 reservas/día normal; hasta 5 000 en campañas de vacunación.

Las once reglas de calidad de los escenarios

Pohl [1] y Robertson & Robertson [52] establecen once reglas que todo escenario bien formado debe satisfacer. Las siguientes están ordenadas de más a menos frecuentemente violadas en

proyectos universitarios:

1. **Flujo básico sin condiciones internas.** El flujo básico solo describe el camino de éxito. No contiene «si», «puede»ni «en caso de». Toda bifurcación va en un flujo alternativo o de excepción separado.
2. **Un actor primario por escenario.** Cada CU tiene exactamente un actor que inicia la interacción. Si hay dos actores de inicio, son dos CU.
3. **La garantía de éxito es observable y verificable.** Debe poderse comprobar en una prueba de aceptación: «El turno queda registrado en la BD», no «El usuario queda satisfecho».
4. **Las precondiciones son verificables.** No se asumen; se especifican explícitamente.
5. **Los flujos alternativos referencian el paso exacto donde divergen.** El nombre del flujo indica: «Paso 2a», no «cuando la cédula es inválida»sin más contexto.
6. **Nivel de abstracción consistente.** No mezclar pasos de negocio («el paciente confirma la reserva») con detalles de implementación («el sistema actualiza el campo `estado` en la tabla `Turno`»).
7. **Atómico: un objetivo de usuario por escenario.** Si CU-01 incluye reservar Y cancelar, hay que dividirlo.
8. **Los flujos de excepción son técnicos, no de negocio.** Los fallos de red, base de datos y servicios externos van en flujos de excepción; las reglas de negocio van en flujos alternativos.
9. **Al menos un RF derivado explícitamente.** Cada flujo básico genera al menos un requisito trazable.
10. **Los actores secundarios están identificados.** Todo sistema externo al que el sistema llama se nombra explícitamente.
11. **Revisado y firmado por el stakeholder propietario.** El actor primario o su representante ha confirmado que el flujo básico es correcto antes de incluirse en el ERS.

Misuse case: derivar requisitos de seguridad

Definición

Misuse case. Extensión del caso de uso propuesta por Sindre y Opdahl [73] que describe cómo un *misactor* (atacante, usuario malintencionado o proceso erróneo) daña el sistema. De cada misuse case se derivan como *contramedidas* los requisitos de seguridad del sistema.

Tabla 6.6. *MC-01: Suplantación de identidad para reservar turno masivamente. Adaptado de [1, 14, 73].*

MC-01: Suplantación de identidad para reserva masiva	
Misactor	Atacante externo con cédulas de terceros obtenidas por medios ilícitos
Meta del misactor	Bloquear la agenda médica mediante reservas masivas automatizadas
Flujo del misuse	1. El atacante obtiene un listado de cédulas válidas (p.ej., filtración de datos). 2. Desarrolla un *bot* que llama al API de reserva de forma automatizada. 3. Realiza 10 000 reservas en menos de una hora en nombre de distintos ciudadanos. 4. La agenda queda bloqueada; los ciudadanos legítimos no pueden obtener turno.
Daño potencial	Denegación de servicio; vulneración de la LOPDP; pérdida de confianza ciudadana [14]
Contramedidas (RF y RNF derivados)	▸ **RNF-SEC-01**: Implementar *rate limiting*: máx. 10 reservas/hora por IP. ▸ **RNF-SEC-02**: Requerir CAPTCHA tras 3 intentos de reserva consecutivos. ▸ **RF-SEC-03**: Limitar a 1 turno activo por cédula y especialidad. ▸ **RF-SEC-04**: Verificar OTP por SMS antes de confirmar la reserva. ▸ **RNF-SEC-05**: Registrar y alertar al administrador cuando una IP realice más de 10 reservas en 5 minutos.
Casos de uso que mitigan	CU-01 (flujo excepción D: sesión bloqueada), CU-09 (gestión de alertas)

6.1.3 Subtema 7.3. Diagramas de CU UML y Message Sequence Charts

Diagrama de casos de uso UML

El diagrama de casos de uso UML muestra los actores externos, los casos de uso del sistema y las relaciones entre ellos. Su función en la IR no es describir los flujos internos (eso lo hace la plantilla Cockburn) sino proporcionar una **vista de alcance** comprensible por todos los stakeholders [1, 69, 70].

Tabla 6.7. *Relaciones en los diagramas de CU UML [1, 69, 70].*

Relación	Semántica	Ejemplo en MSP
`«include»`	El CU base *siempre* invoca al CU incluido, en todos sus flujos.	CU-01 `«include»` CU-08 (Validar cédula): ocurre siempre.
`«extend»`	El CU extensor puede ocurrir opcionalmente bajo cierta condición (*extension point*).	CU-10 (Recordatorio) `«extend»` CU-01: solo si el turno es $>$ 24h.
Generalización (herencia)	Un actor hijo hereda todos los CU del actor padre.	*Médico especialista* hereda de *Médico*.

Advertencia / Error frecuente

Error frecuente. Usar `«include»` y `«extend»` como si fueran equiva
es crítica: `«include»` es obligatorio y siempre se ejecuta; `«extend»` es
condición. Usar el incorrecto genera requisitos erróneos en el ERS [69

Las tres relaciones fundamentales.

Message Sequence Charts

El **Message Sequence Chart** (Message Sequence Chart (Diagrama de Secuencia de Mensajes) (MSC)), equivalente al diagrama de secuencia UML a nivel de sistema, muestra la cronología de mensajes entre actores y sistema. En la fase de requisitos se usa para especificar el protocolo de interacción de un CU sin entrar en el diseño de clases ni métodos [1, 70].

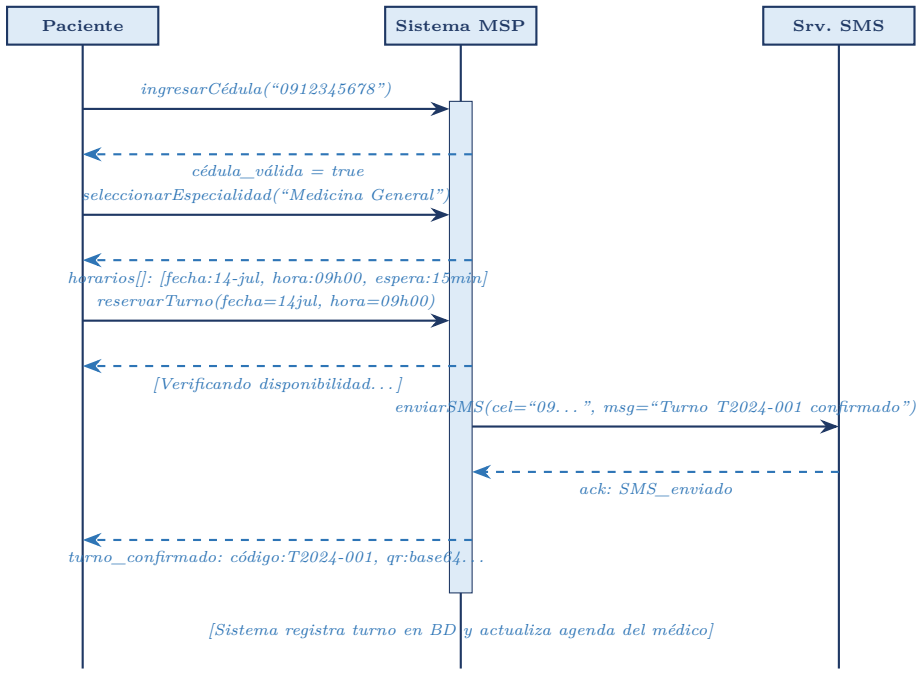


Figura 6.2. *MSC del CU-01: Reservar turno médico (flujo básico completo). Flechas sólidas = mensajes entrantes; flechas discontinuas = respuestas. El rectángulo azul en la lifeline del sistema indica el período de procesamiento. Adaptado de [1, 70].*

6.1.4 Subtema 7.4. Requisitos orientados a solución: los tres modelos

Los **requisitos orientados a solución** complementan las metas y escenarios con tres perspectivas de modelado que describen el sistema de forma complementaria [1, 2]:

Tabla 6.8. *Las tres perspectivas de los modelos orientados a solución [1, 71, 72].*

Perspectiva	Pregunta	Notación	Ejemplo
Datos	¿Qué entidades maneja el sistema y cómo se relacionan?	ER / Clases UML	Turno, Paciente, Médico, Especialidad
Funcional	¿Qué transformaciones aplica el sistema sobre los datos?	DFD / Actividades UML	Proceso de reserva, generación de código
Comportamiento	¿Cómo reacciona el sistema a los eventos en el tiempo?	Statechart / Máq. estados UML	Ciclo de vida del Turno

Modelo de datos: diagrama de clases UML (dominio)

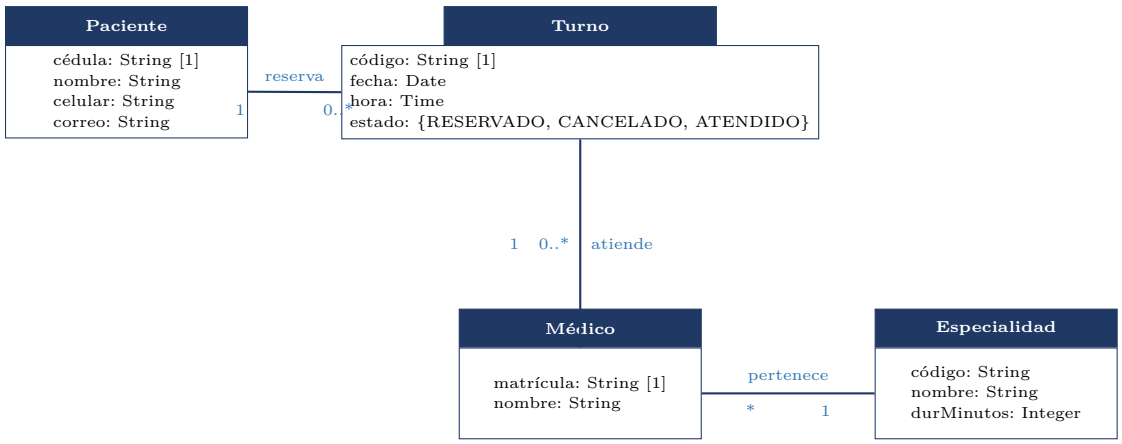


Figura 6.3. *Diagrama de clases UML del dominio del sistema de turnos MSP. Se modela el problema, no el diseño. Las multiplicidades expresan reglas de negocio verificables. Adaptado de [1, 71].*

Modelo de comportamiento: statechart del Turno

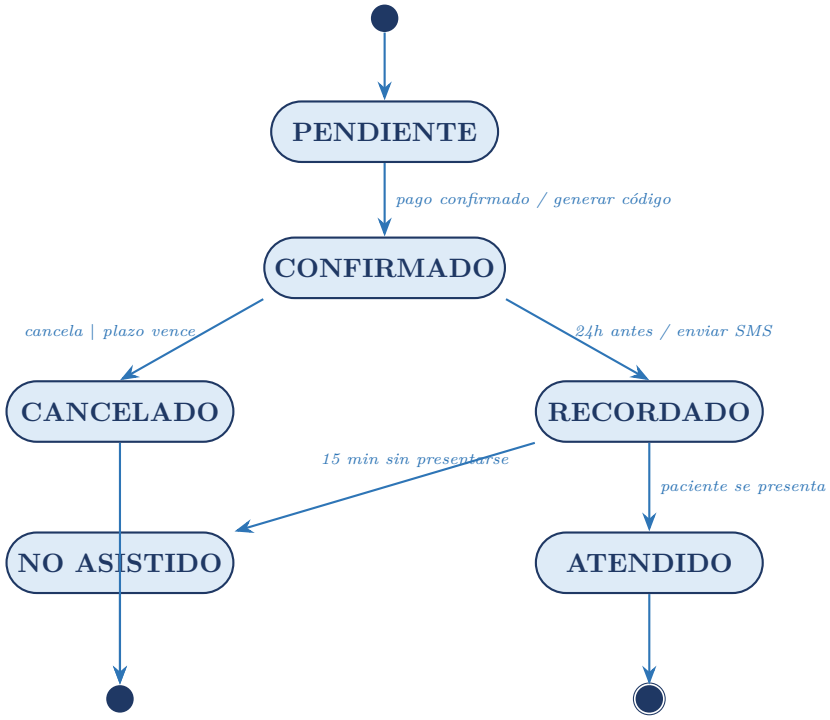


Figura 6.4. *Statechart del ciclo de vida de la entidad* `Turno`*. Cada transición muestra el evento disparador y la acción ejecutada por el sistema (separados por "/"). Adaptado de [1, 72].*

Coherencia entre las tres perspectivas

Tabla 6.9. *Tabla de coherencia entre los tres modelos ROS del sistema de turnos MSP. Cada fila verifica que una entidad o evento aparezca consistentemente en los tres modelos [1, 2].*

Elemento	Clases (datos)	CU (función)	Statechart (comportamiento)
`Turno`	Clase con atributo `estado`.	CU-01 crea; CU-02 cancela; CU-03 consulta.	Estados PENDIENTE→CONFIRMADO→...
`Paciente`	Clase con cédula como PK.	Actor primario CU-01, CU-02, CU-03.	No tiene statechart (actor externo).
Evento «pago confirmado»	Atributo de estado de Turno.	Paso 5 del CU-01.	Transición PENDIENTE→CONFIRMADO.
«Cancelar turno»	Actualiza `estado` a CANCELADO.	CU-02: Cancelar turno (actor: Paciente o Médico).	Transición →CANCELADO.

6.2 Mapa síntesis de la Semana 6

Tabla 6.10. *Mapa de cierre de la Semana 6.*

Subtema	Concepto clave	Artefacto del proyecto
7.1 Metas	7 reglas Pohl; KAOS AND/OR; iStar 2.0 SR; suposiciones.	Grafo KAOS del dominio + modelo SR del actor principal.
7.2 Escenarios	4 tipos; plantilla Cockburn; 11 reglas; misuse case.	3+ CU *fully dressed*; 1 misuse case con RF de seguridad.
7.3 UML / MSC	«include» / «extend»; herencia de actores; MSC.	Diagrama CU + MSC del flujo principal.
7.4 Modelos ROS	Clases (dominio); DFD/actividades; statechart; coherencia.	Modelo de clases + statechart de entidad clave.

6.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 6 (en clase, 2 horas).

Caso: Sistema de gestión de préstamos de la biblioteca de la UTEQ.

Tareas en equipos de 3 estudiantes:

1. Construir el grafo KAOS con meta raíz «Garantizar acceso oportuno al material bibliográfico»: mínimo 2 niveles, 3 hojas con RF derivados.
2. Redactar el CU «Prestar libro»con plantilla Cockburn *fully dressed*: 1 flujo básico, 2 flujos alternativos, 1 excepción.
3. Identificar 1 misuse case («Préstamo fraudulento») con al menos 3 RF/RNF de seguridad derivados.
4. Construir el statechart de la entidad `Préstamo` (estados: SOLICITADO, ACTIVO, RENOVADO, DEVUELTO, VENCIDO).
5. Verificar coherencia entre clases y statechart mediante la tabla de cruce.

Práctica / Actividad

Tarea autónoma n.º 6 (5 horas, equipo del proyecto integrador).

Hito 2 del proyecto integrador:

1. **Grafo KAOS**: 3 niveles, suposiciones identificadas, requisitos operacionalizados en las hojas.
2. **Modelo iStar 2.0 SR** del actor principal del sistema.
3. **3 CU** *fully dressed*: el más crítico, el más complejo en excepciones, y uno de consulta.
4. **Diagrama de CU UML** completo con «include»/«extend»/herencia.
5. **MSC** del CU principal (flujo básico completo).
6. **Diagrama de clases del dominio**: mínimo 5 clases, 4 relaciones.
7. **Statechart** de la entidad central del sistema.
8. **Tabla de coherencia** que muestre la trazabilidad cruzada entre los tres modelos.

6.4 Resumen ejecutivo de la semana

- Las **metas** articulan el *por qué*, los **escenarios** el *cómo*, y los **modelos ROS** el *qué* — los tres niveles son obligatorios en un ERS de calidad [1].
- Las **siete reglas de Pohl** garantizan metas verificables, atómicas, consistentes y trazables [1].
- El grafo **KAOS** con AND/OR-refinement proporciona la jerarquía que vincula objetivos de negocio con RF atómicos [42, 67].
- El modelo **iStar 2.0 SR** [68] es el mecanismo más poderoso para derivar Requisito No Funcional (RNF) desde metas blandas de los actores.
- Los **misuse cases** [73] convierten las amenazas de seguridad en requisitos verificables mediante la técnica de contramedidas.
- Las **once reglas** de calidad [1, 52] son el checklist mínimo para validar un caso de uso antes de incluirlo en el ERS.
- La **coherencia** entre las tres perspectivas de los ROS se verifica explícitamente con la tabla de cruce [1, 2].

Semana 7

Documentación de requisitos

Documentar un requisito no es escribirlo: es convertir una intención difusa en una afirmación precisa, verificable y comprendida por todas las partes. La calidad de un ERS se mide no por su longitud sino por la ausencia de suposiciones implícitas.

adaptado de IEEE [2]

Objetivos de aprendizaje de la semana

Al finalizar esta séptima semana, el estudiante estará en capacidad de:

1. Explicar los **cuatro propósitos** de la documentación de requisitos y clasificar los tres tipos de artefactos [1-3].
2. Seleccionar el **formato de representación** adecuado (lenguaje natural, semiestructurado, estructurado, formal) según el tipo de sistema, la audiencia y el nivel de riesgo [1, 10, 52].
3. Redactar requisitos con la **plantilla EARS** en sus cinco patrones, eliminando los siete tipos de ambigüedad lingüística más frecuentes [1, 15, 74].
4. Detectar **requirement smells** mediante los indicadores de Femmer et al. [63] y proponer correcciones.
5. Completar todos los **diez atributos de requisito** definidos en IEEE 29148:2018 para una ficha de especificación individual [2, 52].
6. Redactar **criterios de aceptación** en formato Dado/Cuando/Entonces para requisitos funcionales y no funcionales [2, 60, 61].
7. Evaluar la calidad de un conjunto de requisitos aplicando las seis características del conjunto de IEEE 29148:2018 [1, 2].

7.1 Tema 8. Documentación de requisitos

7.1.1 Subtema 8.1. Propósito de la documentación y tipos de artefactos

Los cuatro propósitos de la documentación

Definición

Documentación de requisitos. Proceso de registrar los requisitos en artefactos persistentes, legibles y controlados, de modo que puedan ser comunicados, verificados, modificados y rastreados a lo largo del ciclo de vida del sistema [1, 2].

Pohl [1] identifica cuatro propósitos esenciales:

1. **Comunicación**: los requisitos documentados son el medio primario de comunicación entre el cliente, el analista y el equipo de desarrollo. Un requisito no documentado es un requisito

implícito, y los implícitos son la fuente de errores más costosa en la industria [8, 9].

2. **Acuerdo contractual**: el ERS firmado por el cliente y el proveedor es el documento que define el alcance del proyecto, el criterio de aceptación final y la base legal del contrato.
3. **Base para el diseño y las pruebas**: ningún artefacto de diseño o caso de prueba debería existir sin un requisito que lo justifique. La trazabilidad inversa (prueba → requisito) garantiza que nada se prueba gratuitamente y nada se deja sin probar.
4. **Gestión del cambio**: solo lo que está documentado puede cambiarse de forma controlada. Sin un baseline del ERS, cada cambio de alcance es invisible para el CCB.

Los tres tipos de artefactos de documentación

Pohl [1] y la norma IEEE 29148:2018 [2] distinguen tres familias de artefactos de requisitos:

Tabla 7.1. *Tipos de artefactos de documentación de requisitos [1, 2].*

Tipo	Descripción	Ejemplos
Artefactos de **requisitos**	Documentan los requisitos en sí: qué debe hacer el sistema.	ERS / SRS, StRS, SyRS, fichas Volere, historias de usuario.
Artefactos de **contexto**	Documentan el entorno del sistema: quién lo usa, por qué, en qué marco normativo.	Documento de visión, modelos i*, diagramas de contexto, BPMN AS-IS.
Artefactos de **proceso**	Documentan cómo se gestionó el proceso de IR: actas, planes, matrices.	Plan de elicitación, actas de entrevista, RTM, registro de cambios.

El continuum de formalidad

La documentación de requisitos no es binaria (formal o informal). Existe un continuum de formalidad que va del lenguaje natural libre hasta la especificación matemática [1, 2]:

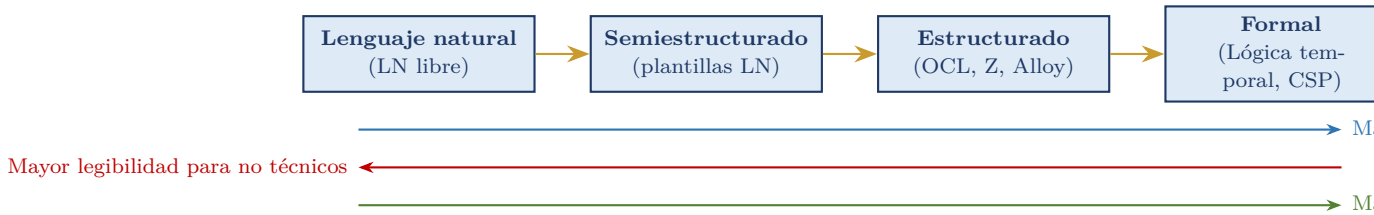


Figura 7.1. *Continuum de formalidad de los formatos de documentación de requisitos. Las tres flechas muestran los tres atributos que varían a lo largo del eje. Adaptado de [1, 2].*

7.1.2 Subtema 8.2. Formatos de representación y técnicas de precisión lingüística

Ambigüedad en el lenguaje natural

El lenguaje natural es el formato más accesible pero el más propenso a la ambigüedad. Pohl [1] y Berry & Kamsties [62] clasifican la ambigüedad en tres tipos:

- **Ambigüedad léxica**: una palabra tiene múltiples interpretaciones. Ejemplo: «El sistema deberá manejar los turnos»— ¿manejar significa crear, modificar, cancelar, asignar?
- **Ambigüedad sintáctica**: la estructura gramatical de la oración admite múltiples análisis. Ejemplo: «El sistema notificará al médico y al paciente cuando cancele»— ¿cancela el médico, el paciente, o el sistema automáticamente?
- **Ambigüedad semántica (pragmática)**: el significado depende del contexto del proyecto, que el lector no comparte. Ejemplo: «El sistema deberá ser seguro»— ¿seguro en términos de autenticación, de cifrado, de disponibilidad o de integridad de datos?

Requirement smells: los indicadores de Femmer

Definición

Requirement smell. Patrón lingüístico o estructural que es un indicador de un posible defecto en el requisito sin ser necesariamente un defecto en sí mismo [63]. Por analogía con los *code smells* de Fowler, su presencia justifica una inspección más profunda.

Tabla 7.2. *Los ocho requirement smells más frecuentes según Femmer et al. [63] y Tiwari et al. [75].*

Smell	Ejemplo	Corrección
Cuantificador vago	«el sistema responderá rápidamente»	Especificar tiempo concreto y percentil.
Lista incompleta («entre otros»)	«el informe incluirá nombre, cédula y otros datos»	Enumerar exhaustivamente todos los campos.
Sustantivo vago	«el usuario podrá consultar la información»	Especificar qué usuario y qué información.
Condición implícita	«si es necesario, el sistema enviará alerta»	Definir explícitamente cuándo es necesario.
Pasiva sin sujeto	«el turno deberá ser confirmado»	«El sistema deberá confirmar el turno. . . »
Opción encubierta («puede»)	«el sistema puede enviar recordatorios»	Decidir: ¿debe o no debe? Usar “deberá”.
Requisito doble (and/or)	«el sistema registrará y enviará el comprobante»	Separar en dos requisitos distintos.
Referencia oscura	«como se indicó anteriormente»	Referenciar el ID del requisito exacto.

Plantilla EARS: Easy Approach to Requirements Syntax

La plantilla **EARS** fue propuesta por Alistair Mavin et al. [15] como respuesta práctica al problema de la ambigüedad en el lenguaje natural, sin el costo de adoptar un lenguaje formal. Define cinco patrones según la naturaleza del requisito:

Tabla 7.3. *Los cinco patrones de la plantilla EARS con ejemplos ecuatorianos [1, 15].*

Patrón	Plantilla	Ejemplo
Ubicuo (*ubiquitous*)	El <sistema> deberá <acción>.	El sistema deberá registrar todos los accesos de usuarios con fecha, hora e IP.
Dirigido por evento (*event-driven*)	**Cuando** <disparador>, el <sistema> deberá <respuesta>.	Cuando el SRI responda con código de rechazo, el sistema deberá notificar al contador en menos de 30 segundos.
No deseado (*unwanted behavior*)	**Si** <condición>, el <sistema> deberá <acción de manejo>.	Si la conexión al servidor del SRI excede 10 segundos sin respuesta, el sistema deberá reintentar hasta 3 veces y registrar el error.
Opcional (*optional feature*)	**Cuando** el sistema incluya <característica>, deberá <acción>.	Cuando el sistema incluya módulo biométrico, deberá validar la huella dactilar en menos de 500 ms.
Complejo (combinado)	**Mientras** <estado>, **cuando** <disparador>, el <sistema> deberá <acción>.	Mientras el período de matrícula esté activo, cuando un estudiante complete el pago, el sistema deberá generar el comprobante en menos de 5 segundos.

Ejemplo en Ecuador

Reescritura de requisitos defectuosos del portal de trámites del MSP usando EARS.

Requisito defectuoso	Requisito EARS corregido
«El sistema deberá responder rápido.»	«Cuando el usuario envíe una solicitud de consulta de turno, el sistema deberá devolver la respuesta en menos de 2 segundos para el 95 % de las solicitudes bajo una carga de 500 usuarios concurrentes.»
«El sistema puede enviar recordatorios.»	«Cuando un turno tenga fecha de atención a menos de 24 horas, el sistema deberá enviar un mensaje SMS de recordatorio al número registrado del paciente.»

7.1.3 Subtema 8.3. Atributos de requisito: la ficha de especificación individual

Los diez atributos de IEEE 29148:2018

La norma *IEEE/ISO/IEC 29148:2018* [2] y Robertson & Robertson [52] coinciden en que cada requisito individual debe llevar consigo un conjunto de metadatos que lo hacen gestionable

a lo largo del ciclo de vida. Los diez atributos más importantes son:

Tabla 7.4. *Los diez atributos de un requisito individual según [2, 10, 52].*

Atributo	Descripción	Ejemplo (RF-01)
ID	Identificador único e inmutable. Esquema: tipo-número.	RF-01
Descripción	Enunciado del requisito en EARS o LN estructurado.	«Cuando el paciente complete la reserva, el sistema deberá…»
Prioridad	MoSCoW o numérica (MoSCoW preferido por claridad).	Must-have
Fuente	Stakeholder, documento o regulación que origina el requisito.	Jefe de Enfermería, MSP-Quevedo; entrevista 14-ene-2025.
Estabilidad	Probabilidad de cambio: Alta / Media / Baja.	Baja (requisito de negocio central)
Estado	PROPUESTO → APROBADO → IMPLEMENTADO → VERIFICADO → ELIMINADO.	APROBADO
Versión	Número de versión del enunciado. Incrementa con cada cambio.	v1.2 (20-feb-2025)
Criterio de aceptación	Condición medible que determina cuándo el requisito está satisfecho.	Ver § 7.1.4.
Trazabilidad hacia arriba	ID del requisito de nivel superior o meta que lo origina.	Meta KAOS: M-03; StRS: STK-07.
Trazabilidad hacia abajo	ID de los elementos de diseño, código y pruebas que lo satisfacen.	UC-01; CLS-Turno; CP-01, CP-02.

La ficha Volere de Robertson

Definición

Plantilla Volere. Sistema de fichas de especificación de requisitos desarrollado por Suzanne y James Robertson [52]. Cada requisito se documenta en una ficha que contiene, además de los atributos estándar, el **fit criterion** (criterio de ajuste) y las métricas de satisfacción e insatisfacción del cliente.

Tabla 7.5. *Ficha Volere para RF-01: Reservar turno médico. Adaptado de [52].*

Ficha Volere — Requisito RF-01	
Número de requisito	RF-01
Tipo de requisito	Funcional (caso de uso CU-01)
Descripción	Cuando el paciente complete la selección de especialidad, fecha y hora, el sistema deberá registrar la reserva del turno y generar el código único de confirmación.
Justificación	Los pacientes pierden entre 45 y 90 minutos en la actualidad desplazándose a las instalaciones del MSP para sacar turno presencialmente. El sistema lo reduce a menos de 5 minutos desde cualquier lugar.
Fuente	Jefa de Enfermería, MSP Quevedo; Oficio N.º 2025-ENF-014 del 15-ene-2025.
Criterio de ajuste (*fit criterion*)	El 95 % de los pacientes que intentan reservar un turno disponible logran completar el proceso en menos de 3 minutos, sin necesitar asistencia de terceros, medido en pruebas de usabilidad con 20 participantes representativos de la población objetivo.
Satisfacción del cliente	5 / 5 (crítico para la adopción del sistema)
Insatisfacción del cliente	5 / 5 (si no existe, el sistema no tiene valor para el ciudadano)
Prioridad	Must-have (MoSCoW)
Conflictos	Ninguno conocido. Interactúa con RNF-SEC-01 (CAPTCHA que no debe frenar la usabilidad).
Historial de cambios	v1.0 (20-ene-2025): enunciado inicial. v1.1 (05-feb-2025): añadido criterio de 3 minutos tras estudio de usuarios.

7.1.4 Subtema 8.4. Criterios de calidad y criterios de aceptación

Criterios de aceptación: el formato Dado/Cuando/Entonces

Definición

Criterio de aceptación. Condición medible y objetiva que determina cuándo un requisito está completamente implementado y satisfecho. Es el contrato de definición de *terminado* (*done*) entre el analista y el equipo de desarrollo [2, 60, 61].

El formato Dado/Cuando/Entonces (GWT: *Given/When/Then*) es el estándar de facto para criterios de aceptación verificables, porque puede convertirse directamente en casos de prueba automatizados con frameworks como Cucumber o Behave [60, 61]:

Tabla 7.6. *Criterios de aceptación en formato GWT para RF-01 y RNF-03. Adaptado de [2, 60].*

Criterio de aceptación CA-RF-01.1: Reserva exitosa	
Dado	El paciente ha iniciado sesión con su cédula válida y existe al menos un turno disponible en la especialidad seleccionada.
Cuando	El paciente selecciona especialidad, fecha y hora, y presiona “Confirmar reserva”.
Entonces	El sistema registra el turno con estado CONFIRMADO, genera el código único T-{AAAA}-{MM}-{DD}-{HH}, envía SMS al número registrado y muestra la confirmación en pantalla en menos de 3 segundos.
Criterio de aceptación CA-RF-01.2: Horario ocupado	
Dado	El paciente ha iniciado sesión y el horario que intenta reservar ya está ocupado.
Cuando	El paciente selecciona ese horario y presiona “Confirmar reserva”.
Entonces	El sistema muestra el mensaje “El horario seleccionado no está disponible” y ofrece los 3 horarios alternativos más cercanos sin salir de la pantalla. No se registra ningún turno.
Criterio de aceptación CA-RNF-03: Tiempo de respuesta	
Dado	El sistema está en operación normal con 500 usuarios concurrentes activos.
Cuando	Un usuario envía cualquier solicitud de consulta o reserva.
Entonces	El 95 % de las respuestas llegan en menos de 2 segundos y el 99 % en menos de 5 segundos, medidos con JMeter durante 30 minutos continuos de carga sostenida.

Métricas de calidad del proceso de IR

Además de la calidad de los artefactos individuales, el proceso de IR debe medirse con indicadores cuantitativos. Los más usados en la industria son [1, 2, 10]:

Tabla 7.7. *Métricas de calidad del proceso de IR más usadas en proyectos medianos [1, 10].*

Métrica	Fórmula / definición	Valor referencia
Densidad de defectos por página	N.º de defectos / páginas del ERS	< 3 defectos/página (objetivo)
Cobertura de trazabilidad	(RF trazados hacia arriba / RF total) $\times$ 100 %	≥ 95 %
Estabilidad de requisitos	(RF no cambiados / RF baseline) $\times$ 100 %	> 80 % en fase de diseño
Tasa de ambigüedad	RF con al menos 1 *smell* detectado / RF total	< 5 % después de revisión
Ratio de RF verificables	RF con criterio de aceptación / RF total	100 % antes del baseline

7.2 Mapa síntesis de la Semana 7

Tabla 7.8. *Mapa de cierre de la Semana 7.*

Subtema	Concepto clave	Artefacto del proyecto
8.1 Propósito y artefactos	4 propósitos; 3 tipos de artefactos; continuum de formalidad.	Clasificar los artefactos del hito 1 en la taxonomía.
8.2 Formatos y precisión	3 tipos de ambigüedad; 8 smells; 5 patrones EARS.	Reescribir los 5 RF más importantes del proyecto en EARS.
8.3 Atributos	10 atributos IEEE 29148; ficha Volere; fit criterion.	Fichas completas de los 10 RF/RNF más prioritarios.
8.4 Calidad y aceptación	GWT; métricas de calidad del proceso.	3+ criterios de aceptación GWT; tabla de métricas.

7.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 7 (en clase, 2 horas).

Objetivo: identificar smells y reescribir en EARS.

Material: el docente entrega un fragmento de ERS real (10 requisitos) de un sistema gubernamental ecuatoriano con defectos deliberados.

Tareas en parejas:

1. Leer los 10 requisitos e identificar el tipo de smell de cada uno (usar la tabla del subtema 8.2).
2. Reescribir los 10 requisitos usando el patrón EARS más apropiado para cada uno.
3. Añadir el atributo *criterio de aceptación* en formato GWT a los 5 RF más importantes.
4. Completar la tabla de métricas de calidad: ¿cuántos de los 10 requisitos originales tenían smells? ¿Qué porcentaje es eso?
5. Comparar con otra pareja y discutir diferencias.

Práctica / Actividad

Tarea autónoma n.º 7 (5 horas, equipo del proyecto integrador).

Hito 2 — ampliación de la documentación de requisitos:

1. **Revisión EARS**: reescribir todos los RF del proyecto usando el patrón EARS correcto para cada uno.
2. **Detección de smells**: pasar el checklist de 8 smells por cada RF; reportar los encontrados y las correcciones.
3. **Fichas Volere**: completar las 10 fichas de los RF/RNF más prioritarios con todos los atributos, incluyendo fit criterion, satisfacción/insatisfacción y conflictos.
4. **Criterios de aceptación GWT**: para cada RF de la Ficha Volere, al menos 2 criterios de aceptación (flujo feliz + flujo alternativo o excepción).
5. **Tabla de métricas de calidad**: calcular las 5 métricas del subtema 8.4 para el conjunto actual de requisitos.

7.4 Resumen ejecutivo de la semana

- La documentación tiene cuatro propósitos: **comunicar, acordar contractualmente, servir de base para diseño/pruebas y gestionar cambios** [1, 2].
- Los tres tipos de ambigüedad (**léxica, sintáctica, semántica**) son la causa más frecuente de requisitos defectuosos [1, 62].
- Los **requirement smells** [63] permiten una revisión sistemática y rápida sin necesitar inspecciones formales completas.
- La plantilla **EARS** [15] reduce la ambigüedad del LN sin el costo de un lenguaje formal, usando cinco patrones estructurados.
- Los **diez atributos** de IEEE 29148:2018 hacen que cada requisito sea rastreable, medible y gestionable a lo largo del ciclo de vida [2].
- La ficha **Volere** [52] añade el *fit criterion* y las métricas de satisfacción, convirtiendo el requisito en un instrumento de negociación y aceptación.
- Los **criterios de aceptación GWT** [60, 61] son el puente directo entre la especificación y las pruebas de aceptación automatizadas.

Semana 8

El documento ERS/SRS

El ERS no es un inventario de funcionalidades; es el contrato técnico del proyecto. Su calidad determina la calidad del software que se construirá sobre él. Un ERS bien escrito es el mejor seguro contra el fracaso del proyecto.

adaptado de Wiegers y Beatty [10]

Objetivos de aprendizaje de la semana

Al finalizar esta octava semana, el estudiante estará en capacidad de:

1. Distinguir los cuatro tipos de especificación de la familia **IEEE 29148:2018**: ConOps, StRS, SyRS y SRS, y seleccionar el apropiado según el tipo de proyecto [1, 2, 66].
2. Construir la **estructura de referencia** de un ERS (6 capítulos) conforme a IEEE 29148:2018, con todos los elementos obligatorios [2, 10, 52].
3. Aplicar correctamente los **verbos modales** en la redacción de requisitos (*deberá, debería, podría, no deberá*) y las ocho técnicas de precisión lingüística [1, 74].
4. Combinar texto estructurado con **modelos UML** en el ERS, vinculando cada modelo con los requisitos que especifica [1, 70, 71].
5. Elaborar una RTM extendida que vincule los requisitos con los modelos UML, casos de prueba y elementos de diseño [2, 10, 66].
6. Definir el proceso de **baseline** del ERS y gestionar su ciclo de vida mediante un CCB básico [1, 2, 10].

8.1 Tema 9. El documento ERS/SRS

8.1.1 Subtema 9.1. Tipos de especificaciones en la familia IEEE 29148:2018

La familia de documentos IEEE 29148

La norma *IEEE/ISO/IEC 29148:2018* [2] define cuatro tipos de especificación que corresponden a los diferentes niveles de la jerarquía de requisitos y a los diferentes artefactos del ciclo de vida de sistemas [66]:

Tabla 8.1. *Familia de documentos IEEE/ISO/IEC 29148:2018 y su posición en la jerarquía de requisitos [2, 66].*

Documento	Contenido	Audiencia	Cuándo producirlo
ConOps	Visión operacional del sistema desde la perspectiva del usuario y de las organizaciones involucradas.	Alta gerencia, patrocinadores, usuarios operacionales.	Antes de comprometerse con un sistema.
StRS	Necesidades, expectativas y restricciones de los stakeholders.	Analistas, cliente, PO.	Fase de elicitación; antes del SyRS.
SyRS	Requisitos del sistema completo (hardware + software + personas + procesos).	Arquitectos de sistema, ingenieros.	En proyectos de sistemas grandes; reemplaza o complementa al SRS.
ERS/SRS	Requisitos del componente de software del sistema.	Equipo de desarrollo, QA, cliente técnico.	Siempre; es el documento central de todo proyecto de software.

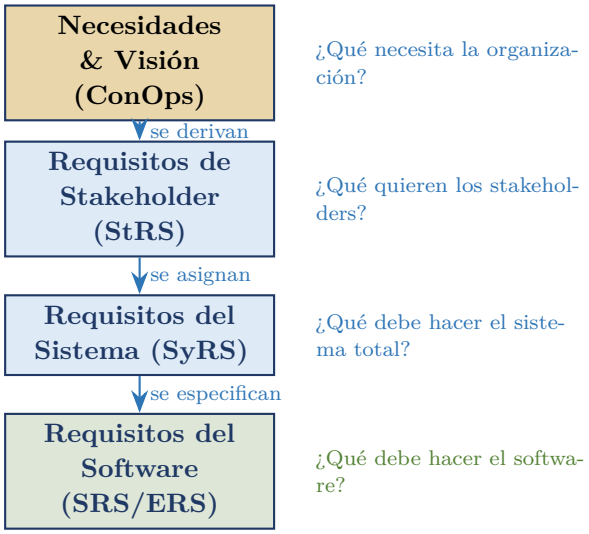


Figura 8.1. *Jerarquía de documentos de requisitos de la familia IEEE 29148:2018. Las flechas indican el flujo de derivación. En proyectos pequeños a medianos, el ERS/SRS (verde) puede ser el único documento formal. Adaptado de [2, 66].*

¿Cuándo producir cada documento?

Tabla 8.2. *Criterios de selección del tipo de especificación según el contexto del proyecto [1, 2].*

Contexto del proyecto	Documentos recomendados
Proyecto universitario o startup pequeña (< 5 personas)	Solo SRS/ERS. El ConOps se puede integrar como capítulo 1 del ERS.
Proyecto SERCOP (contratación pública)	StRS (en el pliego) + SRS (entregable contractual).
Sistema para entidad pública grande (MSP, IESS, SRI)	ConOps + StRS + SRS; el SyRS es opcional si no hay hardware propio.
Sistema embebido / IoT / sistema-de-sistemas	ConOps + StRS + SyRS + SRS (uno por subsistema de software).
Proyecto ágil Scrum	Épicas (ConOps informal) + Backlog priorizado (StRS informal) + Product Backlog con criterios de aceptación (SRS informal).

8.1.2 Subtema 9.2. Estructura de referencia del ERS según IEEE 29148:2018

Los seis capítulos canónicos

La norma IEEE 29148:2018 [2] y Wiegers & Beatty [10] establecen la siguiente estructura de referencia para un ERS completo. Todos los proyectos deben adaptarla, pero ninguno debería eliminar secciones sin justificación explícita:

Tabla 8.3. *Estructura de referencia canónica del ERS/SRS según [2, 10, 52].*

Cap.	Título	Contenido esencial
1	Introducción	Propósito, alcance, acrónimos, referencias, visión general del documento.
2	Descripción general	Perspectiva del producto, clases de usuarios, entorno de operación, supuestos y dependencias.
3	Interfaces externas	Interfaces de usuario (UI), hardware, software y comunicaciones.
4	Requisitos del sistema	RF organizados por función o por caso de uso; cada RF con ID, descripción y criterio de aceptación.
5	Requisitos no funcionales	Rendimiento, seguridad, fiabilidad, mantenibilidad, portabilidad — con criterios cuantificables.
6	Apéndices y matrices	RTM, glosario, modelos UML de referencia, historial de versiones.

SRS del sistema de matrículas UTEQ: ejemplo concreto

A continuación se presenta la estructura del Capítulo 4 (Requisitos del sistema) del ERS del sistema de matrícula en línea de la UTEQ, como ejemplo de organización por función [2, 10]:

Tabla 8.4. *Capítulo 4 del ERS del sistema de matrículas UTEQ. Ejemplo de organización de RF por subsistema funcional.*

ID	Requisito	Criterio de aceptación (resumen)
4.1 Módulo de Autenticación		
RF-A01	Cuando el estudiante ingrese cédula y contraseña válidas, el sistema deberá autenticarlo en menos de 2 s.	CA: Prueba con 100 pares válidos; 100 % autenticados en < 2 s.
RF-A02	Si el estudiante falla la autenticación 3 veces consecutivas, el sistema deberá bloquear la cuenta por 15 minutos.	CA: Dado 3 intentos fallidos, entonces cuenta bloqueada y notificación enviada.
4.2 Módulo de Matrícula		
RF-M01	Cuando el estudiante seleccione una asignatura, el sistema deberá verificar que cumple el prerrequisito antes de registrar la matrícula.	CA: Con estudiante sin prerrequisito, el sistema rechaza y muestra el motivo.
RF-M02	El sistema deberá permitir la matrícula solo dentro del período académico activo definido por el calendario de la UTEQ.	CA: Intento fuera del período genera mensaje de error específico.
RF-M03	Cuando el estudiante complete la matrícula, el sistema deberá generar y mostrar el comprobante en PDF en menos de 5 s.	CA: PDF generado con nombre, período, asignaturas y código de verificación.
4.3 Módulo de Pagos (integración Banco del Pacífico)		
RF-P01	Cuando el estudiante inicie el pago en línea, el sistema deberá redirigirlo a la pasarela del Banco del Pacífico mediante HTTPS.	CA: URL de redirección usa TLS 1.2+ ; sin exponer datos en URL.
RF-P02	Si la pasarela de pagos devuelve código de error, el sistema deberá registrar el intento fallido y ofrecer al estudiante reintentar o pagar presencialmente.	CA: Log de error generado; opción de pago presencial visible en < 3 s.

8.1.3 Subtema 9.3. Lenguaje natural: verbos modales y técnicas de precisión

Los verbos modales en la redacción de requisitos

El uso correcto de los verbos modales en los requisitos es una de las decisiones técnicas más importantes de la redacción. IEEE 29148:2018 [2] y Pohl [1] establecen la siguiente jerarquía de obligatoriedad:

Tabla 8.5. *Jerarquía de verbos modales en la redacción de requisitos según [1, 2, 74].*

Verbo modal (ES)	RFC 2119 (EN)	Significado	Cuándo usar
deberá (*shall*)	MUST / SHALL	Obligatorio absoluto. El sistema no es aceptable sin esto.	RF principales; RNF con criterio cuantificable.
debería (*should*)	SHOULD	Muy recomendable; pueden existir excepciones justificadas.	Buenas prácticas, pero no críticas para aceptación.
podría (*may/can*)	MAY/CAN	Opcional o permisible.	Funcionalidades que el sistema puede ofrecer sin ser obligatorias.
no deberá (*shall not*)	MUST NOT	Prohibición absoluta. Violarlo invalida la aceptación.	Restricciones de seguridad, privacidad, legal.

Advertencia / Error frecuente

Anti-patrón crítico. Usar «debe»y «deberá»como sinónimos. En los estándares IEEE, «shall»(deberá) es obligatorio y verificable; «should»(debería) es una recomendación. Mezclarlos en un mismo ERS produce un documento donde nadie sabe qué es contractualmente exigible. La convención es usar **siempre** «deberá»para todo requisito que el equipo está comprometido a implementar [2].

Las ocho técnicas de precisión lingüística

Pohl [1] y Rupp [74] identifican ocho técnicas concretas para aumentar la precisión del lenguaje natural sin recurrir a notaciones formales:

1. **Definir el glosario de dominio**: cada término técnico del negocio aparece en el glosario del ERS con su definición precisa y unívoca. Ejemplo: «período de matrícula»= rango de fechas definido en el calendario académico UTEQ para cada ciclo.
2. **Usar cuantificadores explícitos**: eliminar «muchos», «pocos», «suficientes», «algunos». Sustituirlos por valores numéricos con unidad y percentil. Ejemplo: «el 95 % de las solicitudes», «en menos de 2 segundos».
3. **Especificar el sujeto siempre**: nunca usar la voz pasiva sin agente. «deberá ser validado»→ «el sistema deberá validar».
4. **Usar la plantilla EARS**: como ya se vio en el subtema 8.2.

5. **Eliminar las opciones encubiertas**: «puede», «debería», «podría»en un RF introduce ambigüedad. Decidir y usar «deberá»o eliminar el requisito.
6. **Referenciar normas y estándares explícitamente**: en lugar de «el sistema deberá cifrar los datos», escribir «el sistema deberá cifrar los datos en reposo usando AES-256 conforme a NIST FIPS 197».
7. **Separar requisito de implementación**: «el sistema deberá usar PostgreSQL»es una restricción de implementación que pertenece al capítulo de restricciones, no al de RF. Un RF dice *qué*, no *cómo*.
8. **Usar tablas de decisión para condiciones complejas**: cuando un requisito tiene más de dos condiciones anidadas, la tabla de decisión elimina la ambigüedad que crearía el texto lineal.

Ejemplo en Ecuador

Tabla de decisión — Validación de matrícula UTEQ.

Condición	R1	R2	R3	R4	
¿Dentro del período?	Sí	Sí	Sí	No	
¿Prerrequisito aprobado?	Sí	Sí	No	—	
¿Cupo disponible?	Sí	No	—	—	
Acción	**R1**	**R2**	**R3**	**R4**	
Registrar matrícula	✓				
Mostrar «sin cupo, lista de espera»		✓			
Mostrar «prerequisito no aprobado»			✓		
Mostrar «período cerrado»				✓	

8.1.4 Subtema 9.4. Combinación de modelos UML y texto en el ERS

Estrategias de integración

Un ERS moderno no es solo texto ni solo diagramas: es la combinación coherente de ambos. Pohl [1] describe cuatro estrategias de integración, ordenadas de menor a mayor acoplamiento:

Tabla 8.6. *Estrategias de integración de modelos UML y texto en el ERS [1, 70, 71].*

Estrategia	Descripción	Cuándo usar
Diagrama ilustrativo	El modelo UML está en el apéndice; el texto no lo referencia directamente.	Proyectos pequeños; audiencia no técnica.
Diagrama referenciado	El texto menciona explícitamente el modelo: «Ver Figura 4.2». El modelo añade precisión; el texto es el elemento normativo.	Recomendado como mínimo para todo ERS profesional.
Modelo primario, texto secundario	El modelo UML es el documento normativo; el texto es un comentario anotado sobre el modelo.	Proyectos con herramienta CASE (Enterprise Architect); equipos con formación UML.
Especificación integrada	Cada elemento de modelo lleva IDs de requisitos anclados (*tagged values*); la RTM se genera automáticamente.	Proyectos certificados (DO-178C, IEC 62304, ISO 26262).

Patrón de integración: ID de requisito anclado al modelo

El patrón más práctico para proyectos universitarios y medianos es el **diagrama referenciado con IDs anclados**: cada elemento UML lleva entre llaves el ID del requisito que especifica [1, 2].

Convención de anclaje de IDs en modelos UML (proyecto UTEQ).
En los diagramas de casos de uso:
- Cada elipse CU lleva: «CU-01 Reservar turno {RF-01, RNF-03}»
- Cada relación `«include»` lleva el ID del RF que la origina.

En el diagrama de clases:
- Cada clase lleva: «Turno {RF-01, RF-02, RF-05}»
- Cada atributo lleva el ID si lo especifica un requisito concreto: «estado: Enum {RF-02.4}»

En el statechart:
- Cada transición lleva el ID: «pago confirmado / generar código {RF-06}»

RTM extendida: vinculando modelos, requisitos y pruebas

La RTM extendida añade columnas para los modelos UML, además de los elementos de diseño y prueba tradicionales. Esto permite navegar en todas las direcciones: desde un requisito a su diagrama, desde un diagrama a sus requisitos, y desde un caso de prueba a su requisito y su modelo [2, 10, 66].

Tabla 8.7. *RTM extendida para los RF más críticos del sistema de matrícula UTEQ. La trazabilidad es bidireccional en todas las columnas [2, 66].*

ID Req.	Enunciado (resumen)	Origen	CU UML	Clase UML	Caso de prue-ba	Estado
RF-A01	Autenticar estudiante < 2 s.	StRS-03	CU-00	Usuario	CP-01	Verificado
RF-A02	Bloquear tras 3 fallos.	StRS-04	CU-00.alt	Sesion	CP-02	En revisión
RF-M01	Verificar prerrequisito.	StRS-11	CU-02	Asignatura	CP-05	Verificado
RF-M02	Matricular solo en período activo.	StRS-12	CU-02	Periodo	CP-06	Pendiente
RF-M03	Generar PDF < 5 s.	StRS-14	CU-04	Comprobante	CP-08	Pendiente
RNF-01	Respuesta < 2 s al P95.	StRS-21	Arch-NFR	—	PR-01	Pendiente
RNF-02	Cifrado TLS 1.2+ en todas las conexiones.	Legal-01	Arch-SEC	—	SP-01	Pendiente

Ciclo de vida del ERS: baseline, CCB y versiones

Definición

Línea base del ERS (*baseline*). Versión del ERS que ha sido formalmente revisada, aprobada por todos los stakeholders relevantes y controlada por el CCB. A partir de este momento, cualquier modificación requiere pasar por el proceso formal de gestión del cambio [1, 2, 66].

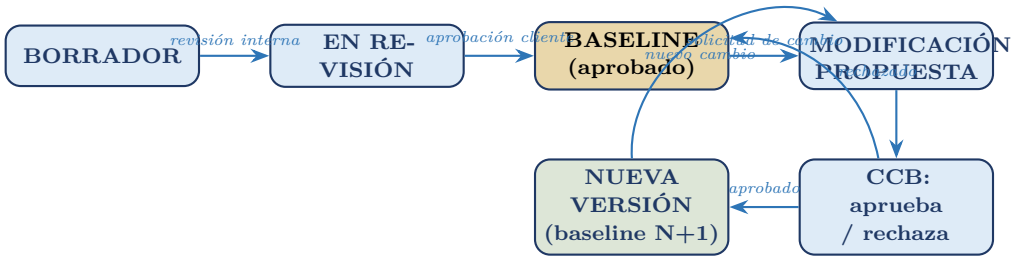


Figura 8.2. *Ciclo de vida del ERS: desde el primer borrador hasta el baseline y la gestión de cambios posterior. El CCB es el árbitro de todos los cambios al baseline. Adaptado de [1, 2, 10].*

Tabla 8.8. *Proceso mínimo del CCB para proyectos universitarios y medianos [2, 10].*

#	Paso	Descripción
1	Solicitud de cambio (CR)	El proponente (cualquier stakeholder) documenta el cambio: qué, por qué, impacto estimado en costo/plazo/alcance.
2	Análisis de impacto	El analista evalúa cuántos RF, RNF, casos de prueba y artefactos de diseño se ven afectados.
3	Decisión del CCB	El CCB (mínimo: cliente + analista + líder técnico) aprueba, rechaza o pospone.
4	Actualización del ERS	Si se aprueba, se actualiza el ERS, se incrementa la versión, se notifica a todos los involucrados.
5	Actualización de la RTM	Los nuevos IDs se incorporan a la RTM; los elementos impactados se marcan como PENDIENTE.

8.2 Mapa síntesis de la Semana 8

Tabla 8.9. *Mapa de cierre de la Semana 8.*

Subtema	Concepto clave	Artefacto del proyecto
9.1 Familia IEEE 29148	ConOps / StRS / SyRS / SRS; cuándo producir cada uno.	Identificar qué documentos aplican al proyecto integrador.
9.2 Estructura ERS	6 capítulos canónicos; Cap. 4 organizado por función.	Documento ERS completo con los 6 capítulos.
9.3 Verbos y precisión	4 verbos modales; tabla de decisión; 8 técnicas.	Revisión final de todo el ERS con checklist de precisión.
9.4 UML + texto + RTM	4 estrategias; IDs anclados; RTM extendida; baseline.	RTM extendida + versión aprobada del ERS.

8.3 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 8 (en clase, 2 horas).

Objetivo: construir la estructura completa del ERS del caso de práctica con integración UML.

Caso: sistema de reservas de la biblioteca UTEQ (continuación de las semanas 5, 6 y 7).

Tareas:

1. Redactar el índice completo del ERS (6 capítulos con secciones).
2. Redactar el Capítulo 2 (Descripción general) con todos sus elementos obligatorios: perspectiva del sistema, clases de usuarios, entorno de operación, supuestos y dependencias.
3. Completar el Capítulo 4 con los 10 RF ya identificados, usando verbos modales correctos y patrón EARS; añadir criterio de aceptación a cada uno.
4. Construir la RTM extendida con los 5 RF más prioritarios: columnas de origen, CU, clase UML, caso de prueba y estado.
5. Simular una sesión de CCB: el docente propone un cambio de alcance y los equipos analizan el impacto en la RTM y el ERS.

Práctica / Actividad

Tarea autónoma n.º 8 (6 horas, equipo del proyecto integrador).

HITO 3 DEL PROYECTO INTEGRADOR — ERS v1.0 completa (primera versión para baseline):

1. **ERS completa** con los 6 capítulos canónicos de IEEE 29148.
2. **Capítulo 1**: propósito, alcance, referencias y acrónimos.
3. **Capítulo 2**: descripción general, perspectiva del sistema, clases de usuarios, entorno de operación, supuestos.
4. **Capítulo 3**: interfaces de usuario (wireframe o mock-up), interfaces de hardware y software externo.
5. **Capítulo 4**: mínimo 25 RF organizados por módulo funcional, con ID, descripción EARS, prioridad MoSCoW y criterio GWT.
6. **Capítulo 5**: mínimo 8 RNF con criterio cuantificable.
7. **Capítulo 6**: RTM extendida + glosario + historial de versiones.
8. **Defensa oral**: 15 minutos por equipo ante tribunal docente. El tribunal realizará al menos 5 solicitudes de cambio simuladas; el equipo deberá analizar el impacto en tiempo real.

8.4 Resumen ejecutivo de la semana

- La familia IEEE 29148:2018 define cuatro tipos de especificación (**ConOps, StRS, SyRS, SRS/ERS**) para diferentes niveles y audiencias; el ERS es obligatorio en todo proyecto de software [2, 66].

- La estructura canónica de **seis capítulos** garantiza que ningún elemento obligatorio quede sin documentar [2, 10].
- El uso correcto de los **cuatro verbos modales** (*deberá, debería, podría, no deberá*) es la base de la legibilidad contractual del ERS [1, 2].
- Las **tablas de decisión** eliminan la ambigüedad de condiciones complejas mejor que cualquier redacción en texto lineal [1, 74].
- La estrategia de **IDs anclados a modelos UML** convierte el ERS en un documento navegable y facilita la generación automática de la RTM [1, 2].
- La **RTM extendida** es el instrumento que hace operativa la trazabilidad bidireccional requerida por IEEE 29148:2018 e ISO 15288:2023 [2, 66].
- El **baseline** del ERS y el proceso del CCB son los mecanismos que protegen el alcance del proyecto y convierten los cambios en decisiones explícitas y rastreables [1, 2, 10].

Semana 9

Modelos de especificación UML: datos y función

Un modelo de datos sin modelo funcional es un almacén sin operarios; un modelo funcional sin modelo de datos es un proceso sin memoria. Los requisitos completos necesitan los dos, y su coherencia mutua es la prueba más rigurosa de que el análisis fue correcto.

adaptado de Pohl [1]

Objetivos de aprendizaje de la semana

Al finalizar esta novena semana, el estudiante estará en capacidad de:

1. Construir **diagramas Entidad-Relación** (ER) en notación Chen y Crow's Foot para sistemas de gestión agropecuaria y veterinaria [1, 2, 76].
2. Traducir un diagrama ER a un **diagrama de clases UML** de dominio, añadiendo operaciones derivadas de los casos de uso [70, 71, 77].
3. Construir **diagramas de flujo de datos** (DFD) nivel 0 y nivel 1 para sistemas IoT y de gestión de haciendas [3, 78, 79].
4. Modelar procesos con **diagramas de actividades UML** incluyendo carriles, decisiones, fork/join y señales [70, 71, 77].
5. Derivar **requisitos funcionales verificables** a partir de cada elemento de los modelos de datos y función [1, 2, 52].
6. Documentar la **trazabilidad** entre modelos de datos, modelos funcionales y el ERS del proyecto integrador [2, 10, 66].

9.1 Subtema 10.1. Modelos de datos: ER y clases UML

9.1.1 El modelo Entidad-Relación como puerta de entrada a los datos

Definición

Modelo Entidad-Relación (ER). Notación gráfica propuesta por Peter Chen en 1976 [76] para representar los conceptos del dominio del problema como entidades, sus propiedades como atributos y sus asociaciones como relaciones. En la fase de requisitos el diagrama ER modela el *mundo del cliente*, no la base de datos del sistema.

El diagrama ER es la técnica más directa para responder la pregunta: *¿sobre qué entidades del negocio debe el sistema mantener información?* Esta pregunta, aparentemente simple, es la que determina el esquema de la base de datos, las entidades del diagrama de clases y los objetos de los casos de uso [1, 76]. Sin responderla correctamente, los modelos de las semanas

anteriores quedan incompletos.

Notaciones ER: Chen y Crow's Foot

Tabla 9.1. *Comparación de las dos notaciones ER más usadas en IR [71, 76].*

Elemento	Notación Chen	Crow's Foot	Uso en IR
Entidad	Rectángulo con nombre.	Rectángulo con atributos dentro.	Ambas; Chen es más didáctica.
Relación	Rombo entre entidades.	Línea directa con patas de cuervo.	Crow's Foot predomina en herramientas CASE.
Cardinalidad	1, N en los extremos.	Símbolos |, <, *O* en los extremos.	Crow's Foot es más precisa.
Atributo clave	Óvalo subrayado.	Campo con PK marcado.	Crow's Foot en herramientas DB.
Entidad débil	Rectángulo doble.	Entidad sin PK propia.	Chen es más explícita.

ER completo: sistema de gestión de ganado bovino (PFC)

Ejemplo en Ecuador

Contexto — PFC: Sistema de Gestión Ganadera para hacienda en Los Ríos. Una hacienda bovina de 450 cabezas en la provincia de Los Ríos necesita un sistema para gestionar el inventario de animales, el calendario de vacunaciones y desparasitaciones, el control reproductivo (inseminaciones, partos, destetes) y los registros de peso. Los stakeholders son el dueño de la hacienda, el mayordomo y el veterinario de campo. El sistema debe funcionar offline en smartphones Android y sincronizar cuando haya señal GSM.

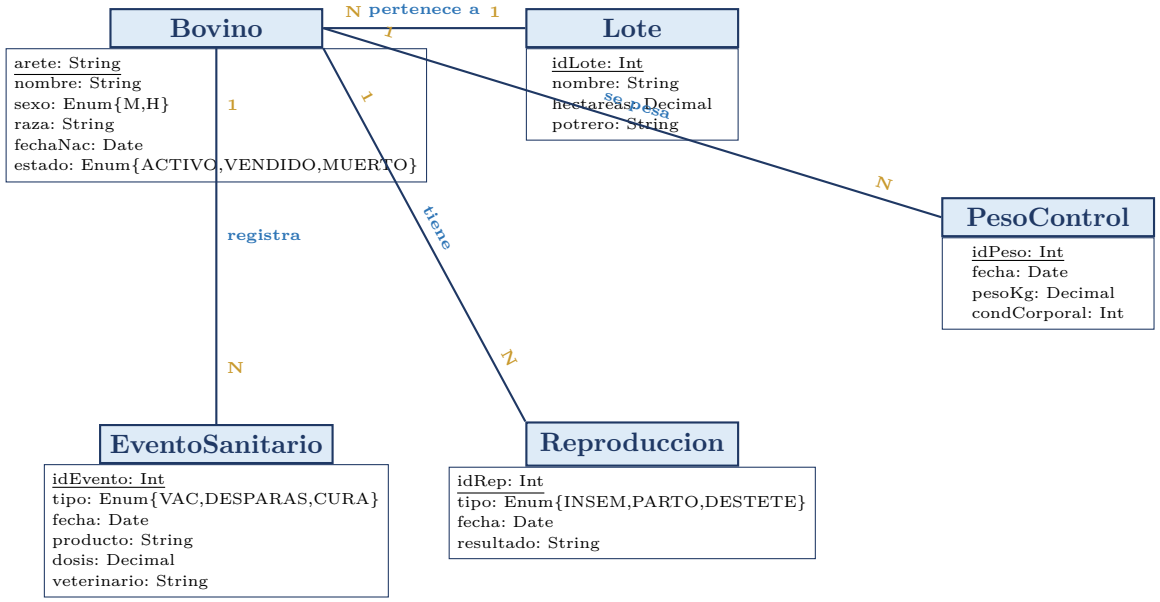


Figura 9.1. *Diagrama ER del sistema de gestión ganadera (PFC — Los Ríos). Notación simplificada Crow's Foot. El arete es la clave primaria del bovino, conforme al sistema de identificación AGROCALIDAD. Adaptado de [1, 76].*

Reglas de mapeo ER → clases UML

El paso de un diagrama ER a un diagrama de clases UML de dominio sigue reglas sistemáticas [70, 71]:

Tabla 9.2. *Reglas de mapeo de ER a diagrama de clases UML [70, 71, 77].*

Elemento ER	Elemento UML	Ejemplo en sistema ganadero
Entidad	Clase	`Bovino`, `Lote`, `EventoSanitario`
Atributo	Atributo de clase (con tipo UML)	`pesoKg: Decimal`, `raza: String`
Atributo clave (PK)	Atributo con estereotipo «PK»	`«PK» arete: String`
Relación 1:N	Asociación con multiplicidad 1..*	Bovino `1 --- 0..*` EventoSanitario
Relación N:M	Clase de asociación o clase intermedia	Bovino–Tratamiento con atributos propios
Entidad débil	Clase dependiente (composición •)	PesoControl depende de Bovino; no existe sin él
Atributo derivado	Operación de clase	`edadMeses(): Int` calculado desde `fechaNac`

Diagrama de clases UML: sistema de clínica veterinaria (PFC)

Ejemplo en Ecuador

Contexto — PFC: Sistema para clínica veterinaria en Quevedo. Una clínica veterinaria atiende perros, gatos, aves y pequeños roedores. Necesita gestionar fichas de pacientes, historial de consultas, recetas, citas y pagos. El dueño de la mascota puede consultar el historial y las citas desde una app móvil.

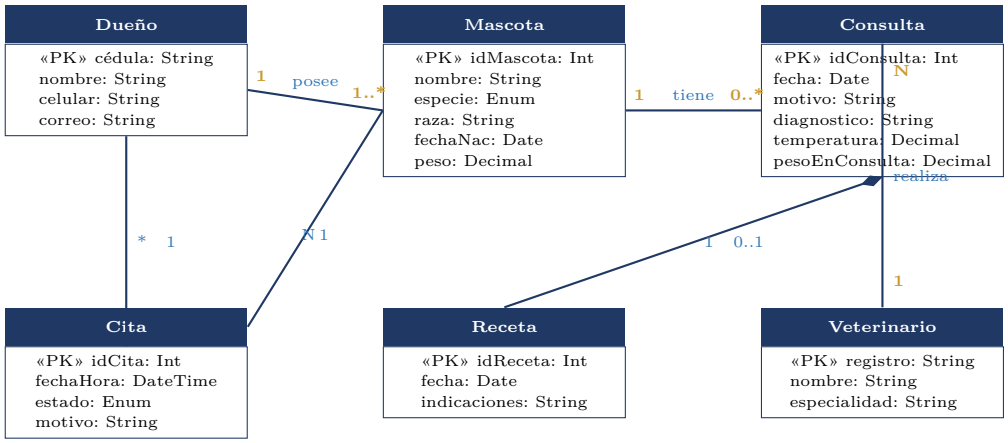


Figura 9.2. *Diagrama de clases UML (dominio) del sistema de clínica veterinaria (PFC — Quevedo). La composición entre Consulta y Receta indica que la receta no existe sin una consulta. Adaptado de [70, 71].*

Derivación de RF desde el modelo de clases. Cada clase, atributo y relación del diagrama de clases de dominio es un candidato a requisito verificable [1, 10]:

- Clase `Mascota`: **RF-01** El sistema deberá permitir registrar una mascota con los atributos: nombre, especie, raza, fecha de nacimiento y peso actual.
- Relación `Dueño posee Mascota`: **RF-02** El sistema deberá asociar cada mascota a exactamente un dueño identificado por su cédula.
- Multiplicidad `0..*`: **RF-03** El sistema deberá admitir que un dueño registre múltiples mascotas.
- Atributo derivado: **RF-04** El sistema deberá calcular y mostrar la edad en meses de cada mascota a partir de su fecha de nacimiento.

9.2 Subtema 10.2. Modelos funcionales: DFD y actividades UML

9.2.1 El diagrama de flujo de datos en IR

Definición

Diagrama de flujo de datos (DFD). Notación propuesta por DeMarco [78] y Yourdon [79] que representa el sistema como un conjunto de procesos que transforman flujos de datos, reciben datos de fuentes externas (entidades) y los almacenan en depósitos. En IR se usa el **DFD esencial** (sin tecnología), que modela la función que el sistema *debe* cumplir, no el cómo lo hace.

Niveles del DFD

Tabla 9.3. *Niveles del DFD y su uso en la especificación de requisitos [3, 78, 79].*

Nivel	Nombre	Contenido	RF derivados
0	Diagrama de contexto	El sistema como una única burbuja; todas las entidades externas y flujos.	Identificar las interfaces del sistema.
1	DFD nivel 1	Los procesos principales del sistema (4–9 burbujas).	RF principales por proceso.
2	DFD nivel 2	Descomposición de cada proceso en sub-procesos.	RF detallados; reglas de negocio.
3+	DFD primitivo	Proceso atómico no descomponible; especificado en mini-spec.	Algoritmo o regla de negocio exacta.

DFD nivel 0 y nivel 1: sistema de gestión de camaroneras (PFC)

Ejemplo en Ecuador

Contexto — PFC: Sistema de gestión de piscinas camaroneras (Provincia de El Oro). Una empresa familiar de acuicultura con 12 piscinas de camarón *Litopenaeus vannamei* necesita controlar: la calidad del agua (oxígeno disuelto, pH, salinidad, temperatura), el ciclo de alimentación, los tratamientos con probióticos y antibióticos, el crecimiento de la biomasa (muestreos semanales) y la cosecha. Los datos de calidad del agua provienen de sensores IoT instalados en cada piscina; los muestreos los registra el técnico acuícola en tablet.

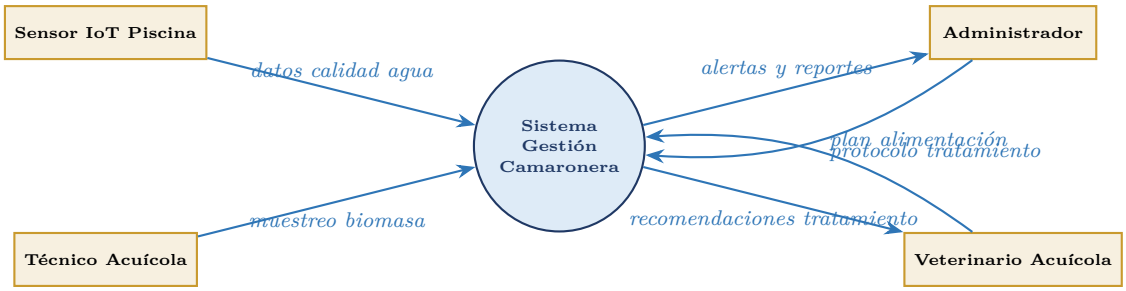


Figura 9.3. *DFD Nivel 0 (diagrama de contexto) del sistema de gestión de camaroneras. Las entidades externas identifican las interfaces del sistema y generan directamente los RF de integración. Adaptado de [78, 79].*

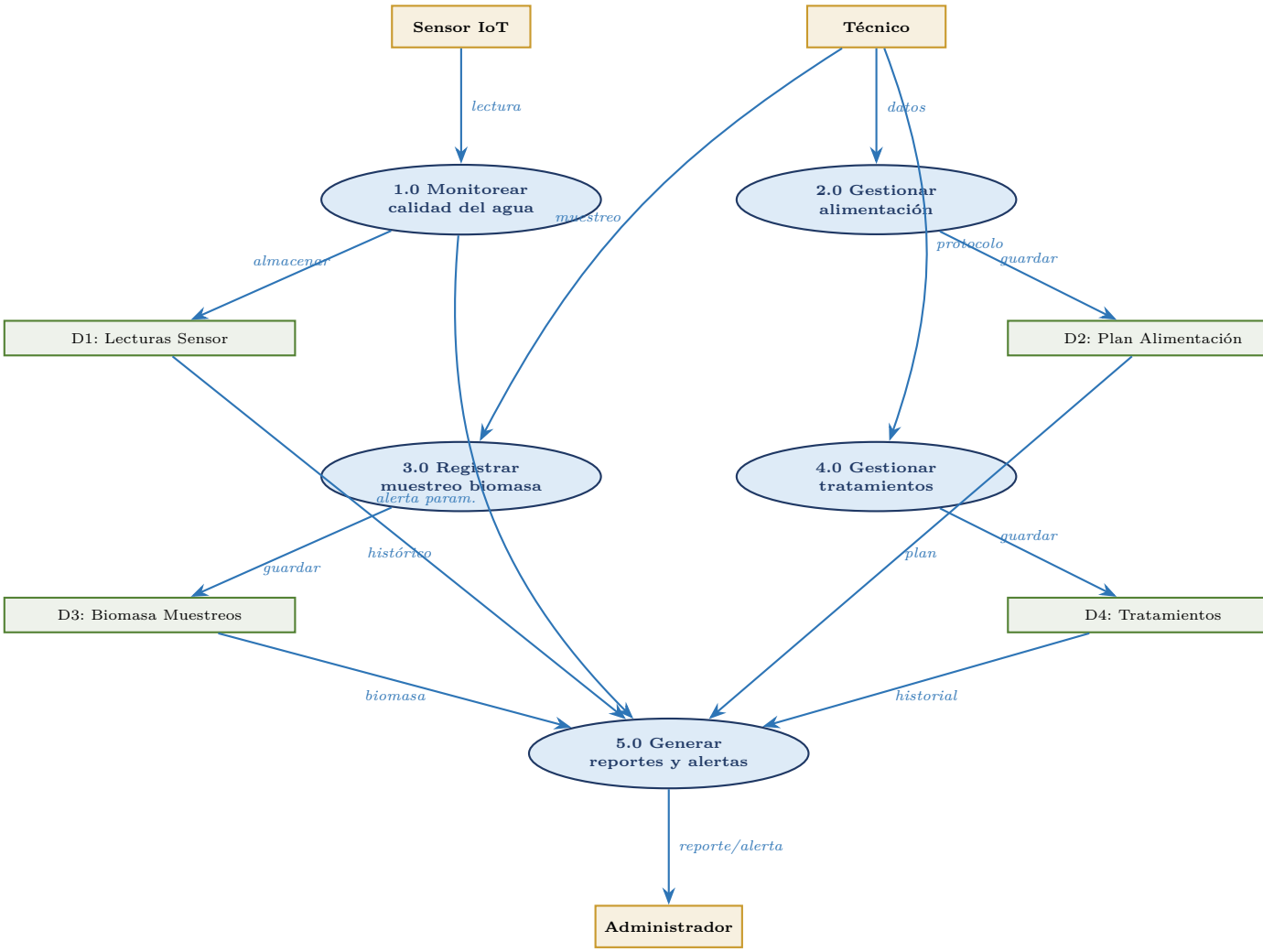


Figura 9.4. *DFD Nivel 1 del sistema de camaroneras. Cada burbuja de proceso genera al menos un RF; cada depósito de datos genera entidades en el ER. Adaptado de [1, 78, 79].*

RF derivados del DFD nivel 1 — camaroneras.

- P1 → **RF-C01**: Cuando el sensor IoT transmita una lectura de oxígeno disuelto inferior a 4,0 mg/L, el sistema deberá generar una alerta crítica en menos de 30 segundos y notificar al técnico de turno por SMS.
- P2 → **RF-C02**: Cuando el técnico registre el plan de alimentación diario, el sistema deberá calcular la ración en kg por piscina usando la fórmula biomasa estimada × tasa de alimentación configurable.
- P3 → **RF-C03**: El sistema deberá permitir al técnico registrar el resultado del muestreo de biomasa (peso promedio en gramos y densidad estimada por m^2).
- P5 → **RF-C04**: El sistema deberá generar semanalmente el reporte de evolución de biomasa por piscina en formato PDF y XLSX.

9.2.2 Diagramas de actividades UML

Definición

Diagrama de actividades UML. Modelo de comportamiento que describe flujos de trabajo (workflows) mediante acciones, decisiones (guards), bifurcaciones paralelas (fork/join) y señales de envío/recepción. En IR se usa como alternativa al DFD nivel 2 para especificar procesos complejos con concurrencia o condiciones múltiples [70, 71, 77].

Diagrama de actividades: proceso de cosecha de palma africana (PFC)

Ejemplo en Ecuador

Contexto — PFC: Sistema para hacienda de palma africana (Esmeraldas). Una hacienda de 800 hectáreas de palma africana (*Elaeis guineensis*) necesita gestionar el ciclo de cosecha: planificación por lotes, registro del corte por cortador, pesaje en báscula, transporte a la extractora y liquidación de pago al cortador (calculado por kg cosechados). El proceso de cosecha involucra acciones paralelas: el cortador trabaja en campo mientras el pesador registra en báscula y el administrador aprueba los reportes.

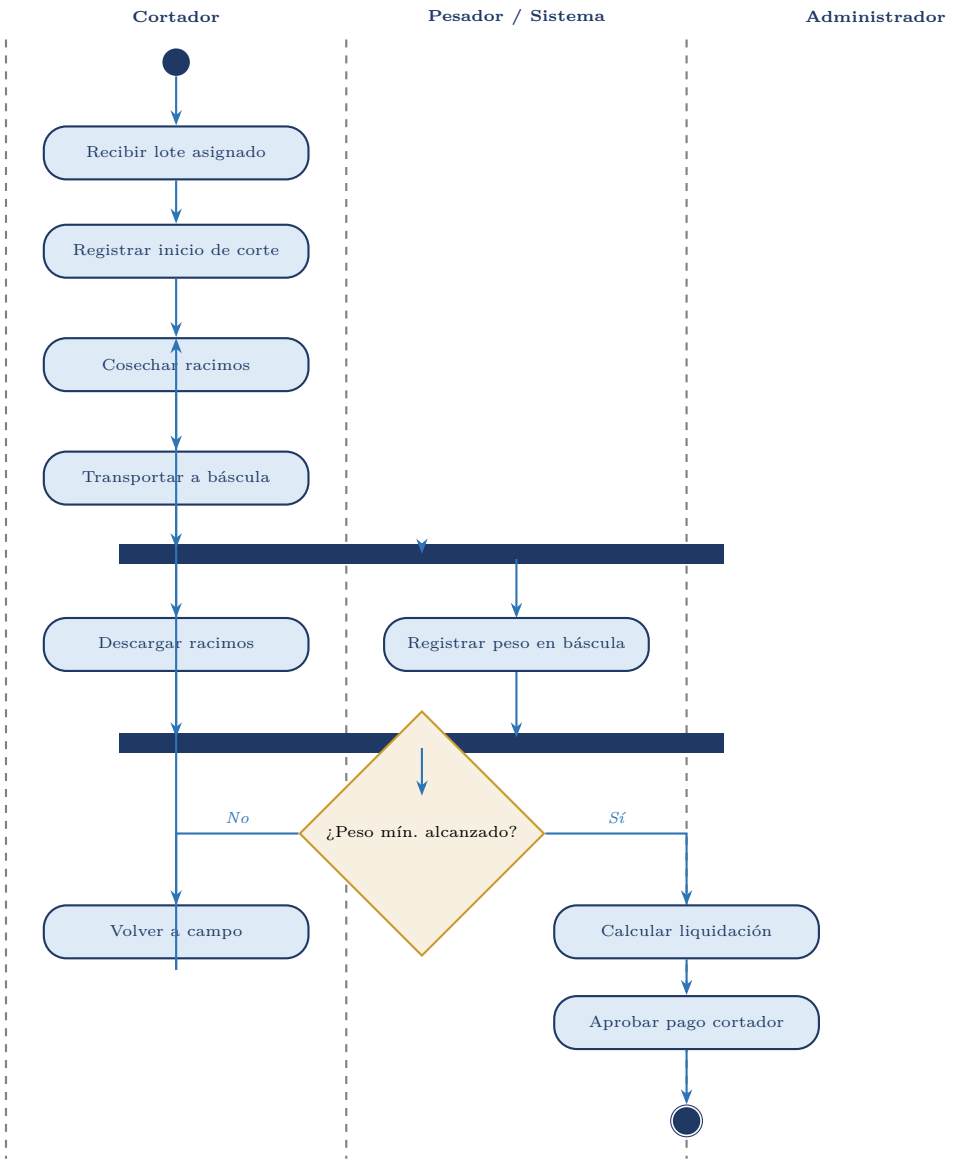


Figura 9.5. *Diagrama de actividades UML del proceso de cosecha de palma africana. El fork indica ejecución paralela (el cortador descarga mientras el pesador registra). La decisión modela la regla de negocio del peso mínimo de cierre de jornada. Cada acción genera RF; el fork genera un RNF de concurrencia. Adaptado de [71, 77].*

RF derivados del diagrama de actividades — palma africana.

- Acción *Registrar inicio de corte*: **RF-P01** El sistema deberá permitir al cortador registrar el inicio de corte mediante código QR del lote, capturando GPS, fecha y hora automáticamente.
- Fork paralelo: **RF-P02** El sistema deberá soportar el registro simultáneo de descargas por múltiples cortadores en la misma báscula.
- Decisión *¿Peso mínimo?*: **RF-P03** El sistema deberá validar al cierre de jornada que el peso total del lote supere el mínimo configurable por el administrador (valor por defecto: 500 kg).
- Acción *Calcular liquidación*: **RF-P04** El sistema deberá calcular el pago del cortador multiplicando el peso neto en kg por la tarifa vigente por tonelada métrica registrada en el sistema.

9.3 Mapa síntesis de la Semana 9

Tabla 9.4. *Mapa de cierre de la Semana 9.*

Subtema	Concepto clave	Artefacto del PFC
10.1 ER	Chen / Crow's Foot; mapeo a clases UML; RF desde entidades.	ER completo del PFC + diagrama de clases.
10.2 DFD	Nivel 0 y 1; DFD esencial; depósitos = entidades ER.	DFD N0 + N1 del proceso principal del PFC.
10.2 Actividades	Carriles, fork/join, decisiones, señales; RF desde acciones.	Diagrama de actividades del proceso clave.

9.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 9 (en clase, 2 horas).

El docente presenta el caso de un sistema de **cuidado de mascotas** a domicilio: una plataforma conecta dueños que necesitan cuidado de sus mascotas con cuidadores certificados, gestiona reservas, seguimiento GPS del paseo, reportes de salud y pagos.

Tareas en equipos de 3:

1. Construir el ER completo con mínimo 6 entidades y todas las cardinalidades.
2. Mapear el ER a diagrama de clases UML añadiendo 3 operaciones derivadas.
3. Construir el DFD nivel 0 (mínimo 4 entidades externas) y nivel 1 (mínimo 5 procesos).
4. Construir el diagrama de actividades del proceso «Reservar servicio de paseo»con carriles para Dueño, Cuidador y Sistema.
5. Derivar al menos 12 RF directamente de los cuatro modelos (3 por modelo).

Práctica / Actividad

Tarea autónoma n.º 9 (5 horas, equipo del proyecto integrador).

Hito 3 del PFC — modelos de datos y función:

1. **Diagrama ER** del dominio del PFC (notación Crow's Foot, mínimo 7 entidades, todas con cardinalidades y atributos clave marcados).
2. **Diagrama de clases UML de dominio** mapeado desde el ER, con multiplicidades UML, tipos de datos y al menos 5 operaciones.
3. **DFD N0 y N1** del proceso principal del PFC.
4. **Diagrama de actividades** del proceso más complejo del PFC (con mínimo 2 carriles, 1 fork/join y 2 decisiones).
5. **Tabla de derivación de RF**: 15 RF derivados de los modelos, con referencia al elemento del modelo que los origina.
6. **Actualizar RTM**: añadir columna de modelo de datos/función.

9.5 Resumen ejecutivo de la semana

- El **ER** modela las entidades del negocio y sus relaciones; el **diagrama de clases UML** de dominio añade operaciones y una semántica orientada a objetos más rica [71, 76].
- El **mapeo ER → clases** sigue reglas sistemáticas: entidad→clase, atributo→atributo, relación 1:N→asociación, relación N:M→clase intermedia [70, 77].
- El **DFD esencial** modela la función sin tecnología; el nivel 0 identifica interfaces, el nivel 1 los procesos principales, y cada burbuja genera al menos un RF [78, 79].
- El **diagrama de actividades UML** extiende el DFD con concurrencia (fork/join) y señales, siendo especialmente útil para procesos IoT y workflows de campo [71, 77].
- Cada elemento de los modelos de datos y función es un **candidato a requisito verificable**, lo que hace que los modelos sean también herramientas de completitud del ERS [1, 2].

Semana 10

Modelos de comportamiento e interrelaciones

El diagrama de clases dice qué existe; el DFD dice qué se transforma; el statechart dice cuándo y cómo reacciona. Solo juntos describen un sistema completamente. La arquitectura de la información, la arquitectura funcional y la arquitectura de comportamiento son las tres dimensiones inseparables de todo ERS.

adaptado de Pohl [1]

Objetivos de aprendizaje de la semana

Al finalizar esta décima semana, el estudiante estará en capacidad de:

1. Construir **autómatas finitos deterministas** (AFD) para modelar el comportamiento de entidades reactivas simples de sistemas IoT [1, 2].
2. Construir **statecharts de Harel** con estados compuestos, transiciones guardadas y acciones de entrada/salida para entidades complejas de sistemas agropecuarios y veterinarios [1, 72].
3. Construir **diagramas de máquina de estados UML** para especificar el ciclo de vida de los objetos clave del PFC [71, 77].
4. Aplicar el **principio de coherencia** entre los tres modelos (datos, función, comportamiento) mediante la tabla de cruce obligatoria [1, 2, 66].
5. Construir la vista de **interrelaciones** del ERS, asegurando que toda entidad del ER tiene su statechart si tiene ciclo de vida relevante [1, 10, 52].
6. Derivar **RF de comportamiento** a partir de transiciones, guardas y acciones de los statecharts del PFC [1, 2].

10.1 Subtema 10.3. Autómatas finitos, statecharts y máquinas de estado UML

10.1.1 De los autómatas finitos a los statecharts

Autómatas finitos deterministas en IR

Definición

Autómata finito determinista (AFD). Modelo matemático de comportamiento compuesto por: un conjunto finito de estados Q, un alfabeto de eventos Σ, una función de transición $\delta : Q \times \Sigma \to Q$, un estado inicial $q_0 \in Q$ y un conjunto de estados de aceptación $F \subseteq Q$. En IR se usa para modelar entidades reactivas con comportamiento simple y sin concurrencia [1, 2].

Autómata finito: sensor IoT de piscina camaronera

Ejemplo en Ecuador

PFC — Camaroneras: comportamiento del nodo sensor IoT. Cada piscina tiene un nodo Arduino con sensores de oxígeno disuelto, pH, salinidad y temperatura. El nodo tiene cuatro estados operacionales y responde a seis eventos: encendido, lectura exitosa, lectura fallida, batería baja, reconexión GSM y apagado.

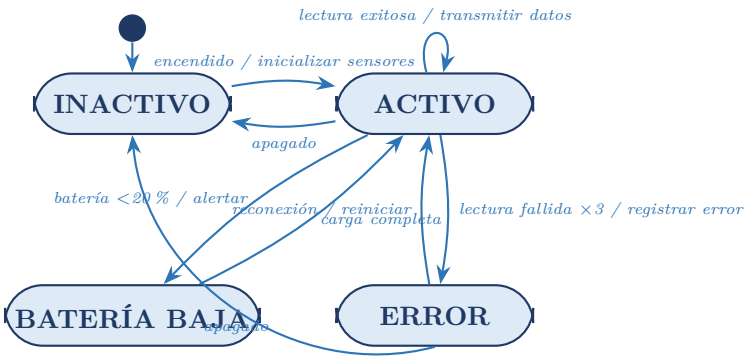


Figura 10.1. *Autómata finito determinista del nodo sensor IoT de la piscina camaronera. Cada transición genera un RF de comportamiento: el evento es el disparador; la acción tras / es lo que el sistema ejecuta. Adaptado de [1, 2].*

Tabla 10.1. *Tabla de transiciones del AFD del sensor IoT y los RF derivados.*

Estado origen	Evento	Estado destino	RF derivado
ACTIVO	lectura exitosa	ACTIVO	Cuando el sensor genere una lectura, el sistema deberá transmitirla al servidor en menos de 5 s.
ACTIVO	lectura fallida ×3	ERROR	Si 3 lecturas consecutivas fallan, el sistema deberá registrar el error con timestamp y piscina.
ACTIVO	batería <20 %	BATERÍA BAJA	Cuando el nivel de batería sea inferior al 20 %, el sistema deberá alertar al técnico por SMS.
ERROR	reconexión	ACTIVO	Cuando la conectividad se restaure, el sistema deberá reintentar la transmisión de las lecturas pendientes.

Statecharts de Harel: estados compuestos y concurrencia

Definición

Statechart (Harel, 1987) [72]. Extensión de los AFD con tres capacidades que los AFD no tienen: (1) **jerarquía**: un estado puede contener sub-estados (estados compuestos), evitando la explosión de transiciones; (2) **concurrencia**: regiones ortogonales que se ejecutan en paralelo; (3) **historia**: pseudoestados H y H* que recuerdan el sub-estado activo antes de salir de un estado compuesto.

Statechart: ciclo de vida del animal bovino (PFC — ganadería)

Ejemplo en Ecuador

PFC — Ganadería: el bovino tiene un ciclo de vida complejo. Desde su nacimiento hasta la venta o muerte, un bovino pasa por fases que determinan qué operaciones puede hacer el sistema sobre él: pesar, vacunar, inseminar, destetar, vender. Las transiciones tienen guardas (condiciones booleanas) y acciones (lo que el sistema ejecuta).

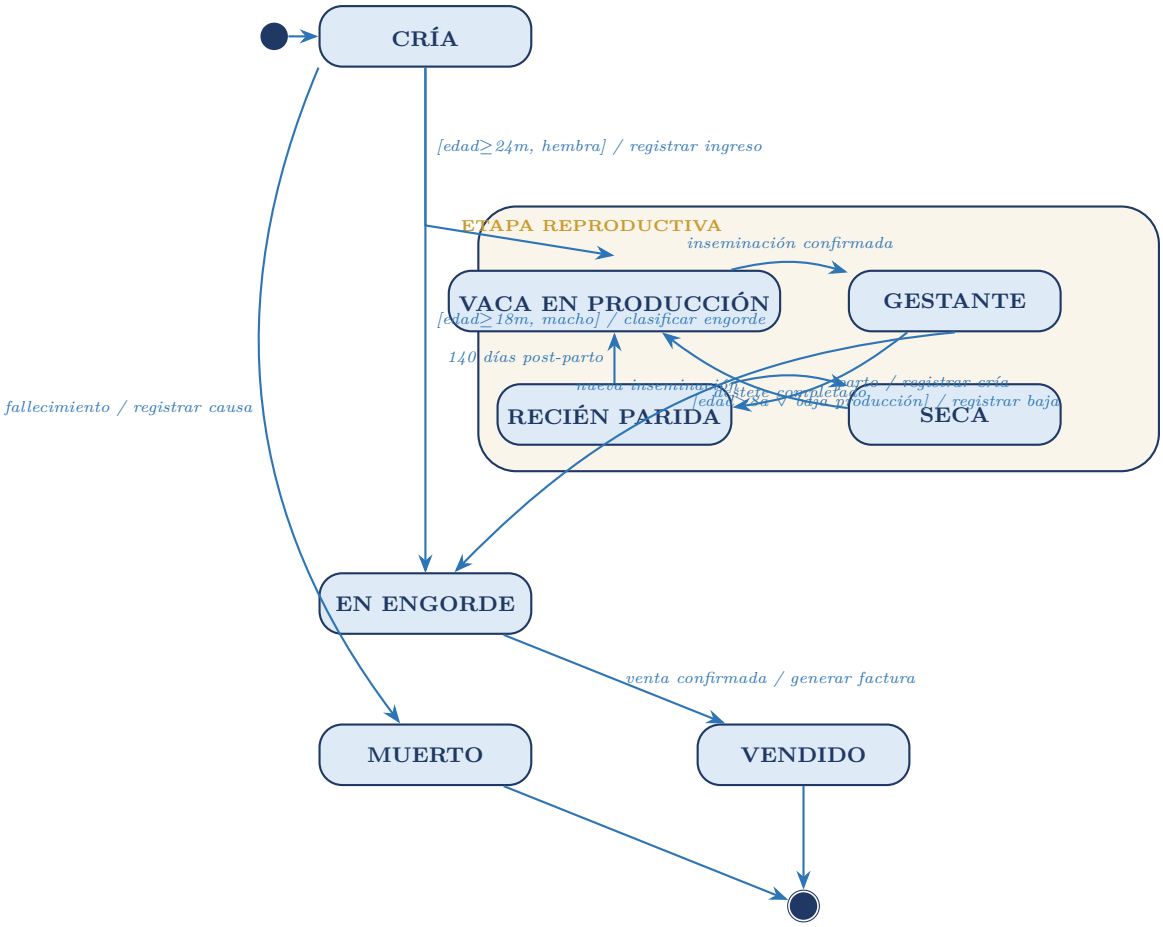


Figura 10.2. *Statechart (Harel) del ciclo de vida del bovino en el PFC de ganadería. El estado compuesto Etapa Reproductiva agrupa los sub-estados reproductivos, evitando repetir transiciones. Las guardas (entre corchetes) generan reglas de negocio; las acciones (tras /) generan RF. Adaptado de [1, 72].*

RF derivados del statechart del bovino.

- Guarda `[edad ≥ 24m, hembra]`: **RF-G01** El sistema deberá verificar que la hembra tiene al menos 24 meses antes de permitir registrar una inseminación.
- Transición *parto / registrar cría*: **RF-G02** Cuando se registre un parto, el sistema deberá crear automáticamente una nueva ficha de bovino para la cría con estado CRÍA.
- Guarda `[edad > 8a ∨ baja producción]`: **RF-G03** El sistema deberá alertar al administrador cuando un bovino supere 8 años o su producción láctea caiga más del 40 % respecto al promedio de los 6 meses anteriores.
- Acción *generar factura*: **RF-G04** Cuando la venta de un bovino sea confirmada, el sistema deberá generar automáticamente la factura electrónica conforme al esquema del SRI [16].

Diagrama de máquina de estados UML: gestión de citas veterinarias

Ejemplo en Ecuador

PFC — Clínica veterinaria: la cita tiene un ciclo de vida bien definido. Una cita puede ser solicitada por el dueño, confirmada o rechazada por la clínica, atendida, marcada como no asistida (no-show) o cancelada en cualquier momento antes de la atención. El sistema debe enviar recordatorios automáticos 24 horas y 1 hora antes.

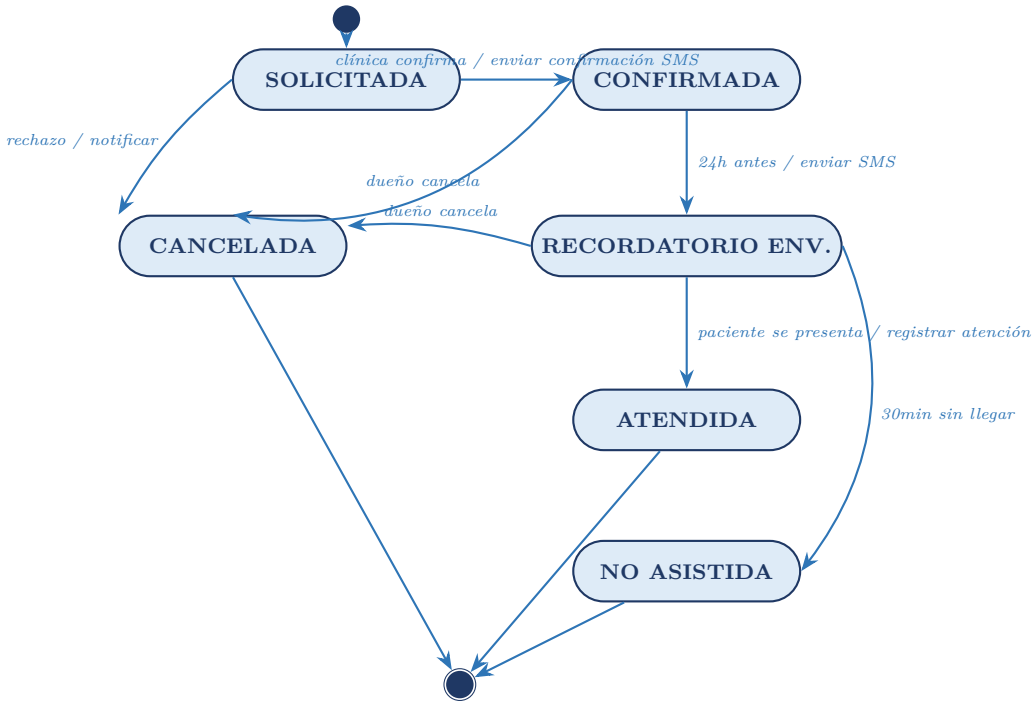


Figura 10.3. *Diagrama de máquina de estados UML para la entidad* `Cita` *en el sistema de clínica veterinaria. Los estados ATENDIDA, NO ASISTIDA y CANCELADA son estados finales distintos, necesarios para las estadísticas de gestión de la clínica. Adaptado de [72, 77].*

10.2 Subtema 10.4. Interrelaciones entre las tres perspectivas

10.2.1 El principio de coherencia de Pohl

Pohl [1] establece que un ERS completo debe ser **coherente** en tres dimensiones: (1) toda entidad del ER con ciclo de vida no trivial tiene un statechart; (2) todo proceso del DFD que transforma datos opera sobre entidades del ER; (3) todo estado del statechart está relacionado con atributos de la clase correspondiente en el diagrama de clases.

Tabla 10.2. *Tabla de interrelaciones entre los tres modelos del PFC de ganadería. Cada celda vacía indica una posible omisión que el analista debe justificar o corregir [1, 2, 66].*

Entidad / Proceso	Clase UML	Proceso DFD	Statechart
Bovino	`Bovino` (Sem. 9)	P2: Registrar muestreo	Fig. 10.2
EventoSanitario	`EventoSanitario`	P3: Gestionar vacunación	No necesita (entidad de registro)
PesoControl	`PesoControl`	P2: Muestreo biomasa	No necesita (dato histórico)
Lote	`Lote`	P1: Gestionar lotes	No necesita (entidad de clasificación)
Alerta	`Alerta`	P5: Generar alertas	PENDIENTE → ENVIADA → RECONOCIDA
Reproduccion	`Reproduccion`	P4: Control reproductivo	Solo si hay flujo de aprobación

10.2.2 Checklist de coherencia del ERS (vista de modelos)

Antes de dar el ERS por completo, el analista debe verificar los siguientes criterios de coherencia entre los tres modelos [1, 2]:

Tabla 10.3. *Checklist de coherencia de los tres modelos del ERS [1, 2, 10].*

#	Criterio de coherencia	Estado
1	Todo depósito de datos del DFD corresponde a una entidad del ER.	✓ / ✗
2	Todo flujo de datos entre procesos corresponde a un atributo de clase.	✓ / ✗
3	Toda entidad con atributo de estado tiene un statechart.	✓ / ✗
4	Todo estado del statechart corresponde a un valor del Enum en la clase.	✓ / ✗
5	Toda transición del statechart corresponde a un proceso del DFD o a una acción del CU.	✓ / ✗
6	Todo RF tiene al menos un elemento de modelo que lo justifica (clase, proceso, estado o transición).	✓ / ✗
7	Todo elemento de modelo tiene al menos un RF trazado en la RTM.	✓ / ✗
8	Las multiplicidades ER son consistentes con las cardinalidades en el DFD (mismas relaciones).	✓ / ✗

10.2.3 Integración de los cuatro modelos en el ERS del PFC

Ejemplo en Ecuador

PFC — Sistema de gestión de aulas inteligentes con IoT (UTEQ). La UTEQ quiere automatizar el control de iluminación, aire acondicionado, proyector y control de acceso en las aulas de la FCI mediante nodos IoT Raspberry Pi. El sistema debe registrar ocupación en tiempo real, ajustar parámetros de confort según el número de ocupantes detectado por sensor PIR y generar reportes de consumo eléctrico para el Departamento de Mantenimiento.

Los cuatro modelos del ERS de este PFC se articulan así:

- **ER**: entidades Aula, Dispositivo (proyector, AC, lámpara), SesiónClase, Reserva, Ocupación, Alerta.
- **DFD N1**: P1 Monitorear ocupación, P2 Controlar dispositivos, P3 Gestionar reservas, P4 Generar reportes de consumo.
- **Statechart Dispositivo**: APAGADO→ENCENDIDO→EN_STANDBY (timer 15 min sin ocupación)→APAGADO.
- **Actividades**: proceso de inicio de sesión de clase (el docente escanea QR → sistema enciende proyector y AC → detecta ocupantes → ajusta intensidad lumínica).

La tabla de interrelaciones cruza estas 6 entidades con los 4 procesos del DFD y los 3 statecharts (Aula, Dispositivo, Reserva), verificando que ningún proceso opera sobre datos sin entidad ER ni ninguna entidad relevante carece de statechart.

10.3 Mapa síntesis de la Semana 10

Tabla 10.4. *Mapa de cierre de la Semana 10.*

Subtema	Concepto clave	Artefacto del PFC
10.3 AFD	Estados, δ, guardas; tabla de transiciones = RF.	AFD del componente IoT más complejo del PFC.
10.3 Statechart	Jerarquía, concurrencia, historia; guardas booleanas.	Statechart de la entidad central del PFC.
10.3 STM UML	Notación UML; pseudoestados; estados finales múltiples.	STM UML de la entidad con ciclo de vida.
10.4 Interrelaciones	Checklist 8 criterios; tabla de cruce; coherencia ER–DFD–SC.	Tabla de interrelaciones completa del PFC.

10.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 10 (en clase, 2 horas).

Caso: sistema de cuidado de mascotas a domicilio (continuación de la Semana 9).

Tareas en equipos de 3:

1. Construir el statechart de la entidad `Servicio` (estados: SOLICITADO, CONFIRMADO, EN CURSO, COMPLETADO, CANCELADO) con guardas y acciones en cada transición.
2. Construir el AFD del nodo GPS del cuidador durante un paseo (estados: REPOSO, RASTREANDO, SIN SEÑAL, ALERTADO) con tabla de transiciones.
3. Completar la tabla de interrelaciones de su PFC aplicando los 8 criterios del checklist de coherencia.
4. Derivar mínimo 8 RF de comportamiento desde los statecharts y el AFD.
5. Actualizar la RTM: añadir columna de statechart/AFD con los IDs de los estados/transiciones que originan cada RF.

Práctica / Actividad

Tarea autónoma n.º 10 (5 horas, equipo del proyecto integrador).

Hito 3 del PFC — modelos de comportamiento e interrelaciones:

1. **AFD** del componente IoT o entidad reactiva más simple del PFC (mínimo 4 estados, 6 transiciones, tabla de transiciones completa).
2. **Statechart** de la entidad central del PFC (mínimo 5 estados, con al menos 1 estado compuesto si aplica, guardas booleanas y acciones).
3. **STM UML** de una segunda entidad con ciclo de vida relevante.
4. **Tabla de interrelaciones** cruzando todas las entidades del ER con los procesos del DFD y los statecharts.
5. **Verificación del checklist de coherencia**: marcar cada uno de los 8 criterios como cumplido o pendiente, con justificación de los pendientes.
6. **RTM actualizada**: todas las columnas (origen, CU, clase, DFD, statechart, caso de prueba, estado).

10.5 Resumen ejecutivo de la semana

- Los **AFD** modelan el comportamiento reactivo simple de entidades y componentes IoT; su tabla de transiciones es una fuente sistemática de RF de comportamiento [1, 2].
- Los **statecharts** [72] superan los AFD con jerarquía (estados compuestos), concurrencia (regiones ortogonales) e historia, permitiendo modelar ciclos de vida complejos sin explosión de transiciones.
- Los **STM UML** [77] son la implementación estándar de los statecharts en las herramientas CASE y en los ERS profesionales.

- El **principio de coherencia** [1] exige que ER, DFD y statecharts sean mutuamente consistentes; la tabla de interrelaciones y el checklist de 8 criterios son las herramientas de verificación.
- El **checklist de coherencia** convierte la revisión de modelos en una actividad sistemática y objetiva, equiparable a una inspección Fagan [65] pero aplicada a los modelos, no al texto.

Semana 11

Validación de requisitos

Verificación pregunta «¿estamos construyendo el sistema correctamente?»; validación pregunta «¿estamos construyendo el sistema correcto?». Solo la validación puede responder si el ERS que hemos escrito es realmente lo que el cliente necesita — y esa respuesta solo la puede dar el cliente.

adaptado de Wiegers y Beatty [10]

Objetivos de aprendizaje de la semana

Al finalizar esta undécima semana, el estudiante estará en capacidad de:

1. Distinguir **verificación** de **validación** de requisitos y aplicar los seis principios de validación de Pohl [1-3].
2. Planificar y facilitar un **walkthrough** y una **revisión estructurada por pares** de un ERS real del PFC [1, 10, 52].
3. Aplicar la **inspección Fagan** a un fragmento del ERS, calculando la densidad de defectos y el ROI de la inspección [10, 65].
4. Usar el **prototipado de validación** para confirmar que los modelos UML del PFC reflejan correctamente los requisitos del cliente [1, 10].
5. Diseñar **pruebas de aceptación** funcionales y no funcionales vinculadas a los RF/RNF del ERS del PFC [2, 10, 60].
6. Calcular **métricas de calidad** del ERS usando densidad de defectos, cobertura de criterios de aceptación y tasa de ambigüedad [1, 10, 63].

11.1 Subtema 11.1. Motivación y principios de validación

11.1.1 ¿Por qué validar los requisitos?

Definición

Validación de requisitos. Proceso sistemático de confirmar que los artefactos de requisitos (ERS, modelos, historias de usuario) representan fielmente las necesidades reales de los stakeholders y que el sistema especificado, si se construye tal como está descrito, producirá el valor de negocio esperado [1-3].

El argumento económico de la validación temprana es el más poderoso para convencer a los clientes de invertir tiempo en ella. Boehm y Turner [7] documentaron que el costo de corregir un defecto en los requisitos crece exponencialmente con el tiempo: un defecto detectado en la revisión del ERS cuesta 1 unidad; en diseño, 6 unidades; en código, 15; en pruebas de sistema, 40; y en producción, hasta 200.

Tabla 11.1. *Costo relativo de corrección de defectos de requisitos según la fase de detección [7, 9, 35].*

Fase de detección	Costo relativo	Ejemplo en PFC ganadero
Revisión del ERS	1×	Se detecta que falta el RF de «exportar a AGROCALIDAD»: 2 horas de trabajo.
Diseño de base de datos	6×	El esquema de la BD no soporta el historial de aretes: refactorizar tablas.
Codificación	15×	El módulo de vacunación no tiene el campo veterinario: reescribir capa de datos.
Pruebas de sistema	40×	El cliente rechaza el prototipo: reescribir módulo completo.
Producción	200×	AGROCALIDAD rechaza los informes: multa + reprogramación urgente.

11.1.2 Los seis principios de validación de Pohl

Pohl [1] establece seis principios que deben guiar toda actividad de validación de requisitos:

1. **Involucrar a los stakeholders correctos.** La validación debe ser realizada por quienes tienen el conocimiento de dominio y la autoridad para confirmar que el requisito es correcto. Validar un ERS ganadero con ingenieros de software en lugar del mayordomo y el veterinario es un error frecuente.
2. **Separar el hallazgo de la corrección.** En una sesión de revisión, el objetivo es *encontrar* defectos, no corregirlos. Mezclar ambas actividades hace las sesiones ineficientes y cargadas de tensión.
3. **Validar en múltiples perspectivas.** El mismo ERS debe ser validado desde la perspectiva de datos (¿el ER es completo?), de función (¿el DFD captura todos los procesos?) y de comportamiento (¿el statechart refleja la realidad del campo?).
4. **Usar representaciones múltiples.** Los stakeholders no técnicos validan mejor a través de prototipos y escenarios narrativos que leyendo tablas de requisitos. Los stakeholders técnicos validan mejor a través de modelos UML y casos de prueba.
5. **Documentar los resultados.** Cada defecto encontrado debe quedar registrado con: ID, descripción, severidad, tipo, responsable de corrección y estado. Sin documentación, la revisión es un evento sin trazabilidad.
6. **Iterar hasta alcanzar el criterio de calidad.** La validación no es un evento único. Se itera hasta que la densidad de defectos quede por debajo del umbral acordado (típicamente <1 defecto/página en ERS de baja criticidad; <0.1 en sistemas médicos o de seguridad).

11.2 Subtema 11.2. Walkthroughs, revisiones estructuradas e inspecciones

11.2.1 Las tres modalidades de revisión de requisitos

Tabla 11.2. *Comparación de las tres modalidades de revisión de requisitos [1, 10, 52, 65].*

Modalidad	Características	Costo	Eficacia
Walkthrough	El autor guía al grupo; informal; sin roles fijos; el autor puede defender.	Bajo (2–4 h)	Media (detecta 30–40 % de defectos)
Revisión por pares estructurada	Revisores leen independientemente; usan checklist; reunión de discusión sin defensa del autor.	Medio (4–8 h)	Alta (50–70 %)
Inspección Fagan [65]	Proceso formal: 6 roles, 4 fases, métricas, criterio de re-inspección.	Alto (8–20 h)	Muy alta (70–90 %)

La inspección Fagan aplicada al ERS del PFC

Definición

Inspección Fagan. Proceso formal de revisión propuesto por Michael Fagan en IBM en 1976 [65], compuesto por seis roles (*moderador, autor, lector, inspectores, registrador*) y cuatro fases (*planning, overview, inspection meeting, rework + follow-up*), con métricas obligatorias de densidad de defectos y tiempo de preparación.

Tabla 11.3. *Las cuatro fases de la inspección Fagan aplicada al ERS del PFC de camaroneras [10, 65].*

#	Fase	Actividades	Salida
1	Planning	El moderador selecciona el fragmento del ERS a inspeccionar (máx. 20 páginas), asigna roles e inspectores.	Paquete de inspección: fragmento + checklist + formularios.
2	Overview	El autor (analista del PFC) explica el contexto del ERS en ≤60 min. Los inspectores no preguntan; solo escuchan.	Comprensión compartida del contexto.
3	Inspection meeting	Los inspectores leen el ERS de forma independiente (mín. 2 h). En la reunión (mín. 90 min) el lector parafrasa, los inspectores reportan defectos. El autor solo escucha.	Lista de defectos con ID, tipo y severidad.
4	Rework + follow-up	El autor corrige. El moderador verifica que las correcciones son correctas. Si la densidad de defectos fue >3/pág., se reinspecciona.	ERS corregido + métricas de inspección.

Checklist de inspección de requisitos para sistemas agropecuarios

Ejemplo en Ecuador

Checklist de inspección Fagan — ERS del PFC de palma africana.

#	Criterio a verificar	¿Cumple?
1	¿Cada RF usa el verbo modal **deberá** y especifica el sujeto (sistema o actor)?	Sí / No
2	¿Cada RNF tiene un criterio cuantificable (número + unidad + percentil)?	Sí / No
3	¿Cada RF tiene al menos un criterio de aceptación GWT verificable?	Sí / No
4	¿El ERS usa terminología del dominio (p.ej., «tonelada métrica», «lote de cosecha») consistente con el glosario?	Sí / No
5	¿Los RF mencionan las integraciones con sistemas externos (báscula, GPS, SRI)?	Sí / No
6	¿El ERS especifica los requisitos de operación offline del sistema en campo?	Sí / No
7	¿Los RF de reportes especifican el formato (PDF, XLSX, CSV), la frecuencia y el destinatario?	Sí / No
8	¿El ERS cubre los escenarios de pérdida de conectividad GSM en campo?	Sí / No

Métricas de la inspección.

- **Densidad de defectos** = defectos encontrados / páginas del ERS. Objetivo: <3 defectos/página.
- **Tiempo de preparación individual** = horas invertidas por cada inspector leyendo el ERS. Referencia: 0.5–2 h/página.
- **Tasa de inyección** = defectos encontrados / hora de inspección. Referencia: 1–3 defectos/hora.
- **Criterio de re-inspección**: si densidad >3/página, la sección se vuelve a inspeccionar tras la corrección [65].

11.3 Subtema 11.3. Prototipado como técnica de validación

11.3.1 El prototipo de validación vs. el prototipo de elicitación

En la Semana 4 presentamos el prototipo como técnica de *elicitación*. En la Semana 11 lo usamos como técnica de *validación*: a partir de un ERS ya redactado, se construye un prototipo que implementa los RF más críticos y se presenta al cliente para confirmar que la especificación fue comprendida correctamente [1, 10].

Tabla 11.4. *Comparación entre prototipo de elicitación y de validación [1, 10, 57].*

Dimensión	Prototipo de elicitación	Prototipo de validación
¿Cuándo se construye?	Antes del ERS, para descubrir requisitos.	Después del ERS, para confirmar que fue bien interpretado.
¿Qué pregunta responde?	¿Qué necesita el cliente?	¿Es esto lo que especificamos?
¿Qué fidelidad necesita?	Baja (paper mock-up, wireframe).	Media-alta (mock-up interactivo, prototipo funcional parcial).
¿Qué se registra?	Requisitos descubiertos.	Discrepancias entre ERS y expectativa real del cliente.

Validación por prototipo: clínica veterinaria (PFC)

Ejemplo en Ecuador

PFC — Clínica veterinaria: sesión de validación por prototipo. El equipo construyó en Figma un mock-up de alta fidelidad de las pantallas del módulo de citas. Durante la sesión de validación con la veterinaria jefe y la recepcionista (45 minutos), se detectaron las siguientes discrepancias entre el ERS y la expectativa real:

1. **RF-C02 (defecto tipo Omisión)**: El ERS especificaba que la confirmación de cita se enviaba solo por SMS. La veterinaria indicó que el 40 % de sus clientes prefieren WhatsApp. Corrección: añadir canal WhatsApp como opción configurable por el dueño.
2. **RF-C05 (defecto tipo Ambigüedad)**: El ERS decía «el sistema mostrará la disponibilidad del veterinario». La recepcionista aclaró que necesita ver también la sala de atención disponible, no solo el veterinario. Corrección: añadir recurso sala como entidad en el ER.
3. **RNF-C01 (defecto tipo Error de concepto)**: El ERS especificaba tiempo de respuesta <3 s. La veterinaria indicó que el sistema suele usarse desde celulares con señal 3G dentro de la clínica, por lo que <5 s es más realista. Corrección: ajustar el RNF con la condición de red especificada.

Las tres discrepancias fueron documentadas como solicitudes de cambio (CR-01, CR-02, CR-03) y procesadas por el CCB antes de actualizar el ERS.

11.3.2 Validación de modelos UML

La validación no se limita al texto del ERS. Los modelos UML también deben ser validados, aunque con técnicas diferentes [1, 71]:

- **Walkthrough del diagrama de clases**: el analista recorre el modelo con el cliente preguntando por cada clase y asociación: «¿Existe esto en su negocio? ¿Con estas reglas de cardinalidad?»
- **Ejecución de escenarios sobre el statechart**: el analista describe una secuencia real de eventos del negocio y traza el camino por el statechart ante el cliente, verificando que cada estado y transición es reconocido como correcto.
- **Verificación del DFD con trazas de datos**: para cada flujo de datos del DFD, el analista traza con el cliente qué dato real entra, qué proceso lo transforma y qué dato sale, verificando que la transformación modela la realidad.

11.4 Subtema 11.4. Pruebas de aceptación y relación con requisitos

11.4.1 Tipos de pruebas de aceptación

Definición

Prueba de aceptación. Prueba formal ejecutada para determinar si el sistema satisface los criterios de aceptación especificados en el ERS, con la participación del cliente o su representante. La superación de las pruebas de aceptación es la condición contractual para la entrega del sistema [2, 10, 60].

Tabla 11.5. *Tipos de pruebas de aceptación y su relación con los artefactos del ERS [2, 10, 60].*

Tipo	Qué verifica	Artefacto origen	Herramienta
Funcional (FAT)	Que cada RF se ejecuta correctamente en el flujo básico y en los flujos alternativos.	CU + criterios GWT	Prueba manual o Selenium/Cypress.
No funcional (NFAT)	Que los RNF cuantificables se cumplen bajo las condiciones especificadas.	RNF con criterio cuantificable	JMeter (rendimiento), OWASP ZAP (seguridad).
Regresión (RAT)	Que los cambios aprobados por el CCB no rompen RF ya verificados.	RTM + historial de cambios	Suite de pruebas automatizadas.
Usuario (UAT)	Que el sistema es aceptable para el usuario real en su entorno de trabajo.	Criterios GWT + escenarios de usuario	Observación con stakeholder real.

Plan de pruebas de aceptación: sistema de gestión de aulas IoT (PFC)

Ejemplo en Ecuador

PFC — Aulas inteligentes UTEQ: plan de pruebas de aceptación. El siguiente fragmento del plan de pruebas muestra cómo cada prueba deriva directamente de un RF o RNF del ERS, trazando la cadena completa: RF → criterio de aceptación → caso de prueba → resultado esperado.

Tabla 11.6. *Fragmento del plan de pruebas de aceptación del PFC de aulas IoT. Cada caso de prueba está vinculado a un requisito del ERS con trazabilidad directa [2, 60].*

ID prue-ba	RF/RI	Descripción	Resultado esperado	Estado
CP-A01	**RF-A01**	El docente escanea el QR del aula. El sistema encien-de el proyector y el AC en menos de 5 s.	Proyector ON; AC ON; luz ajustada al 80 %. Registro de sesión en BD.	Pendiente
CP-A02	**RF-A02**	El sensor PIR detecta 0 per-sonas durante 15 minutos. El sistema entra en standby.	Proyector OFF; AC en 25°C; luz al 30 %. Alerta a admi-nistrador.	Pendiente
CP-A03	**RNF-01**	30 solicitudes simultáneas de 30 aulas distintas.	95 % respuestas en <2 s me-dido con JMeter.	Pendiente
CP-A04	**RNF-02**	Corte de energía del nodo IoT del aula B201.	Sistema detecta nodo offline en <60 s y notifica a mante-nimiento.	Pendiente
CP-A05 (UAT)	**RF-A03**	El Jefe del Departamento de Mantenimiento genera el reporte mensual de consumo eléctrico.	PDF generado con tabla por aula, total y comparativo mensual en <10 s.	Pendiente

11.4.2 Vínculo entre GWT, RTM y plan de pruebas

La cadena de trazabilidad completa del ERS al plan de pruebas es [2, 10, 60]:



Figura 11.1. *Cadena de trazabilidad desde el requisito hasta el resultado de la prueba de aceptación, pasando por la RTM. Adaptado de [2, 60].*

11.5 Mapa síntesis de la Semana 11

Tabla 11.7. *Mapa de cierre de la Semana 11.*

Subtema	Concepto clave	Artefacto del PFC
11.1 Principios	6 principios Pohl; costo exponencial de defectos.	Plan de validación del PFC con roles y criterios.
11.2 Revisiones	Walkthrough / revisión / Fagan; checklist; métricas.	Acta de revisión + lista de defectos del ERS.
11.3 Prototipado	Proto validación vs. elicitación; discrepancias→CR.	Sesión de validación con Figma + acta de hallazgos.
11.4 Pruebas FAT/NFAT/UAT	GWT→CP; plan de pruebas; RTM de pruebas.	Plan de pruebas de aceptación completo del PFC.

11.6 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 11 (en clase, 2 horas).

El docente entrega a cada equipo el ERS de otro equipo (revisión cruzada).

Tareas:

1. Aplicar la fase 3 (Inspection Meeting) de Fagan: cada revisor usa el checklist de 8 criterios y reporta defectos en formulario estándar.
2. Calcular la densidad de defectos del ERS revisado y el tiempo de preparación individual.
3. Redactar 3 pruebas de aceptación (2 FAT + 1 NFAT) para los RF más prioritarios del ERS revisado.
4. Presentar al equipo autor los defectos encontrados sin defensa ni debate (siguiendo el principio 2 de Pohl).
5. El equipo autor registra cada defecto como solicitud de cambio CR.

Práctica / Actividad

Tarea autónoma n.º 11 (6 horas, equipo del proyecto integrador).

Hito 4 del PFC — validación del ERS:

1. **Sesión de validación** con el cliente real del PFC (mínimo 30 min): usar el mock-up o wireframe como soporte visual; documentar el acta con discrepancias y decisiones.
2. **Revisión estructurada por pares**: otro equipo revisa su ERS usando el checklist de Fagan; documentar defectos con ID, tipo y severidad.
3. **Correcciones**: responder a todos los defectos encontrados como CR y actualizar el ERS a la versión 2.0.
4. **Plan de pruebas de aceptación**: mínimo 15 casos de prueba (10 FAT, 3 NFAT, 2 UAT) con criterios GWT y trazados a la RTM.
5. **Métricas de calidad**: calcular densidad de defectos antes y después de la corrección, y la cobertura de criterios de aceptación.

11.7 Resumen ejecutivo de la semana

- La **validación** confirma que el ERS representa lo que el cliente necesita; la **verificación** confirma que el ERS está bien formado. Ambas son obligatorias [1, 2].
- El **costo exponencial** de los defectos justifica económicamente invertir en revisiones tempranas: 1 hora en inspección evita 100 horas en producción [7, 65].
- La **inspección Fagan** [65] es la técnica más eficaz (70–90 % de defectos detectados) aunque la más costosa; debe aplicarse a los módulos de mayor criticidad del PFC.
- El **prototipo de validación** es la técnica más accesible para los stakeholders no técnicos; sus discrepancias se convierten directamente en solicitudes de cambio para el CCB [1, 10].
- Las **pruebas de aceptación** (FAT, NFAT, UAT) cierran el ciclo del ERS: cada RF tiene su criterio GWT, su caso de prueba y su resultado trazado en la RTM [2, 60].

Semana 12

Gestión de requisitos

Los requisitos no son estáticos. Cambian porque el negocio cambia, porque el cliente aprende con los prototipos, porque la legislación se actualiza y porque el equipo descubre restricciones técnicas invisibles al inicio. Gestionarlos no significa resistirse al cambio: significa que cada cambio sea explícito, evaluado, aprobado y trazado.

adaptado de Wiegers y Beatty [10]

Objetivos de aprendizaje de la semana

Al finalizar esta duodécima semana, el estudiante estará en capacidad de:

1. Explicar la **naturaleza iterativa** del proceso de IR y gestionar el contexto y las actividades de IR a lo largo del ciclo de vida del proyecto [1, 2, 66].
2. Aplicar el proceso de **gestión del cambio** mediante solicitudes de cambio (CR), el análisis de impacto y la operación del CCB en proyectos de PFC reales [2, 10, 52].
3. Construir y mantener los **tres tipos de trazabilidad** (pre-trazabilidad, trazabilidad interna, post-trazabilidad) y visualizarlos mediante una RTM extendida [1, 2, 66].
4. Aplicar **métricas de requisitos** para medir el progreso y la estabilidad del ERS del PFC [1, 10, 80].
5. Gestionar la **configuración del ERS**: versiones, baselines y política de versionado semántico [1, 2, 10].
6. Elaborar la **política de gestión de requisitos** del PFC: roles, herramientas, proceso de cambio y criterios de baseline [1, 2].

12.1 Subtema 12.1. Naturaleza iterativa de la IR y gestión del contexto

12.1.1 La IR no termina cuando empieza el desarrollo

Definición

Gestión de requisitos. Conjunto de actividades continuas que mantienen los artefactos de requisitos actualizados, consistentes y trazados a lo largo de todo el ciclo de vida del proyecto, desde la primera entrevista hasta el mantenimiento del sistema en producción [1, 2, 66].

Hofmann y Lehner [35] documentaron que en proyectos de desarrollo de software medianos, los requisitos cambian un promedio de 35 % entre el baseline inicial del ERS y la entrega del sistema. En proyectos agropecuarios y de IoT, este porcentaje puede ser aún mayor porque:

- El cliente descubre necesidades al ver los primeros prototipos.

- Las condiciones del campo (conectividad, hardware, regulación AGROCALIDAD, normativa del SRI) cambian durante el desarrollo.
- Los integrantes del equipo aprenden sobre el dominio e identifican restricciones técnicas no anticipadas.
- En sistemas IoT, los sensores disponibles en el mercado ecuatoriano pueden no coincidir con los supuestos del ERS inicial.

12.1.2 Ciclo de vida iterativo de la IR

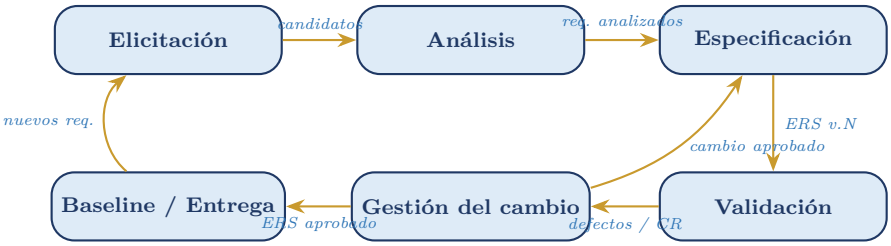


Figura 12.1. *Ciclo de vida iterativo de la IR. La gestión del cambio (centro) actúa como regulador de todas las iteraciones posteriores al primer baseline. Adaptado de [1, 2].*

12.1.3 Gestión del contexto: fuentes de cambio en PFC agropecuarios

Ejemplo en Ecuador

Fuentes de cambio identificadas en PFC de la UTEQ – Los Ríos.

PFC	Fuente de cambio típica	Impacto en el ERS
Gestión ganadera	AGROCALIDAD actualiza el esquema XML del Certificado Zoosanitario.	Cambio en RF-G04 (exportación); posible cambio en ER.
Camaroneras	El proveedor de sensores IoT discontinúa el modelo YSI Pro30; el sustituto usa protocolo MQTT en lugar de HTTP.	Cambio en RNF de integración; nuevo RF de gestión de tópicos MQTT.
Palma africana	La extractora exige reportes de cosecha en formato EXCEL con macros específicas.	Nuevo RF de exportación; cambio en RNF de formato.
Aulas IoT	El Dpto. de TI de la UTEQ exige que todos los nodos IoT usen VPN institucional.	Nuevo RNF de red (VPN); cambio en arquitectura de comunicación.
Clínica veterinaria	La CONEVET publica nuevo reglamento de recetas electrónicas.	Nuevos RF en módulo de recetas; cambio en el ER (Receta).
Mascotas	El cliente añade servicio de guardería nocturna no previsto en el ERS.	Nueva épica; 8 RF nuevos; cambio en el modelo de dominio.

12.2 Subtema 12.2. Gestión del cambio: CCB, proceso y documentación

12.2.1 El proceso de gestión del cambio

Definición

Gestión del cambio de requisitos. Proceso sistemático que garantiza que toda modificación al ERS baseline sea: (1) identificada y documentada formalmente, (2) analizada en cuanto a su impacto, (3) aprobada o rechazada por la autoridad competente (CCB), (4) implementada de forma trazable y (5) comunicada a todos los involucrados [1, 2, 10].

Formulario de solicitud de cambio (CR)

Tabla 12.1. *Formulario de solicitud de cambio (CR) del PFC de clínica veterinaria. Este es el formato mínimo recomendado [2, 10].*

Solicitud de Cambio CR-02 — PFC Clínica Veterinaria Quevedo	
ID	CR-02
Fecha	2025-05-14
Solicitante	Dra. Gisella Moreira, Veterinaria Jefe
Descripción	El sistema debe permitir también la confirmación de cita por WhatsApp Business API, no solo por SMS.
Justificación	El 40 % de los clientes de la clínica prefieren WhatsApp. Se detectó en la sesión de validación del prototipo (acta 2025-05-12).
Requisitos afectados	RF-C02 (canal de confirmación); RNF-C03 (tiempo de notificación).
Modelos afectados	Ninguno (la entidad Cita no cambia; solo el canal de envío).
Impacto estimado	Costo: integración WhatsApp Business API (+3 días). Plazo: sin impacto si se implementa en el sprint 4. Riesgo: bajo.
Decisión del CCB	**APROBADO**. Implementar en sprint 4. Canal WhatsApp es configurable por el dueño de la mascota.
Responsable	Andrés Llerena (desarrollador BackEnd)
Estado	IMPLEMENTADO / VERIFICADO

Proceso completo del CCB para PFC universitario

En un PFC universitario, el CCB mínimo está formado por: el director del PFC (autoridad máxima), el cliente real del proyecto y el líder del equipo de desarrollo. La tabla siguiente muestra el proceso adaptado a este contexto [2, 10]:

Tabla 12.2. *Proceso del CCB adaptado para proyectos de fin de carrera (PFC) de la UTEQ [1, 10].*

#	Paso	Responsable	Tiempo máximo
1	Registrar CR	Cualquier stakeholder	24 h tras identificar el cambio
2	Análisis de impacto	Analista del PFC	48 h: listar RF, modelos, pruebas afectados
3	Reunión CCB	Director + cliente + líder	1 h semanal (viernes)
4	Actualizar ERS	Analista del PFC	72 h tras aprobación
5	Actualizar RTM y notificar	Analista del PFC	Simultáneo con paso 4
6	Verificar corrección	Director del PFC	1 semana

12.3 Subtema 12.3. Seguimiento y trazabilidad de requisitos

12.3.1 Los tres tipos de trazabilidad

Pohl [1] e IEEE 29148:2018 [2] distinguen tres tipos de trazabilidad que toda RTM completa debe implementar:

Tabla 12.3. *Los tres tipos de trazabilidad de requisitos según [1, 2, 66].*

Tipo	Dirección	Pregunta que responde
Pre-trazabilidad (*backward-to*)	Del RF hacia su origen: stakeholder, meta KAOS, documento legal, CR.	¿Por qué existe este requisito? ¿Quién lo originó?
Trazabilidad interna	Del RF hacia otros RF relacionados (generalización, especialización, conflicto).	¿Qué otros requisitos dependen de este o lo contradicen?
Post-trazabilidad (*forward-to*)	Del RF hacia los artefactos que lo satisfacen: modelo UML, código, prueba.	¿Está este requisito implementado y probado?

RTM completa del PFC de mascotas: los tres tipos integrados

Ejemplo en Ecuador

PFC — Plataforma de cuidado de mascotas a domicilio. La plataforma conecta dueños de mascotas con cuidadores certificados en Quevedo. Los servicios incluyen: paseo, guardería diurna, veterinario a domicilio y transporte a clínica. Los pagos se procesan con la pasarela del Banco del Pacífico. Los datos de mascotas están protegidos por la LOPDP.

Tabla 12.4. *RTM completa con los tres tipos de trazabilidad para el PFC de mascotas a domicilio. Cada fila es un RF trazado en las seis dimensiones [1, 2, 66].*

ID	Descripción	Origen (pre)	RF relacionado (int.)	CU/Clas	Prueba	Estado
RF-M01	Reservar servicio de paseo.	StRS-05	**RF-M02**,**RNF-M01**	CU-01, Servicio	CP-M01	Verificado
RF-M02	Rastrear cuidador por GPS durante el paseo.	StRS-06	**RF-M01**	CU-01, AFD-GPS	CP-M02	En revisión
RF-M03	Procesar pago con pasarela BdP.	StRS-12	**RNF-M02**	CU-05, Pago	CP-M03	Pendiente
RF-M04	Registrar mascota con datos básicos.	StRS-02	**RF-M01**	CU-00, Mascota	CP-M04	Verificado
RNF-M01	Tiempo de confirmación de reserva <5 s.	StRS-19	**RF-M01**	Arch-NFR	PR-M01	Pendiente
RNF-M02	Cifrado TLS 1.3 en todos los pagos.	LOPDP-Art15 [14]	**RF-M03**	Arch-SEC	SP-M01	Pendiente

12.3.2 Trazabilidad dirigida por uso

Pohl [1] introduce el concepto de **trazabilidad dirigida por uso**: en lugar de mantener trazas hacia todos los artefactos posibles, se traza solo hacia los artefactos que se usarán activamente para una decisión concreta. Los tres usos más frecuentes son:

1. **Análisis de impacto**: dado un RF que cambia, ¿qué diseño, código y pruebas deben revisarse? → post-trazabilidad.
2. **Auditoría de completitud**: ¿hay pruebas para todos los RF? → cobertura de la columna *Prueba* en la RTM.
3. **Rastreo de origen**: dado un defecto en producción, ¿qué stakeholder originó el RF que lo causó? → pre-trazabilidad.

12.4 Subtema 12.4. Medición, priorización, configuración y baselines

12.4.1 Métricas de gestión de requisitos

Tabla 12.5. *Métricas clave de gestión de requisitos y sus valores de referencia [1, 10, 80].*

Métrica	Fórmula	Valor referencia
Estabilidad de req.	(RF sin cambiar / RF baseline) × 100 %	>80 % en diseño
Tasa de CR aprobados	CR aprobados / CR totales	60–80 %
Cobertura de pruebas	RF con caso de prueba / RF total	100 % antes de entrega
Densidad de defectos	Defectos hallados / páginas del ERS	<3 tras primera revisión
Tiempo medio de CR	∑ (fecha resolución − fecha CR) / N CR	<5 días hábiles
Volatilidad	RF cambiados en el sprint / RF planificados	<15 % por sprint

Monitoreo de estabilidad: PFC de aulas IoT

Ejemplo en Ecuador

Ejemplo de monitoreo de estabilidad de requisitos — PFC Aulas IoT UTEQ. El equipo tenía 42 RF en el baseline v1.0. Al finalizar el sprint 3:

- 6 RF cambiaron (2 por cambio de protocolo IoT, 3 por pedido del DTI, 1 por error de ambigüedad detectado en inspección).
- 4 RF nuevos fueron añadidos (CR aprobados).
- 2 RF fueron eliminados por estar fuera de alcance.

Estabilidad = (42 − 6) / 42 × 100 % = **85,7 %** → dentro del umbral del 80 %.
Volatilidad del sprint = 6 / 42 = **14,3 %** → dentro del umbral.
La conclusión del director del PFC: el ERS es gestionable. Si la volatilidad superara el 20 % en un sprint, se dispararía una revisión del alcance con el cliente.

12.4.2 Gestión de la configuración del ERS

Definición

Gestión de la configuración del ERS. Conjunto de prácticas que garantizan que todas las versiones del ERS y sus artefactos asociados (modelos UML, RTM, actas de revisión) son identificables de forma única, reproducibles y recuperables en cualquier momento del ciclo de vida del proyecto [1, 2].

Política de versionado semántico para el ERS

Se recomienda aplicar el esquema de versionado semántico al ERS del PFC: **Mayor.Menor.Parche**:

Tabla 12.6. *Política de versionado semántico para el ERS del PFC [1, 2, 10].*

Tipo	Cuándo incrementar	Ejemplo
Mayor (X.0.0)	Cambio de alcance que altera el modelo de dominio o más del 20 % de los RF.	ERS 1.0.0 → 2.0.0: se añade módulo de guardería nocturna.
Menor (x.Y.0)	Adición o eliminación de RF sin cambiar el modelo de dominio.	ERS 1.2.0 → 1.3.0: se añade RF de exportación a CSV.
Parche (x.y.Z)	Corrección de ambigüedad, error tipográfico o ajuste de criterio de aceptación.	ERS 1.3.0 → 1.3.1: se precisa el percentil en RNF-04.

Estructura de carpetas para el repositorio del PFC

El repositorio Git del PFC debe seguir esta estructura mínima de gestión de la configuración [1, 10]:

Estructura de repositorio Git recomendada para el ERS del PFC.

```
pfc-nombre-sistema/
+---- docs/
|   +---- ERS/
|   |   +---- ERS_v1.0.0_baseline.pdf  ← primera entrega aprobada
|   |   +---- ERS_v1.3.1_actual.pdf    ← versión en desarrollo
|   |   \---- CHANGELOG.md             ← historial de cambios
|   +---- modelos/
|   |   +---- ER_dominio.drawio
|   |   +---- clases_UML.puml
|   |   \---- statecharts/
|   \---- validacion/
|       +---- actas_revision/
|       +---- solicitudes_cambio/  ← CR-01.md, CR-02.md ...
|       \---- plan_pruebas.xlsx
\---- README.md
```

12.4.3 Baselines del ERS en el PFC

Tabla 12.7. *Los tres baselines del ERS recomendados para un PFC de la UTEQ [1, 2, 10].*

Baseline	Versión	Criterio de aprobación	Momento
Baseline 0 (Visión)	ERS v0.1.0	Director y cliente aprueban el alcance y la lista de stakeholders.	Fin de Semana 5
Baseline 1 (Análisis)	ERS v1.0.0	Director aprueba el ERS con RF, RNF, modelos y RTM; densidad de defectos <3/pág.	Fin de Semana 11
Baseline 2 (Entrega)	ERS v1.N.0	Todas las pruebas de aceptación PASS; cliente firma el acta.	Defensa del PFC

12.5 Mapa síntesis de la Semana 12

Tabla 12.8. *Mapa de cierre de la Semana 12 y de la Unidad 4.*

Subtema	Concepto clave	Artefacto del PFC
12.1 Iteratividad	35 % cambio promedio; fuentes de cambio por dominio.	Registro de cambios del PFC (CHANGELOG).
12.2 Gestión cambio	Formulario CR; proceso CCB; 6 pasos; métricas.	3+ CRs completos del PFC procesados por el CCB.
12.3 Trazabilidad	Pre / interna / post; RTM completa; trazabilidad por uso.	RTM con los tres tipos de trazabilidad.
12.4 Métricas y config.	6 métricas; versionado semántico; 3 baselines.	ERS etiquetado con versión semántica; repositorio organizado.

12.6 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 12 (en clase, 2 horas).

Simulación completa del proceso de gestión del cambio.

El docente entrega a cada equipo 3 solicitudes de cambio simuladas para su PFC (una de cada tipo: nuevo RF, cambio de RNF, eliminación de módulo).

Tareas:

1. Completar el formulario CR para cada solicitud (justificación, RF afectados, modelos afectados, impacto estimado).
2. Simular la reunión del CCB (15 min): cada equipo actúa con los tres roles (director, cliente, desarrollador) y toma decisiones de aprobar, rechazar o posponer.
3. Para cada CR aprobado: identificar qué filas de la RTM deben actualizarse y calcular el nuevo valor de estabilidad del ERS.
4. Calcular las 6 métricas de gestión de requisitos para el estado actual del PFC.
5. Etiquetar la versión actual del ERS del PFC usando el esquema semántico.

Práctica / Actividad

Tarea autónoma n.º 12 (6 horas, equipo del proyecto integrador).

Hito 4 del PFC — ERS v1.0.0 final (baseline 1):

1. **Procesar todos los CR** identificados en la Semana 11: completar formulario, decisión del CCB, actualizar ERS y RTM.
2. **ERS v1.0.0**: versionar el documento PDF con el esquema semántico; etiquetar en el repositorio Git con tag `v1.0.0`.
3. **RTM completa** con los tres tipos de trazabilidad para todos los RF del ERS (mínimo 25 RF).
4. **Calcular las 6 métricas** de gestión de requisitos y redactar el análisis de resultados (1 párrafo por métrica).
5. **CHANGELOG.md**: historial cronológico de todos los cambios desde el ERS v0.1.0 hasta v1.0.0.
6. **Política de gestión de requisitos del PFC** (1 página): roles del CCB, frecuencia de reuniones, criterios de baseline y umbrales de volatilidad.
7. **Preparar la defensa del Hito 4**: presentación de 10 min donde el equipo demuestra que el ERS es completo, consistente, trazable y gestionado.

12.7 Resumen ejecutivo de la semana

- Los requisitos cambian un promedio del **35 %** entre el baseline inicial y la entrega; la gestión del cambio convierte ese hecho inevitable en un proceso controlado [10, 35].
- El **formulario CR** y el proceso del CCB son los mecanismos que protegen el alcance del proyecto sin impedir la evolución necesaria [1, 2].

- Los **tres tipos de trazabilidad** (pre, interna, post) responden a las tres preguntas de gestión más frecuentes: ¿por qué existe?, ¿qué afecta? y ¿está implementado? [1, 2].
- Las **seis métricas** de gestión dan visibilidad objetiva sobre la salud del ERS: estabilidad, cobertura, densidad de defectos, volatilidad, tiempo de CR y tasa de aprobación [10, 80].
- El **versionado semántico** del ERS y la **gestión de la configuración** en Git son la base para que el PFC pueda ser auditado, reproducido y defendido ante el tribunal [1, 2].
- Los **tres baselines** (visión, análisis, entrega) estructuran el ciclo de vida del ERS del PFC y definen los criterios formales de progreso y de aceptación final [2, 10].

Semana 13

Herramientas CASE y estándares para IR

Una herramienta CASE no mejora el proceso de IR por sí sola; es el proceso bien definido el que hace útil a la herramienta. Sin proceso, la herramienta es un almacén de requisitos; con proceso, es un sistema nervioso del proyecto.
adaptado de Wiegers y Beatty [10]

Objetivos de aprendizaje de la semana

Al finalizar esta decimotercera semana, el estudiante estará en capacidad de:

1. Comparar las principales **herramientas CASE para IR** (Jira, Enterprise Architect, Sparx EA, ReqIF) y seleccionar la más apropiada para su PFC [1, 3, 10].
2. Aplicar **IEEE 29148:2018** e **ISO/IEC 25010:2023** de forma práctica en la gestión de requisitos del PFC, más allá de la redacción del ERS [2, 4, 56].
3. Configurar **esquemas de atributos** de requisitos en una herramienta CASE real y gestionar la trazabilidad desde ella [1, 2, 10].
4. Implementar la **gestión de la configuración del ERS** en Git con versionado semántico, ramas y etiquetas de baseline [1, 2, 10].

13.1 Subtema 13.1. Herramientas CASE para IR

Definición

Herramienta CASE para IR. Software especializado en la creación, gestión, trazabilidad y verificación de artefactos de requisitos. Va más allá de un procesador de texto: gestiona las relaciones entre requisitos, soporta múltiples vistas del mismo artefacto, genera métricas y mantiene la RTM de forma automática [1, 10].

13.1.1 Panorama de herramientas

Tabla 13.1. *Comparación de herramientas CASE para IR relevantes para PFC de la UTEQ [1, 3, 10].*

Herramienta	Fortaleza principal	Trazabilic	UML	Costo
Jira + Confluence	Gestión de backlog ágil; integración con repositorios Git.	Básica (via plugins)	No nativa	Gratuito hasta 10 usuarios
Enterprise Architect (Sparx)	Modelado UML completo; RTM automática; generación de ERS.	Completa	Completa	Licencia comercial
draw.io + GitHub	Modelos UML/ER gratuitos; versionado Git nativo.	Manual	Parcial	Gratuito
ReqIF / Polarion	Intercambio de requisitos entre organizaciones (estándar OMG).	Completa	Parcial	Enterprise
Excel + OneDrive	Mínimo costo; accesible para PFC.	Manual (RTM en hoja)	No	Gratuito (UTEQ)
Obsidian + Git	Requisitos como Markdown enlazados; gratuito; trazabilidad con plugins.	Parcial	No	Gratuito

Jira como herramienta de gestión de requisitos ágiles

Jira es la herramienta más usada en proyectos ágiles para la gestión del backlog, pero puede configurarse para soportar los procesos de IR de un PFC [10, 38]:

- **Epics** → metas de negocio o módulos del sistema.
- **Stories** → historias de usuario o RF individuales.
- **Sub-tasks** → criterios de aceptación o tareas de diseño.
- **Labels** → prioridad MoSCoW, tipo (RF/RNF/restricción), estado de validación.
- **Links** → trazabilidad entre RF relacionados (*is related to*, *blocks*, *duplicates*).
- **Confluence** → ERS en formato wiki, vinculado a tickets de Jira.

Ejemplo en Ecuador

Configuración de Jira para el PFC de clínica veterinaria. El equipo configuró el proyecto con tres tipos de *issue*: **Épica** (módulo: Citas, Mascotas, Pagos, Recetas), **Historia** (RF individual en formato EARS) y **Defecto-Requisito** (para CRs aprobados por el CCB). El flujo de estados fue: PROPUESTO → EN ANÁLISIS → APROBADO → EN IMPLEMENTACIÓN → VERIFICADO, coincidiendo exactamente con el atributo Estado del IEEE 29148:2018. La RTM se exportó automáticamente desde Jira a Excel para incluirla en el ERS.

Enterprise Architect / Sparx EA para modelado y trazabilidad

Enterprise Architect (Sparx Systems) es la herramienta CASE de modelado UML más completa del mercado y la estándar en ingeniería de sistemas [1, 10]. Sus capacidades clave para IR son:

- Soporte completo de diagramas UML 2.5.1: clases, CU, actividades, statecharts, secuencia, componentes y despliegue.
- **RTM automática**: al asignar un *tagged value* `«requires»` entre un elemento UML y un requisito, EA genera y actualiza la RTM automáticamente.
- Generación de documentos Word/PDF desde los modelos con plantillas.
- Importación/exportación en formato **ReqIF** (OMG) para intercambiar requisitos con otras herramientas.

ReqIF: el formato de intercambio de requisitos

Definición

ReqIF (Requirements Interchange Format). Estándar XML del OMG para el intercambio de requisitos entre herramientas distintas, sin pérdida de atributos, relaciones ni trazabilidad. Permite que el cliente use IBM DOORS y el proveedor use Polarion, intercambiando el ERS en un formato común [1, 3].

Tabla 13.2. *Recomendación de herramienta CASE según el tipo de PFC de la UTEQ.*

Tipo de PFC	Herramienta recomendada	Justificación
Sistema web o móvil ágil	Jira + Confluence + draw.io	Gratuito, integra backlog y ERS.
Sistema IoT / embebido	draw.io + GitHub + Excel RTM	Flujo Git natural; diagramas técnicos.
Sistema agropecuario con trazabilidad formal	Enterprise Architect (licencia UTEQ)	RTM automática; modelos UML completos.
Cualquier PFC	Excel + PDF + Git mínimo	Fallback siempre disponible.

13.2 Subtema 13.2. IEEE 29148:2018 e ISO/IEC 25010:2023 en proyectos reales

13.2.1 IEEE 29148:2018: más allá del ERS

El uso de IEEE 29148:2018 [2] va más allá de la estructura del documento ERS. Sus cláusulas definen procesos obligatorios que el equipo del PFC debe implementar en su herramienta CASE:

Tabla 13.3. *Cláusulas de IEEE 29148:2018 y su implementación práctica en el PFC de aulas IoT [1, 2].*

Cláusula	Proceso requerido	Implementación en PFC Aulas IoT
5.2.1 Elicitación	Identificar fuentes y aplicar técnicas.	Plan de elicitación documentado en Confluence.
5.2.2 Análisis	Clasificar, priorizar y resolver conflictos.	Tabla MoSCoW en Jira con etiquetas de prioridad.
5.2.3 Especificación	Documentar RF con los 10 atributos.	Plantilla Jira con campos: ID, descripción EARS, prioridad, fuente, criterio GWT.
5.2.4 Validación	Revisar y confirmar con stakeholders.	Acta de revisión + reporte de densidad de defectos.
5.2.5 Gestión	Mantener trazabilidad; gestionar cambios.	RTM en Confluence vinculada a tickets Jira.

13.2.2 ISO/IEC 25010:2023 como criterio de RNF

La norma *ISO/IEC 25010:2023* [4] proporciona el marco de referencia para especificar, clasificar y medir los requisitos no funcionales de cualquier sistema software. Su uso más práctico en el PFC es como **lista de verificación de RNF**: antes de cerrar el ERS, el equipo recorre las 9 características de calidad y verifica que el sistema tiene al menos un RNF documentado para cada característica relevante.

Tabla 13.4. *Aplicación de las características de ISO 25010:2023 al PFC de gestión de ganado bovino [1, 4, 17].*

Característica	RNF del PFC ganadero	¿Cubierto
Adecuación funcional	Los cálculos de ración y peso deben ser exactos al 99,9 %.	✓
Eficiencia de desempeño	Sincronización offline-online <30 s con 450 registros.	✓
Compatibilidad	API REST compatible con sistema AGROCALIDAD.	✓
Interaction capability	App móvil usable sin conexión con una mano en campo.	✓
Fiabilidad	Disponibilidad ≥95 % (opera en zona con intermitencia GSM).	✓
Seguridad	Solo el propietario y el veterinario acceden al historial clínico.	✓
Mantenibilidad	Añadir nueva especie animal sin modificar el módulo de vacunación.	Pendiente
Flexibilidad	Desplegable en Android 10+ sin dependencia de Play Store.	✓
Safety	El sistema no deberá permitir registrar dosis de medicamento por encima del límite AGROCALIDAD.	Pendiente

13.3 Subtema 13.3. Atributos y trazabilidad en herramientas CASE

13.3.1 Esquema de atributos configurado en Jira para el PFC

La tabla siguiente muestra el esquema de atributos que cada equipo del PFC debe configurar en su herramienta CASE, mapeando los 10 atributos de IEEE 29148:2018 a los campos de Jira [2, 10]:

Tabla 13.5. *Mapeo de los 10 atributos IEEE 29148:2018 a campos de Jira. Este esquema es el mínimo exigible en el PFC de la UTEQ.*

Atributo IEEE 29148	Campo en Jira	Configuración
ID	Issue Key (ej: PFC-42)	Automático; no modificar.
Descripción	Summary + Description	Usar plantilla EARS en el campo Description.
Prioridad	Priority (custom: Must/Should/Could/Wont)	Lista desplegable personalizada.
Fuente	Reporter + campo custom *Stakeholder origen*	Texto libre.
Estabilidad	Campo custom *Estabilidad* (Alta/Media/Baja)	Lista.
Estado	Status (workflow: Propuesto→Aprob	Workflow personalizado.
Criterio de aceptación	Campo custom *Acceptance Criteria*	Texto en formato GWT.
Trazabilidad ↑	Link type *traces to*	Enlace a Epic de origen.
Trazabilidad ↓	Link type *is implemented by*	Enlace a subtarea o PR de código.
Versión	Fix Version + campo *ERS Version*	Versionado semántico.

13.4 Subtema 13.4. Gestión de la configuración del ERS en Git

13.4.1 Flujo de trabajo Git para el ERS del PFC

Definición

Gestión de la configuración del ERS. Aplicación de los principios de control de versiones y configuración a todos los artefactos del ERS: documento principal, modelos UML, RTM, actas de revisión y formularios CR. En el contexto del PFC de la UTEQ, se implementa con Git y GitHub/GitLab [1, 2].

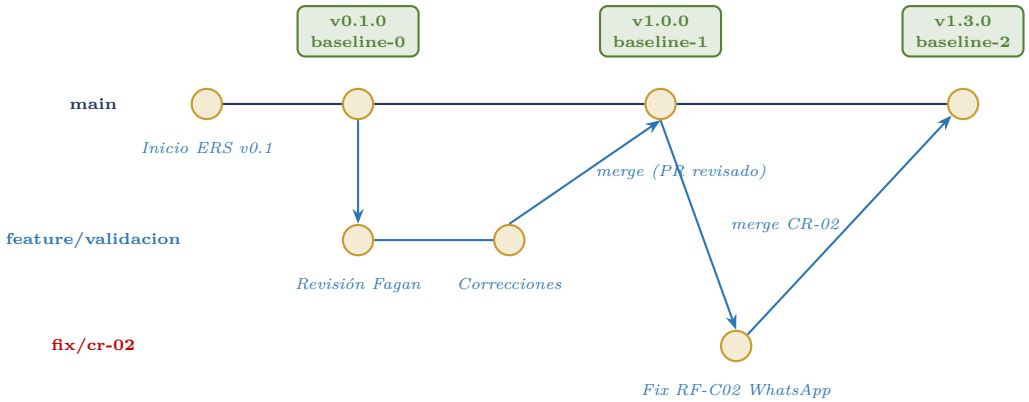


Figura 13.1. *Flujo de trabajo Git recomendado para el ERS del PFC. La rama* `main` *contiene solo versiones aprobadas por el CCB; los cambios entran via Pull Request revisado. Los tags marcan los baselines. Adaptado de [1, 2].*

Ejemplo en Ecuador

Política Git mínima para el ERS del PFC de la UTEQ:

1. La rama `main` es protegida: no se hace *push* directo.
2. Cada CR aprobado crea una rama `fix/cr-NN`.
3. Cada sprint de análisis crea una rama `feature/semana-NN`.
4. Los merges a `main` requieren revisión de un segundo miembro.
5. Cada baseline del ERS se etiqueta con tag semántico: `v1.0.0`.
6. El archivo `CHANGELOG.md` se actualiza en cada merge.

13.5 Mapa síntesis de la Semana 13

Tabla 13.6. *Mapa de cierre de la Semana 13.*

Subtema	Concepto clave	Artefacto del PFC
13.1 CASE	Jira, EA/Sparx, ReqIF; tabla de selección por tipo de PFC.	Herramienta CASE configurada para el PFC.
13.2 Estándares	IEEE 29148 en 5 procesos; ISO 25010 como lista RNF.	Tabla ISO 25010 aplicada al PFC.
13.3 Atributos	10 atributos IEEE 29148 → campos Jira; trazabilidad CASE.	Esquema de atributos configurado.
13.4 Git ERS	Ramas, tags, PRs, CHANGELOG; flujo de trabajo PFC.	Repositorio Git del ERS con baseline-1 etiquetado.

13.6 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 13 (en clase, 2 horas).

Taller: configurar Jira para el PFC en 2 horas.

1. Crear proyecto en Jira (gratuito cloud): configurar tipos de issue (Épica, Historia, Defecto-Requisito), workflow y campos custom.
2. Importar los 15 RF más prioritarios del ERS del PFC en el formato correcto (Summary = ID + enunciado EARS; Description = criterio GWT).
3. Configurar las relaciones de trazabilidad entre issues.
4. Aplicar la tabla ISO 25010:2023 al PFC: identificar qué características no tienen RNF cubierto y redactar al menos 2 RNF nuevos.
5. Etiquetar el estado actual del ERS como `v1.0.0` en el repositorio Git del PFC y verificar que el CHANGELOG está actualizado.

Práctica / Actividad

Tarea autónoma n.º 13 (4 horas, equipo del PFC).

1. **ERS completo en Jira**: todos los RF/RNF migrados con los 10 atributos y las relaciones de trazabilidad configuradas.
2. **Tabla ISO 25010:2023**: las 9 características aplicadas al PFC con al menos un RNF por característica relevante.
3. **Repositorio Git limpio**: estructura de carpetas estándar, tag `v1.0.0` en baseline-1, CHANGELOG.md actualizado.
4. **Reporte de configuración**: 1 página describiendo la herramienta elegida, la justificación y las decisiones de configuración tomadas.

13.7 Resumen ejecutivo de la semana

- Las herramientas CASE multiplican la eficiencia del proceso de IR pero no lo reemplazan: se necesita el proceso primero [1, 10].
- **Jira** es la opción más accesible para PFC ágiles; **Enterprise Architect** es la más completa para sistemas complejos con trazabilidad formal [3].
- IEEE 29148:2018 define **cinco procesos** que deben implementarse, no solo la estructura del documento ERS [2].
- ISO 25010:2023 es la **lista de verificación de RNF** más poderosa disponible: garantiza que ninguna característica de calidad quede sin especificar [4, 17].
- La gestión de la configuración del ERS en **Git** con versionado semántico, ramas protegidas y tags de baseline es la práctica profesional estándar [1, 2].

Semana 14

Ingeniería de Requisitos en procesos ágiles

Los métodos ágiles no eliminan los requisitos; eliminan el documento de requisitos monolítico. Los requisitos siguen existiendo, solo que viven en el backlog, en las historias de usuario y — sobre todo — en las conversaciones continuas entre el equipo y el cliente.
adaptado de Cohn [37]

Objetivos de aprendizaje de la semana

Al finalizar esta decimocuarta semana, el estudiante estará en capacidad de:

1. Gestionar el **product backlog** como artefacto central de IR ágil, con épicas, historias de usuario y criterios de aceptación [37, 38, 56].
2. Aplicar **BDD** (Behavior Driven Development) para convertir los criterios de aceptación en especificaciones ejecutables con Gherkin/Cucumber [36, 60, 61].
3. Integrar las actividades de IR con los **eventos y artefactos de Scrum**: Sprint Planning, Refinement, Review y retrospectiva [1, 38, 56].
4. Comparar **IR tradicional vs. IR ágil** y decidir cuándo aplicar cada enfoque en el PFC [1, 37, 38].

14.1 Subtema 14.1. IR en procesos ágiles: backlog, historias y criterios

14.1.1 El product backlog como ERS ágil

Definición

Product Backlog. Lista ordenada y emergente de todo lo que se conoce que es necesario en el producto. Es el único artefacto de requisitos en Scrum y el responsable de su calidad es el **Product Owner** (PO). No es una lista estática: se refina continuamente en cada sprint [37, 38].

La jerarquía del backlog en un PFC ágil se compone de tres niveles:

Tabla 14.1. *Jerarquía de artefactos del backlog ágil y su equivalente en IR tradicional [1, 37, 38].*

Nivel ágil	Descripción	Equivalente IR tradicional	Ejemplo (PFC camaroneras)
Épica	Capacidad grande que tarda más de un sprint.	Módulo o subsistema del ERS.	*Monitoreo de calidad del agua en tiempo real.*
Historia	Entregable en un sprint; criterio INVEST.	RF individual en EARS.	*Como técnico, quiero ver la temperatura de la piscina 3 en el dashboard.*
Tarea	Unidad de trabajo técnico dentro de la historia.	Subtarea de diseño o implementación.	*Crear endpoint REST GET /piscinas/id/sensores.*

Refinamiento del backlog: de épica a historia INVEST

El **Backlog Refinement** (o Grooming) es el evento de Scrum donde el PO y el equipo descomponen épicas en historias, añaden criterios de aceptación y estiman el esfuerzo en puntos de historia [38]. Para el PFC de camaroneras:

Ejemplo en Ecuador

Refinamiento de épica → historias INVEST (PFC camaroneras).
Épica: Monitorear calidad del agua en tiempo real.
Historia US-C01 (Must-have, 5 puntos): *Como técnico acuícola, quiero ver en el dashboard el valor de oxígeno disuelto de cada piscina actualizado cada 30 segundos, para detectar condiciones críticas antes de que los camarones comiencen a morir.*
Criterios de aceptación (Gherkin):

- **Dado** que el dashboard está abierto y el sensor de la piscina 3 está activo,
  cuando la lectura de O2 cambia en el sensor,
  entonces el dashboard muestra el nuevo valor en menos de 35 segundos.
- **Dado** que el O2 cae por debajo de 4.0 mg/L,
  cuando el sistema detecta la lectura crítica,
  entonces el dashboard muestra la alerta en rojo y suena la alarma sonora.

Historia US-C02 (Must-have, 3 puntos): *Como administrador, quiero exportar el historial de lecturas de oxígeno de los últimos 30 días por piscina en formato CSV, para el reporte semanal a la extractora.*

14.2 Subtema 14.2. BDD: especificación ejecutable de requisitos

Definición

Behavior Driven Development (BDD). Técnica de desarrollo propuesta por Dan North [61] que extiende el TDD (Test-Driven Development) al nivel de los requisitos: los criterios de aceptación se escriben como **escenarios Gherkin** *antes* de implementar la historia, y se convierten automáticamente en pruebas ejecutables con frameworks como Cucumber (Java/Ruby), Behave (Python) o Pytest-BDD (Python) [36, 60, 61].

14.2.1 La anatomía completa de un escenario Gherkin

Tabla 14.2. *Palabras clave de Gherkin y su semántica [60, 61].*

Palabra clave	Semántica	En el PFC de mascotas
Feature	El nombre de la funcionalidad (épica).	`Feature: Reservar servicio de paseo.`
Background	Precondiciones comunes a todos los escenarios del Feature.	`Given el dueño ha iniciado sesión.`
Scenario	Un caso de prueba de aceptación concreto.	`Scenario: Reserva exitosa con cuidador disponible.`
Given	Estado del sistema antes de la acción.	`Given existe un cuidador certificado disponible.`
When	La acción del usuario o del sistema.	`When el dueño selecciona fecha, hora y cuidador.`
Then	El resultado observable verificable.	`Then el sistema confirma la reserva y envía SMS.`
And / But	Condiciones adicionales encadenadas.	`And el cuidador recibe notificación push.`
Scenario Outline	Escenario parametrizado con tabla de ejemplos.	`Outline: Reservar servicio de tipo <tipo>.`

Feature file completo: PFC mascotas a domicilio

Ejemplo en Ecuador

Archivo `reserva_paseo.feature` — PFC Mascotas.

```
Feature: Reservar servicio de paseo de mascota

  Background:
    Given el dueño "Carlos Ponce" ha iniciado sesión
    And tiene registrada la mascota "Simba" (golden retriever, 4 años)

  Scenario: Reserva exitosa con cuidador disponible
    Given existe el cuidador "María Loor" disponible el 2025-06-15 a las 08h00
    When Carlos selecciona a María para el paseo de 60 minutos
    And confirma la reserva
    Then el sistema registra la reserva con estado CONFIRMADO
    And envía SMS a Carlos: "Reserva confirmada con María el 15/06 08h00"
    And envía notificación push a María con los datos de Simba

  Scenario: Sin cuidadores disponibles
    Given no hay cuidadores disponibles el 2025-06-15 a las 08h00
    When Carlos intenta reservar a esa hora
    Then el sistema muestra "No hay cuidadores disponibles en ese horario"
    And sugiere los 3 horarios más próximos con disponibilidad

  Scenario Outline: Reservar distintos tipos de servicio
    When Carlos selecciona el servicio "<tipo>"
    Then el sistema aplica la tarifa de <precio> dólares por hora
    Examples:
      | tipo          | precio |
      | Paseo         | 5.00   |
      | Guardería día | 8.00   |
      | Vet. domicilio| 15.00  |
```

14.2.2 De Gherkin a prueba ejecutable con Pytest-BDD

Ejemplo en Ecuador

Step definitions en Python (Pytest-BDD) para el PFC mascotas.

```
# test_reserva_paseo.py
from pytest_bdd import given, when, then, scenario

@scenario('reserva_paseo.feature',
          'Reserva exitosa con cuidador disponible')
def test_reserva_exitosa():
    pass

@given('existe el cuidador "María Loor" disponible el 2025-06-15 a las 08h00')
def cuidador_disponible(db):
    db.insert_cuidador('María Loor', fecha='2025-06-15', hora='08:00')

@when('Carlos selecciona a María para el paseo de 60 minutos')
def seleccionar_cuidadora(client, reserva_data):
    reserva_data['response'] = client.post('/api/reservas', json={
        'cuidador_id': 'maria-loor',
        'duracion_min': 60,
        'fecha': '2025-06-15',
        'hora': '08:00'
    })

@then('el sistema registra la reserva con estado CONFIRMADO')
def verificar_estado(reserva_data):
    assert reserva_data['response'].json()['estado'] == 'CONFIRMADO'
```

14.3 Subtema 14.3. Integración de IR con Scrum

14.3.1 El rol del Product Owner en la IR ágil

En Scrum, el **Product Owner** es el responsable único del Product Backlog y actúa como el analista de requisitos del equipo [38]. Sus responsabilidades de IR son:

- Redactar o aprobar todas las historias de usuario antes de que entren al Sprint Planning.
- Mantener el backlog priorizado por valor de negocio (no por preferencias técnicas del equipo).
- Participar en el Refinement al menos 2 horas por semana.
- Definir el criterio de **Definition of Done** (DoD) que incluye al menos: criterios GWT pasados, RTM actualizada y documentación del usuario actualizada.
- Aceptar o rechazar el incremento en la Sprint Review.

14.3.2 IR en cada evento Scrum

Tabla 14.3. *Actividades de IR en cada evento de Scrum [1, 38, 56].*

Evento Scrum	Actividad de IR	Artefacto producido
Sprint Planning	El PO presenta las historias del sprint; el equipo verifica INVEST y estima esfuerzo.	Sprint Backlog con historias listas para desarrollar.
Daily Scrum	¿Hay bloqueos por ambigüedad de requisitos? El PO resuelve en el día.	Decisión de aclaración documentada en el issue de Jira.
Backlog Refinement	Desglose de épicas, escritura de criterios GWT, estimación.	Historias refinadas con criterios de aceptación.
Sprint Review	El PO valida el incremento contra los criterios de aceptación.	Historias aceptadas/rechazadas; nuevas historias si surgen.
Retrospectiva	¿El proceso de escritura de historias funcionó? ¿Hubo muchos rechazos por criterios mal escritos?	Mejoras al proceso de IR del equipo.

14.4 Subtema 14.4. XP y comparación IR tradicional vs. IR ágil

14.4.1 IR en Extreme Programming (XP)

Extreme Programming (XP) [36] tiene su propio conjunto de prácticas de IR que difieren de Scrum pero comparten el énfasis en la comunicación continua:

- **Customer on site**: el cliente (o su representante) está físicamente presente con el equipo para responder preguntas de requisitos en tiempo real. En el PFC, el dueño del negocio puede participar en sesiones semanales presenciales de 2 horas.
- **User stories**: el cliente escribe las historias en tarjetas físicas o en Jira; el equipo las estima; se negocian alcance y prioridad en cada iteración.
- **Acceptance tests**: el cliente escribe los criterios de aceptación antes de la implementación (similar a BDD).
- **Small releases**: entregas cada 1–2 semanas validan continuamente que el sistema en construcción coincide con lo que el cliente necesita.

14.4.2 Comparación IR tradicional vs. IR ágil

Tabla 14.4. *Comparación sistemática entre IR tradicional e IR ágil [1, 37, 38, 56].*

Dimensión	IR Tradicional	IR Ágil
Artefacto central	ERS/SRS con los 10 atributos IEEE 29148.	Product Backlog con historias INVEST.
Momento de especificar	Antes del desarrollo; baseline aprobado.	Iterativo; just-in-time por sprint.
Criterio de calidad	Densidad de defectos; cobertura RTM.	Criterios GWT; Definition of Done.
Gestión del cambio	Proceso CCB formal; CR documentado.	Reordenamiento del backlog; PO decide.
Participación del cliente	Revisiones planificadas en hitos.	Continua; Review cada sprint.
Trazabilidad	RTM bidireccional formal.	Links en Jira; BDD tests.
Cuándo aplicar	Alta criticidad; contrato fijo; regulado.	Requisitos cambiantes; cliente disponible.
Cuándo NO aplicar	Equipos pequeños; requisitos cambiantes.	Sistemas de seguridad crítica; pliego SERCOP.

⚠ Advertencia / Error frecuente

Falacia frecuente. "Somos ágiles, no necesitamos documentar requisitos." Los métodos ágiles *no* eliminan la necesidad de especificar requisitos con criterios de aceptación verificables: la eliminan del documento monolítico y la distribuyen en el backlog. Un backlog sin criterios de aceptación es igualmente inútil que un ERS sin criterios de aceptación [1, 37, 38].

Ejemplo en Ecuador

¿Qué enfoque debe usar el PFC de la UTEQ?
La respuesta depende de tres factores:

1. **Disponibilidad del cliente**: si el cliente (dueño de la camaronera, mayordomo de la hacienda) puede reunirse semanalmente, el enfoque ágil con Scrum + BDD es más eficiente.
2. **Requisito del pliego SERCOP**: si el PFC es parte de una contratación pública, el pliego exige un ERS formal (IR tradicional).
3. **Criticidad**: para el sistema de aulas IoT de la UTEQ con control de acceso, se recomienda un enfoque híbrido: ERS formal para los RF de seguridad (IEEE 29148); backlog ágil para los RF de funcionalidad de usuario.

El **enfoque híbrido** (ERS mínimo + backlog) es la recomendación para la mayoría de PFC de la UTEQ [1, 56].

14.5 Mapa síntesis de la Semana 14

Tabla 14.5. *Mapa de cierre de la Semana 14.*

Subtema	Concepto clave	Artefacto del PFC
14.1 Backlog ágil	Épica→Historia→Tarea; INVEST; refinement.	Backlog priorizado del PFC en Jira.
14.2 BDD / Gherkin	Feature, Scenario, Given/When/Then; step defs.	3+ feature files con criterios ejecutables.
14.3 Scrum + IR	PO como analista; IR en cada evento Scrum.	Sprint Backlog del sprint 1 del PFC.
14.4 XP vs. Ágil	Customer on site; comparación 8 dimensiones.	Decisión documentada de enfoque para el PFC.

14.6 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 14 (en clase, 2 horas).

Taller BDD: de historia de usuario a feature file ejecutable.

1. Tomar 3 historias del PFC del equipo y escribir sus feature files completos en Gherkin (mínimo 2 escenarios por historia, incluyendo flujos alternativos y un Scenario Outline).
2. Implementar los step definitions vacíos en Python (Behave) o Java (Cucumber): solo la firma de cada step, sin lógica de negocio.
3. Ejecutar los tests en modo "pendiente" y verificar que el framework los reconoce correctamente.
4. Completar la tabla de comparación IR tradicional vs. ágil para el PFC propio: ¿qué enfoque usarán y por qué?

Práctica / Actividad

Tarea autónoma n.º 14 (4 horas, equipo del PFC).

1. **Backlog completo en Jira**: todas las épicas del PFC descompuestas en historias INVEST con criterios GWT y puntos de historia estimados.
2. **Feature files**: al menos 5 feature files cubrien los RF más críticos del PFC, con step definitions vacíos en el lenguaje del proyecto.
3. **Sprint Backlog del Sprint 1**: definir cuáles historias entran al primer sprint de implementación, con la Definition of Done del equipo.
4. **Decisión de enfoque**: documento de 1 página justificando la elección entre IR tradicional, IR ágil o híbrido para el PFC, con referencia a los tres factores de decisión.

14.7 Resumen ejecutivo de la semana

- El **Product Backlog** es el artefacto central de IR en Scrum; su calidad depende del PO y de los criterios de aceptación [37, 38].
- **BDD** convierte los criterios de aceptación en especificaciones ejecutables que sirven simultáneamente como pruebas de regresión automáticas [60, 61].
- En Scrum, **cada evento tiene actividades de IR**: el Refinement es la sesión de análisis de requisitos, la Review es la validación [1, 38].
- La IR ágil **no elimina** los requisitos: los distribuye en el tiempo y los mantiene en el backlog en lugar de en un documento monolítico [1, 37].
- El **enfoque híbrido** (ERS mínimo + backlog) es la recomendación práctica para la mayoría de PFC de la UTEQ [56].

Semana 15

Proyecto integrador: planificación, levantamiento y análisis

Un proyecto de fin de carrera no es un ejercicio académico sobre datos ficticios. Es la primera vez que el estudiante negocia requisitos con un cliente real, que puede cambiar de opinión, que tiene agendas propias y que firmará — o no — el acta de aceptación.
docentes de IR, UTEQ

Objetivos de aprendizaje de la semana

Al finalizar esta decimoquinta semana, el estudiante estará en capacidad de:

1. Seleccionar y delimitar el **sistema objetivo** del PFC con criterios técnicos y de viabilidad [1, 10, 52].
2. Construir el **mapa de stakeholders** del PFC real identificando roles, intereses, nivel de influencia y estrategia de comunicación [1, 2, 52].
3. Aplicar **técnicas reales de elicitación** con el cliente del PFC: entrevista semiestructurada, taller y análisis documental [1, 10, 50].
4. Clasificar, priorizar y negociar los requisitos levantados aplicando **MoSCoW** y el modelo **Kano** sobre datos reales del PFC [10, 37, 64].

15.1 Subtema 15.1. Selección del sistema, stakeholders y plan de IR

15.1.1 Criterios para seleccionar el sistema objetivo del PFC

La selección del sistema objetivo del PFC no es trivial. El sistema debe cumplir los siguientes criterios mínimos [1, 52]:

Tabla 15.1. *Criterios de selección del sistema objetivo para el PFC de la UTEQ con ejemplos de los seis dominios propuestos.*

Criterio	Descripción y ejemplo
Cliente real accesible	Existe una persona o empresa que tomará decisiones sobre el sistema y con quien se puede acordar al menos 4 sesiones de elicitación de 1 hora. Ejemplo: propietario de la hacienda ganadera en Quinsaloma.
Alcance implementable	El equipo puede implementar el 70 % de los RF Must-have en el tiempo del PFC. Ejemplo: la clínica veterinaria puede acotarse al módulo de citas y fichas, dejando el módulo de facturación para versión 2.
Dominio con documentación	Existen documentos del dominio analizables: manual actual, formularios, reportes del SRI. Ejemplo: la camaronera tiene bitácoras físicas de calidad del agua de los últimos 3 años.
Al menos 3 stakeholders	El sistema tiene al menos tres roles distintos con perspectivas diferentes. Ejemplo: hacienda de palma → cortador, pesador, administrador.
Innovación técnica	El sistema incorpora al menos un elemento de innovación: IoT, IA, app móvil offline, integración API gubernamental.

15.1.2 Mapa de stakeholders del PFC

El mapa de stakeholders es el primer artefacto que el equipo produce para su PFC. Combina la identificación de actores con el análisis de su influencia e interés [1, 2, 52]:

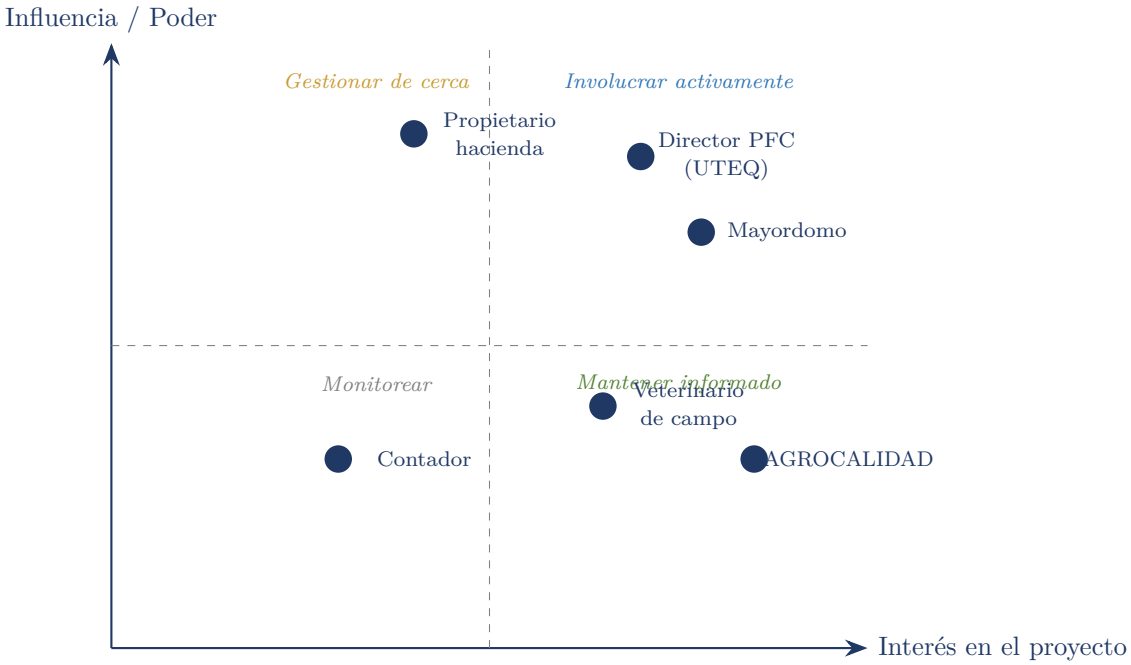


Figura 15.1. *Mapa de stakeholders del PFC de gestión ganadera. Los stakeholders en el cuadrante superior derecho (involucrar activamente) deben participar en todas las sesiones de elicitación y revisión del ERS. Adaptado de [1, 52].*

15.1.3 Plan de IR del PFC

Tabla 15.2. *Plan de IR mínimo para el PFC de la UTEQ (adaptable a los 6 dominios propuestos) [1, 2, 52].*

Seman	Actividad	Entregable	Responsab
15	Elicitación inicial	Plan de elicitación + acta entrevista 1 + mapa stakeholders.	Todo el equipo.
15	Análisis documental	Lista candidatos a requisitos (mín. 30).	Analista.
16	Análisis y clasificación	Candidatos clasificados (tipo/nivel/persp.) + conflictos resueltos.	Analista.
16	Modelado UML	ER + clases + DFD N1 + statechart entidad central.	Modelador.
16	Especificación	ERS v1.0.0 con 6 capítulos + RTM completa.	Todo el equipo.
16	Validación	Revisión cruzada (peer review) + sesión con cliente + plan de pruebas.	Todo el equipo.
16	Gestión y defensa	ERS v1.0.0 etiquetado en Git + métricas de calidad + defensa ante tribunal.	Todo el equipo.

15.2 Subtema 15.2. Elicitación con usuarios reales

15.2.1 Guía de entrevista real para el PFC agropecuario

Ejemplo en Ecuador

Guía de entrevista semiestructurada — PFC hacienda de palma africana.
Objetivo: Entender el proceso actual de cosecha y los principales problemas que el sistema debe resolver.

Duración: 60–75 minutos. **Entrevistado**: Administrador de la hacienda.

Bloque 1 — El proceso AS-IS (20 min)

1. ¿Puede describirme paso a paso cómo funciona un día de cosecha, desde que el cortador sale al lote hasta que se genera el pago? (*walk-me-through*)
2. ¿Qué documentos o registros físicos se generan durante ese proceso? (solicitar verlos)
3. ¿Qué hace cuando hay un desacuerdo entre el peso registrado por el cortador y el registrado en la báscula? (*caso excepcional*)

Bloque 2 — Problemas y necesidades (20 min)

4. ¿Cuáles son los tres problemas más frecuentes con el sistema actual? (*ranking de pain points*)
5. ¿Cuánto tiempo le toma calcular la liquidación de todos los cortadores al final de la semana? ¿Le parece razonable?
6. Si pudiera cambiar *una sola cosa* del proceso actual, ¿qué sería? (*one thing question*)

Bloque 3 — Expectativas del sistema TO-BE (15 min)

7. ¿Tiene experiencia con apps similares en otras haciendas de la zona? ¿Qué funcionó y qué no?
8. ¿Qué dispositivos usa en la hacienda? ¿Hay señal celular en los lotes? (restricciones técnicas)
9. ¿Necesita que el sistema se integre con la extractora o con el SRI?

Bloque 4 — Cierre (5 min)

10. ¿Hay algún otro aspecto del proceso que no hayamos cubierto y que sea importante para el sistema?
11. ¿Podemos volver la próxima semana con un primer esquema del sistema para validarlo con usted?

15.2.2 Taller de requisitos real: estructura para el PFC

Cuando el cliente tiene múltiples stakeholders con perspectivas distintas (por ejemplo, cortadores, pesador y administrador en la hacienda), el taller es la técnica más eficiente para elicitar y resolver conflictos en una sola sesión [10, 52]:

Tabla 15.3. *Agenda del taller de requisitos del PFC (adaptación de la estructura de Robertson & Robertson [52]).*

Bloque	Duración	Actividad
Apertura	15 min	Presentar el objetivo del taller; establecer normas (un hablante a la vez; no hay respuestas incorrectas; el analista toma nota, no juzga).
Visión compartida	20 min	Presentar el mapa de stakeholders y el alcance propuesto; obtener confirmación o ajuste.
Brainstorming	45 min	Elicitación abierta: post-its con candidatos a RF (un color por módulo); el facilitador los agrupa en el pizarrón.
Descanso	15 min	—
MoSCoW grupal	30 min	Votación por puntos (cada participante tiene 10 puntos para repartir); el facilitador revela los resultados y discute los conflictos.
RNF	20 min	El analista pregunta por cada característica ISO 25010; los participantes cuantifican.
Cierre	15 min	Resumen de acuerdos; próximos pasos; fecha de la siguiente sesión.

15.3 Subtema 15.3. Clasificación, priorización y negociación

15.3.1 Clasificación de los candidatos a requisitos del PFC

Una vez concluidas las sesiones de elicitación, el equipo clasifica todos los candidatos en las tres dimensiones presentadas en la Semana 5 [1, 2]. El resultado de este ejercicio para el PFC de la clínica veterinaria es:

Ejemplo en Ecuador

Resultado del análisis de candidatos — PFC clínica veterinaria (fragmento).

ID	Candidato (elicitado)	Tipo	Nivel	Perspectiva
C-01	“El sistema tiene que guardar los datos del perro.”	Funcional	Software	Datos
C-02	“Quiero que sea rápido.”	Calidad	Sistema	Función (RNF ambiguo → reescribir)
C-03	“El veterinario necesita ver el historial completo.”	Funcional	Software	Función
C-04	“Debe funcionar en el celular del recepcionista.”	Restricción	Sistema	—
C-05	“No podemos compartir datos sin permiso del dueño.”	Funcional/Legal	Software	Comportamiento (LOPDP)

15.3.2 Negociación de conflictos reales del PFC

Ejemplo en Ecuador

Conflicto real resuelto en el PFC de aulas IoT.
Conflicto: El Departamento de TI de la UTEQ requería que todos los nodos IoT usen VPN institucional (mayor seguridad). El equipo de mantenimiento requería acceso directo a los nodos sin VPN (mayor facilidad de mantenimiento en campo). Los dos requisitos eran incompatibles.
Resolución win-win: Se definieron dos modos de acceso: (a) acceso de lectura sin VPN para el dashboard de monitoreo en tiempo real (satisface a Mantenimiento); (b) acceso de escritura (configuración de parámetros) solo con VPN (satisface a TI). La solución se documentó como CR-03 y fue aprobada por el CCB en 3 días.
Lección aprendida: los conflictos de stakeholders casi siempre tienen solución cuando se identifican los *intereses* reales en lugar de negociar sobre *posiciones* [1, 52].

15.4 Mapa síntesis de la Semana 15

Tabla 15.4. *Mapa de cierre de la Semana 15 — inicio del Proyecto Integrador.*

Subtema	Concepto clave	Entregable del PFC
15.1 Selección y plan	Criterios 5; mapa stakeholders; plan IR.	Mapa de stakeholders + Plan de IR del PFC.
15.2 Elicitación real	Guía de entrevista; taller con agenda completa.	Actas de 2+ sesiones con cliente real.
15.3 Clasificación	3 dimensiones; MoSCoW real; negociación win-win.	Lista de 30+ candidatos clasificados + acta CCB de conflictos.

15.5 Actividades del proyecto integrador

Práctica / Actividad

Hito 5 del PFC (semana 15) — Planificación y elicitación con cliente real.

1. **Mapa de stakeholders**: identificar mínimo 5 stakeholders; ubicarlos en la matriz poder/interés; definir estrategia de comunicación.
2. **Plan de elicitación**: técnicas seleccionadas, cronograma de sesiones, roles del equipo.
3. **Sesión 1 — Entrevista**: conducir entrevista semiestructurada (mínimo 60 min) con el stakeholder principal usando la guía de la sección 15.2.1; documentar acta con grabación o notas.
4. **Sesión 2 — Taller o análisis documental**: según la disponibilidad del cliente; documentar acta.
5. **Lista de candidatos**: mínimo 30 candidatos clasificados en las 3 dimensiones; lista priorizada MoSCoW; acta de resolución de al menos 2 conflictos.
6. **Revisión del director**: breve sesión (30 min) para validar que el alcance es coherente con el tiempo disponible del PFC.

15.6 Resumen ejecutivo de la semana

- La selección del sistema objetivo del PFC debe cumplir cinco criterios: cliente real accesible, alcance implementable, documentación disponible, mínimo tres stakeholders e innovación técnica [1, 52].
- El **mapa de stakeholders** es el primer artefacto del PFC y determina quiénes participarán en cada sesión de elicitación [2, 52].
- La **entrevista walk-me-through** y el **taller con agenda estructurada** son las técnicas más efectivas para clientes de sectores agropecuarios y de servicios locales [10, 50].
- La negociación **win-win** basada en intereses (no posiciones) es la estrategia más efectiva para resolver conflictos entre stakeholders con perspectivas opuestas [1].

Semana 16

Especificación, modelado, validación y defensa del PFC

La defensa del PFC no es el final del proceso de IR; es la demostración pública de que se lo ha hecho correctamente. Un tribunal que no puede encontrar ambigüedades, omisiones ni inconsistencias en el ERS es la mejor validación posible.
docentes de IR, UTEQ

Objetivos de aprendizaje de la semana

Al finalizar esta decimosexta semana, el estudiante estará en capacidad de:

1. Entregar el **ERS v1.0.0 completo** con los 6 capítulos IEEE 29148, todos los modelos UML y la RTM extendida [1, 2, 10].
2. Aplicar la **revisión cruzada** (peer review) entre equipos del PFC usando el checklist de inspección Fagan [10, 52, 65].
3. Calcular y reportar las **métricas de calidad del ERS** con ISO 25010 como criterio de evaluación [1, 4, 10].
4. Gestionar la trazabilidad del PFC en la herramienta CASE elegida y generar la RTM final exportada [1, 2].
5. Preparar y ejecutar la **defensa oral** del ERS ante el tribunal docente, respondiendo solicitudes de cambio en tiempo real.

16.1 Subtema 16.1. ERS/SRS completo según IEEE 29148

16.1.1 Lista de verificación del ERS del PFC

Antes de entregar el ERS, el equipo debe completar la siguiente lista de verificación obligatoria [1, 2, 10]:

Tabla 16.1. *Lista de verificación del ERS del PFC — criterios de completitud. Todos los ítems deben estar marcados como ✓ antes de la entrega.*

#	Criterio obligatorio	Estado
1	El ERS tiene portada con: nombre del sistema, versión, fecha, equipo, director y cliente.	✓ / ✗
2	El Cap. 1 incluye propósito, alcance, glosario y referencias.	✓ / ✗
3	El Cap. 2 incluye perspectiva del sistema, clases de usuario, entorno operacional y supuestos.	✓ / ✗
4	El Cap. 3 describe las interfaces de usuario (wireframes), hardware y software externo.	✓ / ✗
5	El Cap. 4 tiene mínimo 25 RF con ID, descripción EARS, prioridad MoSCoW y criterio GWT.	✓ / ✗
6	El Cap. 5 tiene mínimo 8 RNF con criterio cuantificable (número + unidad + percentil).	✓ / ✗
7	El Cap. 6 incluye RTM completa con pre-trazabilidad, trazabilidad interna y post-trazabilidad.	✓ / ✗
8	El ER del dominio tiene mínimo 7 entidades, todas con cardinalidades y PK marcados.	✓ / ✗
9	El diagrama de clases UML mapea el ER con operaciones derivadas de los CU.	✓ / ✗
10	El DFD N0 y N1 muestran todas las entidades externas y procesos del sistema.	✓ / ✗
11	El statechart de la entidad central tiene mínimo 5 estados y guardas booleanas.	✓ / ✗
12	La tabla de interrelaciones verifica los 8 criterios de coherencia.	✓ / ✗
13	El plan de pruebas tiene mínimo 15 casos (10 FAT, 3 NFAT, 2 UAT) trazados a la RTM.	✓ / ✗
14	El ERS está etiquetado como `v1.0.0` en Git con CHANGELOG actualizado.	✓ / ✗
15	La tabla ISO 25010 muestra cobertura de las 9 características con RNF cuantificables.	✓ / ✗

16.2 Subtema 16.2. Modelos UML: revisión final y coherencia

16.2.1 Revisión de coherencia de los modelos del PFC

La revisión final de modelos aplica el checklist de coherencia de la Semana 10 sobre los modelos reales del PFC [1, 2]. Para los seis dominios del PFC de la UTEQ, los elementos críticos a revisar son:

Tabla 16.2. *Elementos UML críticos a revisar por dominio de PFC.*

Dominio PFC	Entidad central y su statechart	Proceso DFD crítico a revisar
Gestión ganadera	Bovino: CRÍA→REPRODUCTIVO→ENGORDE→VENDIDO/MUERTO.	P4: Control reproductivo — verificar que cubre los umbrales de parto y destete.
Camaroneras IoT	Piscina: SEMBRADA→EN CULTIVO→EN COSECHA→VACÍA.	P1: Monitoreo calidad agua — verificar alertas por umbral por parámetro.
Palma africana	OrdenCosecha: PLANIFICADA→EN CAMPO→PESADA→LIQUIDADA.	P3: Liquidación — verificar fórmula de pago por kg y deducciones.
Aulas IoT	Dispositivo: APAGADO→ENCENDIDO→STAN	P2: Control dispositivos — verificar lógica de activación por sensor PIR.
Mascotas	Servicio: SOLICITADO→CONFIRMADO→EN CURSO→COMPLETADO/CANCELADO.	P1: Reservar servicio — verificar flujo de pago y asignación de cuidador.
Clínica veterinaria	Cita: SOLICITADA→CONFIRMADA→F ASISTIDA.	P3: Historial clínico — verificar trazabilidad consulta→receta→medicamento.

16.3 Subtema 16.3. Revisión cruzada entre equipos

16.3.1 Protocolo de peer review entre PFC

La revisión cruzada es la actividad de validación más valiosa de la Unidad 5: cada equipo revisa el ERS de otro equipo como si fuera un inspector externo. El protocolo es [10, 52, 65]:

1. **Asignación**: el docente asigna los pares de revisión de forma que equipos del mismo dominio no se revisen entre sí (más perspectiva independiente).
2. **Preparación individual** (90 min): cada revisor lee el ERS completo usando el checklist de 15 criterios y el checklist Fagan de 8 criterios; registra cada defecto con ID, tipo, severidad y ubicación exacta.
3. **Reunión de revisión** (60 min): los revisores presentan sus hallazgos; el equipo autor *solo escucha y registra* (sin defender). Principio 2 de Pohl [1].
4. **Lista consolidada de defectos**: el equipo autor recibe la lista y tiene 48 horas para procesar cada defecto como CR y actualizar el ERS.
5. **Cálculo de métricas**: densidad de defectos, tiempo de preparación de cada revisor y tasa de detección.

Ejemplo en Ecuador

Resultados reales de la revisión cruzada entre PFC (ejemplo):
El equipo de *clínica veterinaria* revisó el ERS del equipo de *camaroneras*. En 90 minutos de preparación individual, cada revisor encontró entre 8 y 14 defectos. Los cinco más frecuentes fueron:

1. **Ambigüedad** (4 ocurrencias): "El sistema deberá mostrar la calidad del agua" sin especificar qué parámetros.
2. **RNF sin criterio cuantificable** (3): "El sistema deberá ser confiable" sin percentil ni condición de carga.
3. **Falta de fuente en RF** (3): varios RF no tienen el nombre del stakeholder que los originó.
4. **Statechart incompleto** (2): la entidad Piscina carecía del estado FALLA por fallo del sensor.
5. **DFD inconsistente con ER** (2): el depósito D3 del DFD no tenía entidad correspondiente en el diagrama ER.

Densidad de defectos: 11 defectos / 18 páginas = **0,61 defectos/página**. Resultado: ERS de alta calidad (umbral: <3/página).

16.4 Subtema 16.4. Métricas, trazabilidad en CASE y rubrica de evaluación

16.4.1 Métricas de calidad del ERS del PFC con ISO 25010

El tribunal evaluará el ERS del PFC usando las siguientes métricas combinadas de IEEE 29148 y ISO 25010 [1, 2, 4, 10]:

Tabla 16.3. *Rúbrica de evaluación del ERS del PFC — Ingeniería de Requisitos UTEQ. Puntaje total: 100 puntos.*

Criterio	Peso	Indicadores de logro
Completitud del ERS	20 pts	6 capítulos completos; mínimo 25 RF y 8 RNF; glosario; todas las secciones IEEE 29148 presentes.
Calidad de RF/RNF	20 pts	100 % con criterio GWT; 0 % con smells detectables; verbos modales correctos; patrón EARS usado.
Modelos UML	20 pts	ER + clases + DFD N0/N1 + statechart + tabla de coherencia; todos consistentes entre sí.
Trazabilidad	15 pts	RTM con 3 tipos de trazabilidad; cobertura 100 % (todo RF tiene origen y prueba).
Validación y pruebas	15 pts	Acta de revisión con cliente real; revisión cruzada completada; plan de pruebas 15+ casos.
Gestión de la configuración	5 pts	Git con etiqueta v1.0.0; CHANGELOG; estructura de carpetas estándar.
Defensa oral	5 pts	Responde correctamente al menos 3 de 5 solicitudes de cambio simuladas en tiempo real.

16.5 Subtema 17.1–17.3. Defensa final del PFC

16.5.1 Estructura de la presentación oral del ERS

La defensa del ERS del PFC tiene una duración de **20 minutos de presentación + 15 minutos de preguntas del tribunal**. La estructura recomendada es:

Tabla 16.4. *Estructura recomendada de la presentación oral del ERS del PFC.*

Tiempo	Sección	Contenido
3 min	Contexto y stakeholders	Dominio del PFC; mapa de stakeholders; alcance del sistema.
3 min	Proceso de elicitación	Técnicas usadas; número de sesiones; hallazgos clave.
4 min	Modelos UML	ER + diagrama de clases + DFD N1 + statechart de la entidad central.
4 min	ERS: RF y RNF destacados	Los 5 RF más críticos con criterio GWT; tabla ISO 25010.
3 min	Validación y métricas	Resultados de la revisión cruzada; métricas de calidad alcanzadas.
3 min	Gestión del cambio	CRs procesados; versión final del ERS; estado del repositorio Git.

16.5.2 Simulacro de solicitudes de cambio en la defensa

El tribunal presentará **cinco solicitudes de cambio simuladas** durante los 15 minutos de preguntas. El equipo debe, en tiempo real: (1) evaluar el impacto del cambio en el ERS, los modelos UML y la RTM; (2) determinar si el cambio es un CR de tipo parche, menor o mayor; (3) estimar el esfuerzo adicional.

Ejemplo en Ecuador

Ejemplos de solicitudes de cambio en la defensa (por dominio).

PFC	Solicitud del tribunal
Ganadería	"AGROCALIDAD ahora exige el campo *geolocalización del parto* en el certificado zoosanitario. ¿Cuántos RF, modelos y casos de prueba se ven afectados?"
Camaroneras	"El nuevo sensor IoT soporta MQTT en lugar de HTTP. ¿Qué cambia en el ERS, el DFD y los RNF de integración?"
Palma africana	"La extractora exige además del peso bruto el peso neto (descontando humedad). ¿Cómo impacta esto al modelo ER y a los RF del módulo de liquidación?"
Aulas IoT	"El Dpto. de TI exige que los logs del sistema IoT se exporten al SIEM institucional en formato syslog. ¿Nuevo RF o cambio a RNF existente?"
Mascotas	"El cliente quiere añadir el servicio de veterinario a domicilio nocturno (23h–07h) con tarifa distinta. ¿Qué tan grande es el impacto?"
Clínica vet.	"La CONEVET exige que las recetas electrónicas incluyan el número de registro de medicamento del ARCSA. ¿Cuántas entidades y RF cambian?"

16.6 Mapa síntesis de la Semana 16 y cierre del curso

Tabla 16.5. *Mapa de cierre del curso ISR-401 Ingeniería de Requisitos.*

Semanas	Unidad	Hito del PFC alcanzado
1–2	U1: Fundamentos	Comprensión del proceso y del ciclo de vida de IR.
3–8	U2–U3: Elicit./Espec.	ERS v0.1 con RF, RNF, CU, ER, DFD, statechart.
9–12	U3–U4: Modelos/Valid.	ERS v1.0.0 con modelos coherentes, validado y gestionado.
13–14	U4: CASE y ágil	Herramienta CASE configurada; BDD implementado.
15–16	U5: Proyecto integrador	ERS v1.0.0 final + defensa oral ante tribunal.

16.7 Actividades finales del proyecto integrador

Práctica / Actividad

Hito 6 (final) del PFC — Entrega y defensa del ERS v1.0.0.

1. **Revisión cruzada**: recibir y procesar todos los defectos del equipo revisor; documentar las correcciones realizadas.
2. **ERS v1.0.0 definitivo**: documento PDF con portada formal, todos los capítulos, modelos en alta resolución y RTM completa.
3. **Repositorio Git**: tag `v1.0.0`, CHANGELOG actualizado, estructura de carpetas estándar verificada.
4. **Presentación oral**: 20 diapositivas máximo; ensayo previo de al menos una vez completo.
5. **Simulacro de CCB**: el equipo practica responder solicitudes de cambio en tiempo real con los ejemplos de la tabla 16.4.
6. **Métricas finales**: calcular y reportar en la presentación las 6 métricas de gestión de requisitos; mostrar la evolución desde la primera versión del ERS.

16.8 Resumen ejecutivo de la semana y del curso

- El **ERS final del PFC** integra todo el conocimiento del curso: elicitación → análisis → especificación → modelado UML → validación → gestión del cambio → configuración [1, 2].
- La **revisión cruzada** es el mejor simulacro de una inspección real: detecta defectos que el propio equipo no puede ver por el sesgo del autor [10, 65].
- Las **métricas de calidad** (densidad de defectos, cobertura, estabilidad, volatilidad) son la evidencia objetiva de que el proceso de IR fue riguroso [1, 80].
- La **defensa ante el tribunal** con solicitudes de cambio en tiempo real es la demostración más exigente y auténtica de la competencia en IR: el estudiante debe pensar como analista, no solo como redactor de documentos [1, 52].
- Un ERS de calidad es el mejor punto de partida para el desarrollo: el equipo que lo construyó bien no necesitará reescribir la mitad del código durante la implementación [7, 35].

Semana 17

Métricas, trazabilidad y preparación de la defensa

La semana antes de la defensa no es para terminar el ERS; es para demostrar que ya estaba terminado. El equipo que llega con métricas calculadas, trazabilidad verificada y un simulacro de preguntas del tribunal no teme a la defensa: la disfruta.
docentes de IR, UTEQ

Objetivos de aprendizaje de la semana

Al finalizar esta decimoséptima semana, el estudiante estará en capacidad de:

1. Exportar y verificar la **trazabilidad completa** del ERS del PFC desde la herramienta CASE elegida (Jira, draw.io+Git o EA), confirmando cobertura bidireccional del 100 % [1, 2, 66].
2. Aplicar la **rúbrica de evaluación** institucional del ERS del PFC, calculando el puntaje de cada criterio y detectando brechas pendientes de cerrar antes de la defensa [2, 4, 10].
3. Calcular y reportar el conjunto completo de **métricas de calidad del ERS**: densidad de defectos, estabilidad, cobertura de pruebas, volatilidad y tiempo medio de CR [1, 10, 80].
4. Preparar la **presentación oral** del ERS del PFC con la estructura de 20 minutos, incluyendo el simulacro de solicitudes de cambio del tribunal [1, 2, 52].
5. Entregar el **repositorio Git del PFC** con la versión definitiva etiquetada, el CHANGELOG completo y los artefactos de entrega organizados [1, 2, 10].

17.1 Subtema 17.1. Trazabilidad en CASE y configuración final del ERS

17.1.1 Verificación final de la trazabilidad en la herramienta CASE

La semana 17 es la semana de **cierre y verificación**: el equipo no produce nuevos requisitos, sino que verifica que todo lo producido está correctamente enlazado, versionado y documentado. La primera tarea es exportar la RTM desde la herramienta CASE y verificar que ningún requisito queda sin traza en ninguna dirección [1, 2].

Tabla 17.1. *Verificación de la cobertura de trazabilidad por tipo — PFC semana 17. Toda celda en rojo indica una brecha que debe cerrarse antes de la defensa [1, 2, 66].*

Tipo de requisito	Total	Con origen (pre)	Con modelo UML	Con prueba	
RF Must-have	N	N (100 %)	N (100 %)	N (100 %)	
RF Should-have	N	N (100 %)	N (≥80 %)	N (≥80 %)	
RF Could-have	N	N (100 %)	opcional	opcional	
RNF con criterio cuant.	N	N (100 %)	Arch. o NFR	N (100 %)	
RNF sin criterio	0	—	—	—	
TOTAL	**N**	**100 %**	**≥90 %**	**100 % Must**	

Exportar la RTM desde Jira

Para equipos que usan Jira, la exportación de la RTM se realiza con la función **Issues** → **Export** → **Excel CSV** filtrando por el proyecto del PFC. Los campos a incluir son: Issue Key, Summary, Priority, Status, Reporter, Acceptance Criteria (custom), Linked Issues (para trazabilidad interna) y Fix Version. La hoja exportada se formatea como la RTM del Cap. 6 del ERS y se adjunta al repositorio Git bajo `docs/validacion/RTM_v1.0.xlsx` [2, 10].

Trazabilidad visual en Enterprise Architect

En Sparx EA, la trazabilidad se verifica desde el menú **Project** → **Impact Analysis** sobre cualquier elemento: el sistema muestra un grafo completo de los artefactos vinculados al elemento seleccionado en ambas direcciones. Para el PFC, el equipo debe generar esta vista para los 5 RF más críticos y capturar pantallas para incluirlas en los apéndices del ERS [1, 10].

17.1.2 Estado final del repositorio Git del PFC

Lista de verificación del repositorio Git del PFC — semana 17.

1. `git tag v1.0.0` aplicado al commit de entrega final.
2. `CHANGELOG.md` actualizado con todas las versiones desde v0.1.0 hasta v1.0.0, con fecha y descripción de cada cambio.
3. Estructura de carpetas estándar completa: `docs/ERS/`, `docs/modelos/`, `docs/validacion/`, `docs/validacion/actas/`, `docs/validacion/solicitudes_cambio/`.
4. `README.md` con: nombre del sistema, integrantes, director, cliente, herramienta CASE usada, versión del ERS e instrucciones para reproducir los modelos.
5. Todos los modelos en formatos editables: `.drawio` (ER, DFD, actividades), `.puml` (clases, secuencia, statechart).
6. Plan de pruebas en formato `.xlsx` con los 15+ casos de prueba trazados a la RTM.
7. Todas las actas de revisión y los formularios CR en PDF dentro de `docs/validacion/`.

17.2 Subtema 17.2. Evaluación del ERS con rúbrica: autoevaluación final

17.2.1 Autoevaluación del ERS con la rúbrica institucional

Antes de la defensa, el equipo debe aplicar la rúbrica de evaluación a su propio ERS y calcular el puntaje estimado. Esto permite identificar brechas a cerrar en las últimas 48 horas [1, 2, 10].

Tabla 17.2. *Rúbrica de autoevaluación del ERS del PFC — semana 17. El equipo completa las columnas de puntaje propio y evidencia antes de la defensa.*

Criterio	Peso	Indicador de logro pleno (100 %)	Puntaje propio
Completitud del ERS	20	6 capítulos; ≥25 RF; ≥8 RNF; glosario; referencias.	/20
Calidad RF/RNF	20	100 % con GWT; sin smells; EARS; verbos modales correctos.	/20
Modelos UML	20	ER + clases + DFD N0/N1 + statechart + coherencia verificada.	/20
Trazabilidad	15	RTM 3 tipos; cobertura 100 % Must-have; exportada de CASE.	/15
Validación y pruebas	15	Acta cliente; peer review completado; 15+ casos de prueba.	/15
Gestión de configuración	5	Git tag v1.0.0; CHANGELOG; estructura estándar; README.	/5
Defensa oral	5	Estructura 20 min; responde 3/5 CRs en tiempo real.	/5
TOTAL ESTIMADO			**/100**

17.2.2 Errores más frecuentes detectados en defensas anteriores

Con base en la experiencia de defensas de PFC en la Facultad de Ciencias de la Computación de la UTEQ, los errores más frecuentes que el tribunal detecta son [1, 10, 65]:

Tabla 17.3. *Los diez errores más frecuentes detectados por tribunales en defensas de ERS de PFC, organizados por frecuencia.*

#	Error	Ejemplo	Cómo evitarlo
1	RNF sin criterio cuantificable	"El sistema deberá ser rápido."	Especificar número + unidad + percentil + condición.
2	RF sin criterio de aceptación	RF sin sección GWT.	Aplicar el checklist de 15 ítems de la Semana 16.
3	Modelos UML inconsistentes entre sí	Entidad en ER que no aparece en DFD.	Aplicar la tabla de coherencia de la Semana 10.
4	RTM incompleta o sin pretrazabilidad	RF sin fuente (stakeholder).	Verificar columna "Origen" de cada fila.
5	ERS sin acta de validación con el cliente real	Solo revisión interna.	Documentar al menos 1 sesión con cliente real.
6	Verbos modales incorrectos	"El sistema puede" para un Must-have.	Auditoría de verbos: buscar "puede" en todo el ERS.
7	Alcance no delimitado en Cap. 2	El sistema "gestionará todo".	Especificar explícitamente qué queda fuera del alcance.
8	Smells no corregidos	Listas con "entre otros", cuantificadores vagos.	Pasar el checklist de 8 smells de la Semana 7.
9	Git sin tag de baseline	Solo rama main sin etiquetas.	Verificar `git tag` antes de la defensa.
10	Respuesta deficiente a CRs del tribunal	El equipo no puede estimar el impacto.	Practicar el simulacro con los 6 CRs de la Semana 16.

17.2.3 Completitud, consistencia y verificabilidad: checklist final

Pohl [1] e IEEE 29148:2018 [2] definen tres propiedades que el ERS debe exhibir como conjunto antes de cualquier proceso formal de revisión:

Tabla 17.4. *Verificación de completitud, consistencia y verificabilidad del ERS del PFC — checklist final de la semana 17 [1, 2, 10].*

Propiedad	Criterio verificable	Estado
	No existe ningún caso de uso del diagrama UML sin su RF correspondiente en el ERS.	✓/✗
	No existe ningún depósito de datos del DFD sin su entidad en el ER.	✓/✗
	No existe ningún stakeholder en el mapa que no haya aportado al menos un requisito.	✓/✗
	Todas las características ISO 25010:2023 relevantes tienen al menos un RNF.	✓/✗
	No existen dos RF que se contradigan en el mismo ERS.	✓/✗
	Los estados del statechart coinciden exactamente con los valores del atributo Enum en la clase correspondiente.	✓/✗
	Las multiplicidades del ER son las mismas que las cardinalidades del diagrama de clases.	✓/✗
Verificabilidad	Todo RF Must-have tiene criterio de aceptación GWT con resultado esperado medible.	✓/✗
	Todo RNF tiene valor numérico, unidad y condición de medición especificados.	✓/✗
	El plan de pruebas tiene un caso de prueba para cada criterio de aceptación.	✓/✗

17.3 Subtema 17.3. Preparación y estructura de la defensa final

17.3.1 Estructura de la presentación: los 20 minutos más importantes

Tabla 17.5. *Guión detallado de la presentación oral del ERS del PFC por minuto. La columna "Evidencia" indica el artefacto que debe estar visible en la pantalla durante ese segmento.*

Minutos	Sección	Contenido	Evidencia visible
0–3	Contexto y dominio	Nombre del sistema; dominio PFC; cliente real; problema que resuelve; mapa de stakeholders.	Mapa de stakeholders.
3–6	Proceso de elicitación	Técnicas usadas (entrevista, taller, observación); número de sesiones; hallazgos clave; conflictos resueltos.	Acta de entrevista (portada) + mapa de actores i*.
6–10	Modelos UML	ER del dominio; diagrama de clases; DFD N1; statechart de la entidad central; tabla de coherencia.	Modelos en draw.io o EA.
10–14	ERS: RF y RNF	Los 5 RF más críticos con criterio GWT; 3 RNF con criterio cuantificable; tabla ISO 25010.	Fragmento del ERS Cap. 4 y 5.
14–17	Validación y métricas	Resultados de la revisión cruzada (densidad de defectos); sesión con cliente (discrepancias CR); métricas finales.	Acta de revisión + tabla de métricas.
17–20	Gestión y cierre	ERS v1.0.0 en Git; número de CRs procesados; versión final; repositorio accesible.	Pantalla GitHub/GitLab con tag v1.0.0.

Consejos de presentación oral para la defensa del ERS

- **Dominar el glosario del dominio.** Si el PFC es sobre camaroneras, el equipo debe saber qué es oxígeno disuelto, tasa de alimentación y biomasa. El tribunal hará preguntas en el lenguaje del cliente, no en el lenguaje de la ingeniería de software.
- **Llevar el ERS impreso o en tablet.** Cuando el tribunal pida ver un RF específico, el equipo debe encontrarlo en menos de 20 segundos. Si no puede localizar su propio ERS, el mensaje es que no lo conoce.
- **Preparar 5 respuestas estándar para las preguntas más frecuentes del tribunal**: (1) ¿Por qué eligieron ese sistema?, (2) ¿Cuántas sesiones tuvieron con el cliente?, (3) ¿Cómo resolvieron el conflicto de requisitos más difícil?, (4) ¿Qué queda fuera del alcance?, (5) ¿Qué cambiaría si tuviera que repetir el proceso?
- **No memorizar el ERS: comprenderlo.** El tribunal detecta en segundos si el equipo

recita o razona. La diferencia es que quien razona puede responder solicitudes de cambio; quien recita, no.

- **Dividir los roles de la presentación** con anticipación: quién habla sobre elicitación, quién sobre modelos, quién sobre validación. Los cambios de orador no deben durar más de 5 segundos.

17.3.2 Simulacro de respuesta a solicitudes de cambio

El simulacro se realiza con el equipo completo en la sala de trabajo, un miembro actuando como tribunal y el resto respondiendo. Para cada CR simulado, el equipo debe responder en **menos de 90 segundos**:

1. **Clasificar el CR**: ¿es un parche (typo, criterio), menor (RF adicional) o mayor (nuevo módulo)?
2. **Identificar el impacto**: ¿qué RF, entidades del ER, estados del statechart y casos de prueba se ven afectados?
3. **Estimar el esfuerzo**: ¿cuántos días de trabajo adicional?
4. **Proponer la acción**: aprobar, rechazar (justificando) o posponer a la versión 2.0.

Ejemplo en Ecuador

Simulacro resuelto — PFC sistema de gestión ganadera.
CR del tribunal: "AGROCALIDAD acaba de publicar la Resolución DAJ-2025-001 que exige incluir la coordenada GPS del evento sanitario (vacunación, desparasitación) en el certificado zoosanitario. ¿Cuál es el impacto en su ERS?"
Respuesta del equipo (75 segundos):

- **Tipo**: CR menor (nuevo atributo, no nuevo módulo).
- **Impacto en el ER**: añadir atributo `gpsLat: Decimal` y `gpsLng: Decimal` a la entidad `EventoSanitario`.
- **Impacto en RF**: modificar **RF-G05** (registrar evento sanitario) añadiendo la captura del GPS; nuevo criterio de aceptación GWT.
- **Impacto en pruebas**: actualizar CP-G05 para verificar que las coordenadas se registran y exportan correctamente.
- **Esfuerzo**: 1 día de análisis + actualización del ERS.
- **Acción**: aprobado para la versión actual; implementar en el módulo de campo de la app móvil.

17.4 Mapa síntesis de la Semana 17

Tabla 17.6. *Mapa de cierre de la Semana 17 — semana de verificación y ensayo.*

Subtema	Tarea concreta	Entregable antes de la defensa
17.1 Trazabilidad CASE	Exportar RTM; verificar cobertura; organizar repositorio Git.	RTM exportada + Git con tag v1.0.0 + README.
17.2 Rúbrica y checklist	Autoevaluación 100 pts; checklist completitud/consistencia/verificabilidad.	Autoevaluación firmada + lista de brechas cerradas.
17.3 Preparación defensa	Guión 20 min; 5 respuestas estándar; simulacro de 6 CRs.	Presentación final + simulacro completado.

17.5 Actividades de la semana 17

Práctica / Actividad

Sesión de taller en clase (2 horas) — Semana 17.

Ensayo general de la defensa.

1. Cada equipo presenta su ERS en exactamente 20 minutos ante el resto de la clase. El docente actúa como moderador del tiempo.
2. Los otros equipos actúan como tribunal: cada uno prepara 2 preguntas y 1 solicitud de cambio del dominio del PFC presentado.
3. El equipo responde en tiempo real a las preguntas y al menos 2 CRs.
4. Retroalimentación pública (5 min por equipo): el docente y los «tribunales»señalan fortalezas y hasta 3 aspectos a mejorar.
5. El equipo registra la retroalimentación en el repositorio Git (`docs/validacion/retroalimentacion_ensayo.md`) y actualiza el ERS si detecta brechas en las últimas 48 horas.

Práctica / Actividad

Tarea autónoma — Semana 17 (cierre del PFC, 4 horas).

1. **Autoevaluación**: completar la rúbrica de 100 puntos y el checklist de completitud/consistencia/verificabilidad; documentar cada brecha encontrada con la acción correctiva tomada.
2. **RTM exportada**: generar el archivo Excel/PDF desde la herramienta CASE y verificar que el 100 % de los RF Must-have tienen origen, modelo UML y caso de prueba.
3. **Git final**: verificar tag v1.0.0, CHANGELOG, estructura de carpetas, README y accesibilidad del repositorio para el tribunal.
4. **Presentación final**: 20 diapositivas máximo; ensayar la presentación completa al menos dos veces en equipo.
5. **Simulacro de CRs**: practicar los 6 CRs de la Semana 16 y los CRs específicos del dominio propio hasta poder responder cada uno en menos de 90 segundos.

17.6 Resumen ejecutivo de la semana

- La semana 17 es de **verificación, no de producción**: todo lo que no estaba en el ERS antes del lunes no debería añadirse bajo presión [1, 10].
- La **autoevaluación con la rúbrica** antes de la defensa elimina las sorpresas: el equipo llega sabiendo exactamente qué puntaje espera y por qué [2, 4].
- Los **diez errores más frecuentes** (RNF sin criterio, RF sin GWT, modelos inconsistentes, RTM incompleta) se detectan con el checklist de completitud en menos de 30 minutos [1, 10, 65].
- El **simulacro de CRs** es el mejor entrenamiento para la defensa: convierte el miedo a las preguntas del tribunal en una competencia verificable [1, 52].
- Un **repositorio Git ordenado** con tag, CHANGELOG y README es en sí mismo evidencia de madurez en el proceso de IR; el tribunal lo verifica durante la defensa [2].

Semana 18

Defensa formal, evaluación y cierre del semestre

El ingeniero de software que puede escribir un ERS completo, coherente y verificable sobre el sistema de una camaronera, una hacienda ganadera o una clínica veterinaria — y defenderlo ante un tribunal exigente — ha demostrado que puede hacerlo para cualquier sistema en cualquier dominio. Eso es lo que hace un ingeniero.
docentes de IR, UTEQ

Objetivos de aprendizaje de la semana

Al finalizar esta decimoctava semana, el estudiante estará en capacidad de:

1. **Defender** el ERS del PFC ante el tribunal docente, respondiendo preguntas técnicas y solicitudes de cambio en tiempo real con argumentos fundamentados en las normas IEEE 29148:2018 e ISO 25010:2023 [1, 2, 4].
2. **Evaluar** el ERS de otro equipo aplicando la rúbrica institucional como miembro del tribunal [10, 65].
3. **Reflexionar** sobre su propio proceso de IR: qué técnicas funcionaron mejor, qué errores se cometieron y qué haría diferente en un proyecto real [1, 3, 52].
4. **Identificar** las competencias de IR adquiridas durante el semestre y vincularlas con las demandas del mercado laboral ecuatoriano e internacional [3, 56].

18.1 Protocolo de la defensa formal del PFC

18.1.1 Estructura de la sesión de defensa

La sesión de defensa del ERS del PFC es un acto académico formal con la siguiente estructura [1, 2, 52]:

Tabla 18.1. *Protocolo completo de la sesión de defensa del PFC de IR — Semana 18.*

Tiempo	Fase	Descripción
0–2 min	Apertura formal	El moderador (docente titular) presenta al equipo, al cliente real (si asiste) y a los miembros del tribunal. Enuncia las reglas: tiempo estricto, preguntas al final, grabación permitida solo con consentimiento.
2–22 min	Presentación oral	El equipo presenta el ERS siguiendo el guión de la Semana 17 (6 secciones de 20 minutos). El tribunal escucha sin interrumpir.
22–37 min	Preguntas del tribunal	Cada miembro del tribunal formula hasta 2 preguntas técnicas y 1 solicitud de cambio. El equipo responde en tiempo real con referencia explícita al ERS, los modelos o la RTM.
37–42 min	Deliberación privada	El tribunal se reúne a puerta cerrada para deliberar y completar la rúbrica. El equipo y el público esperan fuera.
42–47 min	Comunicación del resultado	El moderador anuncia el puntaje final y la calificación. El tribunal entrega retroalimentación verbal positiva y constructiva.
47–50 min	Cierre	El moderador felicita al equipo, agradece al cliente real (si asiste) y declara concluida la defensa.

18.1.2 Rúbrica del tribunal: cómo se aplica la calificación

Cada miembro del tribunal completa independientemente la rúbrica de evaluación y los puntajes se promedian. La calificación final del PFC de IR se calcula como [2, 10]:

$$\text{Nota final} = \frac{\sum_{i=1}^{N_T} \text{Puntaje}_i}{N_T} \times \frac{10}{100} \tag{18.1}$$

donde N_T es el número de miembros del tribunal (típicamente 3 en la UTEQ). La tabla siguiente muestra cómo el tribunal interpreta cada nivel de logro:

Tabla 18.2. *Niveles de logro de la rúbrica del tribunal para la defensa del ERS del PFC.*

Puntaje	Nivel	Descriptor
90–100	Excelente	ERS completo, consistente y verificable; modelos UML coherentes entre sí; RTM con cobertura 100 %; responde todos los CRs con precisión técnica; repositorio Git organizado.
75–89	Muy bueno	ERS con mínimas brechas ($\leq$2 ítems del checklist); modelos generalmente coherentes; RTM $\geq$90 %; responde la mayoría de los CRs; repositorio correcto.
60–74	Bueno	ERS funcional pero con algunas omisiones relevantes o RNF sin criterio cuantificable; trazabilidad parcial; responde los CRs más simples.
40–59	Suficiente	ERS básico: menos de 25 RF o modelos inconsistentes; RTM incompleta; dificultades para responder CRs; repositorio desorganizado.
$<$40	Insuficiente	ERS no cumple los criterios mínimos de IEEE 29148:2018; modelos ausentes o incorrectos; no hay trazabilidad; no puede responder preguntas del tribunal.

18.1.3 Preguntas más frecuentes del tribunal

Con base en la experiencia de defensas de PFC en la FCI–UTEQ, el tribunal suele hacer las siguientes preguntas. El equipo debe preparar respuestas sólidas para cada una:

Tabla 18.3. *Las doce preguntas más frecuentes del tribunal en defensas de ERS de PFC, organizadas por tema.*

Tema	Pregunta del tribunal	Lo que el tribunal evalúa
Alcance	¿Qué queda explícitamente fuera del alcance de este sistema?	Si el equipo delimitó el alcance conscientemente.
Elicit.	¿Cuántas sesiones tuvieron con el cliente real y cómo las documentaron?	Rigor del proceso de elicitación.
Conflictos	¿Cuál fue el conflicto de requisitos más difícil que tuvieron y cómo lo resolvieron?	Capacidad de negociación técnica.
Modelos	¿Por qué en el ER la entidad X tiene cardinalidad N y no 1?	Comprensión real del dominio del negocio.
Statechart	¿Puede mostrarme en el statechart qué ocurre cuando [evento inesperado]?	Completitud del modelo de comportamiento.
RNF	¿Cómo van a medir el cumplimiento del RNF de rendimiento especificado?	Verificabilidad de los RNF.
Trazab.	¿Puede mostrarme la trazabilidad del RF-X hasta su prueba de aceptación?	Integridad de la RTM.
Validació	¿Qué encontraron en la revisión cruzada con el otro equipo?	Actitud ante los defectos propios.
IoT/campo	¿Qué pasa si el sistema pierde la señal GSM en campo por 12 horas?	Requisitos de operación offline.
Segurida	¿Cómo garantizan que los datos del cliente cumplen la LOPDP?	Requisitos legales y de seguridad.
Cambio	[CR en tiempo real adaptado al dominio del PFC]	Capacidad de análisis de impacto.
Reflexió	¿Qué cambiarían del proceso de IR si tuvieran que repetirlo?	Madurez y capacidad de aprendizaje.

18.2 Evaluación final y coevaluación entre equipos

18.2.1 Coevaluación: los estudiantes como tribunal

En la sesión de defensa, los estudiantes de otros equipos participan como **tribunal externo** usando la misma rúbrica institucional. Su calificación tiene un peso del 20 % en la nota final, promediada con la del tribunal docente (80 %). Esta práctica desarrolla la competencia de evaluación técnica entre pares, que es clave en el perfil del Ingeniero de Software [10, 52, 65].

Las reglas de la coevaluación son:

- El equipo coevaluador no puede pertenecer al mismo dominio de PFC que el equipo evaluado (para garantizar perspectiva independiente).
- La coevaluación debe estar fundamentada: cada puntaje inferior a 80 debe ir acompañado de un ejemplo específico del ERS.
- El resultado de la coevaluación se entrega al docente por escrito antes de que el tribunal delibere.

18.3 Reflexión final: el proceso de IR visto desde el final del semestre

18.3.1 ¿Qué hemos aprendido sobre IR en 18 semanas?

La tabla siguiente resume las competencias desarrolladas a lo largo del semestre y las vincula con los perfiles del mercado laboral ecuatoriano e internacional [1, 3, 56]:

Tabla 18.4. *Competencias de IR adquiridas en el semestre y su valoración en el mercado laboral del sector tecnológico ecuatoriano.*

Competencia	Semanas donde se desarrolló	Rol laboral que la requiere
Elicitar requisitos con clientes reales	3–5, 15	Analista de sistemas, Product Owner, Scrum Master.
Modelar con UML (ER, clases, DFD, statechart)	9–10	Arquitecto de software, Analista funcional.
Redactar ERS con IEEE 29148	7–8, 16	Consultor de calidad, Auditor de software.
Validar requisitos con Fagan e inspecciones	11	QA Lead, Responsable de calidad.
Gestionar cambios con CCB	12	Project Manager, Release Manager.
Usar herramientas CASE (Jira, EA, Git)	13	Cualquier rol de desarrollo ágil.
Aplicar BDD con Gherkin	14	QA Automation, DevOps.
Defender un ERS ante un tribunal técnico	17–18	Senior Analyst, Tech Lead.

18.3.2 Las diez lecciones del semestre

1. **Los requisitos no se recolectan: se construyen.** La elicitación es un proceso activo de comunicación, no de transcripción. El analista que solo escucha y escribe produce requisitos incompletos [1, 51].
2. **El cliente no sabe lo que quiere hasta que lo ve.** El prototipo de validación es imprescindible: convierte especificaciones abstractas en representaciones concretas que el cliente puede confirmar o corregir [10, 57].
3. **Un RNF sin criterio cuantificable no es un requisito: es un deseo.** "El sistema deberá ser rápido" es tan inútil como no escribir nada. La cuantificación es la diferencia entre un ERS profesional y un listado de intenciones [2, 17].
4. **Los modelos UML no son decoración.** Cada entidad del ER sin RF correspondiente es un agujero en el ERS; cada estado del statechart sin transición documentada es un comportamiento indefinido del sistema [1, 71, 72].
5. **El conflicto de requisitos es inevitable: su gestión, no.** Los proyectos fracasan no por tener conflictos sino por no detectarlos a tiempo o por resolverlos con autoridad en lugar de con argumentos técnicos [1, 52].

6. **La trazabilidad no es un lujo: es una infraestructura de seguridad.** Sin trazabilidad, cada cambio de requisito es una operación a ciegas; con ella, el impacto de cualquier cambio se calcula en minutos [2, 66].
7. **El ERS que no ha sido revisado por el cliente no existe como documento de IR: es ficción técnica.** La validación con el stakeholder real es la única prueba de que el analista entendió correctamente [1, 10].
8. **Ágil no significa sin documentación: significa documentación justa y oportuna.** Un backlog sin criterios de aceptación es tan deficiente como un ERS sin criterios cuantificables [36-38].
9. **La calidad del ERS se mide, no se opina.** Densidad de defectos, estabilidad de requisitos, cobertura de trazabilidad: estas métricas convierten la calidad en un dato gestionable, no en una sensación subjetiva [10, 80].
10. **La ingeniería de requisitos es la actividad más humana de la ingeniería de software.** Requiere escuchar, negociar, comunicar, facilitar y convencer. El ingeniero que solo sabe programar no puede hacer IR; el que sabe hacer IR puede coordinar cualquier equipo de desarrollo [1, 3, 51].

18.3.3 El camino hacia la certificación IREB

El egresado que concluye esta asignatura con un ERS de calidad tiene las bases para obtener la certificación **IREB CPRE Foundation Level** [56], que es el estándar internacional de competencia en IR más reconocido en Europa y Latinoamérica.

Tabla 18.5. *Correspondencia entre los contenidos del semestre ISR-401 y los bloques de conocimiento del IREB CPRE v3.2.0 [1, 56].*

Bloque IREB CPRE v3.2.0	Contenido del examen	Semanas ISR-401
1. Fundamentos de IR	Definición, tipos, proceso, rol del analista.	1–2
2. Elicitación	Técnicas, fuentes, documentación.	3–5
3. Documentación	ERS, LN estructurado, EARS, atributos.	6–8
4. Modelos de sistemas	ER, clases, CU, actividades, statechart.	9–10
5. Validación	Revisiones, prototipos, pruebas.	11
6. Gestión de requisitos	Cambios, trazabilidad, métricas, baselines.	12–13
7. IR ágil	Backlog, historias, BDD, Scrum.	14

18.4 Cierre del semestre: evaluación del proceso de aprendizaje

18.4.1 Autoevaluación y coevaluación del proceso

La última actividad del semestre es la **autoevaluación del proceso de aprendizaje**: cada estudiante reflexiona honestamente sobre su contribución al PFC y sobre las competencias que desarrolló [1, 52]:

Práctica / Actividad

Instrumento de autoevaluación — ISR-401 Ingeniería de Requisitos.

Responde individualmente (no en equipo). No hay respuestas incorrectas; el objetivo es la honestidad reflexiva.

1. ¿Qué técnica de elicitación te resultó más difícil de aplicar en el PFC real y por qué?
2. ¿Cuál fue el momento en que más claramente entendiste la diferencia entre un RF bien escrito y uno mal escrito? Describe la situación.
3. ¿Qué aspecto del ERS de tu PFC mejorarías si tuvieras una semana más? ¿Qué te faltó para completarlo en el tiempo disponible?
4. Si mañana comenzaras un proyecto de IR en una empresa real, ¿qué harías en la primera semana? ¿Y en la segunda?
5. En una escala de 1 a 10, ¿qué tan preparado te sientes para trabajar como analista de sistemas o Product Owner? ¿Por qué ese número y no uno mayor?

18.4.2 Mensaje de cierre

A los estudiantes de ISR-401 Ingeniería de Requisitos, UTEQ.

Hace 18 semanas, un sistema de gestión de ganado, una plataforma de camaroneras o una red de aulas inteligentes eran solo una idea de PFC. Hoy son especificaciones formales: entidades con cardinalidades, procesos con flujos de datos, entidades reactivas con ciclos de vida definidos, requisitos con criterios que el cliente puede verificar y el desarrollador puede implementar.

Han aprendido que los requisitos son la primera forma de código que existe en un proyecto: el código que describe el problema antes de que alguien escriba la solución. Y que ese código — si está bien escrito — tiene la misma importancia que cualquier línea de Python, Java o C++ que produzcan en su carrera.

El dominio cambiará. Las técnicas evolucionarán. Los estándares se actualizarán. Pero la competencia de escuchar al cliente, modelar su mundo, especificar con precisión y defender con argumentos: esa competencia no expira.

Ahora: a aplicarla.

Los docentes de Ingeniería de Requisitos — UTEQ, FCI.

18.5 Mapa de cierre del semestre completo

Tabla 18.6. *Mapa completo del semestre ISR-401 — de la Semana 1 a la Semana 18.*

Semanas	Unidad / Tema	Competencia central	Hito del PFC
1–2	U1: Fundamentos	Entender el proceso de IR y los tipos de requisitos.	Visión del sistema + mapa de stakeholders.
3–5	U2: Elicitación	Elicitar con técnicas clásicas y avanzadas.	Plan de elicitación + candidatos a RF.
6–8	U3: Especificación	Especificar metas, escenarios y modelos.	ERS v0.1 con CU, ER y DFD.
9–10	U3: Modelos UML	Modelar datos, función y comportamiento.	ER + clases + DFD N1 + statechart.
11–12	U4: Valid./Gestión	Validar con Fagan y gestionar cambios con CCB.	ERS v1.0.0 baseline-1.
13–14	U4: CASE y ágil	Usar herramientas CASE; aplicar BDD y Scrum.	Backlog en Jira + feature files.
15–16	U5: PFC	Elicitar, especificar y modelar con cliente real.	ERS v1.0.0 completo.
17	U5: PFC cierre	Verificar, medir y preparar la defensa.	Git tag v1.0.0 + rúbrica autoevaluada.
18	U5: Defensa	Defender y evaluar el ERS ante tribunal.	Nota final + certificado de competencia IR.

Glosario de términos

Modelo Kano Modelo de priorización que clasifica los requisitos según el tipo de satisfacción que producen: básicos (must-be), de desempeño, atractivos (delighters), indiferentes y contraindicados.

Product backlog Lista priorizada y dinámica de todo el trabajo (historias, requisitos, defectos, tareas técnicas) pendiente para un producto en metodología Scrum, gestionada por el Product Owner.

Requisito funcional Requisito que describe una función, servicio o tarea que el sistema debe ejecutar. Responde a la pregunta «¿qué debe hacer el sistema?»

Requisito no funcional Requisito que describe atributos de calidad, restricciones o condiciones del sistema, en lugar de una función específica. Habituales son los requisitos de rendimiento, seguridad, fiabilidad, usabilidad o portabilidad. Pueden modelarse mediante el catálogo ISO/IEC 25010:2023.

Stakeholder Individuo, grupo u organización que tiene un derecho, una participación, una reclamación o un interés legítimo en el sistema o en sus capacidades. Incluye usuarios finales, clientes, operadores, desarrolladores, mantenedores, reguladores y entidades afectadas directa o indirectamente por el sistema. Equivalente en español: interesado, parte interesada.

Referencias bibliográficas

[1] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.ª ed. Berlin, Heidelberg, Germany: Springer, 2025, pág. 907, Originally published 2024 (Springer online), print edition April 2025, ISBN: 978-3-662-69204-2. DOI: 10.1007/978-3-662-69205-9. dirección: https://link.springer.com/book/9783662692042.

[2] IEEE, *IEEE/ISO/IEC 29148-2018 – ISO/IEC/IEEE International Standard – Systems and software engineering – Life cycle processes – Requirements engineering*, Institute of Electrical y Electronics Engineers (IEEE), New York, NY, USA, nov. de 2018, ISBN: 978-1-5044-5302-8. DOI: 10.1109/ieeestd.2018.8559686. dirección: https://ieeexplore.ieee.org/document/8559686.

[3] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge V4.0a*, ed. por H. Washizaki, IEEE Computer Society, Los Alamitos, CA, USA, 2024. dirección: https://www.computer.org/education/bodies-of-knowledge/software-engineering.

[4] ISO/IEC, *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, 2.ª ed., International Organization for Standardization, Geneva, Switzerland, nov. de 2023. dirección: https://www.iso.org/standard/78176.html.

[5] Universidad Técnica Estatal de Quevedo, *Programa Analítico de la Asignatura Ingeniería de Requisitos (ISR-401)*, Facultad de Ciencias de la Computación, Carrera de Ingeniería de Software, Quevedo, Los Ríos, Ecuador, 2025.

[6] M. Daun, A. M. Grubb, V. Stenkova y B. Tenbergen, «A systematic literature review of requirements engineering education,» *Requirements Engineering*, vol. 28, págs. 145-175, 2023. DOI: 10.1007/s00766-022-00381-9.

[7] B. W. Boehm y V. R. Basili, «Software Defect Reduction Top 10 List,» *Computer*, vol. 34, n.º 1, págs. 135-137, ene. de 2001. DOI: 10.1109/2.962984.

[8] R. N. Charette, «Why Software Fails,» *IEEE Spectrum*, vol. 42, n.º 9, págs. 42-49, 2005. DOI: 10.1109/MSPEC.2005.1502528.

[9] The Standish Group, «CHAOS Report 2020: Beyond Infinity,» The Standish Group International, inf. téc., 2020. dirección: https://www.standishgroup.com/.

[10] K. E. Wiegers y J. Beatty, *Software Requirements*, 3.ª ed. Redmond, WA, USA: Microsoft Press, 2013, ISBN: 978-0-7356-7966-5.

[11] I. Sommerville, *Ingeniería del software*, 10.ª ed. Madrid, España: Pearson Educación, 2016, ISBN: 978-1-292-09613-1.

[12] MINTEL, *Plan Nacional de Gobierno Electrónico 2018-2021*, Ministerio de Telecomunicaciones y de la Sociedad de la Información, Quito, Ecuador, 2018. dirección: https://www.gobiernoelectronico.gob.ec/.

[13] Secretaría Nacional de Planificación, *Plan Nacional de Desarrollo «Creando Oportunidades 2021-2025»*, Gobierno del Ecuador, Quito, Ecuador, 2021. dirección: https://www.planificacion.gob.ec/.

[14] Asamblea Nacional del Ecuador, *Ley Orgánica de Protección de Datos Personales*, Registro Oficial Suplemento No. 459, Quito, Ecuador, mayo de 2021. dirección: https://www.telecomunicaciones.gob.ec/.

[15] A. Mavin, P. Wilkinson, A. Harwood y M. Novak, «EARS (Easy Approach to Requirements Syntax),» en *17th IEEE International Requirements Engineering Conference*, 2009, págs. 317-322. DOI: 10.1109/RE.2009.9.

[16] Servicio de Rentas Internas (SRI), *Ficha Técnica de Comprobantes Electrónicos – Versión 2.27*, SRI Ecuador, Quito, Ecuador, 2024. dirección: https://www.sri.gob.ec/comprobantes-electronicos.

[17] M. Glinz, «On Non-Functional Requirements,» *15th IEEE International Requirements Engineering Conference (RE'07)*, págs. 21-26, 2007. DOI: 10.1109/RE.2007.45.

[18] L. Chung, B. A. Nixon, E. Yu y J. Mylopoulos, *Non-Functional Requirements in Software Engineering*. Boston, MA, USA: Kluwer Academic Publishers, 2000, ISBN: 978-0-7923-8666-7.

[19] J. Eckhardt, A. Vogelsang y D. M. Fernández, «Are "Non-functional" Requirements really Non-functional? An Investigation of Non-functional Requirements in Practice,» *Proceedings of the 38th International Conference on Software Engineering*, págs. 832-842, 2016. DOI: 10.1145/2884781.2884788.

[20] ISO/IEC, *ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*, International Organization for Standardization, Geneva, Switzerland, 2011. dirección: https://www.iso.org/standard/35733.html.

[21] I. Inayat, S. S. Salim, S. Marczak, M. Daneva y S. Shamshirband, «A systematic literature review on agile requirements engineering practices and challenges,» *Computers in Human Behavior*, vol. 51, págs. 915-929, 2015. DOI: 10.1016/j.chb.2014.10.046.

[22] K. Curcio, T. Navarro, A. Malucelli y S. Reinehr, «Requirements engineering: A systematic mapping study in agile software development,» *Journal of Systems and Software*, vol. 139, págs. 32-50, 2018. DOI: 10.1016/j.jss.2018.01.036.

[23] ISO/IEC/IEEE, *ISO/IEC/IEEE 12207:2017 – Systems and software engineering – Software life cycle processes*, International Organization for Standardization, Geneva, Switzerland, 2017. dirección: https://www.iso.org/standard/63712.html.

[24] P. Sawyer, I. Sommerville y S. Viller, «Requirements Process Improvement Through the Phased Introduction of Good Practice,» en *Software Process: Improvement and Practice*, vol. 3, 1997, págs. 19-34. DOI: 10.1002/(SICI)1099-1670(199703)3:1<19::AID-SPIP66>3.0.CO;2-X.

[25] ISO/IEC, *ISO/IEC 33001:2015 – Information technology – Process assessment – Concepts and terminology*, International Organization for Standardization, Geneva, Switzerland, 2015. dirección: https://www.iso.org/standard/54175.html.

[26] M. Niazi, K. Cox y J. Verner, «A Maturity Model for the Implementation of Software Process Improvement: an empirical study,» *Journal of Systems and Software*, vol. 74, n.º 2, págs. 155-172, 2005. DOI: 10.1016/j.jss.2003.10.017.

[27] B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi y J. Magee, «Software Engineering for Self-Adaptive Systems: A Research Roadmap,» en *Software Engineering for Self-Adaptive Systems*, ép. Lecture Notes in Computer Science, vol. 5525, Springer, 2009, págs. 1-26. DOI: 10.1007/978-3-642-02161-9_1.

[28] Z. Yang, Z. Li, Z. Jin e Y. Chen, «A Systematic Literature Review of Requirements Modeling and Analysis for Self-Adaptive Systems,» en *Requirements Engineering: Foundation for Software Quality (REFSQ 2014)*, ép. Lecture Notes in Computer Science, vol. 8396, Springer, 2014, págs. 55-71. DOI: 10.1007/978-3-319-05843-6_5.

[29] G. Kotonya e I. Sommerville, *Requirements Engineering: Processes and Techniques*. Chichester, England: John Wiley & Sons, 1998, ISBN: 978-0-471-97208-2.

[30] CMMI Institute, *CMMI Model V3.0 – ISACA*, ISACA / CMMI Institute, 2023. dirección: https://cmmiinstitute.com/cmmi.

[31] R. S. Pressman y B. R. Maxim, *Ingeniería del software: un enfoque práctico*, 8.ª ed. Ciudad de México, México: McGraw-Hill, 2015, ISBN: 978-607-15-1118-8.

[32] W. W. Royce, «Managing the Development of Large Software Systems,» *Proceedings of IEEE WESCON*, págs. 1-9, 1970.

[33] B. W. Boehm, «A Spiral Model of Software Development and Enhancement,» *Computer*, vol. 21, n.º 5, págs. 61-72, 1988. DOI: 10.1109/2.59.

[34] L. Cao y B. Ramesh, «Agile Requirements Engineering Practices: An Empirical Study,» en *IEEE Software*, vol. 25, 2008, págs. 60-67. DOI: 10.1109/MS.2008.1.

[35] H. F. Hofmann y F. Lehner, «Requirements Engineering as a Success Factor in Software Projects,» *IEEE Software*, vol. 18, n.º 4, págs. 58-66, 2001. DOI: 10.1109/MS.2001.936219.

[36] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland y D. Thomas, «Manifesto for Agile Software Development,» The Agile Alliance, 2001. dirección: https://agilemanifesto.org/.

[37] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley Professional, 2004, ISBN: 978-0-321-20568-1.

[38] K. Schwaber y J. Sutherland, *The Scrum Guide – The Definitive Guide to Scrum: The Rules of the Game*. 2020. dirección: https://scrumguides.org/scrum-guide.html.

[39] D. Méndez Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetrò, T. Conte, M.-T. Christiansson, D. Greer, C. Lassenius, T. Männistö, M. Nayebi, M. Oivo, B. Penzenstadler, D. Pfahl, R. Prikladnicki, G. Ruhe, A. Schekelmann, S. Sen, R. Spinola, A. Tuzcu, J. L. de la Vara y R. Wieringa, «Naming the pain in requirements engineering: contemporary problems, causes, and effects in practice,» *Empirical Software Engineering*, vol. 22, págs. 2298-2338, 2017. DOI: 10.1007/s10664-016-9451-7.

[40] E. Bjarnason, K. Wnuk y B. Regnell, «Requirements are slipping through the gaps – A case study on causes & effects of communication gaps in large-scale software development,» *Empirical Software Engineering*, vol. 19, págs. 1107-1146, 2014. DOI: 10.1007/s10664-013-9255-y.

[41] E. S. K. Yu, «Towards modelling and reasoning support for early-phase requirements engineering,» en *Proceedings of ISRE '97: 3rd IEEE International Symposium on Requirements Engineering*, 1997, págs. 226-235. DOI: 10.1109/ISRE.1997.566873.

[42] A. van Lamsweerde, «Goal-Oriented Requirements Engineering: A Guided Tour,» *Proceedings Fifth IEEE International Symposium on Requirements Engineering*, págs. 249-262, 2001. DOI: 10.1109/ISRE.2001.948567.

[43] A. Ferrari, S. Abualhaija y C. Arora, «Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review,» *arXiv preprint arXiv:2509.11446*, 2025. DOI: 10.48550/arXiv.2509.11446.

[44] H. Cheng, J. S. P. P. Husen, Y. Lu, T. Racharak, N. Yoshioka, N. Ubayashi y H. Washizaki, «Generative AI for Requirements Engineering: A Systematic Literature Review,» *arXiv preprint arXiv:2409.06741*, 2024. DOI: 10.48550/arXiv.2409.06741.

[45] C. Arora, J. Grundy y M. Abdelrazek, «Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs,» *Generative AI for Effective Software Development*, págs. 129-148, 2024. DOI: 10.1007/978-3-031-55642-5_6.

[46] S.-C. Necula, F. Dumitriu y V. Greavu-Serban, «A Systematic Literature Review on Using Natural Language Processing in Software Requirements Engineering,» *Electronics*, vol. 13, n.º 11, pág. 2055, 2024. DOI: 10.3390/electronics13112055.

[47] E. W. Pujiharto, E. Tikasni, R. Lewu, S. Sudirman y E. Utami, «Systematic Literature Review on Software Requirement Engineering in 5.0 Industry: Current Practices and Future Challenges,» *International Journal of Advanced Science Computing and Engineering*, vol. 6, n.º 3, págs. 104-108, 2024. DOI: 10.62527/ijasce.6.3.152.

[48] INCOSE, *INCOSE Systems Engineering Handbook, 5th Edition*, 5.ª ed., International Council on Systems Engineering, Wiley, 2023, ISBN: 978-1-119-81429-0. dirección: https://www.incose.org/products-and-publications/se-handbook.

[49] B. Penzenstadler y H. Femmer, «A Generic Model for Sustainability with Process- and Product-Specific Instances,» en *Workshop on Green by Software Engineering*, 2013, págs. 3-8. DOI: 10.1145/2451605.2451609.

[50] D. Zowghi y C. Coulin, «Requirements Elicitation: A Survey of Techniques, Approaches, and Tools,» en *Engineering and Managing Software Requirements*, A. Aurum y C. Wohlin, eds., Berlin, Heidelberg, Germany: Springer, 2005, págs. 19-46. DOI: 10.1007/3-540-28244-0_2.

[51] B. Nuseibeh y S. Easterbrook, «Requirements Engineering: A Roadmap,» *Proceedings of the Conference on The Future of Software Engineering (ICSE 2000)*, págs. 35-46, 2000. DOI: 10.1145/336512.336523.

[52] S. Robertson y J. Robertson, *Mastering the Requirements Process: Getting Requirements Right*, 3.ª ed. Boston, MA, USA: Addison-Wesley Professional, 2012, pág. 592, ISBN: 978-0-321-81574-3.

[53] J. Kontio, J. Bragge y L. Lehtola, «The Focus Group Method as an Empirical Tool in Software Engineering,» en *Guide to Advanced Empirical Software Engineering*, Springer, 2008, págs. 93-116. DOI: 10.1007/978-1-84800-044-5_4.

[54] D. R. Millen, «Rapid Ethnography: Time Deepening Strategies for HCI Field Research,» *Proceedings of the 3rd Conference on Designing Interactive Systems (DIS)*, págs. 280-286, 2000. DOI: 10.1145/347642.347763.

[55] H. Sharp, H. Robinson, J. Segal y D. Furniss, «The Role of Story Cards and the Wall in XP Teams: A Case Study,» en *Proceedings of PPIG 2003*, See also: Sharp, H. et al., "Ethnographically-informed analysis for software engineers", *Int. J. Hum.-Comput. Stud.*, vol. 53, no. 1, pp. 33–52, 2000, 2003.

[56] IREB, *IREB CPRE Foundation Level – Syllabus, Version 3.2.0*, Latest English version as of May 2026; v3.3.0 available in German (April 2026), International Requirements Engineering Board, Karlsruhe, Germany, feb. de 2024. dirección: https://www.ireb.org/downloads/\#cpre-foundation-level-syllabus.

[57] C. Floyd, «A Systematic Look at Prototyping,» *Approaches to Prototyping*, R. Budde, K. Kuhlenkamp, L. Mathiassen y H. Züllighoven, eds., págs. 1-18, 1984. DOI: 10.1007/978-3-642-69796-8_1.

[58] E. Yu, «Modeling Strategic Relationships for Process Reengineering,» en *Social Modeling for Requirements Engineering*, E. Yu, P. Giorgini, N. Maiden y J. Mylopoulos, eds., Cambridge, MA, USA: MIT Press, 2011, págs. 3-50, ISBN: 978-0-262-01388-8. dirección: https://direct.mit.edu/books/edited-volume/3344/.

[59] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0.2*, Object Management Group, 2013. dirección: https://www.omg.org/spec/BPMN/2.0.2/.

[60] G. Adzic, *Specification by Example: How Successful Teams Deliver the Right Software*. Shelter Island, NY, USA: Manning Publications, 2011, ISBN: 978-1-617-29008-4.

[61] D. North, *Introducing BDD*. Better Software Magazine, 2006. dirección: https://dannorth.net/introducing-bdd/.

[62] D. M. Berry, E. Kamsties y M. M. Krieger, «From Contract Drafting to Software Specification: Linguistic Sources of Ambiguity,» *Technical Report, University of Waterloo*, 2003. dirección: https://cs.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf.

[63] H. Femmer, D. Méndez Fernández, S. Wagner y S. Eder, «Rapid Quality Assurance with Requirements Smells,» *Journal of Systems and Software*, vol. 123, págs. 190-213, 2017. DOI: 10.1016/j.jss.2016.10.Generic.

[64] C. Berger, R. Blauth, D. Boger, C. Bolster, G. Burchill, W. DuMouchel, F. Pouliot, R. Richter, A. Rubinoff, D. Shen, M. Timko y D. Walden, «Kano's Methods for Understanding Customer-defined Quality,» *Center for Quality of Management Journal*, vol. 2, n.º 4, págs. 3-36, 1993.

[65] M. E. Fagan, «Design and Code Inspections to Reduce Errors in Program Development,» *IBM Systems Journal*, vol. 15, n.º 3, págs. 182-211, 1976. DOI: 10.1147/sj.153.0182.

[66] ISO/IEC/IEEE, *ISO/IEC/IEEE 15288:2023 – Systems and software engineering – System life cycle processes*, 2.ª ed., Cancels and replaces ISO/IEC/IEEE 15288:2015, International Organization for Standardization, Geneva, Switzerland, mayo de 2023. DOI: 10.1109/IEEESTD.2023.10123367. dirección: https://www.iso.org/standard/81702.html.

[67] A. van Lamsweerde, *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Chichester, England: Wiley, 2009, pág. 706, ISBN: 978-0-470-01270-3.

[68] C. Cares, X. Franch, G. Grau, J. Horkoff, N. Maiden, J. Mylopoulos, J. C. Nardi, O. Pastor, A. Perini, A. Susi y E. Yu, *iStar 2.0 Language Guide*, arXiv preprint arXiv:1605.07767, 2016. dirección: https://arxiv.org/abs/1605.07767.

[69] A. Cockburn, *Writing Effective Use Cases*. Boston, MA, USA: Addison-Wesley Professional, 2001, ISBN: 978-0201702255.

[70] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3.ª ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2005, ISBN: 978-0-13-148906-6.

[71] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3.ª ed. Boston, MA, USA: Addison-Wesley Professional, 2004, ISBN: 978-0-321-19368-1.

[72] D. Harel, «Statecharts: A Visual Formalism for Complex Systems,» *Science of Computer Programming*, vol. 8, n.º 3, págs. 231-274, 1987. DOI: 10.1016/0167-6423(87)90035-9.

[73] G. Sindre y A. L. Opdahl, «Eliciting Security Requirements with Misuse Cases,» *Requirements Engineering*, vol. 10, n.º 1, págs. 34-44, 2005. DOI: 10.1007/s00766-004-0194-4.

[74] C. Rupp y SOPHISTen, die, «Requirements-Engineering und -Management,» *Hanser Verlag*, 2009, German reference; defines the "shall/should/will" template for requirements.

[75] S. Tiwari y S. S. Rathore, «A Systematic Literature Review on Requirements Smells,» *Journal of Software: Evolution and Process*, vol. 34, n.º 11, e2499, 2022. DOI: 10.1002/smr.2499.

[76] P. P.-S. Chen, «The Entity-Relationship Model — Toward a Unified View of Data,» *ACM Transactions on Database Systems*, vol. 1, n.º 1, págs. 9-36, 1976. DOI: 10.1145/320434.320440.

[77] Object Management Group, *Unified Modeling Language (UML), Version 2.5.1*, Object Management Group, 2017. dirección: https://www.omg.org/spec/UML/2.5.1/.

[78] T. DeMarco, *Structured Analysis and System Specification*. New York, NY, USA: Yourdon Press, 1978, ISBN: 978-0-917072-07-3.

[79] E. Yourdon, *Modern Structured Analysis*. Englewood Cliffs, NJ, USA: Prentice Hall, 1989, ISBN: 978-0-13-598632-0.

[80] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell y A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg, Germany: Springer, 2012, ISBN: 978-3-642-29043-5. DOI: 10.1007/978-3-642-29044-2.